Harmonia

Health Information Exchange Platform

Architecture, Implementation, and Deployment Reference

Harmonia Platform Architecture Team
m.a.hunter@outlook.com

September 2026 — Version 0.6.0

Executive Summary

The Harmonia Platform Overview

The **Harmonia Health Information Exchange (HIE)** is a modular, high-availability, distributed healthcare integration platform engineered for HL7 FHIR Release 5 (R5) compliance, real-time HL7 v2 clinical data ingestion, sub-millisecond in-memory cached access, and resilient, audit-compliant workflow execution.

Modern healthcare ecosystems demand zero-loss event streaming, high-throughput message translation, resilient task state persistence, and fine-grained clinical data access across disparate clinical systems. Harmonia addresses these challenges through a cohesive five-tier architectural design and cross-cutting governance framework:

- **Tier 1 — Presentation Services (*Iris*)**: Decoupled Vue 3 / TypeScript single-page applications (`iris-clinical`, `iris-console`, `iris-administration`) supported by a dual-port Jakarta EE Backend-For-Frontend (BEFE) gateway separating clinical read interfaces from administrative operational controls.
- **Tier 2 — Interface & Gateway Services (*Pylai*)**: High-throughput bidirectional MLLP and REST ingress and egress dispatch gateways, featuring dedicated destination-isolated queues to eliminate head-of-line blocking across external hospital systems (PAS, EMR, LIS, RIS-PACS).
- **Tier 3 — Transport & Workflow Services (*Petasos* & *Energeia*)**: An active-active Apache ActiveMQ Artemis messaging cluster providing sub-2-second failover and zero message loss, paired with the Ponos/Praxis/Erga modular Apache Camel workflow execution engine operating on canonical task instances (*Pragma*).
- **Tier 4 — Distributed Cache Services (*Hestia Mneme*)**: An in-memory Infinispan 15+ distributed cache grid delivering sub-millisecond retrieval of active clinical tasks, transaction states, and resource models with asynchronous write-behind persistence.
- **Tier 5 — Persistence Services (*Hestia Mnemosyne*)**: Durable long-term relational persistence uniting PostgreSQL 16+ database engines with Spring Boot HAPI FHIR JPA and Operations JPA servers.
- **Cross-Cutting Governance & Security (*Themis* & *Calliope*)**: Zero-trust role- and attribute-based access control (RBAC/ABAC), dual-gate Protected Health Information (PHI) audit logging, and centralized canonical domain data schemas (DTOs).

Structure and Scope of the Reference Publication

This document serves as the authoritative Harmonia Architecture, Implementation, and Deployment Reference. To address the diverse requirements of enterprise architects, integration engineers, system administrators, and clinical application developers, the documentation is divided into nine major Parts and twelve comprehensive Appendices:

- **PART I: Foundations (Chapters 1–4)**: Formulates platform motivation, the healthcare integration context, classical conceptual taxonomy, and business/clinical architecture.
- **PART II: Platform Architecture (Chapters 5–12)**: 5-Tier application component architecture, boundary gateways (Pylai), messaging transport (Petasos), collaboration (Agora), workflow orchestration (Energeia), presentation portals (Iris), and FHIR R5 Provider Registry core services.
- **PART III: Information & Integration (Chapters 13–15)**: Canonical information models, FHIR R5 schemas, HL7 v2 transformations, and MLLP integration guide.
- **PART IV: Middleware Architecture (Chapters 16–20)**: Clustered ActiveMQ Artemis HA, Infinispan distributed caching, PostgreSQL/HAPI FHIR R5 persistence, and Matrix Synapse homeserver architecture.
- **PART V: Runtime Architecture (Chapters 21–26)**: Container packaging, MicroK8s pod scheduling, single-node resilience, physical deployment, Themis security architecture, and governed

PHI logging.

- **PART VI: Normal Deployment (Chapters 27–32)**: Declarative configuration hierarchy, properties reference, Ubuntu host preparation, normal MicroK8s deployment runbooks, and undeployment/retention.
- **PART VII: Paradeigma Simulation (Chapters 33–35)**: Synthetic clinical generators, scenario framework, workload deployment runbooks, and strict production isolation guardrails.
- **PART VIII: Development (Chapters 36–38)**: Development environment, architecture invariants, and implementation guides for Ergon activity modules and Praxis workflow sequences.
- **PART IX: Operations (Chapters 39–40)**: Failure operations, recovery guarantees, operational runbooks, and PHI-aware logging implementation guide.
- **APPENDICES (Appendices A–L)**: Authoritative registers and checklists: Appendix A (Glossary), Appendix B (System Inventory), Appendix C (Module Register), Appendix D (Middleware Matrix), Appendix E (Configuration Register), Appendix F (Port/Protocol Register), Appendix G (Image Register), Appendix H (Kubernetes Register), Appendix I (Persistence Register), Appendix J (Dependency Matrix), Appendix K (Normal Deployment Checklist), Appendix L (Paradeigma Deployment Checklist).

ArchiMate 3.2 Architectural Modeling Framework

To ensure complete structural rigor, unambiguous domain boundaries, and formal traceability across business, application, and infrastructure tiers, this specification is authored strictly in accordance with **The Open Group ArchiMate 3.2 Standard**.

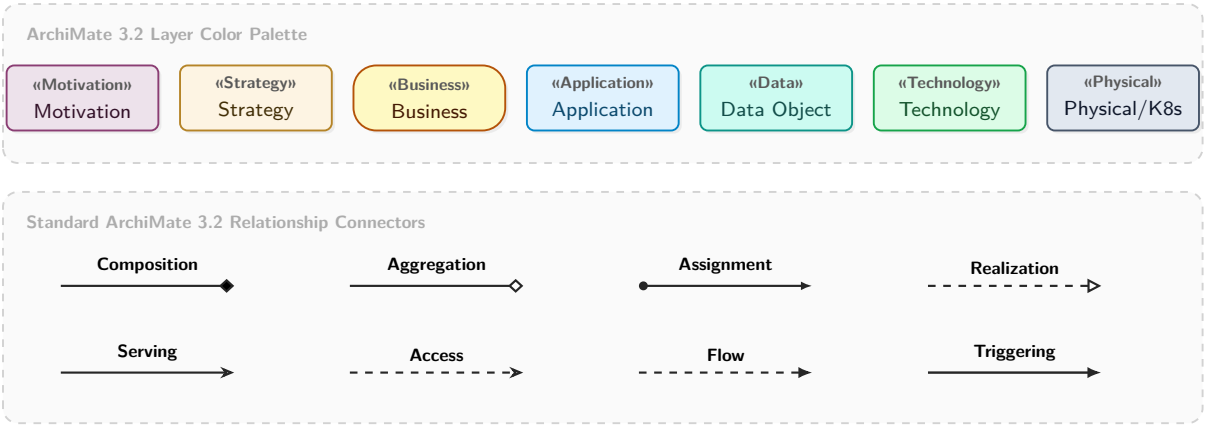


Figure 1: ArchiMate 3.2 Visual Modeling Notation, Layer Color Palette, and Relationship Types

Classical Architectural Nomenclature & System Meanings

Harmonia adopts naming conventions rooted in classical Greek mythology and philosophy. Each name explicitly reflects the functional responsibility and domain boundary of the underlying software component.

Subsystem	Architectural Role	Classical Origin	Platform Responsibilities
`Harmonia`	Health Integration Environment	*Harmonia* — Goddess of concord and balance.	Overall platform uniting clinical protocols, streaming messaging, cache grids, and relational storage.
`Petasos`	Messaging Transport Subsystem	*Petasos* — Winged hat of Hermes, symbol of swift dispatch.	Clustered ActiveMQ Artemis messaging, durable queues, client failover, and `PetasosMessage` envelopes.
`Energeia`	Workflow Services	*Energeia* — Aristotelian actuality and continuous activity.	Top-level workflow services aggregator containing Ponos, Praxis, Erga, and Ponos CLI.
`Ponos`	WorkEngine Execution Container	*Ponos* — Personification of industrious toil and labor.	WildFly 31 Jakarta EE 10 WAR container hosting Camel contexts, Petasos conduits, and task worker managers.
`Erga`	Modular Task Activities	*Erga* — Plural of *Ergon*, works, deeds, or tasks.	Single-responsibility Camel route components (`ErgonBase`) executing discrete clinical mappings.
`Praxis`	Workflow Sequence Orchestrator	*Praxis* — Structured action or enactment of theory.	Sequence orchestrator chaining ordered Ergon steps and recording multi-phase state checkpoints.
`Pragma`	Task Instance / State Entity	*Pragma* — That which has been done, concrete deed.	Canonical task state model holding inputs, outputs, checkpoints, and bidirectional FHIR `Task` mapping.
`Calliope`	Canonical Schema Repository	*Calliope* — Chief Muse of eloquence and harmonious order.	Shared information models, domain DTOs, schemas, and payload envelopes used across all modules.
`Themis`	Policy Governance Subsystem	*Themis* — Titaness of divine law and order.	Default-deny ABAC/RBAC authorization engine, PEP interceptors, and non-PHI audit event logging.
`Mneme`	In-Memory Distributed Cache	*Mneme* — Muse of active memory and rapid recall.	Infinispan distributed L1 cache grid providing low-latency `task-cache` and `tasksequence-cache`.
`Mnemosyne`	Relational Persistence Tier	*Mnemosyne* — Titaness of durable memory.	Long-term PostgreSQL storage comprising Spring Boot FHIR JPA and Operations JPA servers.
`Hestia`	Data Services Subsystem	*Hestia* — Goddess of the hearth and foundational stability.	Data services aggregator managing caching (Mneme) and persistence (Mnemosyne).
`Iris`	Presentation Services	*Iris* — Goddess of the rainbow and divine messenger.	Decoupled multi-portal presentation tier comprising Jakarta EE dual-port BEFE gateway, Clinical UI SPA, Console UI SPA, and Administration Dual-Portal SPA.
`Agora`	Collaboration Services	*Agora* — Ancient assembly and civic gathering place.	Collaboration gateway projecting clinical events into Matrix Synapse Spaces, child rooms, and multidisciplinary care team chats.
`Pylai`	Gateway / Interface Services	*Pylai* — Gateways, portals of entry and exit.	Boundary interface services including inbound MLLP gateways (`pylai-mllp-in`), multi-instance outbound MLLP gateways (`pylai-mllp-out`), and FHIR R5 Provider Registry REST gateway (`pylai-fhir-registry`).
`Paradeigma`	Clinical Simulation & Testbed	*Paradeigma* — Model, pattern, exemplar.	Reference clinical simulation subsystem providing deterministic hospital actor emulators (PAS, EMR, LMS, RIS-PAC), multi-system Patient Journey orchestration, and automated fault injection testbed.

Table 1: Harmonia Classical Architectural Nomenclature and System Meaning Matrix

Contents

Executive Summary **i**
ArchiMate 3.2 Architectural Modeling Framework . . . ii
Classical Architectural Nomenclature & System Meanings . . . ii

I Foundations **1**

1 Motivation & Strategy Architecture **2**
1.1 Stakeholders & Key Drivers . . . 2
1.1.1 Stakeholder Analysis . . . 2
1.1.2 Core Drivers . . . 3
1.2 Strategic Goals . . . 3
1.3 Core Architectural Principles . . . 3
1.4 Strategic Capabilities & End-to-End Value Stream . . . 4
1.4.1 Strategic Value Stream Flow . . . 4

2 Platform Motivation, HIE Context & Architectural Principles **7**
2.1 Healthcare Interoperability Context & Stakeholders . . . 7
2.2 Core Architectural Drivers . . . 8
2.3 Core Architectural Principles . . . 8
2.4 Strategic Value Stream Flow . . . 9

3 Conceptual Taxonomy & Subsystem Domains **10**
3.1 Core Conceptual Domains . . . 10
3.1.1 Presentation Services (`Iris`) . . . 10
3.1.2 Gateway & Protocol Services (`Pylai`) . . . 11
3.1.3 Messaging & Transport Subsystem (`Petasos`) . . . 11
3.1.4 Workflow & Execution Engine (`Energeia`) . . . 11
3.1.5 Data, Caching & Persistence Services (`Hestia`) . . . 11
3.1.6 Canonical Schemas & Security Governance (`Calliope` & `Themis`) . . . 11
3.1.7 Collaboration Services (`Agora`) . . . 11
3.1.8 Synthetic Clinical Simulation (`Paradeigma`) . . . 12

4 Business & Clinical Architecture **13**
4.1 Healthcare Business Actors & Operational Roles . . . 13
4.2 Clinical Business Services . . . 13
4.2.1 Patient Admission & Movement Service (`HL7-ADT`) . . . 13
4.2.2 Practitioner & Staff Directory Service (`HL7-MFN`) . . . 15
4.2.3 FHIR R5 Clinical Data Discovery Service . . . 15
4.2.4 Operations Telemetry & Workflow Administration Service . . . 15
4.2.5 Clinical Outbound Notification & Egress Service (`HL7-MLLP-OUT`) . . . 15
4.2.6 FHIR R5 Provider Directory Management Service . . . 15
4.3 Clinical Business Process Workflows . . . 15
4.3.1 Process: Ingest and Reconcile HL7 ADT Trigger . . . 15
4.3.2 Process: Outbound MLLP Dispatch and ACK Validation Lifecycle . . . 15
4.3.3 Process: Governed Provider Registry Change Ingestion & Verification . . . 16
4.4 Core Business Objects . . . 16

II Platform Architecture **17**

5 Application Architecture **18**
5.1 Presentation Tier: `Iris` Subsystem . . . 18
5.1.1 `iris-befe` (Backend-For-Frontend Gateway) . . . 18
5.1.2 `iris-clinical` (Clinical Single-Page Application) . . . 19
5.1.3 `iris-console` (Operations & Workflow Console) . . . 19
5.1.4 `iris-administration` (Provider Directory Administration & Self-Service Portal) . 19
5.2 Interface & Gateway Tier: `Pylai` Subsystem . . . 19
5.2.1 `pylai-mllp-in` (Inbound Clinical MLLP Gateway) . . . 19
5.2.2 `pylai-mllp-out` (Resilient Multi-Instance Outbound MLLP Gateway) . . . 19
5.2.3 `pylai-mllp-base` (Shared Gateway Integration Library) . . . 20
5.2.4 `pylai-mllp-cli` (Synthetic Test & Diagnostic Utility) . . . 20
5.2.5 `pylai-fhir-registry` (FHIR R5 Provider Registry REST Gateway) . . . 20
5.3 Policy & Authorisation Subsystem: `Themis` . . . 20
5.3.1 `themis-api` (Domain Contracts & Security Models) . . . 20
5.3.2 `themis-core` (Deterministic Evaluation Engine) . . . 21
5.3.3 `themis-audit` (Structured Security Decision Auditing) . . . 21
5.3.4 Multi-Tier Boundary Enforcement & Non-Amplification Invariant . . . 22
5.4 Messaging Transport Subsystem: `Petasos` . . . 22
5.4.1 `petasos-api` (Core Abstractions) . . . 22
5.4.2 `petasos-core` (Serialization & Deduplication) . . . 22
5.4.3 `petasos-artemis` (Artemis Adapter) . . . 23
5.5 Workflow Services Subsystem: `Energeia` . . . 23
5.5.1 `energeia/ponos` (WorkEngine Runtime Container) . . . 23
5.5.2 `energeia/praxis` (Workflow Sequence Orchestration) . . . 23
5.5.3 `energeia/erga` (Modular Task Activities) . . . 24
5.6 Data Services Subsystem: `Hestia` . . . 24
5.7 Canonical Schema Repository: `Calliope` . . . 24

6 External Gateways (`Pylai`) **25**
6.1 MLLP Inbound Ingress Gateway (`pylai-mllp-in`) . . . 25
6.1.1 MLLP Framing and Transport . . . 25
6.1.2 Ingress Dual-Write Safety (REC-001) . . . 25
6.2 MLLP Outbound Egress Dispatcher (`pylai-mllp-out`) . . . 26
6.2.1 Destination-Isolated Queuing . . . 26
6.2.2 Synchronous ACK / NACK Evaluation & Fan-Out Tracking (REC-002) . . . 26
6.3 FHIR Provider Registry Gateway (`pylai-fhir-registry`) . . . 26

7 Messaging & Event Transport (`Petasos`) **28**
7.1 Transport Abstraction Invariant (`petasos-api`) . . . 28
7.2 Core Routing and Deduplication (`petasos-core`) . . . 29
7.2.1 Sliding-Window Deduplication . . . 29
7.3 ActiveMQ Artemis HA Adapter (`petasos-artemis`) . . . 29
7.3.1 Failover Connection URL Configuration . . . 29

8 Collaboration & Matrix Integration (`Agora`) **30**
8.1 Subsystem Decomposition & Layering . . . 30
8.2 Key Architectural Decisions (ADRs) . . . 30
8.3 Collaboration Projections & Room Lifecycles . . . 30
8.3.1 Patient Collaboration Space Hierarchy . . . 31
8.3.2 Practitioner & Group Spaces . . . 31
8.4 Security Governance & Reconciliation . . . 31
8.4.1 Themis Default-Deny Authorization . . . 31
8.4.2 Membership Drift Reconciliation . . . 31
8.5 Application Service Endpoint & Petasos Integration . . . 32

9 Workflow & Task Processing Engine (Energeia) 33
9.1 Ponos Task Processor (`energeia-ponos`) . . . 33
9.2 Modular Activity Units (`energeia-erga`) . . . 33
9.2.1 Standard Ergon Activity Implementations . . . 33
9.3 Workflow Orchestration Sequences (`energeia-praxis`) . . . 34
9.3.1 Dynamic Route Synthesis & Checkpointing . . . 35

10 Presentation Services & User Interfaces (`Iris`) 36
10.1 Presentation Single-Page Applications (SPAs) . . . 36
10.1.1 1. `iris-clinical` (Clinical Portal) . . . 36
10.1.2 2. `iris-console` (Operations & Telemetry Portal) . . . 36
10.1.3 3. `iris-administration` (Directory Administration Portal) . . . 37
10.2 Backend-For-Frontend Gateway (`iris-befe`) . . . 38
10.2.1 Operations Backend Architecture (Port 8090) . . . 38
10.3 The Iris Design System (`@harmonia/iris-befe`) . . . 38
10.3.1 Architecture and Three-Tier Provenance . . . 38
10.3.2 Design Tokens and Light Theme Baseline . . . 39
10.4 Future Migration Assessment: `iris-clinical` & `iris-administration` . . . 39

11 Iris Presentation Tier and User Interfaces Specification & User Guide 40
11.1 Presentation Tier Overview & Architectural Principles . . . 40
11.1.1 Multi-Portal Topology and Separation of Concerns . . . 40
11.1.2 Architectural Principles . . . 40
11.2 Iris BEFE Dual-Port Gateway Specification . . . 41
11.2.1 Dual-Port Architecture: Port 8080 vs Port 8090 . . . 42
11.2.2 BEFE JAX-RS Resource Endpoints (Port 8080) . . . 42
11.2.3 CORS and Security Headers . . . 43
11.3 Iris Clinical Portal (`iris-clinical`) Specification & User Guide . . . 43
11.3.1 Component Topology and Layout Architecture . . . 43
11.3.2 Comprehensive Clinical View Catalog . . . 43
11.3.3 Pinia State Management: Clinical Store Pattern . . . 43
11.4 Iris Operations Console (`iris-console`) Specification & User Guide . . . 45
11.4.1 Architectural Perspectives & Shell Topology . . . 45
11.4.2 The Six Canonical Operational Perspectives . . . 45
11.4.3 Telemetry Gaps Register (`IRIS-API-GAP`) . . . 46
11.4.4 Operations Backend REST API Catalog (Port 8090) . . . 47
11.5 Iris Provider Administration (`iris-administration`) Specification & User Guide . . . 47
11.5.1 Dual-Portal Architecture and Route Guarding . . . 48
11.5.2 The Governed Change Workflow Lifecycle . . . 48
11.6 Themis Security & RBAC Integration . . . 48
11.6.1 Themis Authority Scopes . . . 48
11.6.2 Client-Side Router Navigation Guards . . . 48
11.6.3 Visual Security Badging . . . 50
11.7 Concrete Operational Examples & Step-by-Step Walkthroughs . . . 50
11.7.1 Example 1: Clinician Locating an On-Call Specialist and Endpoint . . . 50
11.7.2 Example 2: Operations Engineer Diagnosing Queue Backpressure and Workflow Stalls 51
11.7.3 Example 3: Practitioner Requesting Address Change and Admin Approval . . . 51

12 Provider Registry Core Services 53
12.1 Overview & Engineering Objectives . . . 53
12.2 Architectural Topology & 5-Tier Interaction Model . . . 53
12.2.1 Synchronous Read and Search Data Path . . . 54
12.2.2 Asynchronous Governed Change Path . . . 54
12.3 FHIR R5 Provider Registry Resource Model & Relationship Topology . . . 54
12.3.1 Relationship Graph and Referential Topology . . . 54
12.3.2 Endpoint as an Independent First-Class Entity . . . 54
12.4 REST API Gateway Contract & Interaction Matrix . . . 56
12.4.1 Multi-Parameter Search Capabilities . . . 56
12.4.2 CapabilityStatement Specification (`GET /metadata`) . . . 56

12.4.3 Asynchronous Task Polling (`GET /Task/{id}`) . . . 56
12.5 Governed Change Processing Lifecycle & Pragma State Machine . . . 57
12.5.1 State Machine Transition Rules . . . 57
12.6 Per-Resource Ergon Processing Activities & Referential Integrity Engine . . . 58
12.6.1 `ProviderRegistryReferenceValidator` Engine . . . 58
12.7 Authoritative Persistence, Optimistic Concurrency & Versioning Model . . . 59
12.7.1 Monotonic Version Incrementing . . . 59
12.7.2 Optimistic Concurrency Control (`If-Match` / `ETag`) . . . 59
12.8 Security, Themis Governance & Multi-Tier Authorization . . . 59
12.8.1 Mnemonic Roles and Granular Authority Matrix . . . 59
12.8.2 `FhirSecurityInterceptor` & Defence-in-Depth Pipeline . . . 59
12.9 Paradeigma Exemplar Generation & Verification Testbed . . . 60
12.9.1 `SyntheticProviderRegistryGenerator` . . . 60
12.9.2 Automated Integration & Verification Test Suites . . . 60
12.10Summary & Engineering Checklist . . . 61

III Information & Integration 62

13 Information Architecture & Standard Message Envelopes 63
13.1 Canonical Domain Models (`Calliope`) . . . 63
13.1.1 1. `Pragma` (Task State Entity) . . . 63
13.1.2 2. `PragmaCheckpoint` (Execution Audit Snapshot) . . . 64
13.1.3 3. `ProviderRegistryChangePragma` (Directory Change Wrapper) . . . 64
13.1.4 4. Canonical Provider Registry Validation Codes . . . 64
13.2 Canonical Outbound Models (`pylai-mllp-base`) . . . 64
13.3 Petasos Messaging Transport Envelopes (`petasos-api`) . . . 65
13.4 HL7 FHIR Release 5 (R5) Resource Schemas . . . 65

14 Data & Information Architecture 66
14.1 Canonical Domain Models (`Calliope`) . . . 66
14.1.1 1. `Pragma` (Task State Entity) . . . 66
14.1.2 2. `PragmaCheckpoint` (Execution Audit Snapshot) . . . 66
14.1.3 3. `ProviderRegistryChangePragma` (Directory Change Wrapper) . . . 66
14.1.4 4. Canonical Provider Registry Validation Codes . . . 67
14.2 Canonical Outbound Models (`pylai-mllp-base`) . . . 67
14.2.1 1. `OutboundMllpRequest` . . . 67
14.2.2 2. `OutboundMllpResponse` . . . 67
14.2.3 3. `MllpDestinationConfig` . . . 68
14.3 The 5-Phase Checkpoint Lifecycle State Flow . . . 68
14.4 `PetasosMessage` Transport Envelope . . . 69
14.5 Supported FHIR R5 Resource Schemas . . . 69
14.6 Relational Persistence Schema (`Mnemosyne-Clinical`) . . . 69

15 Implementation Guide for MLLP Ingress and Egress Services 71
15.1 Overview & Engineering Objectives . . . 71
15.2 Ingress Architectural Blueprint & Framing Protocols . . . 71
15.2.1 MLLP Transport Framing . . . 72
15.2.2 The 8-Phase Ingress Processing Lifecycle . . . 72
15.3 Ingress Component Implementation . . . 72
15.3.1 Camel RouteBuilder Configuration . . . 72
15.3.2 Processor Wrapper & Context Management . . . 73
15.3.3 HL7 v2 Data Extraction with HAPI Terser . . . 73
15.4 Canonical FHIR Model Mapping & Resource Builders . . . 74
15.4.1 FHIR Communication Resource Construction . . . 74
15.4.2 FHIR Task (Pragma) Resource Construction . . . 75
15.5 Persistence, Event Dispatch & Synchronous Acknowledgement . . . 75
15.5.1 Persistence Integration with Mneme & Mnemosyne . . . 76
15.5.2 Synchronous Acknowledgement Mapping Rules . . . 76

15.5.3 REST Management & CDI Deployment Configuration . . . 77
15.6 Multi-Instance Egress Architecture & Queue Partitioning . . . 77
15.6.1 Head-of-Line Blocking Mitigation & Destination Isolation . . . 77
15.6.2 Outbound Configuration & Destination Registry . . . 78
15.6.3 Outbound Data Transfer Models . . . 78
15.7 Outbound Queue Consumer & Camel Dynamic MLLP Routes . . . 78
15.7.1 Dedicated Task Queue Consumer . . . 78
15.7.2 Camel Dynamic Outbound Route Builder . . . 79
15.7.3 Outbound MLLP Processor Header Injection . . . 80
15.8 Downstream HL7 ACK/NACK Deep Inspection . . . 81
15.8.1 MSA & ERR Segment Parsing Specification . . . 81
15.8.2 Downstream ACK Code Decision Matrix . . . 82
15.9 State, Task, and Provenance Lifecycle Management . . . 82
15.9.1 Outbound State Lifecycle Synchronization . . . 82
15.9.2 FHIR Provenance Resource Construction . . . 83
15.10 Testing, Verification Harnesses & Containerization . . . 84
15.10.1 Tier 1: Unit Testing with HAPI HL7 v2 Parsers . . . 84
15.10.2 Tier 2: Camel Route Harness Testing . . . 84
15.10.3 Tier 3: Embedded Mock MLLP Server Integration Testing . . . 85
15.10.4 Tier 4: Multi-Instance Docker Compose Deployment Topology . . . 85

IV Middleware Architecture 87

16 Technology & Infrastructure Architecture 88
16.1 System Software Runtimes . . . 88
16.2 In-Memory Cache Grid & Persistence SPI (`Hestia`) . . . 89
16.3 Security Governance, Authorization & Audit Infrastructure (`Themis`) . . . 89
16.3.1 Data Classification & Security Tagging (`HarmoniaSecurityLabelEnum`) . . . 89
16.3.2 Identity Context Transport & Serialization . . . 90
16.3.3 Non-PHI Decision Audit Trail Architecture (`ThemisAuditService`) . . . 90
16.3.4 Fail-Closed Determinism & Reason Code Diagnostics . . . 90
16.4 PHI-Aware Diagnostic Logging Runtime & Appender Isolation . . . 90
16.4.1 Dual-Logger Runtime Framework (`Calliope` / `net.fhirfactory.harmonia.logging`) 90
16.4.2 Dual-Gate Security Evaluation Mechanics . . . 90
16.4.3 SLF4J Namespace Routing & Non-Additive Appender Isolation . . . 91
16.5 Master Platform Port Allocation Specification . . . 91

17 Apache ActiveMQ Artemis Topology 92
17.1 4-Broker HA Clustering Topology . . . 92
17.1.1 Symmetric Server-Side Clustering (`petasos-cluster`) . . . 92
17.1.2 Synchronous Journal Replication . . . 92
17.2 Address & Queue Settings (`broker.xml`) . . . 93
17.3 Durable Journal Persistence & Storage Requirements . . . 93
17.4 Broker-Level Message Deduplication . . . 93

18 Infinispan Distributed Cache Grid 94
18.1 Clustered Grid Topology & Protocols . . . 94
18.2 Named Cache Configurations & Semantics . . . 94
18.2.1 Write-Behind Store Architecture (`mneme-persistence`) . . . 94
18.3 Cache Classification Invariant . . . 95

19 PostgreSQL & HAPI FHIR R5 Persistence Architecture 96
19.1 Database Separation: Clinical vs Operations vs Collaboration . . . 96
19.1.1 1. Clinical Database (`postgres-1` & `postgres-2`) . . . 96
19.1.2 2. Operations Database (`postgres-ops-1` & `postgres-ops-2`) . . . 96
19.1.3 3. Collaboration Database (`postgres-synapse`) . . . 96
19.2 JPA Server Deployment Architecture . . . 96
19.3 Optimistic Concurrency & Referential Integrity . . . 96

20 Matrix Synapse Homeserver & Collaboration Middleware **98**
20.1 Homeserver Clustered Topology & Network Architecture . . . 98
20.2 Application Service Registration (`appservice-agora.yaml`) . . . 98
20.2.1 Registration Invariants . . . 98
20.3 Spaces Hierarchy & Zero-PHI Metadata Invariant . . . 99
20.4 Ephemeral Retention & Database Archival . . . 99

V Runtime Architecture **100**

21 Container Packaging & Runtime Profiles **101**
21.1 Container Image Hierarchy & Registry Taxonomy . . . 101
21.1.1 1. Upstream Middleware Base Images . . . 101
21.1.2 2. Java Application Runtimes . . . 101
21.2 JVM Resource Profiling & Memory Ergonomics . . . 101

22 MicroK8s Pod Scheduling, Services & Ingress Routing **102**
22.1 Service Discovery & Internal DNS Mesh . . . 102
22.2 Ingress Routing & NodePort Mapping . . . 103

23 Single-Node Resilience vs Host-Level Availability **104**
23.1 Resilience Semantics on Single-Node MicroK8s . . . 104
23.2 Host-Level Failure Domain Boundaries . . . 104
23.3 Production Multi-Node Scaling Pathway . . . 104

24 Physical & High Availability Deployment **105**
24.1 Clustering vs. High Availability (HA) Replication . . . 105
24.1.1 1. Server-Side Clustering (`petasos-cluster`) . . . 105
24.1.2 2. Journal Replication Groups (`group-a`, `group-b`) . . . 105
24.2 Isolated Persistent Storage Ownership . . . 106
24.3 Client Reconnection & Failover Configuration . . . 106
24.4 In-Memory Cache Grid HA Topology (`Mneme`) . . . 106
24.5 Docker Compose Multi-Container Deployment . . . 106
24.5.1 Provider Registry High Availability & Crash Recovery . . . 107
24.5.2 Egress Failure Domain Isolation . . . 107

25 Themis Policy Evaluation & Security Boundaries **108**
25.1 Default-Deny Policy Invariant . . . 108
25.2 The 4-Gate Defence-in-Depth Pipeline . . . 108

26 PHI-Sanitized Logging & Audit Lineage Tracking **110**
26.1 Zero-PHI Logging Invariant . . . 110
26.2 Dual-Logger Pattern & Non-Additive Routing . . . 110

VI Normal Deployment **111**

27 Declarative Configuration Hierarchy **112**
27.1 The 5-Stage Configuration Hierarchy . . . 112
27.2 Secret Management & Zero-Plaintext Policy . . . 112

28 Configuration Register & Property Cross-Reference **113**
28.1 Core Platform Properties Matrix . . . 113

29 Ubuntu Host Preparation & MicroK8s Add-ons **114**
29.1 Hardware Prerequisites & OS Requirements . . . 114
29.2 Kernel Parameter & Sysctl Tuning . . . 114
29.3 MicroK8s Installation & Required Add-ons . . . 114

30 Normal Workload Deployment & Verification Runbook **115**
30.1 Phase 1: Build & Container Image Push . . . 115
30.2 Phase 2: Apply Kustomize Overlay Manifests . . . 115
30.3 Phase 3: Verification & Health Validation Runbook . . . 115

31 Safe Undeployment & Storage Retention Mechanics **117**
31.1 Workload Termination (Data-Preserving) . . . 117
31.2 Authoritative Storage Wipe Protocol . . . 117

32 Platform Infrastructure, MicroK8s & Middleware Configuration **118**
32.1 Platform Overview & Infrastructure Separation Strategy . . . 118
32.1.1 Architectural Separation of Concerns . . . 118
32.1.2 Single-Node MicroK8s Resilience Semantics vs Multi-Node Scaling . . . 118
32.2 Platform & Host Infrastructure Prerequisites . . . 119
32.2.1 Supported Operating Systems & Architecture Support . . . 119
32.2.2 Hardware Sizing Profiles & Resource Allocation Matrix . . . 119
32.2.3 Linux Kernel Networking & Sysctl Configuration . . . 119
32.2.4 Host Firewall, Ingress Routing & Port Exposure Matrix . . . 120
32.3 MicroK8s Component Architecture & Add-on Specification . . . 120
32.3.1 Snap Packaging, Confinement & Permission Management . . . 120
32.3.2 Authoritative Add-on Matrix . . . 120
32.3.3 CoreDNS Add-on (`dns`) Configuration . . . 120
32.3.4 Nginx Ingress Controller (`ingress`) Specification . . . 122
32.3.5 Dynamic Storage Provisioner (`hostpath-storage`) Specification . . . 123
32.3.6 Resource Telemetry Add-on (`metrics-server`) Specification . . . 123
32.4 Kubernetes Middleconfiguration & Cluster Architecture . . . 123
32.4.1 Declarative Kustomize Base and Overlay Hierarchy . . . 123
32.4.2 Namespace Governance & Multi-Tier NetworkPolicies . . . 124
32.4.3 Secret Management, Sensitive Credentials & Ansible Vault . . . 125
32.4.4 Container Health Probes & Resource Allocation Matrix . . . 125
32.5 Tier-by-Tier Subsystem Middleware Configuration Catalog . . . 125
32.5.1 Petasos Messaging: Apache ActiveMQ Artemis 2.33.0 HA Cluster . . . 125
32.5.2 Hestia Mneme: Infinispan 15 Distributed In-Memory Cache Grid . . . 126
32.5.3 Hestia Mnemosyne & PostgreSQL 16 Relational Persistence . . . 126
32.5.4 Energeia Ponos: WildFly 31 Workflow Execution Engine . . . 126
32.5.5 Pylai Gateways: Inbound & Outbound MLLP Interfaces . . . 126
32.5.6 Iris Presentation: BEFE Gateway & Vue 3 SPAs . . . 127
32.6 Ansible Deployment Automation, Storage Lifecycle & Operational Runbooks . . . 127
32.6.1 Automated Playbook Architecture & Workflow Pipeline . . . 127
32.6.2 Persistent Storage Lifecycle Governance: Safe Teardown vs Destructive Purge . . . 127
32.6.3 Operational Health Check Runbooks & Diagnostic Commands . . . 127
32.6.4 Disaster Recovery & Single-Node Node Restoration Procedures . . . 128

VII Paradeigma Simulation **130**

33 Synthetic Clinical Generators & Scenario Framework **131**
33.1 Synthetic Actor Simulators & Module Inventory . . . 131
33.2 Deterministic Patient Journeys & Scenario Engine . . . 132
33.2.1 REST Scenario Orchestration API . . . 132

34 Paradeigma Workload Deployment & Isolation Guardrails **133**
34.1 Strict Production Isolation Invariants . . . 133
34.2 Automated ArchUnit Verification . . . 133
34.3 Normal Production vs Paradeigma Simulation Deployment Matrix . . . 134
34.4 Paradeigma Simulation Profile Deployment . . . 134

35 Paradeigma Clinical Simulation Subsystem **135**
35.1 Subsystem Overview & Architectural Principles . . . 135
35.1.1 Core Design Principles . . . 135
35.2 ArchiMate Application & Interface Landscape . . . 135
35.3 Formal Interface Catalogue (PD-01 through PD-09) . . . 136
35.3.1 MLLP Framing & Transport Protocol . . . 136
35.3.2 Synchronous Acknowledgement Contract . . . 136
35.4 Hospital Simulator Microservice Specifications . . . 137
35.4.1 Patient Administration System Simulator (`paradeigma-pas`) . . . 137
35.4.2 Electronic Medical Record Simulator (`paradeigma-emr`) . . . 137
35.4.3 Laboratory Information System Simulator (`paradeigma-lms`) . . . 137
35.4.4 Radiology Information System & PACS Simulator (`paradeigma-rispac`) . . . 138
35.5 Synthetic Data Models & Deterministic Generators . . . 138
35.5.1 Canonical Domain Entities . . . 138
35.5.2 Seed-Based Deterministic Generation Engine . . . 139
35.6 Scenario Orchestration Engine & Patient Journey Workflows . . . 140
35.6.1 9-Step Canonical Patient Journey Sequence . . . 140
35.6.2 Execution Profiles . . . 140
35.6.3 Scenario Control REST API . . . 140
35.7 Failure Simulation, Fault Injection & Automated Recovery . . . 140
35.7.1 Fault Injection Modes (`FailureSimulator`) . . . 140
35.7.2 Automated Recovery Acceptance Test Suite . . . 140
35.8 Deployment Topology & Docker Orchestration . . . 141
35.9 Provider Registry Simulation & Synthetic FHIR R5 Generators . . . 141
35.9.1 Standardized Australian Digital Health Identifiers . . . 142
35.9.2 Dedicated Synthetic Generators Catalog . . . 142
35.9.3 Interconnected Graph Builders (`SyntheticProviderRegistryGenerator`) . . . 142
35.10 Themis Security Simulation & Authorization Fixtures . . . 143
35.10.1 Standard Security Actors Catalog . . . 143
35.10.2 Security Context Orchestration (`SecurityScenarioContext`) . . . 143
35.10.3 Negative Testing Authorization Seams . . . 145
35.11 PHI-Aware Logging Simulation & Credential Protection . . . 145
35.11.1 Dual-Gate Verification Probe (`PhiLogTestProbe`) . . . 145
35.11.2 Synthetic Secret Leakage Protection (`SecretLeakageAssertion`) . . . 146
35.11.3 Integrated Multi-Capability Verification Listing . . . 146
35.12 Comprehensive Configuration Reference Matrix . . . 147
35.12.1 Simulator Service Properties & Port Allocations . . . 147
35.12.2 Security & PHI Logging Governance Configuration . . . 148
35.12.3 FailureSimulator Fault Injection Knobs . . . 148
35.13 Step-by-Step CLI & REST Execution Instructions . . . 148
35.13.1 Building the Solution via Maven . . . 148
35.13.2 Running Simulators as Standalone Microservices . . . 149
35.13.3 Docker Compose Topology Orchestration . . . 149
35.13.4 REST API Scenario Orchestration (cURL Snippets) . . . 149
35.14 Automated Scenario & Integration Test Suite Reference . . . 150

VIII Development **151**

36 Platform Development, Architecture Invariants & Rules **152**
36.1 Build Environment & Toolchain Prerequisites . . . 152
36.2 Maven Multi-Module Reactor Architecture . . . 152
36.3 The Eight Mandatory Architectural Invariants . . . 152
36.4 Continuous ArchUnit Verification . . . 153

37 Implementation and Deployment Guide for Ergon Workflow Activity Modules 154
37.1 Overview & Engineering Objectives . . . 154
37.2 Architectural Blueprint & the ErgonBase Contract . . . 154
37.2.1 RouteBuilder Inheritance and Processing Topologies . . . 154
37.2.2 Exchange Headers and Properties Invariants . . . 155
37.2.3 The Pragma Task Envelope Model . . . 155
37.2.4 Security Tagging with FhirSecurityTagManager . . . 156
37.3 Step-by-Step Code Development Blueprint . . . 156
37.3.1 Step 1: Maven Project Configuration . . . 156
37.3.2 Step 2: Subclassing ErgonBase & Constructor Wiring . . . 156
37.3.3 Step 3: Defensive Payload Extraction . . . 157
37.3.4 Step 4: Clinical Transformation & Resource Generation . . . 157
37.3.5 Step 5: Pragma Output & Checkpoint Recording . . . 157
37.3.6 Step 6: Structured Exception Handling . . . 157
37.4 Production Reference Implementation: ObservationEnrichmentErgon . . . 157
37.5 Praxis Workflow Orchestration & Step Chaining . . . 160
37.5.1 Linear Workflow Sequences . . . 160
37.5.2 Conditional Branching and Error Recovery . . . 160
37.6 CDI Configuration, Dependency Injection & Scoping . . . 161
37.6.1 Bean Scoping: @Dependent vs. @ApplicationScoped . . . 161
37.6.2 CDI Deployment Descriptor (beans.xml) . . . 161
37.7 Testing Strategies, Test Fixtures & Validation . . . 161
37.7.1 CamelTestSupport Unit Testing Pattern . . . 161
37.8 Deployment, Containerization & Operational Runbook . . . 162
37.8.1 WildFly & Ponos Container Architecture . . . 162
37.8.2 Dynamic Live Reload via ponos-cli . . . 162
37.8.3 Concurrency Tuning and Thread Pool Sizing . . . 162
37.8.4 Dead-Letter Queue (DLQ) Recovery Operations . . . 163
37.9 Summary & Best Practices Checklist . . . 163

38 Implementation and Deployment Guide for Praxis Workflow Sequences 165
38.1 Overview & Engineering Objectives . . . 165
38.2 Architectural Blueprint & the Praxis Paradigm . . . 165
38.2.1 Decoupling Definition from Runtime Execution . . . 166
38.2.2 Topic Subscription Matching & Hierarchical Routing . . . 166
38.2.3 Audit Checkpoint Lifecycle across Execution Boundaries . . . 167
38.2.4 Dual-Cache Architecture in the Mneme Grid . . . 167
38.3 Step-by-Step Praxis Creation Blueprint . . . 167
38.3.1 Phase 1: Defining Declarative Workflow Metadata . . . 167
38.3.2 Phase 2: Activity Ordering and Sequence Validation . . . 168
38.3.3 Phase 3: Route Synthesis and Ingress Conduits . . . 168
38.3.4 Phase 4: Error Boundary & Exception Trapping . . . 168
38.4 Production Reference Implementation: PatientCareAdmissionPraxis . . . 169
38.5 Dynamic Route Generation & Camel Pipeline Topology . . . 170
38.5.1 Route Synthesis Architecture . . . 170
38.5.2 Exchange Property Invariants and Header Preservation . . . 171
38.6 Dynamic Discovery, Seeding & Runtime Registration . . . 171
38.6.1 CDI Bean Resolution via Jakarta EE Instance . . . 171
38.6.2 Initial Seeding via TaskSequenceDefaultSeeder . . . 172
38.7 Multi-Tier Testing Strategies . . . 173
38.7.1 CamelTestSupport Unit Test Harness . . . 173
38.7.2 Failure Snapshotting and Checkpoint Assertions . . . 174
38.8 Deployment, Cluster Management & Operational Runbook . . . 174
38.8.1 Containerization within Ponos WildFly Container . . . 174
38.8.2 REST Administrative API: /workflow . . . 174
38.8.3 Dynamic Live Reload via ponos-cli . . . 174
38.9 Summary & Best Practices Checklist . . . 174

IX Operations **176**

39 Failure Scenarios, Recovery Guarantees & Operations **177**
39.1 Platform Failure Scenario Matrix . . . 177
39.2 Dead-Letter Queue (DLQ) Governance . . . 177
39.3 Kubernetes Health & Readiness Probes . . . 177

40 PHI-Aware Logging Architecture & Developer Implementation Guide **179**
40.1 Overview & Regulatory Objectives . . . 179
40.1.1 The Logging Policy Matrix . . . 179
40.2 Architectural Model & Security Gates . . . 180
40.2.1 Dual-Gate Security Evaluation Flow . . . 180
40.3 Configuration & Environment Variables . . . 180
40.3.1 Startup Warning Banner . . . 181
40.4 Developer Guide & API Usage Blueprint . . . 181
40.4.1 The Dual-Logger Pattern . . . 181
40.4.2 Compile-Time Safety and API Contract . . . 182
40.4.3 Lazy Evaluation with Functional Suppliers . . . 182
40.4.4 Categorization: PHI vs Non-PHI Operational Metadata . . . 183
40.4.5 Absolute Prohibition on Secrets and Credentials . . . 183
40.4.6 Exception Handling & Stack Trace Sanitization . . . 183
40.4.7 Mapped Diagnostic Context (MDC) Invariants . . . 183
40.5 Subsystem Integration Patterns . . . 184
40.5.1 Gateway Tier: Pylai Ingress/Egress . . . 184
40.5.2 Workflow Tier: Energeia & Ergon Activities . . . 184
40.5.3 Persistence Tier: Hestia & Mnemosyne . . . 184
40.6 Logback Configuration & Appender Isolation . . . 184
40.6.1 Physical Storage & Retention Invariants . . . 185
40.7 Testing & Verification Strategies . . . 185

A Glossary of Classical Nomenclature & Terminology **187**
A.1 Core Greek Metaphor Register . . . 187
A.2 Healthcare & Enterprise Integration Terminology . . . 187

B Authoritative Harmonia System Inventory **190**
B.1 Subproject and Leaf Module Inventory . . . 190
B.2 Infrastructure and Middleware Workloads . . . 192

C Authoritative Subproject & Leaf Module Register **194**
C.1 Module Taxonomy & Architectural Classification . . . 194

D Middleware Capability Matrix **196**
D.1 Middleware Technology Summary . . . 196
D.2 Middleware Capabilities: Used vs. Avoided Analysis . . . 197
D.2.1 ActiveMQ Artemis 2.33.0 Messaging Engine . . . 197
D.2.2 Infinispan 15.0.3 Distributed In-Memory Data Grid . . . 197
D.2.3 PostgreSQL 16 Relational Persistence . . . 197
D.2.4 WildFly 31.0.1 Jakarta EE Runtime (Iris BEFE Gateway & Ponos) . . . 197
D.2.5 Matrix Synapse 1.120.0 Collaboration Homeserver . . . 197

E Configuration Property Register **199**
E.1 Dual-Dimension Configuration Discipline . . . 199
E.2 Global Infrastructure & Environment Parameters . . . 199
E.3 Subsystem Application Parameters . . . 200

F Port and Network Protocol Register **201**
F.1 The 18 Platform Network Listeners . . . 201

G Container and Image Register **202**
G.1 Container Image Register . . . 202

H Kubernetes Workload and Resource Register **203**
H.1 Workload Allocations & Persistent Claims . . . 203

I Persistence Register: Relational Databases & Cache Grid **205**
I.1 Relational Database Schema Register . . . 205
I.1.1 Relational Table Definitions . . . 205
I.2 Distributed Cache Grid Register (Infinispan 15.0.3) . . . 205

J Module Dependency Matrix **207**

K Normal Deployment Verification Checklist **208**
K.1 Phase 1: Host Preparation & MicroK8s Setup . . . 208
K.2 Phase 2: Persistent Storage & Volume Claims . . . 208
K.3 Phase 3: Stateful Tier Verification . . . 208
K.4 Phase 4: Stateless Workloads & Gateways . . . 208

L Paradeigma Simulation Deployment Checklist **209**
L.1 Phase 1: Production Isolation Pre-Flight Checks . . . 209
L.2 Phase 2: Simulation Workload Deployment . . . 209
L.3 Phase 3: Scenario Execution & Journey Verification . . . 209

M Architecture Requirements **210**
M.1 Requirement Conventions . . . 210
M.2 Core Platform Requirements . . . 210
M.2.1 CORE-001 — Information Authority Classification . . . 210
M.2.2 CORE-002 — Separation of Architectural Concerns . . . 210
M.2.3 CORE-003 — Governed Canonical Semantics . . . 210
M.2.4 CORE-004 — Correlation and Causation . . . 210
M.3 Security Requirements . . . 211
M.3.1 SEC-001 — Trusted Authentication . . . 211
M.3.2 SEC-002 — Default-Deny Authorisation . . . 211
M.3.3 SEC-003 — Authentication and Authorisation Separation . . . 211
M.3.4 SEC-004 — Contextual Resource Authorisation . . . 211
M.3.5 SEC-005 — End-to-End Security Context . . . 211
M.3.6 SEC-006 — Initiating and Executing Identity . . . 211
M.3.7 SEC-007 — Anti-Spoofing . . . 211
M.3.8 SEC-008 — Security Metadata Preservation . . . 211
M.3.9 SEC-009 — Least Privilege . . . 211
M.3.10 SEC-010 — PHI-Safe Operational Logging . . . 211
M.4 Provenance and Audit Requirements . . . 211
M.4.1 PROV-001 — End-to-End Provenance . . . 211
M.4.2 PROV-002 — Source Context . . . 212
M.4.3 PROV-003 — Dual-Actor Provenance . . . 212
M.4.4 PROV-004 — Provenance and Authority Independence . . . 212
M.4.5 PROV-005 — Immutable Audit . . . 212
M.4.6 PROV-006 — Audit Discoverability . . . 212
M.5 Data, Persistence and Recovery Requirements . . . 212
M.5.1 DATA-001 — Durable Governed State . . . 212
M.5.2 DATA-002 — Authoritative Read Semantics . . . 212
M.5.3 DATA-003 — Cache Transparency . . . 212
M.5.4 DATA-004 — Search Scalability . . . 212
M.5.5 DATA-005 — Concurrency and Version Integrity . . . 212
M.5.6 DATA-006 — Referential Integrity . . . 212
M.5.7 DATA-007 — Persistence Failure Safety . . . 212
M.5.8 DATA-008 — Recovery Fidelity . . . 213
M.6 Integration and Interoperability Requirements . . . 213
M.6.1 INT-001 — Standards-Based Healthcare Interoperability . . . 213
M.6.2 INT-002 — Reliable Messaging . . . 213
M.6.3 INT-003 — Messaging Recovery . . . 213

M.6.4 INT-004 — FHIR R5 Boundary . . . 213
M.6.5 INT-005 — Opaque Resource Identity . . . 213
M.6.6 INT-006 — Governed Validation . . . 213
M.6.7 INT-007 — Terminology Abstraction . . . 213
M.7 Provider Registry Requirements . . . 213
M.7.1 DIR-001 — Standards-Based Provider Directory . . . 213
M.7.2 DIR-002 — Provider Directory Discovery . . . 213
M.7.3 DIR-003 — Governed Directory Change . . . 213
M.7.4 DIR-004 — Directory Referential Integrity . . . 213
M.7.5 DIR-005 — Healthcare Identifier Governance . . . 214
M.7.6 DIR-006 — Provider Self-Service Governance . . . 214
M.8 Clinical Information and Longitudinal Record Requirements . . . 214
M.8.1 CLIN-001 — Vendor-Neutral Clinical Information Layer . . . 214
M.8.2 CLIN-002 — FHIR-Based Longitudinal Clinical Representation . . . 214
M.8.3 CLIN-003 — Medication as a First-Class Clinical Domain . . . 214
M.8.4 CLIN-004 — Source-System Independence . . . 214
M.8.5 CLIN-005 — Source Authority Preservation . . . 214
M.8.6 CLIN-006 — Authority-Aware Clinical Reconciliation . . . 214
M.8.7 CLIN-007 — Preservation of Conflicting Assertions . . . 214
M.8.8 CLIN-008 — Clinical Source Context . . . 214
M.8.9 CLIN-009 — Governed Patient Identity . . . 214
M.8.10 CLIN-010 — Read-Oriented Initial LHR . . . 215
M.8.11 CLIN-011 — Selective Clinical Authoring . . . 215
M.9 Human-Facing Presentation Requirements . . . 215
M.9.1 UX-001 — Consistent Iris Experience . . . 215
M.9.2 UX-002 — Information-Dense Operational Design . . . 215
M.9.3 UX-003 — Honest System State . . . 215
M.9.4 UX-004 — Explicit Availability States . . . 215
M.9.5 UX-005 — Patient Search and Workspace . . . 215
M.9.6 UX-006 — Longitudinal Clinical Navigation . . . 215
M.9.7 UX-007 — Persistent Patient Context . . . 215
M.9.8 UX-008 — Provenance-Aware Clinical Presentation . . . 215
M.9.9 UX-009 — Initial Clinical Scope . . . 215
M.10 Operational and Resilience Requirements . . . 215
M.10.1 OPS-001 — Reproducible Deployment . . . 215
M.10.2 OPS-002 — Durable Infrastructure State . . . 216
M.10.3 OPS-003 — Meaningful Health and Readiness . . . 216
M.10.4 OPS-004 — Operational Observability . . . 216
M.10.5 OPS-005 — Production Resilience . . . 216
M.10.6 OPS-006 — Architecture Conformance . . . 216
M.11 Requirement Evolution . . . 216

N Architecture Conformance Findings 217
N.1 Open / Recently Identified Findings . . . 217
N.1.1 FND-001 — Clinical Search Has Been Cache-Dependent . . . 217
N.1.2 FND-002 — Volatile Clinical Fallback Path . . . 217
N.1.3 FND-003 — Clinical API Authorisation Gap . . . 217
N.1.4 FND-004 — Clinical Browser Authentication Gap . . . 217
N.1.5 FND-005 — Credentialed Wildcard CORS . . . 217
N.1.6 FND-006 — Mutable AuditEvent Surface . . . 217
N.1.7 FND-007 — Unbounded Iris Collection Loading . . . 217
N.1.8 FND-008 — Pylai Header Trust Risk . . . 217
N.1.9 FND-009 — Legacy Embedded-Broker Assumptions . . . 218
N.1.10 FND-010 — Partial Queue Telemetry Population . . . 218
N.2 Finding Lifecycle . . . 218

O Architecture and Implementation Backlog 219
O.1 Backlog Conventions . . . 219
O.2 Foundation Convergence . . . 219
O.2.1 BL-SEC-001 — Complete Clinical Authorisation Boundary . . . 219
O.2.2 BL-SEC-002 — Complete Security Context Propagation . . . 219
O.2.3 BL-SEC-003 — Close Internal Trust-Boundary Spoofing Paths . . . 219
O.2.4 BL-SEC-004 — Restrict Browser CORS . . . 219
O.2.5 BL-PROV-001 — Complete End-to-End Provenance Framework . . . 219
O.2.6 BL-PROV-002 — Enforce AuditEvent WORM Behaviour . . . 219
O.2.7 BL-DATA-001 — Remove Volatile Clinical Persistence Fallback . . . 220
O.2.8 BL-DATA-002 — Make Clinical Search Authoritative-Backed . . . 220
O.2.9 BL-DATA-003 — Implement Server-Side FHIR Paging and Totals . . . 220
O.2.10 BL-DATA-004 — Validate Write-Behind Failure and Recovery . . . 220
O.2.11 BL-DATA-005 — Preserve Context During Persistence Recovery . . . 220
O.3 Architecture Technical Debt . . . 220
O.3.1 Execution Context and Context Propagation . . . 220
O.4 Messaging and Runtime Backlog . . . 220
O.4.1 BL-MSG-001 — Preserve Petasos/Ponos Separation . . . 220
O.4.2 BL-MSG-002 — Standardise Broker Configuration . . . 221
O.4.3 BL-MSG-003 — Complete Petasos Runtime Telemetry . . . 221
O.4.4 BL-MSG-004 — Verify Broker Persistence and Failover . . . 221
O.4.5 BL-MSG-005 — Petasos Message Replay . . . 221
O.5 Provider Registry Backlog . . . 221
O.5.1 BL-DIR-001 — Complete Provider Registry Validation . . . 221
O.5.2 BL-DIR-002 — Complete Governed Change Workflow . . . 221
O.5.3 BL-DIR-003 — Provider Registry Syndication . . . 221
O.6 Iris Baseline Backlog . . . 221
O.6.1 BL-IRIS-001 — Maintain Shared Iris Design System . . . 221
O.6.2 BL-IRIS-002 — Remove Fabricated Runtime Presentation . . . 221
O.6.3 BL-IRIS-003 — Complete Monitor Runtime Drill-Down . . . 221
O.6.4 BL-IRIS-004 — Make Audit UI Read-Only . . . 222
O.6.5 BL-IRIS-005 — Eliminate Unbounded Dashboard Searches . . . 222
O.7 Clinical / Longitudinal Health Record Backlog . . . 222
O.7.1 BL-CLIN-001 — Clinical FHIR Information Layer . . . 222
O.7.2 BL-CLIN-002 — Core Clinical Resource Model . . . 222
O.7.3 BL-CLIN-003 — Medication Information Model . . . 222
O.7.4 BL-CLIN-004 — Information Authority Framework . . . 222
O.7.5 BL-CLIN-005 — LHR Reconciliation and Update Processing . . . 222
O.7.6 BL-CLIN-006 — Clinical Source-System Ingestion . . . 222
O.7.7 BL-CLIN-007 — Iris-Clinical LHR Viewer . . . 222
O.7.8 BL-CLIN-008 — Clinical Authoring and Terminology Services . . . 222
O.7.9 BL-CLIN-009 — Patient Identity / Golden Record Processing . . . 223
O.8 Agora Backlog . . . 223
O.8.1 BL-AGORA-001 — Complete Matrix/Synapse Clinical Mapping . . . 223
O.8.2 BL-AGORA-002 — Align Agora Security Context . . . 223
O.9 Deployment and Operations Backlog . . . 223
O.9.1 BL-OPS-001 — Maintain Docker Compose Known-Good Baseline . . . 223
O.9.2 BL-OPS-002 — Complete MicroK8s Deployment Automation . . . 223
O.9.3 BL-OPS-003 — Architecture Regression Guardrails . . . 223
O.10 Harmonia - Audit . . . 223
O.10.1 BL-AUDIT-01 — Introduce a typed Harmonia Security Domain . . . 223
O.10.2 BL-AUDIT-02 — Relocate immutable audit evidence to dedicated Operations/Audit persistence . . . 223
O.10.3 BL-AUDIT-03 — Database-level audit immutability hardening . . . 224
O.11 Backlog Sequencing . . . 224

P Architecture Decisions **225**
P.1 Current Decisions . . . 225
P.1.1 ADR-001 — Petasos Owns Messaging Runtime . . . 225
P.1.2 ADR-002 — Pylai Owns External Interface and Protocol Behaviour . . . 225
P.1.3 ADR-003 — Mnemosyne Owns Durable Application State . . . 225
P.1.4 ADR-004 — Calliope Owns Canonical Models and Semantic Governance . . . 225
P.1.5 ADR-005 — Themis Owns Security Policy Decisions . . . 225
P.1.6 ADR-006 — FHIR R5 Is the Clinical Interoperability Boundary . . . 225
P.1.7 ADR-007 — Harmonia Is an Interoperability Authority, Not Universal Clinical Truth 225
P.1.8 ADR-008 — Iris-Clinical Begins as a Longitudinal Record Viewer . . . 225
P.1.9 ADR-009 — Information Authority Is Cross-Cutting . . . 225
P.1.10 ADR-010 — Clinical Search Must Be Authoritative-Backed . . . 226
P.1.11 ADR-011 — Agora Uses Restricted Self-Hosted Matrix/Synapse . . . 226
P.1.12 ADR-012 — Paradeigma Is an Isolated Exemplar/Test Subsystem . . . 226
P.1.13 ADR-013 — Kleio Owns Audit Evidence and Provenance . . . 226
P.1.14 ADR-014 — Petasos Owns Durable Processing Transition Boundaries . . . 226
P.1.15 ADR-015 — Replay Is Anchored to Explicit Durable Transitions . . . 226
P.1.16 ADR-016 — Audit Is Anchored to Architecturally Significant Transitions . . . 226
P.1.17 ADR-017 — Processing Pressure Is Expressed as Durable Backlog . . . 226
P.1.18 ADR-018 — Mnemosyne Defines the Authoritative Durable State Boundary . . . 226
P.1.19 ADR-019 — Mneme Owns Distributed Resource Access and Coordination . . . 230
P.1.20 ADR-020 — Governed Information Uses Lifecycle State Rather Than Physical Deletion 234

List of Figures

1 ArchiMate 3.2 Visual Modeling Notation, Layer Color Palette, and Relationship Types . . ii

1.1 ArchiMate Motivation and Strategy Architecture View for Harmonia HIE 2

2.1 ArchiMate Motivation and Strategy Architecture View for Harmonia HIE 7

3.1 Harmonia Conceptual Layering Stack and Subsystem Interactions 10

4.1 ArchiMate Business and Clinical Architecture View for Harmonia HIE 13

5.1 ArchiMate 5-Tier Application Component Landscape for Harmonia HIE 18
5.2 ArchiMate Themis Policy & Multi-Tier Defence-in-Depth Security Architecture 21
5.3 ArchiMate Petasos Messaging Subsystem & Clustered Transport Architecture 22
5.4 ArchiMate Energeia Workflow Engine & Sequential Erga Pipeline View 23

6.1 ArchiMate Component Flow for MLLP Ingress Service Processing Pipeline 25
6.2 ArchiMate Component Flow for MLLP Egress Dispatch Pipeline 26

7.1 Petasos Messaging Framework and Artemis Cluster Architecture 28
7.2 ActiveMQ Artemis HA Replication Pairs and Symmetric Cluster Mesh 29

9.1 Energeia Workflow Architecture: Ponos WorkEngine, Praxis Sequences & Ergon Activities 33
9.2 Ergon Modular Activity Architecture and Checkpoint Lifecycle 34
9.3 Praxis Dynamic Route Synthesis and Multi-Stage Pipeline Execution 34

10.1 Iris Presentation Tier and BEFE Gateway Architecture 36
10.2 Iris UI Federation, Hot Rod Caching & REST Endpoint Integration 39

11.1 ArchiMate 3.2 Component and Interaction View for the Iris Presentation Tier and BEFE Gateway 40
11.2 Iris Operations Console Shell and Global Navigation Layout 45

12.1 FHIR R5 Provider Registry Entity Relationship and Referential Dependency Graph . . . 55
12.2 Provider Registry Change Request State Progression 57

13.1 Canonical `Pragma` State Machine and Lifecycle Progression 63

14.1 ArchiMate Pragma Task State Machine and 5-Phase Checkpoint Lifecycle 68

15.1 ArchiMate 3.2 Component Flow for the MLLP Ingress Service Processing Pipeline 71
15.2 ArchiMate 3.2 Component Flow for the MLLP Egress Multi-Instance Dispatch Pipeline . 77

16.1 ArchiMate Technology Infrastructure, System Software & Communication Paths 88
16.2 ArchiMate Hestia In-Memory Data Grid & Write-Behind Persistence SPI Architecture . . 89

17.1 ActiveMQ Artemis 2.33.0 4-Broker Clustered HA Messaging Topology and Journal Replication 92

18.1 Infinispan 15.0.3 Distributed Cache Grid Topology and Hot Rod Integration 94

22.1 MicroK8s 5-Tier Deployment Topology and Add-on Integration 102

24.1 ArchiMate Physical Deployment & Multi-Broker ActiveMQ Artemis HA Architecture . . 105

25.1 Themis 4-Gate Defence-in-Depth Authorization Pipeline and PDP Evaluation 108

26.1 Dual-Gate PHI-Aware Logging Architecture and Isolated Logback Routing 110

32.1 ArchiMate 3.2 Physical Infrastructure, MicroK8s Add-ons, and 5-Tier Workload Deployment Topology 119

33.1 Paradeigma Simulation Architecture: Generators, Personas & Scenario Engine 131

34.1 Normal Production vs Paradeigma Simulation Deployment Isolation Boundary 133

35.1 ArchiMate 3.2 Component Landscape for the Paradeigma Simulation Subsystem 136

37.1 ArchiMate 3.2 Processing Topology and Component Interaction View for an Ergon Activity Module 155
37.2 Docker Compose Topology for the Ponos Task Processor Container 163

38.1 ArchiMate 3.2 Praxis Workflow Sequence Architecture and Dynamic Route Synthesis Topology 166

40.1 ArchiMate 3.2 View: PHI-Aware Logging Security Architecture and Appender Isolation . 180

List of Tables

1 Harmonia Classical Architectural Nomenclature and System Meaning Matrix iii

1.1 Harmonia Strategic Capabilities and Subsystem Realization 5

4.1 Harmonia Business Actors, Assigned Roles, and Operational Responsibilities 14

8.1 Agora Architecture Decision Records (ADRs) 31

11.1 Iris Presentation Tier Subsystem and Portal Topology Matrix 41
11.2 Iris BEFE Clinical JAX-RS REST Endpoint Catalog (Port 8080) 42
11.3 Iris Clinical Portal View Matrix and FHIR Resource Mapping 44
11.4 Operations Backend REST Endpoint Catalog (iris-befe Port 8090) 47
11.5 Iris Provider Administration Portal Route and Authority Matrix 49

12.1 FHIR R5 Provider Registry Managed Resource Model Catalog 55
12.2 FHIR R5 Provider Registry REST Interaction Matrix 56
12.3 Provider Registry Per-Resource Ergon Activity Responsibilities 58
12.4 Provider Registry Mnemonic Role and Granular Authority Matrix 60
12.5 Production Readiness Checklist for FHIR R5 Provider Registry Core Services 61

13.1 Canonical Provider Registry Validation and Error Diagnostic Codes 64

14.1 Canonical Provider Registry Validation and Error Diagnostic Codes 67
14.2 Petasos Standard Message Envelope Attribute Specification 69
14.3 Harmonia Supported FHIR R5 Resource Schema Catalog 70

15.1 HL7 v2 to FHIR R5 Task Canonical Mapping Specifications 75
15.2 Synchronous HL7 Acknowledgement Response Contracts 76
15.3 MLLP Outbound Gateway Environment Configuration Parameters 78
15.4 Downstream HL7 Acknowledgement Code Decision Matrix 82

16.1 Harmonia Master Technology Infrastructure Port Allocation Matrix 91

18.1 Infinispan 15.0.3 Replicated Named Cache Definitions and Write-Behind Store Bindings . 95

24.1 ActiveMQ Artemis Broker Storage Isolation and Persistent Volume Mapping 106

28.1 Primary Platform Configuration Parameters Cross-Reference 113

32.1 Harmonia Host Platform Sizing Profiles and Resource Allocation Matrix 120
32.2 Authoritative Linux Kernel Parameters and Sysctl Network Tuning 121
32.3 Host Network Port Matrix and Firewall Exposure Rules 121
32.4 Authoritative MicroK8s Add-on Specification and Capabilities 122
32.5 Kubernetes Container Probe Specifications and Compute Resource Allocations 125
32.6 Harmonia Ansible Lifecycle Playbooks and Operational Capabilities 128

34.1 Authoritative Normal Production vs Paradeigma Simulation Deployment Matrix 134

35.1 Paradeigma Interface Catalogue and Port Mappings 136
35.2 Paradeigma Synthetic Pathology LOINC Catalog 138
35.3 Scenario Engine REST Control Endpoints 141
35.4 FailureSimulator Fault Injection Capabilities 141
35.5 Paradeigma Docker Compose Deployment Configuration 142
35.6 Dedicated FHIR R5 Synthetic Resource Generators 143
35.7 Paradeigma Themis Security Actors Reference Catalog 144

35.8 Paradeigma Simulator Microservice Configuration Properties . . . 147
35.9 Security & PHI Logging Governance Properties . . . 148
35.10FailureSimulator Fault Injection Configuration Knobs . . . 148

37.1 Standardized Exchange Headers and Properties Managed by ErgonBase . . . 155
37.2 Production Readiness Checklist for New Ergon Workflow Activity Modules . . . 164

38.1 Praxis Workflow Execution Checkpoint Stages and Lifecycle Semantics . . . 167
38.2 Mandatory Apache Camel Exchange Properties and Headers in Praxis Pipelines . . . 172
38.3 Production Readiness Checklist for Praxis Workflow Sequences . . . 175

39.1 Platform Failure Scenario Matrix and Automated Recovery Guarantees . . . 177

40.1 Harmonia Platform Logging Policy Matrix . . . 179
40.2 PHI Logging Configuration Properties . . . 181
40.3 PhiLogger Interface API Contract . . . 182
40.4 Classification of Operational Metadata vs. Restricted PHI . . . 183

A.1 Harmonia Classical Architectural Nomenclature and Etymology Register . . . 188
A.2 Healthcare and Enterprise Integration Terminology Register . . . 189

B.1 Harmonia System Inventory: Foundation, Security, Caching, and Transport Modules . . . 190
B.2 Harmonia System Inventory: Workflow, Gateways, Presentation, and Simulation Modules 191
B.3 Harmonia 25-Workload Infrastructure and Middleware Inventory . . . 193

C.1 Authoritative 37-Leaf-Module Register Across 9 Subprojects . . . 195

D.1 Harmonia Middleware Technology Capability Matrix . . . 196
D.2 ActiveMQ Artemis Capability Analysis: Used vs. Avoided . . . 197
D.3 Infinispan Capability Analysis: Used vs. Avoided . . . 197
D.4 PostgreSQL Capability Analysis: Used vs. Avoided . . . 198
D.5 WildFly Capability Analysis: Used vs. Avoided . . . 198
D.6 Matrix Synapse Capability Analysis: Used vs. Avoided . . . 198

E.1 Harmonia Global Infrastructure Configuration Parameters . . . 199
E.2 Harmonia Subsystem Application Configuration Properties . . . 200

F.1 Authoritative Harmonia 18-Port Network Listener Register . . . 201

G.1 Harmonia Container Image and Runtime Security Register . . . 202

H.1 Harmonia Kubernetes 25-Workload Resource Allocations and Storage Claims . . . 204

I.1 Harmonia Relational Database Register . . . 205
I.2 Infinispan In-Memory Replicated Cache Grid Register . . . 206

J.1 Harmonia Subproject Allowed Dependencies and Unidirectional Invariants . . . 207

Part I

Foundations

Chapter 1

Motivation & Strategy Architecture

The Motivation and Strategy layer articulates the fundamental drivers, stakeholder concerns, strategic goals, architecture principles, and business capabilities that shape the Harmonia Health Information Exchange platform.

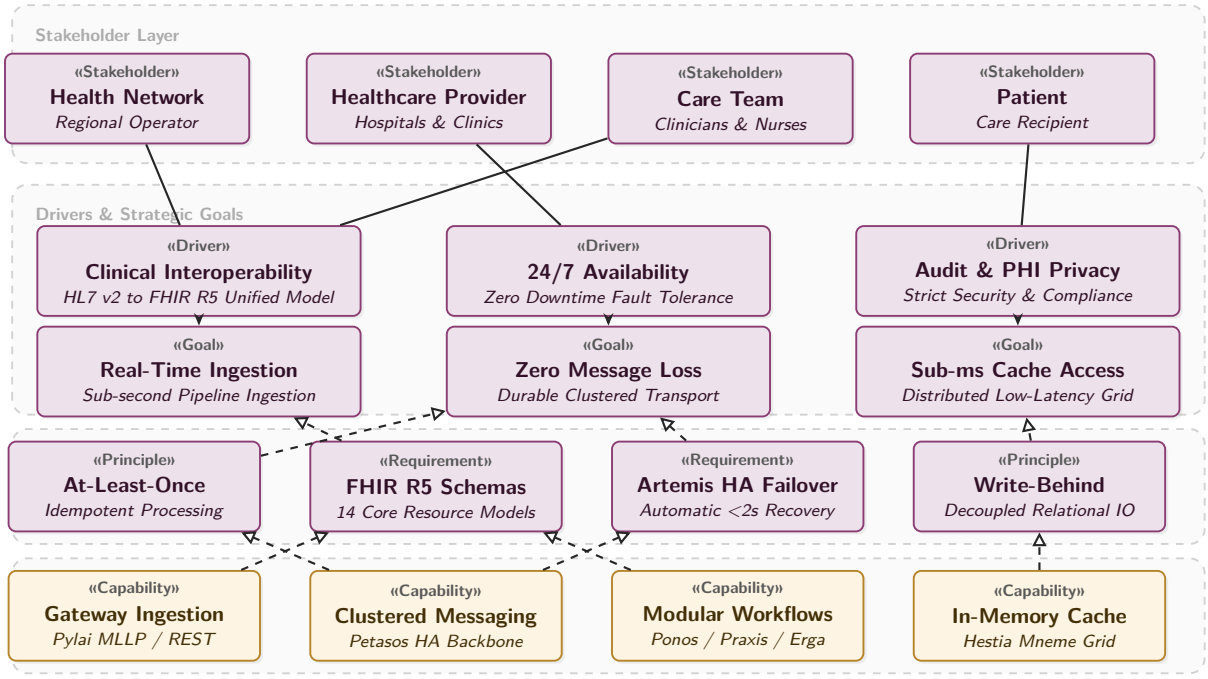


Figure 1.1: ArchiMate Motivation and Strategy Architecture View for Harmonia HIE

1.1 Stakeholders & Key Drivers

Harmonia is designed to serve its primary healthcare stakeholders, addressing their distinct operational, clinical, governance, and regulatory drivers:

1.1.1 Stakeholder Analysis

Regional Health Network Operator Responsible for governing regional health data exchange, ensuring service-level agreement (SLA) adherence, managing cross-facility data governance, and maintaining uninterrupted interoperability bridges.

Healthcare Providers (Hospitals, Health Systems, Diagnostic Labs) Organizations that produce and consume high volumes of clinical messages (e.g. ADT admissions, ORU lab observations, MDM clinical notes) via legacy MLLP streams and modern REST APIs.

Healthcare Directory Stewards & Credentialing Authorities Governance officers, medical registrars, and directory administrators charged with validating practitioner qualifications, managing organizational hierarchies, verifying electronic service endpoints, and maintaining master practitioner registries.

Clinicians & Care Teams Physicians, nurses, and allied health staff requiring immediate, accurate, and consolidated longitudinal patient records and up-to-date provider directories at the point of care.

Patients & Care Recipients The ultimate beneficiaries of timely, coordinated healthcare delivery whose Protected Health Information (PHI) must be safeguarded with absolute confidentiality.

1.1.2 Core Drivers

- **Clinical Interoperability Mandates**: The necessity to bridge heterogeneous health protocols (HL7 v2.3–v2.5, HL7 v3, CDA) into standardized HL7 FHIR Release 5 schemas without loss of clinical nuance.
- **National Healthcare Directory Interoperability**: The requirement to maintain synchronized, standards-compliant FHIR R5 provider directories (`Practitioner`, `PractitionerRole`, `Organization`, `Location`, `HealthcareService`, `Endpoint`, `Group`) with strict referential integrity and national provider identifier governance (e.g., Australian HPI-I, HPI-O).
- **Clinical Information Independence**: Clinical information must remain accessible through open, standards-based interfaces independently of proprietary Electronic Medical Record (EMR), laboratory, diagnostic, and departmental system data models and vendor-specific licensing constraints. Harmonia provides a durable, vendor-neutral FHIR representation of clinically relevant information while preserving the authority and provenance of the originating source systems.
- **24/7/365 High-Availability Operations**: Mission-critical healthcare environments cannot tolerate system outages, message loss, or pipeline stalls during emergency admissions or acute care events.
- **Patient Safety & Identity Integrity**: Mismatched patient identities lead to severe clinical errors. Automated, deterministic master patient index (EMPI) reconciliation is paramount.
- **PHI Privacy & Regulatory Compliance**: Strict adherence to applicable privacy and healthcare information-protection obligations requires encrypted transport, controlled access, comprehensive audit trails (`AuditEvent`, `Provenance`), and prevention of inappropriate PHI leakage into operational logs.

1.2 Strategic Goals

To satisfy these drivers, Harmonia establishes six measurable architectural goals:

1. **Goal 1: Real-Time Ingestion & Transformation**: Ingest, parse, transform, and persist clinical trigger events in under 500 milliseconds end-to-end.
2. **Goal 2: Zero In-Flight Message Loss**: Guarantee that acknowledged messages survive hardware failures, broker restarts, network partitions, and process crashes.
3. **Goal 3: Sub-Millisecond Cache Response**: Deliver read query response times under 1 millisecond for active clinical entities via distributed in-memory data grids.
4. **Goal 4: Automated Identity & Demographics Reconciliation**: Resolve and merge patient identifiers, practitioner roles, and organizational hierarchies across facilities automatically.
5. **Goal 5: Governed Healthcare Directory Lifecycle**: Expose standards-compliant FHIR R5 Provider Registry REST endpoints with sub-50ms synchronous read/search response and asynchronous, resilient change request workflows enforcing referential validation, optimistic concurrency control, and auditability.
6. **Goal 6: Vendor-Neutral Longitudinal Clinical Record**: Maintain a durable, standards-based longitudinal representation of clinically relevant patient information aggregated from source systems, exposed through FHIR R5 APIs and human-facing clinical views without requiring consumers to depend upon proprietary source-system interfaces.

1.3 Core Architectural Principles

Architectural Principle: At-Least-Once Delivery & Idempotent Processing

The platform assumes at-least-once message delivery across all messaging transports. Consuming components must be idempotent. Deduplication is enforced at broker level (via ActiveMQ Artemis `_AMQ_DUPL_ID` caches), client transport level (`DuplicateDetector`), and business persistence level.

Architectural Principle: Separation of Transport from Business Logic

The messaging framework (`Petasos`) treats all payloads as opaque byte arrays. Transport routes are unaware of clinical schemas. Transformation logic is strictly isolated in modular task activities (`Erga`).

Architectural Principle: Governed Asynchronous Change Management for Directory Entities

HTTP `POST` and `PUT` interactions on directory resources represent requests for change rather than immediate direct mutations of authoritative state. Every write request generates an immutable `ProviderRegistryChangePragma`, undergoes validation and automated referential verification in the `Energeia` pipeline, and commits only upon successful verification.

Architectural Principle: Write-Behind In-Memory Cache Persistence

Workflow pipelines write state snapshots exclusively to the in-memory cache grid (`Mneme`) to prevent disk I/O bottlenecks. Mneme asynchronously persists snapshots to relational storage (`Mnemosyne`) via non-blocking write-behind cache stores.

Architectural Principle: Canonical Modeling with Standardized FHIR R5 Boundaries

Internal workflow state is represented using the canonical `Pragma` domain model in `Calliope`. All external boundary interactions and REST APIs adhere strictly to HL7 FHIR R5 resource definitions.

Architectural Principle: Provenance and Information Authority

All information entering Harmonia must retain sufficient provenance to identify its origin and processing history. Information authority is evaluated separately from provenance and security.
Sources, ingress packets, and individual information assertions may be classified as `AUTHORITATIVE`, `INFORMATIONAL`, or `ANECDOTAL` within a defined information context.
Authority classification does not assert clinical truth. It expresses the degree of authority Harmonia assigns to an assertion for reconciliation, longitudinal record maintenance, and workflow processing. A source may be authoritative for one information domain and informational or anecdotal for another. Source-level defaults may be supplemented or overridden by ingress-packet or assertion-level classifications where permitted by governance.
Ponos and Erga processing rules determine how information is accepted, reconciled, retained, superseded, referred for review, or rejected according to resource type, source, authority, provenance, current state, and processing context. The authority classifications themselves do not prescribe fixed processing outcomes.

Architectural Principle: Governed PHI Logging & Dual-Logger Privacy Controls

Operational log streams (`INFO`, `WARN`, `ERROR`) must remain strictly devoid of Protected Health Information (PHI), personally identifiable demographics, and security credentials under all conditions. Clinical payload data and diagnostic identifiers are permitted exclusively at `DEBUG` and `TRACE` severity levels via the dedicated `PhiLogger` dual-gate security evaluation mechanism, routing non-additively to isolated, encrypted diagnostic storage (documented in Appendix 40).

1.4 Strategic Capabilities & End-to-End Value Stream

Harmonia delivers seven core business capabilities organized into a strategic value stream:

1.4.1 Strategic Value Stream Flow

The end-to-end flow of clinical information across Harmonia realizes the following sequential value stream:

Capability	Owning Subsystem	Business Value Delivered
Clinical Gateway Interfaces	`Pylai`	Multi-protocol MLLP/TCP and REST ingress and reliable egress dispatch accepting and delivering continuous HL7 v2 and FHIR feeds to/from external hospital systems.
FHIR Provider Registry	`Pylai` / `Energeia` / `Mnemosyne`	Standards-compliant FHIR R5 Provider Registry exposing synchronous read/search and governed asynchronous change APIs for practitioners, roles, organizations, locations, services, endpoints, and groups.
Clinical FHIR Information Service	`Pylai` / `Mnemosyne` / `Calliope`	Vendor-neutral, durable longitudinal FHIR representation of clinically relevant information with standards-based read/search APIs, governed semantics, provenance, and information-authority context independent of proprietary source-system interfaces.
High-Availability Messaging	`Petasos`	Clustered, replicated ActiveMQ Artemis transport ensuring message durability, load redistribution, and automatic failover.
Modular Task Execution	`Energeia` (`Ponos`/`Praxis`/`Erga`)	Composable Apache Camel processing routes performing parsing, validation, enrichment, transformation, reconciliation, and governed longitudinal-record update processing on canonical `Pragma` tasks.
In-Memory Cache Grid	`Hestia` (`Mneme`)	Low-latency Infinispan data grid maintaining active clinical state, task sequences, and write-behind persistence.
Longitudinal Clinical Record Presentation	`Iris-Clinical`	Human-facing, primarily read-oriented longitudinal patient record providing consolidated access to demographics, flags, allergies/intolerances, episodes and encounters, problems, medications, observations, diagnostics, referrals, and source/provenance context.

Table 1.1: Harmonia Strategic Capabilities and Subsystem Realization

Ingest Trigger $\longrightarrow$ Transport & Deduplicate $\longrightarrow$ Orchestrate Erga
$\longrightarrow$ Validate & Reconcile $\longrightarrow$ Update Clinical State $\longrightarrow$ Dispatch Egress / Serve FHIR APIs

Chapter 2

Platform Motivation, HIE Context & Architectural Principles

The Motivation and Strategy layer articulates the fundamental healthcare interoperability drivers, stakeholder concerns, strategic goals, architecture principles, and business capabilities that shape the Harmonia Health Information Exchange (HIE) platform.

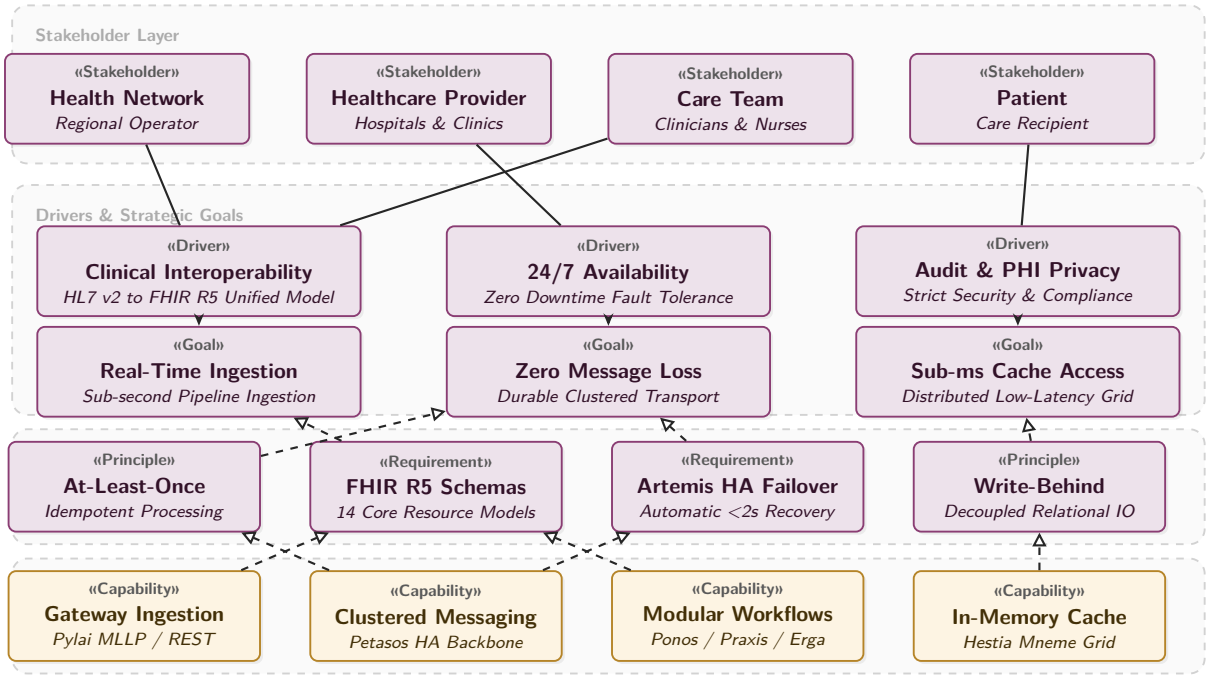


Figure 2.1: ArchiMate Motivation and Strategy Architecture View for Harmonia HIE

2.1 Healthcare Interoperability Context & Stakeholders

Modern regional healthcare networks operate in heterogeneous environments characterized by disparate legacy protocols (HL7 v2.3–v2.5 over MLLP/TCP), emergent web standards (HL7 FHIR R5 over HTTPS/REST), and distributed operational requirements. Harmonia provides an enterprise-grade integration bus and canonical state engine engineered to serve key stakeholders:

Regional Health Network Operator Responsible for governing regional health data exchange, ensuring service-level agreement (SLA) adherence, managing cross-facility data governance, and maintaining uninterrupted interoperability bridges.

Healthcare Providers (Hospitals, Health Systems, Diagnostic Labs) Organizations that produce and consume high volumes of clinical messages (e.g. ADT admissions, ORU lab observations, MDM clinical notes) via legacy MLLP streams and modern REST APIs.

Healthcare Directory Stewards & Credentialing Authorities Governance officers, medical registrars, and directory administrators charged with validating practitioner qualifications, managing organizational hierarchies, verifying electronic service endpoints, and maintaining master practitioner registries.

Clinicians & Care Teams Physicians, nurses, and allied health staff requiring immediate, accurate,

and consolidated longitudinal patient records and up-to-date provider directories at the point of care.

Patients & Care Recipients The ultimate beneficiaries of timely, coordinated healthcare delivery whose Protected Health Information (PHI) must be safeguarded with absolute confidentiality.

2.2 Core Architectural Drivers

The platform architecture resolves five foundational drivers:

1. **Clinical Interoperability Mandates**: Seamless translation of legacy HL7 v2 triggers into standardized HL7 FHIR Release 5 schemas without loss of clinical nuance or diagnostic context.
2. **National Healthcare Directory Interoperability**: Maintenance of synchronized, standards-compliant FHIR R5 provider directories (`Practitioner`, `PractitionerRole`, `Organization`, `Location`, `HealthcareService`, `Endpoint`, `Group`) with strict referential integrity and national provider identifier governance (e.g., Australian HPI-I, HPI-O).
3. **24/7/365 High-Availability Operations**: High-availability messaging clusters and cache grids ensuring zero message loss, sub-second failover, and zero pipeline stalls during emergency admissions.
4. **Patient Safety & Identity Integrity**: Automated, deterministic Enterprise Master Patient Index (EMPI) reconciliation preventing duplicate or mismatched patient identities.
5. **PHI Privacy & Regulatory Compliance**: Strict adherence to HIPAA, GDPR, and regional privacy frameworks requiring encrypted transport, audit trails (`AuditEvent`, `Provenance`), and zero PHI leakage into operational logs.

2.3 Core Architectural Principles

Architectural Principle: At-Least-Once Delivery & Idempotent Processing

The platform assumes at-least-once message delivery across all messaging transports. Consuming components must be idempotent. Deduplication is enforced at broker level (via ActiveMQ Artemis `_AMQ_DUPL_ID` caches), client transport level (`DuplicateDetector`), and business persistence level.

Architectural Principle: Separation of Transport from Business Logic

The messaging framework (`Petasos`) treats all payloads as opaque binary/text streams. Transport routes are unaware of clinical schemas. Transformation logic is strictly isolated in modular task activities (`Erga`).

Architectural Principle: Governed Asynchronous Change Management for Directory Entities

HTTP `POST` and `PUT` interactions on directory resources represent requests for change rather than immediate direct mutations of authoritative state. Every write request generates an immutable `ProviderRegistryChangePragma`, undergoes validation and automated referential verification in the `Energeia` pipeline, and commits only upon successful verification.

Architectural Principle: Write-Behind In-Memory Cache Persistence

Workflow pipelines write state snapshots exclusively to the in-memory cache grid (`Mneme`) to prevent disk I/O bottlenecks. Mneme asynchronously persists snapshots to relational storage (`Mnemosyne`) via non-blocking write-behind cache stores.

Architectural Principle: Canonical Modeling with Standardized FHIR R5 Boundaries

Internal workflow state is represented using the canonical `Pragma` domain model in `Calliope`. All external boundary interactions and REST APIs adhere strictly to HL7 FHIR R5 resource definitions.

Architectural Principle: Governed PHI Logging & Dual-Logger Privacy Controls

Operational log streams (`INFO`, `WARN`, `ERROR`) must remain strictly devoid of Protected Health Information (PHI), personally identifiable demographics, and security credentials under all conditions. Clinical payload data and diagnostic identifiers are permitted exclusively at `DEBUG` and `TRACE` severity levels via the dedicated `PhiLogger` dual-gate security evaluation mechanism, routing non-additively to isolated, encrypted diagnostic storage.

2.4 Strategic Value Stream Flow

The platform executes clinical event workflows across five coordinated processing phases:

$$\text{Ingest Trigger} \longrightarrow \text{Transport \& Deduplicate} \longrightarrow \text{Orchestrate Erga} \longrightarrow \text{Update Cache Grid} \longrightarrow \text{Dispatch Egress / Serve FHIR APIs}$$

Chapter 3

Conceptual Taxonomy & Subsystem Domains

Harmonia enforces strict separation of concerns across its conceptual domain landscape. Each domain encapsulates distinct operational responsibilities, persistence boundaries, and protocol lifecycles.

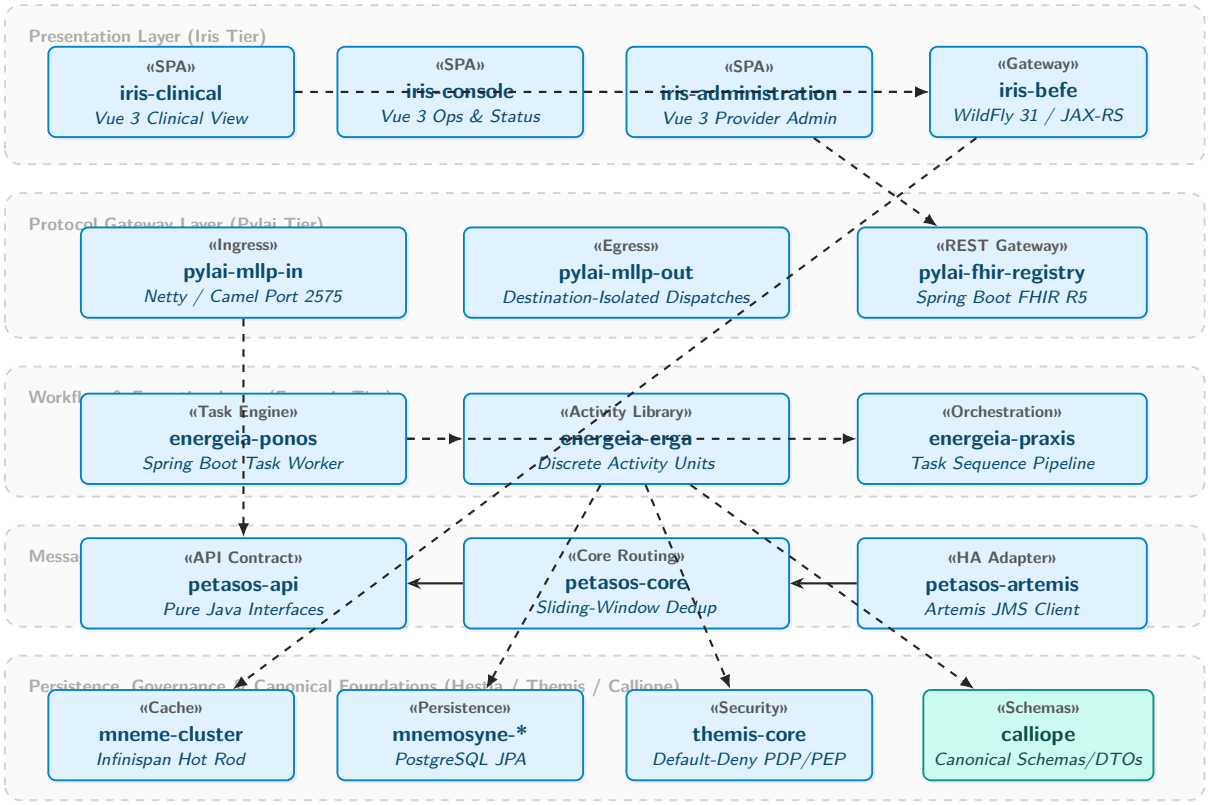


Figure 3.1: Harmonia Conceptual Layering Stack and Subsystem Interactions

3.1 Core Conceptual Domains

3.1.1 Presentation Services (`Iris`)

The presentation layer delivers decoupled clinical and administrative web applications supported by a dual-port Backend-For-Frontend (BEFE) gateway.

- **iris-clinical**: Clinical single-page application (Vue 3 / TypeScript) providing patient record exploration, search, and longitudinal timeline visualization.
- **iris-console**: Operational telemetry portal (Vue 3 / TypeScript) exposing message broker queue depths, cache statistics, and cluster health metrics. (Reconciled from legacy terminology `iris-monitor`).
- **iris-administration**: Administrative portal (Vue 3 / TypeScript) providing provider directory management and governance controls. Crucially, `iris-administration` is a presentation client consuming REST endpoints and does not implement server-side registry persistence or state machines.
- **iris-befe**: WildFly 31 Jakarta EE gateway exposing JAX-RS REST endpoints on Port 8080 and administrative telemetry on Port 8090, connecting to downstream services via Infinispan Hot Rod

and Petasos JMS.

3.1.2 Gateway & Protocol Services (`Pylai`)

Boundary protocol interface services providing bidirectional translation between external healthcare wire formats and internal event buses.

- **pylai-mllp-in**: High-throughput MLLP/TCP ingress gateway (Port 2575) consuming HL7 v2 triggers, validating framing, performing security screening, and publishing durable tasks to Petasos.
- **pylai-mllp-out**: Multi-instance MLLP egress gateway consuming outbound dispatch queues, transmitting frames over TCP sockets, and evaluating synchronous HL7 ACK/NACK responses.
- **pylai-fhir-registry**: Spring Boot REST gateway exposing standard HL7 FHIR R5 Provider Directory endpoints (`/fhir/r5/Practitioner`, etc.), translating write interactions into asynchronous change requests.

3.1.3 Messaging & Transport Subsystem (`Petasos`)

Transport abstraction and resilient messaging layer.

- **petasos-api**: Pure Java abstraction layer declaring contracts (`Petasos`, `PetasosProducer`, `PetasosConsumer`, `PetasosMessage`, `PetasosDestination`) strictly decoupled from JMS and ActiveMQ Artemis.
- **petasos-core**: Message routing, metrics aggregation, and sliding-window deduplication logic.
- **petasos-artemis**: Apache ActiveMQ Artemis adapter implementing failover connection pools and JMS transport bridges.

3.1.4 Workflow & Execution Engine (`Energeia`)

Composable clinical workflow and activity execution engine.

- **energeia-ponos**: WildFly Jakarta EE 10 WAR task execution engine consuming Petasos queue tasks and executing dynamic Camel route pipelines.
- **energeia-erga**: Library of discrete, single-responsibility Ergon activities executing specific transformations, validation rules, and schema mappings.
- **energeia-praxis**: Workflow orchestration engine chaining sequential Ergon steps into auditable execution graphs.
- **Pragma**: Canonical task state entity holding inputs, outputs, execution checkpoints, and bidirectional FHIR `Task` mappings.

3.1.5 Data, Caching & Persistence Services (`Hestia`)

Unified data services aggregator dividing ephemeral operational cache from durable relational persistence.

- **Mneme (`hestia/mneme-cluster`)**: Infinispan 15+ distributed cache grid providing sub-millisecond retrieval of active clinical tasks (`task-cache`), task sequences (`sequence-cache`), security pragmas, and module statuses over Hot Rod (Port 11222).
- **Mnemosyne (`hestia/mnemosyne-clinical` & `mnemosyne-operations`)**: Durable PostgreSQL 16 relational storage comprising Spring Boot HAPI FHIR JPA servers (clinical store) and Operations JPA servers (task audit and workflow lineage).

3.1.6 Canonical Schemas & Security Governance (`Calliope` & `Themis`)

Cross-cutting shared libraries and policy engines.

- **Calliope**: Centralized canonical schema definitions, domain DTOs, and FHIR converter models used across all modules.
- **Themis**: Default-deny Attribute- and Role-Based Access Control (ABAC/RBAC) security engine, 4-gate enforcement interceptors, and non-PHI audit event logging.

3.1.7 Collaboration Services (`Agora`)

Clinical collaboration subsystem bridging Harmonia clinical events into Matrix Synapse.

- **agora-api**: Pure Java domain contracts, event models, and Petasos destination constants.

- **agora-matrix**: Encapsulated Matrix Client-Server REST and Synapse Admin API client adapters.
- **agora-core**: Lifecycle coordination, deterministic identity provisioning, care team reconciliation, and Themis default-deny policy evaluation.
- **agora-service**: Spring Boot 3.2.5 microservice exposing the Matrix Application Service transaction endpoint on Port 8092 (management on Port 9992).

3.1.8 Synthetic Clinical Simulation (`Paradeigma`)

Isolated synthetic simulation subsystem providing deterministic hospital actor emulators (PAS, EMR, LMS, RIS-PAC) and scenario drivers for integration and resilience testing. Production modules maintain strict zero-dependency isolation from Paradeigma.

Chapter 4

Business & Clinical Architecture

The Business Layer specifies the healthcare business environment, defining external business actors, internal operational roles, business services exposed to healthcare facilities, business processes, and the core business objects exchanged across clinical pathways.

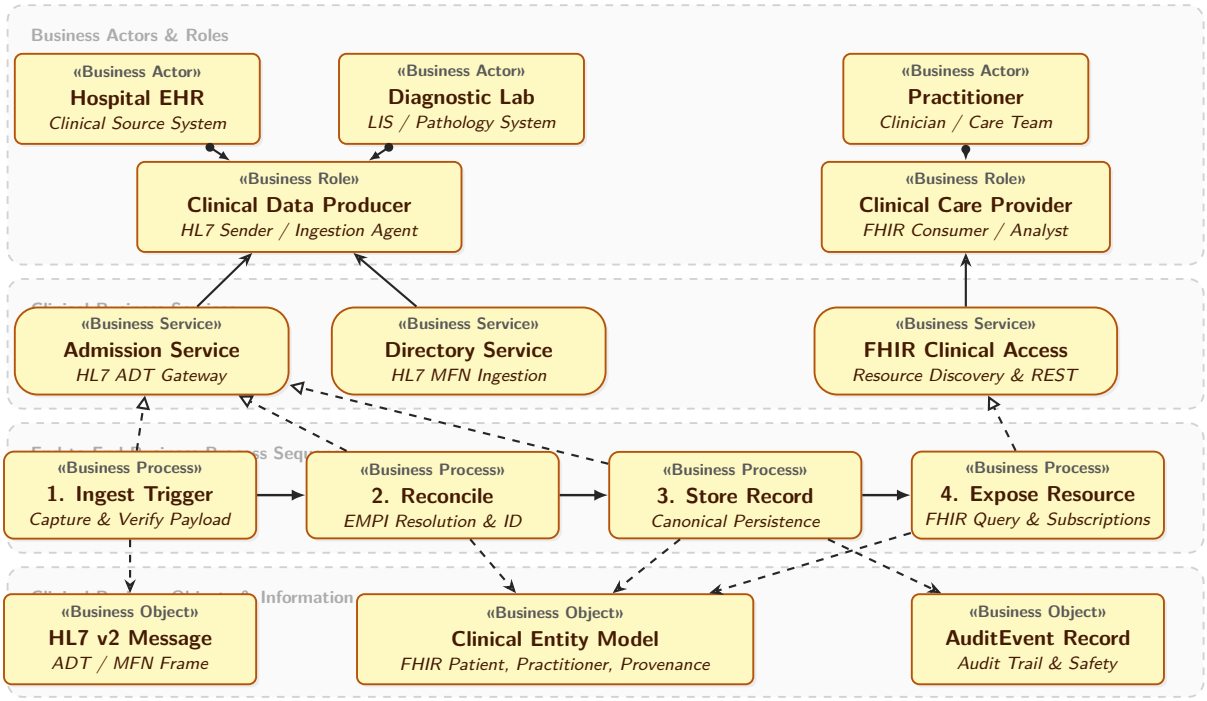


Figure 4.1: ArchiMate Business and Clinical Architecture View for Harmonia HIE

4.1 Healthcare Business Actors & Operational Roles

Harmonia interacts with a diverse ecosystem of clinical actors, abstracting their interactions through formal business roles:

4.2 Clinical Business Services

Harmonia exposes six core business services to external healthcare organizations:

4.2.1 Patient Admission & Movement Service (`HL7-ADT`)

Accepts real-time patient admission, discharge, and transfer (ADT) event streams adhering to HL7 v2.3–v2.5 standards across standard trigger events:

- **ADT^A01 / A04**: Patient Admission / Outpatient Registration.
- **ADT^A03**: Patient Discharge / End of Encounter.
- **ADT^A08**: Patient Demographics / Record Update.
- **ADT^A40**: Merge Patient Identifiers (MRN / EMPI consolidation).

Business Actor	Assigned Business Role	Healthcare Operational Scope
Hospital EHR / PAS	*Clinical Data Producer*	Sends continuous real-time HL7 v2 ADT feeds (admit, discharge, transfer, demographic updates) over MLLP connections.
Diagnostic Laboratory	*Diagnostic Result Producer*	Dispatches lab order notifications and observation reports (HL7 ORU / FHIR `Observation`) to the platform.
External System (HIS / LIS)	*Clinical Data Consumer*	Receives outbound HL7 v2 triggers, ADT notifications, and MFN directory updates via MLLP connections and returns synchronous acknowledgments (`ACK`).
Practitioner / Clinician	*Clinical Care Provider*	Queries longitudinal patient records, reviews historical clinical encounters, inspects diagnostic summaries, and looks up provider directory entries via web UI or FHIR APIs.
Directory Steward	*Directory Governance Authority*	Governs healthcare provider directory entries, verifies practitioner credentialing, oversees organizational hierarchies, and reviews flagged directory change requests.
Credentialing Authority	*Authoritative Identity Source*	National or state medical boards (e.g. AHPRA, HI Service) publishing authoritative provider identifiers (HPI-I, HPI-O) and qualification updates.
Health Network Admin	*Operations Administrator*	Monitors message queue depths, inspects Artemis cluster topologies, triggers task sequence reload, and audits failed pipelines.

Table 4.1: Harmonia Business Actors, Assigned Roles, and Operational Responsibilities

4.2.2 Practitioner & Staff Directory Service (HL7-MFN)

Maintains an authoritative, synchronized regional directory of healthcare practitioners, specialties, role assignments, and organizational facility hierarchies via HL7 MFN master file notifications.

4.2.3 FHIR R5 Clinical Data Discovery Service

Provides RESTful search, retrieval, and subscription capabilities across 15 FHIR R5 resource types, enabling authorized clinical applications to query patient histories, encounters, and audit logs.

4.2.4 Operations Telemetry & Workflow Administration Service

Exposes real-time health metrics, ActiveMQ Artemis broker statistics, Infinispan cache hit rates, and Ponos task execution statuses to health network operators.

4.2.5 Clinical Outbound Notification & Egress Service (HL7-MLLP-OUT)

Dispatches transformed, enriched clinical trigger events (ADT, MFN, ORU) and outbound clinical notifications to remote hospital information systems and diagnostic endpoints over resilient MLLP/TCP connections, enforcing synchronous HL7 ACK/NACK validation, automatic retries, and audit tracking.

4.2.6 FHIR R5 Provider Directory Management Service

Exposes synchronous read/search and governed asynchronous change management APIs for healthcare directory entities (`Practitioner`, `PractitionerRole`, `Organization`, `Location`, `HealthcareService`, `Endpoint`, `Group`). It enforces automated referential verification, optimistic concurrency locking, duplicate identifier protection, and auditable FHIR `Task` tracking.

4.3 Clinical Business Process Workflows

4.3.1 Process: Ingest and Reconcile HL7 ADT Trigger

The primary clinical ingestion workflow executes through five discrete business stages:

- **Stage 1: Capture MLLP Frame**: Inbound MLLP gateway receives the TCP stream, validates the MLLP envelope framing (`<VT>` and `<FS><CR>`), and returns an immediate HL7 `ACK` to release the source hospital system.
- **Stage 2: Envelope Encapsulation**: The raw message is wrapped into a `PetasosMessage` and published onto the clustered Petasos queue (`clinical.tasks.inbound`).
- **Stage 3: Patient Identity Resolution (EMPI)**: The `PatientIdentityUpdateErgon` activity extracts the Medical Record Number (MRN), queries the Enterprise Master Patient Index (EMPI) in Mneme cache, and assigns a canonical patient identifier.
- **Stage 4: Demographics Merge & Enrichment**: The `PatientDemographicsUpdateErgon` merges telecom, residential addresses, contact persons, and marital status into a consolidated FHIR `Patient` resource.
- **Stage 5: Audit & Persistence**: A `Provenance` and `AuditEvent` record is linked to the transaction, and the resulting resources are stored in Mneme cache and Mnemosyne PostgreSQL.

4.3.2 Process: Outbound MLLP Dispatch and ACK Validation Lifecycle

The outbound clinical egress workflow executes through five structured business stages:

1. **Stage 1: Egress Queue Enqueue**: The Ponos WorkEngine or upstream routing activity publishes an outbound dispatch task to the endpoint's dedicated Petasos queue (`petasos.queue.mllp.outbound.<endpoint-id`
2. **Stage 2: Endpoint Resolution & Dequeue**: The dedicated `pylai-mllp-out` instance consumes the task and verifies target destination parameters (host, port, timeouts) via the dynamic `MllpDestinationRegistry`.
3. **Stage 3: MLLP Framing & Wire Transmission**: The Apache Camel MLLP client component frames the HL7 v2 payload with standard delimiters (`<VT>` and `<FS><CR>`) and transmits the message over a dedicated TCP socket.
4. **Stage 4: Synchronous ACK / NACK Evaluation**: The gateway awaits the remote system's acknowledgment frame, parsing the `MSH` and `MSA` segments to validate acknowledgment codes (`AA`/`CA`

for success vs AE/AR/CE/CR for application/commit errors).

5. **Stage 5: State Machine Finalization & Audit**: The task status is updated in the Mneme cache grid, recording FHIR Communication, Task (Pragma), and Provenance records with remote acknowledgment details and transmission metrics.

4.3.3 Process: Governed Provider Registry Change Ingestion & Verification

The governed provider directory change workflow executes through five structured business stages:

1. **Stage 1: Ingestion & Pragma Generation**: The pylai-fhir-registry gateway receives a POST or PUT change request, performs structural validation, generates a ProviderRegistryChangePragma, and dispatches a PetasosMessage to Artemis queue harmonia.provider.registry.change.request, returning HTTP 202 Accepted with Location: /Task/{id}.
2. **Stage 2: Sequence Orchestration**: The Ponos WorkEngine consumes the change request and triggers the Praxis sequence seq-provider-registry-change-pipeline, routing the task to the dedicated resource Ergon activity.
3. **Stage 3: Referential & Business Rule Verification**: The Ergon activity queries ProviderRegistryReferenceVal to verify that all referenced target entities (Practitioner, Organization, Location, HealthcareService, Endpoint) exist and are active in Mnemosyne storage, while verifying identifier uniqueness.
4. **Stage 4: Automated Approval & Versioned Persistence**: If verification passes, the change is approved and committed to Mnemosyne PostgreSQL (hie_fhir_resources), incrementing the resource version_id and updating last_updated. If verification fails, the request transitions to REJECTED with an embedded FHIR OperationOutcome.
5. **Stage 5: Task Status Reflection**: The underlying Pragma transitions to COMPLETED (or REJECTED), making the updated resource immediately visible on synchronous GET read/search paths and reflecting completion on GET /Task/{id}.

4.4 Core Business Objects

HL7 v2 Message Stream The raw wire-format clinical transmission received from legacy hospital interface engines.

Canonical Patient Record (FHIR::Patient) The consolidated, deduplicated clinical identity representing the care recipient.

Task State Record (Pragma) The authoritative audit representation of an in-flight integration task, including all historical state checkpoints.

Provider Registry Change Pragma Specialized canonical task entity capturing directory change requests, target resources, correlation IDs, and validation outcomes.

Authoritative Provider Graph Interconnected FHIR R5 directory resources representing practitioners, roles, organizations, locations, healthcare services, endpoints, and groups.

Audit & Provenance Trail (FHIR::AuditEvent / FHIR::Provenance) Immutable compliance records establishing source attribution, timestamping, and processing integrity.

Part II

Platform Architecture

Chapter 5

Application Architecture

The Application Architecture layer models the core software components, internal subsystem collaborations, application services, and application interfaces across Harmonia's 26 Maven modules and presentation SPAs.

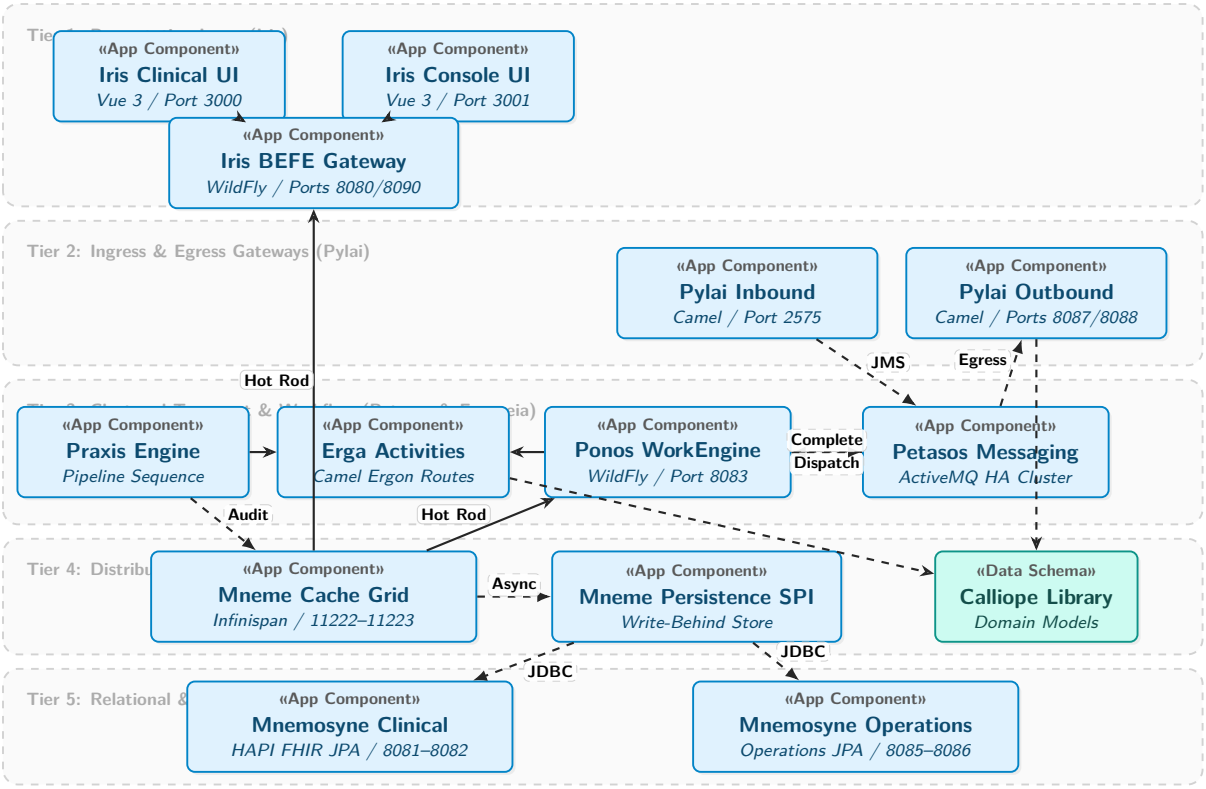


Figure 5.1: ArchiMate 5-Tier Application Component Landscape for Harmonia HIE

5.1 Presentation Tier: `Iris` Subsystem

The `Iris` subsystem delivers user interfaces and API mediation for clinicians, operations engineers, provider directory administrators, and healthcare practitioners. A comprehensive architectural specification, interface catalog, and operational manual for all Iris presentation applications is provided in Appendix 11.

5.1.1 `iris-befe` (Backend-For-Frontend Gateway)

`iris-befe` is a Jakarta EE 10 microprofile application deployed on WildFly providing dual-port isolated REST endpoints:

- **Clinical FHIR Port 8080** (`/api/v1/{resourceType}`): Handles clinical resource queries, FHIR search operations, and governed change submissions, routing requests directly to the `Mneme` Infinispan data grid via HotRod protocol and evaluated against Themis RBAC security policies.
- **Operations Port 8090** (`/api/operations/{resourceType}`): Dedicated socket-based operations HTTP server exposing operational telemetry, `Pragma` task audit histories, ActiveMQ Artemis queue monitoring, Infinispan cache topologies, and dynamic online `Praxis` sequence reload endpoints.

5.1.2 `iris-clinical` (Clinical Single-Page Application)

A modern web application built with Vue 3, Vite, TypeScript, and Pinia operating on port 3000. Provides clinicians with real-time patient lookup, longitudinal medical history timelines, encounter summaries, provider directory lookups, care team navigation, and diagnostic observation views.

5.1.3 `iris-console` (Operations & Workflow Console)

An administrative single-page application built with Vue 3 and TypeScript operating on port 3001. Enables systems engineers to inspect ActiveMQ Artemis broker queue depths, view Infinispan cache topologies, trigger live reload of task sequence configurations, and diagnose failing `Pragma` pipelines.

5.1.4 `iris-administration` (Provider Directory Administration & Self-Service Portal)

A dual-portal single-page application built with Vue 3, TypeScript, and Pinia operating on port 3002. It serves both individual healthcare providers through a Self-Service Portal (profile inspection, credential review, change proposal submissions) and credentialing staff through a Departmental Administration Portal (work queue triage, directory lifecycle management, data quality validation, and Themis security governance).

5.2 Interface & Gateway Tier: `Pylai` Subsystem

`Pylai` serves as the secure edge gateway through which external healthcare facilities exchange data with Harmonia, providing bidirectional ingestion and egress capabilities.

5.2.1 `pylai-mllp-in` (Inbound Clinical MLLP Gateway)

Hosts an Apache Camel MLLP listener on TCP port `2575`. It captures HL7 v2 frames (`<VT>...<FS><CR>`), verifies message integrity, issues immediate HL7 commit acknowledgements (`ACK`), wraps the payload into a `PetasosMessage`, and publishes it to the Petasos messaging cluster for asynchronous workflow execution.

5.2.2 `pylai-mllp-out` (Resilient Multi-Instance Outbound MLLP Gateway)

`pylai-mllp-out` is a dedicated, high-reliability outbound integration gateway deployed as a WildFly Jakarta EE 10 microservice. It provides comprehensive HL7 v2 message transmission to external clinical systems (e.g. Hospital Information Systems, Laboratory Information Systems, and Picture Archiving and Communication Systems).

Key architectural characteristics include:

- **Multi-Instance Egress Architecture**: Each external clinical destination or tenant is serviced by an independent, isolated gateway instance (e.g., `mllp-outbound-his` on port `8087` and `mllp-outbound-lis` on port `8088`). This multi-instance model provides hardware and process isolation across remote clinical endpoints.
- **Independent Petasos Queues per Destination**: Outbound tasks are partitioned into dedicated, destination-specific messaging queues (e.g., `petasos.queue.mllp.outbound.<endpoint-id>`). Slow, lagging, or offline hospital destinations cannot cause head-of-line blocking or degrade dispatch performance for other destinations.
- **Apache Camel MLLP Client Routes**: The `OutboundMllpRouteBuilder` configures synchronous Camel MLLP producer endpoints (`direct:mllp-outbound-send`), managing TCP socket connections, socket timeouts, keep-alives, and automatic reconnection policies.
- **Outbound Processing & ACK/NACK Validation**: The `OutboundMllpProcessor` serializes payloads into standard MLLP frames, while the `Hl7AckProcessor` performs deep inspection of the returned HL7 `ACK` frame (`MSH` and `MSA` segments), validating acknowledgment codes (`AA`/`CA` for success, `AE`/`AR`/`CE`/`CR` for error states) and extracting error condition descriptions.
- **Synchronous REST Dispatch APIs**: The `OutboundMllpResource` exposes high-performance JAX-RS REST endpoints:
  - `POST /api/mllp/outbound/send`: Generic HL7 v2 outbound dispatch endpoint.
  - `POST /api/mllp/outbound/adt/send`: Specialized clinical ADT trigger event dispatch.

  - `POST /api/mllp/outbound/mfn/send`: Master file notification directory dispatch.
  - `GET /api/mllp/outbound/health`: Real-time destination connectivity and circuit health status.

- **Dynamic Destination Management**: The `MllpDestinationRegistry` resolves target endpoint hostnames, TCP ports, timeout thresholds, and retry policies dynamically via `MllpDestinationConfig`.
- **Mneme Cache State Auditing**: Every outbound dispatch updates the `Mneme` Infinispan data grid with FHIR `Communication` (sent message, response ACK, latency), `Task` (`Pragma` state transition), and `Provenance` resources.

5.2.3 `pylai-mllp-base` (Shared Gateway Integration Library)

Provides core domain models and shared infrastructure for the Pylai gateway tier, including the `OutboundMllpRequest` and `OutboundMllpResponse` DTOs, `MllpDestinationConfig`, `MllpDestinationRegistry`, MLLP framing utilities, and `Mneme` cache connectors.

5.2.4 `pylai-mllp-cli` (Synthetic Test & Diagnostic Utility)

A command-line diagnostic tool used by integration engineers to generate and dispatch parameterized HL7 v2.4/v2.5 ADT (A01–A40) messages to Pylai gateways for end-to-end functional, egress verification, and throughput benchmarking.

5.2.5 `pylai-fhir-registry` (FHIR R5 Provider Registry REST Gateway)

`pylai-fhir-registry` provides an authoritative, standards-compliant HL7 FHIR Release 5 (R5) REST interface for managing healthcare provider directory entities (`Practitioner`, `PractitionerRole`, `Organization`, `Location`, `HealthcareService`, `Endpoint`, `Group`).

Key architectural components and capabilities include:

- **`FhirRestGatewayController`**: Exposes synchronous `GET` read and multi-parameter search endpoints delegating directly to Mnemosyne persistence, and asynchronous governed write endpoints (`POST` and `PUT`) that return `HTTP 202 Accepted` with `Location: /Task/{id}`.
- **`CapabilityStatementProvider`**: Dynamically generates and serves the server's official FHIR R5 `CapabilityStatement` on `GET /metadata`, describing supported resource schemas, search parameters, REST interactions, and security mechanisms.
- **`ProviderRegistryTaskResourceProvider`**: Implements standard FHIR `GET /Task/{id}` polling, mapping the underlying `Pragma` lifecycle state (`ACCEPTED`, `IN_PROGRESS`, `COMPLETED`, `REJECTED`, `FAILED`) to FHIR `Task.status` and attaching `OperationOutcome` diagnostics.
- **`FhirSecurityInterceptor`**: Fine-grained Spring/JAX-RS security interceptor delegating to `ThemisService` to enforce default-deny authorization policies, evaluate caller roles (`HarmoniaRoleEnum`), and map required granular authorities (`HarmoniaAuthorityEnum`, e.g. `provider.read`, `provider.change.submit`).
- **`ChangeRequestSubmissionService`**: Packages incoming write payloads into canonical `ProviderRegistryChangePra` envelopes, attaching immutable caller security context, and publishes `PetasosMessage` instances to ActiveMQ Artemis.

5.3 Policy & Authorisation Subsystem: `Themis`

`Themis` is Harmonia's centralized policy and authorization engine, delivering deterministic default-deny policy evaluation, role-to-authority mapping, data security labeling, immutable caller context propagation, and non-PHI decision auditing.

5.3.1 `themis-api` (Domain Contracts & Security Models)

Provides framework-independent, lightweight domain models and interfaces decoupled from Spring, Artemis, or WildFly:

- **`ThemisService`**: Core authorization contract defining `authorize(ThemisAuthorizationRequest)` and `evaluate(ThemisAuthorizationRequest)`.
- **`ThemisPrincipal`**: Immutable identity model capturing `principalId`, `PrincipalType` (`HUMAN`, `SYSTEM`, `SERVICE`, `PROCESS`), `sourceDomain`, and security attributes.

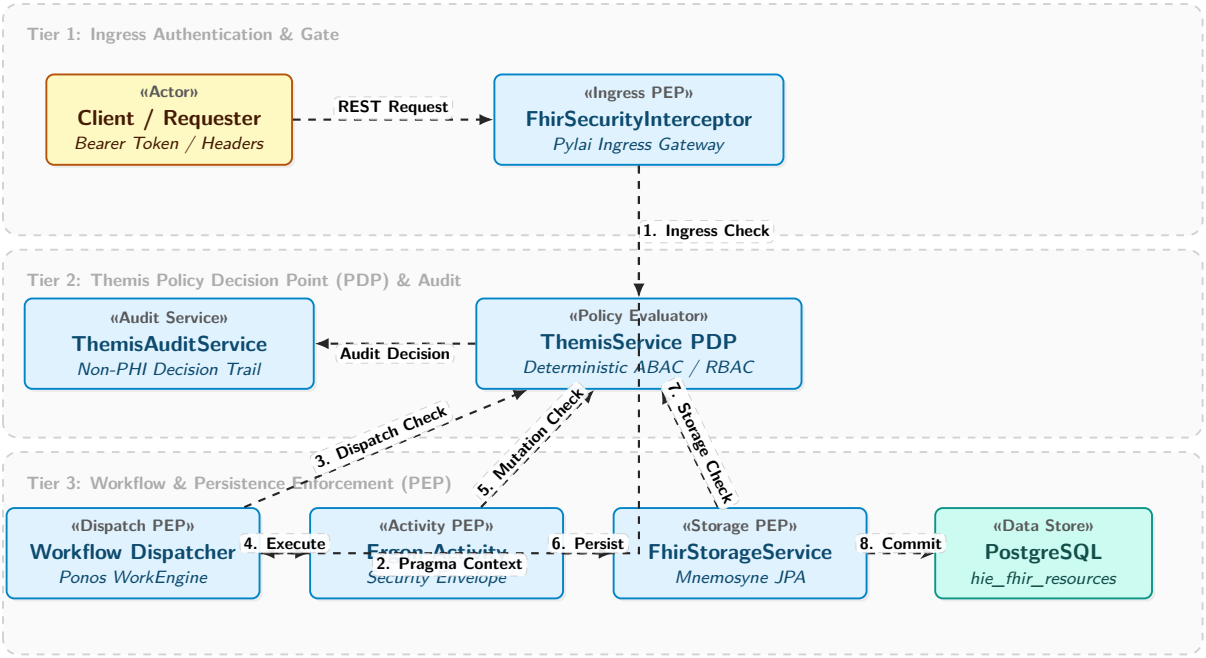


Figure 5.2: ArchiMate Themis Policy & Multi-Tier Defence-in-Depth Security Architecture

- **`ThemisAuthority`**: Granular capability token representing atomic operations (e.g., `provider.read`, `provider.change.submit`, `provider.resource.update`).
- **`ThemisDecisionReason`**: Exhaustive decision reason enumeration (`ALLOWED_BY_POLICY`, `EXPLICIT_DENY`, `DEFAULT_DENY`, `POLICY_NOT_FOUND`, `AUTHORITY_MISSING`, etc.).
- **`ThemisSecurityLabel`**: Data classification tags (`HarmoniaSecurityLabelEnum`) serializable to FHIR R5 `meta.security`.

5.3.2 `themis-core` (Deterministic Evaluation Engine)

- **Deterministic 3-Stage Evaluation Pipeline**:
  1. **Explicit Denials First**: Evaluates explicit prohibition rules; any matching denial terminates evaluation immediately with `DENY`.
  2. **Ordered Policy Matching**: Evaluates registered policies (`ThemisPolicy`) sorted by order against target domain, resource type, action, and required authorities.
  3. **Default-Deny Fallback**: If no active policy explicitly grants access, the engine fails closed to `DENY` with reason `POLICY_NOT_FOUND` or `AUTHORITY_MISSING`.
- **Controlled Mnemonic Roles (`HarmoniaRoleEnum`)**: Maps high-level role codes (`PRV_RDR`, `PRV_SUB`, `PRV_PROC`, `PRV_APR`, `PRV_ADM`, `AUD_RDR`, `SYS_INT`, `SYS_ADM`) to sets of granular `ThemisAuthority` tokens.
- **Built-in Policy Catalog**: Implements domain policies including `ProviderRegistryReadPolicy`, `ProviderRegistrySubmitPolicy`, `ProviderRegistryProcessPolicy`, `ProviderRegistryPersistPolicy`, and `SystemAdminPolicy`.
- **Controlled Service Identities (`HarmoniaServiceIdentities`)**: Defines explicit least-privilege authority sets for internal services (`service:pylai`, `service:ponos`, `service:mnemosyne`, etc.).

5.3.3 `themis-audit` (Structured Security Decision Auditing)

- **`ThemisAuditService`**: Records every authorization decision as an immutable `ThemisAuditEvent`.
- **Strict PHI & Secret Sanitization**: Audit records capture only structural metadata (`decisionId`, `principalId`, `action`, `resourceType`, `resourceId`, `correlationId`, `reason`), strictly omitting clinical payload bodies, bearer tokens, or user passwords.
- **Security Classification**: Audit streams carry the `AUDIT` security label and require the `audit.read` authority for inspection.

5.3.4 Multi-Tier Boundary Enforcement & Non-Amplification Invariant

Harmonia prevents privilege escalation by enforcing independent Themis checkpoints across each tier:

1. **Pylai Ingress Gate**: Validates that the external caller possesses submission authority (`provider.change.submit`).
2. **Pragma Context Serialization**: Originating caller credentials and authorities are immutably captured into the `Pragma` and serialized to FHIR R5 Task extensions.
3. **Ponos Dispatch Gate**: Validates both the originating requester claims and the executing Ergon processor authority (`provider.change.process`).
4. **Ergon Execution Envelope**: The activity executes within its declared `ErgonSecurityDefinition`.
5. **Mnemosyne Persistence Gate**: The persistence layer independently verifies storage mutation authority (`provider.resource.update`) before writing to PostgreSQL.

5.4 Messaging Transport Subsystem: `Petasos`

`Petasos` provides reliable, durable, and asynchronous message transportation between all Harmonia subsystems, fully encapsulating Apache ActiveMQ Artemis JMS and Core protocol complexity behind clean domain interfaces.

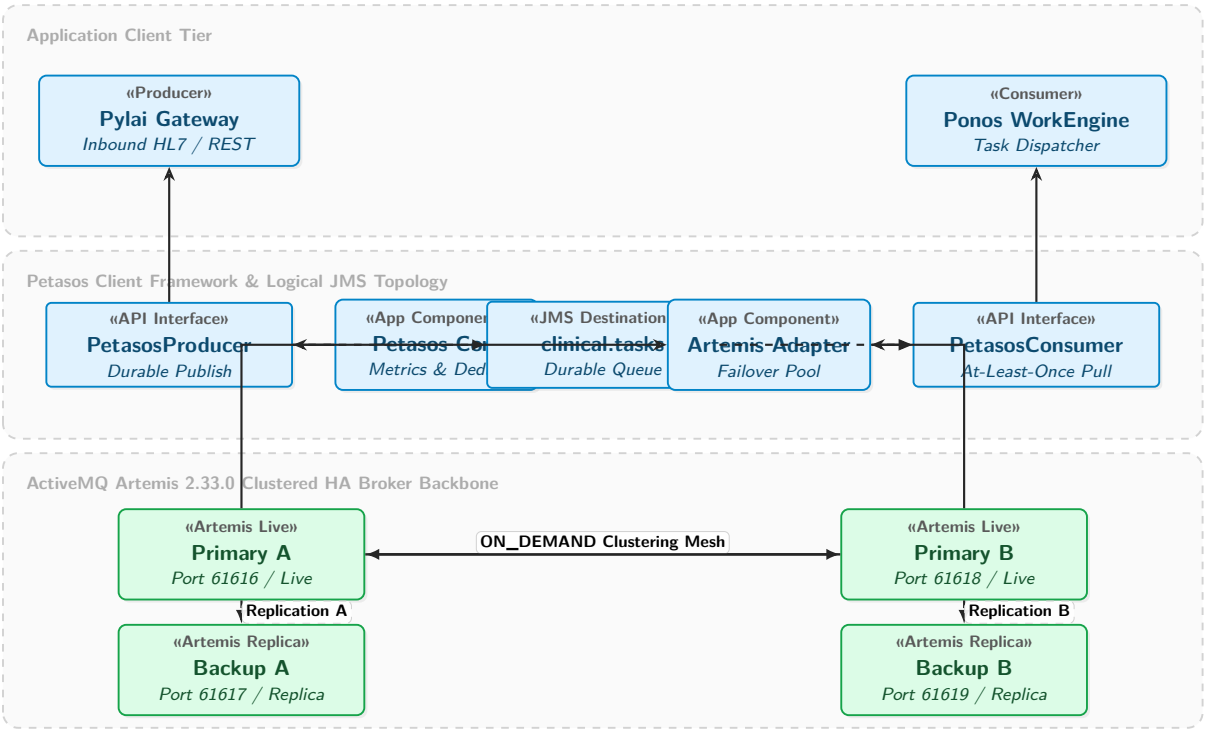


Figure 5.3: ArchiMate Petasos Messaging Subsystem & Clustered Transport Architecture

5.4.1 `petasos-api` (Core Abstractions)

Defines the client-facing contracts used across Harmonia:

- `PetasosProducer`: Synchronous and asynchronous durable message publishing.
- `PetasosConsumer`: Thread-safe queue polling and event subscription with lifecycle context hooks.
- `PetasosMessage`: Immutable metadata envelope encapsulating `messageId`, `correlationId`, `causationId`, schema URIs, and payload byte arrays.
- `PetasosDestination`: Logical address and queue routing identifiers.

5.4.2 `petasos-core` (Serialization & Deduplication)

- **JSON Envelope Serialization**: High-speed Jackson-based serialization of message headers and payloads.

- **DuplicateDetector**: An in-memory sliding window cache tracking message IDs to suppress client-side duplicates.
- **PetasosMetricsCollector**: Thread-safe metrics registry tracking message send/receive counters, failure rates, and reconnection events.

5.4.3 petasos-artemis (Artemis Adapter)

- **ArtemisConnectionManager**: Manages pooled JMS connections with automatic reconnect loops, failover listeners (FAILURE_DETECTED, FAILOVER_COMPLETED), and exponential backoff retry policies.
- **ArtemisMessageConverter**: Bidirectional mapping between PetasosMessage and Artemis Core/JMS messages, automatically setting _AMQ_DUPL_ID for broker-level deduplication.

5.5 Workflow Services Subsystem: Energeia

Energeia hosts the task execution and workflow pipeline framework.

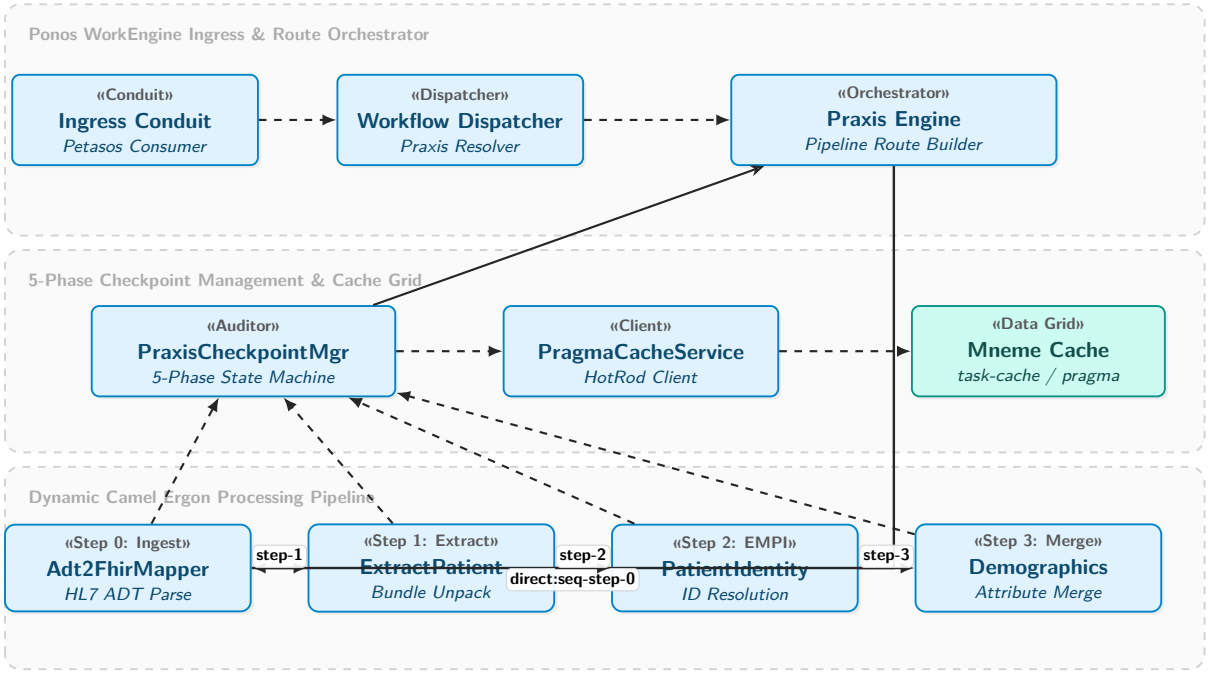


Figure 5.4: ArchiMate Energeia Workflow Engine & Sequential Erga Pipeline View

5.5.1 energeia/ponos (WorkEngine Runtime Container)

Deployed on WildFly Jakarta EE 10 (port 8083), Ponos hosts the execution environment:

- **PetasosQueueToExchangeConduit**: Ingress consumer listening on Petasos queues (including clinical ingestion queues and harmonia.provider.registry.change.request), extracting incoming Pragma payloads, creating Camel exchanges, and managing acknowledgement lifecycles.
- **PragmaWorkflowDispatcher**: Resolves matching Praxis workflow definitions based on task metadata and routes exchanges to entry direct endpoints.
- **CamelContextManager**: Manages the lifecycle of Apache Camel contexts and thread pools.

5.5.2 energeia/praxis (Workflow Sequence Orchestration)

- **PraxisImplementation**: Orchestrates ordered sequences of Ergon activities over synchronous Camel direct endpoints (direct:seq-<praxisId>-step-<N>).
- **ProviderRegistrySequenceBuilder**: Configures and seeds the canonical seq-provider-registry-change-pipeli sequence, orchestrating per-resource validation, reference verification, and persistence.
- **PraxisCheckpointManager**: Records the 5-phase execution audit checkpoints (PIPELINE_INGRESS, PRE_ERGON, POST_ERGON, PIPELINE_COMPLETE, PIPELINE_FAILED) and snapshots state to Mneme

cache.

- `PragmaCacheService`: Interfaces directly with Mneme's `task-cache`.

5.5.3 `energeia/erga` (Modular Task Activities)

- `ErgonBase`: Abstract Camel `RouteBuilder` enforcing the single-responsibility contract: reads from `pragma.getInput()`, executes domain logic via `processErgon(Pragma, Exchange)`, and appends resulting artifacts to `pragma.getOutput()`.
- **Clinical Ingestion Catalog**: Includes `Adt2FhirMapper`, `Mfn2FhirBundle`, `ExtractPatientFromBundle`, `PatientIdentityUpdateErgon`, and `PatientDemographicsUpdateErgon`.
- **Provider Registry Erga Suite**: 7 dedicated per-resource change processing activities:
  - `PractitionerChangeErgon`: Validates individual practitioner names, qualifications, and unique HPI-I/NPI identifiers.
  - `PractitionerRoleChangeErgon`: Validates reference links to Practitioner, Organization, Location, HealthcareService, and Endpoint.
  - `OrganizationChangeErgon`: Validates organization identifiers (HPI-O), active status, and parent/child hierarchies.
  - `LocationChangeErgon`: Validates physical facility coordinates, addresses, and managing organization references.
  - `HealthcareServiceChangeErgon`: Validates provided-by organization and operating location references.
  - `EndpointChangeErgon`: Validates technical connection types, address URIs, and payload MIME types.
  - `GroupChangeErgon`: Validates group member references and membership types.

5.6 Data Services Subsystem: `Hestia`

`Hestia` delivers multi-tier caching and durable relational persistence:

- `mneme-cluster`: 2-node Infinispan 15+ replicated data grid (ports `11222`–`11223`) maintaining `task-cache`, `tasksequence-cache`, and `communication-cache`.
- `mneme-persistence`: Custom `NonBlockingStore` SPI implementations (`OperationsRestCacheStore`, `FhirRestCacheStore`) providing asynchronous write-behind synchronization to Mnemosyne.
- `mnemosyne-clinical`: Spring Boot HAPI FHIR JPA server (ports `8081`–`8082`) backed by PostgreSQL storing canonical FHIR R5 resources in `hie_fhir_resources`. Implements `EndpointResourceProvider`, multi-parameter search filtering in `FhirStorageService`, deep referential integrity checks via `ProviderRegistryReferenceValidator`, and optimistic concurrency control (`If-Match`/`ETag`).
- `mnemosyne-operations`: Spring Boot JPA server (ports `8085`–`8086`) backed by PostgreSQL storing operational task sequences, Pragmas, and audit logs.

5.7 Canonical Schema Repository: `Calliope`

`Calliope` is the shared canonical library defining domain DTOs, the `Pragma` model, `ErgonPayload`, `ProviderRegistryChangePragma`, canonical constants (`ProviderRegistryConstants`), validation error codes (`PR-VAL-001` through `PR-VAL-010`), and bidirectional converters for FHIR R5 `Task` resources, guaranteeing type safety across all platform tiers.

Chapter 6

External Gateways (`Pylai`)

The Gateway and Interface layer (`Pylai`) serves as the boundary interface of Harmonia, terminating external protocols, enforcing security screening, isolating network failures, and providing bidirectional translation between external healthcare wire formats and internal event buses.

6.1 MLLP Inbound Ingress Gateway (`pylai-mllp-in`)

The inbound MLLP gateway terminates inbound TCP connections from hospital Electronic Medical Records (EMR), Patient Administration Systems (PAS), Laboratory Information Systems (LIS), and Radiology Information Systems (RIS).

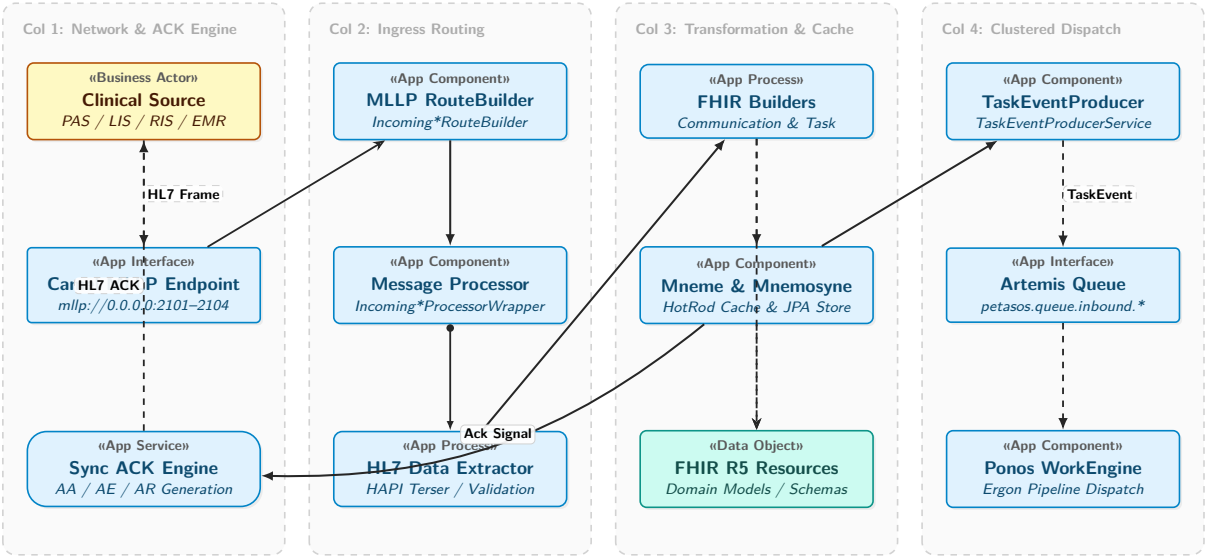


Figure 6.1: ArchiMate Component Flow for MLLP Ingress Service Processing Pipeline

6.1.1 MLLP Framing and Transport

MLLP provides minimal framing over stream-oriented TCP sockets:

$$\langle \text{Frame} \rangle ::= \texttt{0x0B}\ (\text{Start-of-Block}) + \langle \text{HL7 v2 Payload} \rangle + \texttt{0x1C}\ (\text{End-of-Block}) + \texttt{0x0D}\ (\text{CR}) \tag{6.1}$$

In standard Kubernetes deployment, `pylai-mllp-in` listens on Port 2575 (exposed via NodePort 30575).

6.1.2 Ingress Dual-Write Safety (REC-001)

In accordance with architectural invariant REC-001, inbound gateways must guarantee end-to-end downstream message acceptance before returning an `AA` (Application Accept) ACK to the upstream sender:

1. When an inbound message is received, it is framed and validated.
2. The gateway attempts to publish the task event to the downstream Petasos ActiveMQ Artemis queue.
3. If downstream publishing succeeds, the gateway returns an HL7 `AA` ACK.

4. If downstream publishing or cache write fails, the exception is caught, and the gateway immediately returns an HL7 `AE` (Application Error) or `AR` (Application Reject) NACK frame, prompting the upstream hospital system to retain and retry the message.

6.2 MLLP Outbound Egress Dispatcher (`pylai-mllp-out`)

The outbound MLLP dispatcher manages reliable delivery of clinical notifications and ADT/MFN messages to external hospital systems.

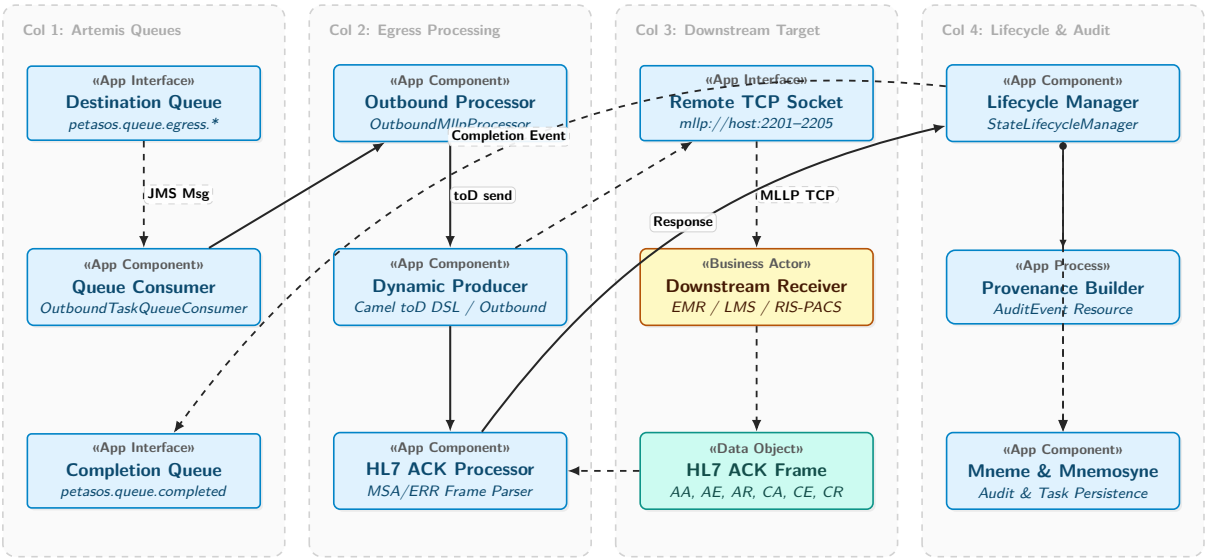


Figure 6.2: ArchiMate Component Flow for MLLP Egress Dispatch Pipeline

6.2.1 Destination-Isolated Queuing

To prevent head-of-line blocking when an external hospital endpoint experiences downtime, Harmonia provisions dedicated Artemis queues per destination endpoint:

```
petasos.queue.mllp.outbound.<destination-id>
```

Each queue is consumed by a dedicated outbound worker instance, guaranteeing that a stalled destination does not delay traffic intended for healthy external systems.

6.2.2 Synchronous ACK / NACK Evaluation & Fan-Out Tracking (REC-002)

When an outbound frame is transmitted over TCP, the dispatcher awaits the remote system's acknowledgment frame:

- **AA / CA**: Remote application accepted the message; task transitions to `COMPLETED`.
- **AE / CE**: Remote application error; task is scheduled for retry or transitioned to `FAILED`.
- **AR / CR**: Remote application reject; task transitions to `FAILED`.

In multi-destination fan-out scenarios (REC-002), individual destination delivery checkpoints (`destinationId`, `status`, `ackCode`, `timestamp`) are recorded into the parent Task's output extensions.

6.3 FHIR Provider Registry Gateway (`pylai-fhir-registry`)

The `pylai-fhir-registry` gateway is a Spring Boot REST microservice exposing HL7 FHIR Release 5 Provider Directory endpoints:

- **Synchronous Read/Search (`GET`)**: Direct, sub-50ms read queries against `mnemosyne-clinical` PostgreSQL JPA storage for `/Practitioner`, `/PractitionerRole`, `/Organization`, `/Location`, `/HealthcareService`, `/Endpoint`, and `/Group`.

- **Governed Asynchronous Writes (`POST/PUT`)**: Enforces the governed change model. Submissions return `HTTP 202 Accepted` with `Location:/Task/{id}` and enqueue a `ProviderRegistryChange Pragma` onto Petasos for validation and approval.

Chapter 7

Messaging & Event Transport (Petasos)

The Petasos subsystem provides a resilient, high-throughput messaging abstraction layer backed by an active-active clustered Apache ActiveMQ Artemis messaging fabric.

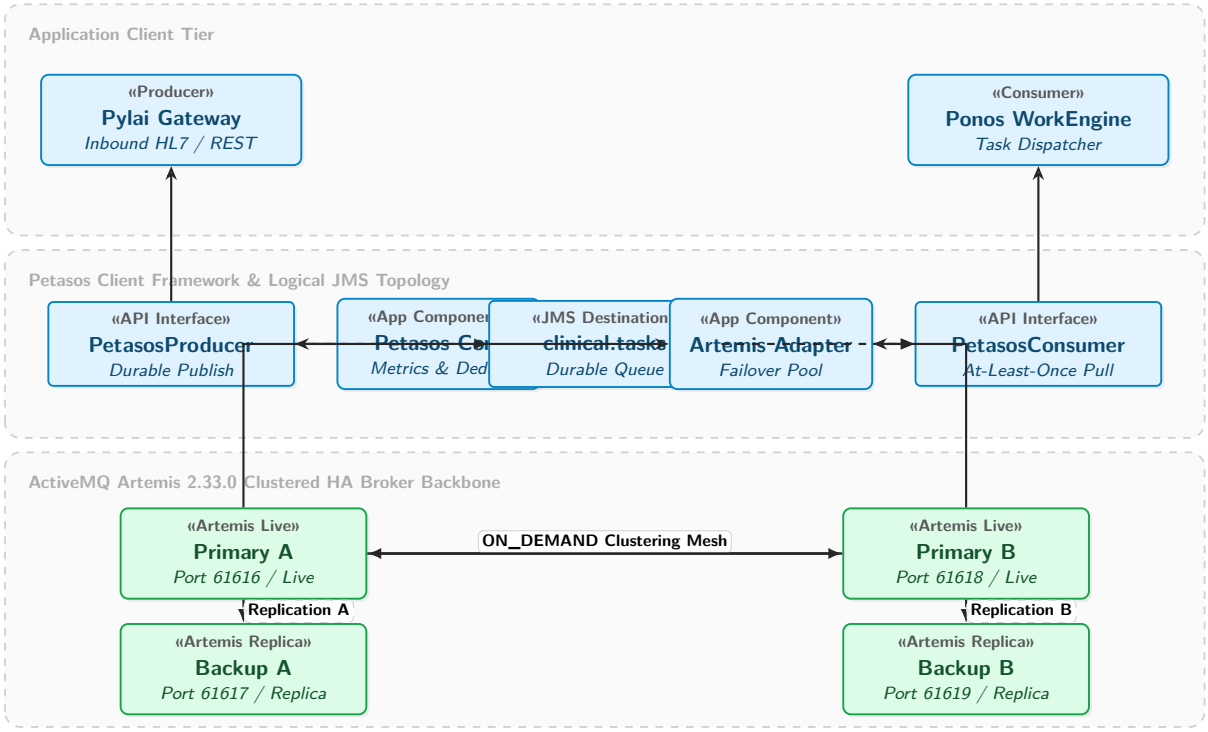


Figure 7.1: Petasos Messaging Framework and Artemis Cluster Architecture

7.1 Transport Abstraction Invariant (`petasos-api`)

To prevent vendor lock-in and preserve strict architectural boundaries, the `petasos-api` module declares pure Java interfaces:

- `Petasos`: Main engine facade for producer and consumer lifecycle management.
- `PetasosProducer`: Asynchronous publish operations returning `CompletableFuture<PetasosPublishResult>`.
- `PetasosConsumer`: At-least-once message subscription contracts supporting acknowledgement semantics (`acknowledge()`, `reject()`).
- `PetasosDestination`: Logical queue and topic destination abstractions.
- `PetasosMessage`: Generic message envelope carrying binary or text payloads.

Architectural Principle: Zero JMS / Artemis Leakage in Petasos API

In accordance with Invariant 2, `petasos-api` contains zero imports or references to `org.apache.activemq..`, `jakarta.jms..`, or `javax.jms...` Artemis client libraries are strictly encapsulated in `petasos-artemis`.

7.2 Core Routing and Deduplication (petasos-core)

petasos-core implements transport routing, metric instrumentation, and multi-tier message deduplication.

7.2.1 Sliding-Window Deduplication

To prevent duplicate processing caused by network retransmissions, DuplicateDetector maintains an in-memory sliding-window cache of recently processed message control identifiers (MSH-10 and PetasosMessage.messageId). Duplicate messages within the sliding window (default: 30 minutes) are logged, acknowledged, and discarded before reaching workflow execution.

7.3 ActiveMQ Artemis HA Adapter (petasos-artemis)

petasos-artemis provides the enterprise JMS driver connecting Harmonia microservices to the underlying broker cluster.

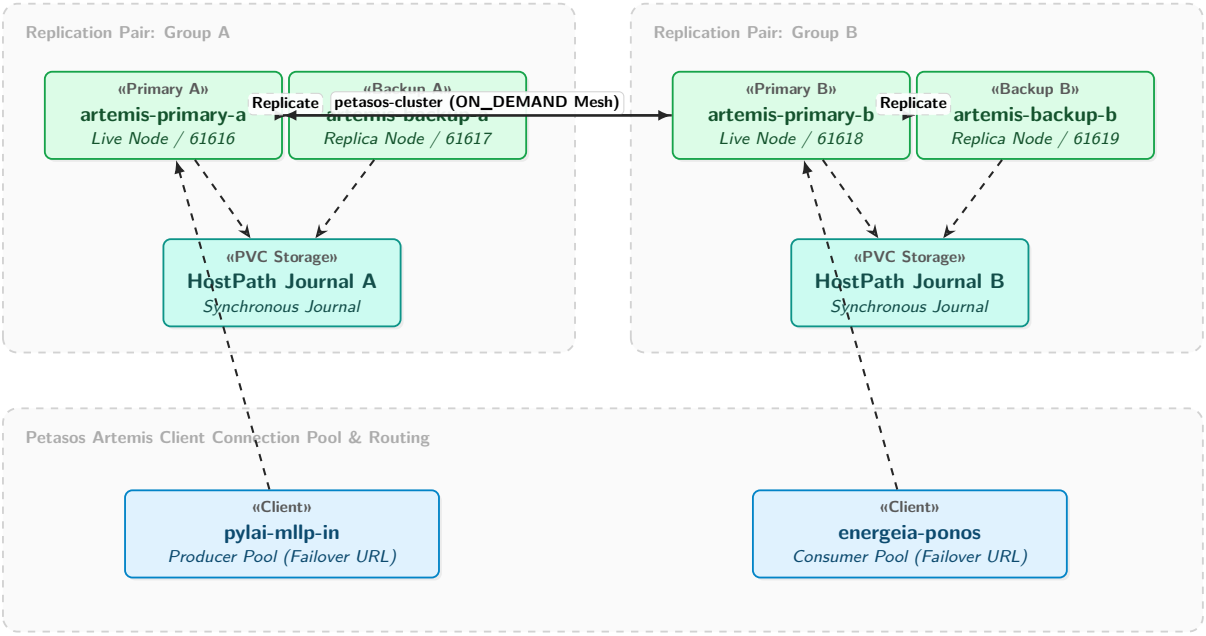


Figure 7.2: ActiveMQ Artemis HA Replication Pairs and Symmetric Cluster Mesh

7.3.1 Failover Connection URL Configuration

Petasos clients connect using multi-endpoint failover URIs enabling automatic, transparent client reconnection:

```
1 (tcp://artemis-primary-a:61616,tcp://artemis-backup-a:61617,tcp://artemis-primary-b
      :61618,tcp://artemis-backup-b:61619)?ha=true&retryInterval=1000&
      retryIntervalMultiplier=1.5&maxRetryInterval=5000&reconnectAttempts=-1
```

Listing 7.1: Petasos Artemis Multi-Endpoint Failover URL

Chapter 8

Collaboration & Matrix Integration (Agora)

The Agora subsystem establishes an authoritative, governed architectural boundary between the Harmonia Health Integration Environment and the Matrix Synapse collaboration ecosystem. Acting primarily as a Matrix Application Service (AS), Agora projects authorized clinical collaboration contexts into Matrix Spaces and Rooms, ingests room interactions, and translates them into canonical Harmonia message envelopes over the Petasos messaging backbone.

Architectural Principle: Matrix as a Capability, Not Architecture

Agora makes Matrix a Harmonia collaboration capability; it does **not** make Matrix the Harmonia architecture. Harmonia remains authoritative for all healthcare information, FHIR resources, Provider Registry state, identity relationships, security policies, and clinical workflows. Matrix provides authorized collaboration and communication surfaces only.

8.1 Subsystem Decomposition & Layering

To guarantee strict architectural separation and prevent Matrix protocol leaks into the clinical domain, Agora is partitioned across four decoupled Maven modules under `agora/`:

- **`agora-api`**: Declares pure Java domain abstractions, event models (`AgoraCollaborationEvent`), space/room/membership requests, and Petasos destination constants (`AgoraTopics`). Free of any Matrix, Spring, or persistence dependencies.
- **`agora-matrix`**: Encapsulates the Matrix Client-Server REST client (`MatrixClientAdapter`), the Synapse Administration Gateway (`SynapseAdministrationGateway`), and Application Service protocol DTOs. Matrix protocol types are strictly quarantined within this module.
- **`agora-core`**: Implements core domain orchestration, including `AgoraCollaborationLifecycleService`, deterministic identity provisioning (`AgoraIdentityService`), care team reconciliation (`AgoraMembershipReconciliationService`), Mnemosyne PostgreSQL persistence entities, and Petasos event bridges.
- **`agora-service`**: Spring Boot 3.2.5 deployment runtime packaging the Application Service REST controller (`ApplicationServiceTransactionEndpoint`), security authentication filters, and Spring Actuator health indicators.

Architectural Note

In accordance with Invariant 8, direct dependencies from Agora to Ponos (`net.fhirfactory.harmonia.energeia.ponos..`) are strictly forbidden. Agora coordinates with clinical workflows exclusively via asynchronous Petasos messaging queues.

8.2 Key Architectural Decisions (ADRs)

The design of Agora is governed by eight platform Architecture Decision Records:

8.3 Collaboration Projections & Room Lifecycles

Agora manages structured collaboration spaces projected from authoritative Harmonia contexts:

Table 8.1: Agora Architecture Decision Records (ADRs)

Identifier	Decision Title	Architectural Summary & Rationale
AGORA-ADR-001	Identity Provisioning	Dedicated local Matrix accounts (`@_harmonia_p_<uuid>:synapse`) are deterministically provisioned via the Synapse Admin REST API, backed by credentials in Mnemosyne.
AGORA-ADR-002	Transport TLS Non-E2EE	Collaboration rooms operate without Olm/Megolm encryption, protected by TLS 1.3 within a closed Kubernetes boundary to allow server-side governance and Petasos dispatch.
AGORA-ADR-003	Authoritative ID Mapping	Mappings between Harmonia entities and Matrix IDs are durably stored in PostgreSQL table `agora_resource_mappings`.
AGORA-ADR-004	Conversation Retention	Synapse enforces a 90-day maximum message lifetime. Clinical decisions must be committed back to Mnemosyne FHIR storage prior to expiration.
AGORA-ADR-005	PHI Minimisation	Room aliases, Space names, and topics must never include patient names, DOB, or MRNs. Opaque UUIDs are strictly enforced.
AGORA-ADR-006	Closed Deployment	Inter-server federation, public registration, and room directories are disabled (`federation.enabled:false`).
AGORA-ADR-007	Transaction Idempotency	Incoming homeserver transactions (`txnId`) are persisted in `agora_as_transactions` to guarantee idempotent processing.
AGORA-ADR-008	Soft Room Archival	Concluded encounters result in read-only power levels, member removal, and an `ARCHIVED` status record rather than a destructive purge.

8.3.1 Patient Collaboration Space Hierarchy

When a patient encounter is initiated, Agora provisions a private Matrix Space linked to four standard child rooms via `m.space.child` state events:

1. **Patient Statistics**: Vitals, clinical trajectory summaries, and encounter metrics.
2. **Patient Tasks**: Clinical workflow assignments, task status transitions, and action notifications.
3. **Patient Discussion**: Real-time multidisciplinary clinical team communication.
4. **Patient Diagnostics**: Laboratory result notifications and diagnostic imaging links.

8.3.2 Practitioner & Group Spaces

In addition to Patient Spaces, Agora provisions:

- **Practitioner Spaces**: Individual practitioner spaces containing child rooms partitioned by `PractitionerRole`.
- **Group Collaboration Rooms**: Multidisciplinary care team coordination surfaces.

8.4 Security Governance & Reconciliation

8.4.1 Themis Default-Deny Authorization

Every collaboration request—including Space creation, room provisioning, user invitation, join approval, and participant kick—is evaluated against Themis. If `ThemisDecision.isAllowed()` is false, the action is rejected immediately, and a sanitized, non-PHI security audit event is emitted.

8.4.2 Membership Drift Reconciliation

Care team membership is authoritatively governed in Harmonia's Provider Registry. The `AgoraMembershipReconciliationService` periodically compares the authoritative roster against active room members retrieved via `GET/_matrix/client/v3/rooms/{roomId}/members`. Any unauthorized participant is ejected via `POST/kick`, and newly authorized practitioners are issued invites via `POST/invite`.

8.5 Application Service Endpoint & Petasos Integration

Agora exposes the Matrix Application Service transaction endpoint:

```
PUT /_matrix/app/v1/transactions/{txnId}
Authorization: Bearer <hs_token>
```

Listing 8.1: Application Service Transaction API

Upon receiving a transaction, Agora executes the following pipeline:

1. **Authentication**: Validates the Bearer token against configured `agora.security.hs-token`.
2. **Deduplication**: Queries `AgoraTransactionRepository` for `txnId`. If already processed, returns HTTP 200 immediately.
3. **Evaluation & Dispatch**: Evaluates Themis authorization for each contained event, translates allowed events into `AgoraCollaborationEvent`, and publishes them onto Petasos queue `petasos.queue.agora.inbound`.
4. **Commit**: Records `txnId` in PostgreSQL table `agora_as_transactions` and acknowledges Synapse with HTTP 200 .

For outbound notifications, `AgoraPetasosEventConsumer` listens on `petasos.queue.agora.outbound`, resolves the target Matrix room ID from Mnemosyne, and dispatches the formatted message via `MatrixClientAdapter.sendMessage()`.

Chapter 9

Workflow & Task Processing Engine (Energeia)

The Energeia subsystem is Harmonia's composable, high-performance workflow execution engine. It coordinates discrete clinical processing activities, manages the lifecycle of canonical integration tasks (`Pragma`), and records end-to-end execution audit trails.

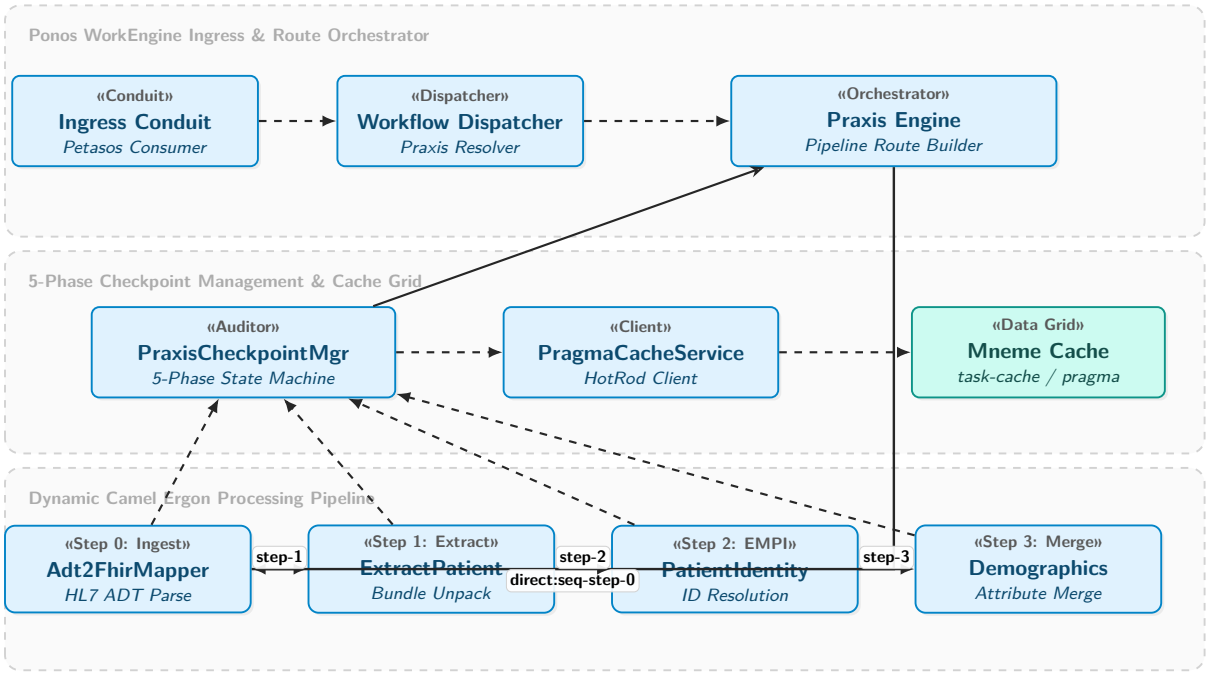


Figure 9.1: Energeia Workflow Architecture: Ponos WorkEngine, Praxis Sequences & Ergon Activities

9.1 Ponos Task Processor (`energeia-ponos`)

`energeia-ponos` is the runtime worker container responsible for:

- Consuming inbound integration tasks from Petasos queues (`clinical.tasks.inbound`, etc.).
- Instantiating and orchestrating Apache Camel execution contexts.
- Interacting with the `Mneme` Infinispan cache grid to store task states and checkpoints.
- Enforcing concurrency controls, transaction boundaries, and thread pool scaling.

9.2 Modular Activity Units (`energeia-erga`)

The `energeia-erga` module provides a library of discrete, reusable `Ergon` activity units. Each Ergon implements a single clinical or technical responsibility adhering to the `ErgonBase` contract:

9.2.1 Standard Ergon Activity Implementations

- **`Hl7v2ToFhirPatientErgon`**: Parses raw HL7 v2 ADT segments (PID, PD1, NK1) and constructs canonical FHIR R5 `Patient` resources.

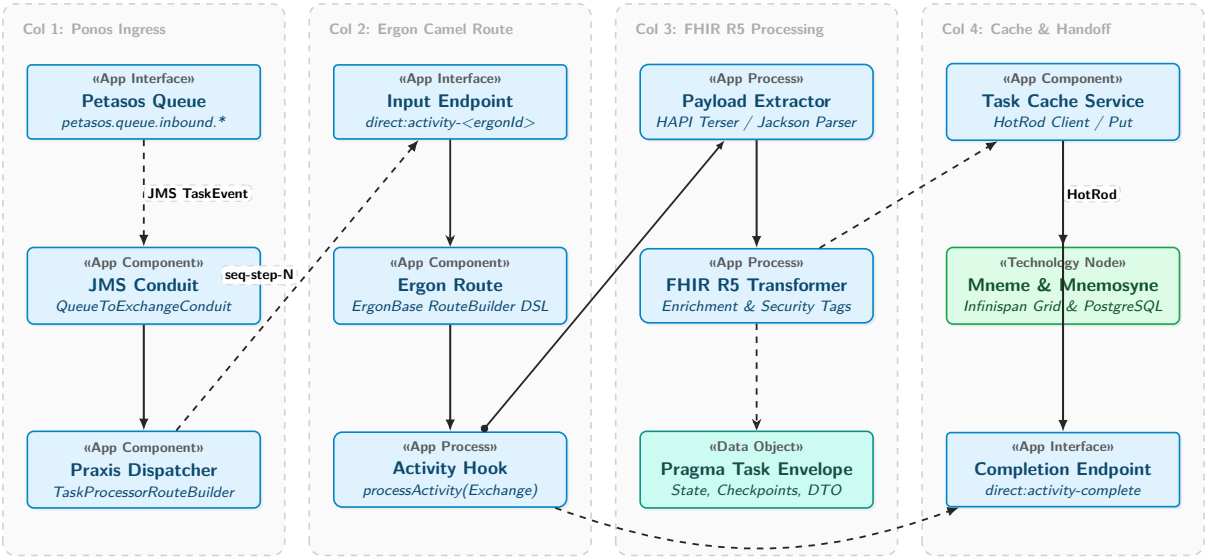


Figure 9.2: Ergon Modular Activity Architecture and Checkpoint Lifecycle

- `PatientIdentityUpdateErgon`: Queries the Enterprise Master Patient Index (EMPI) in Mneme cache to reconcile Medical Record Numbers (MRNs) into master patient IDs.
- `PatientDemographicsUpdateErgon`: Merges updated demographic data (names, addresses, telecoms) into existing patient records.
- `AdtDistributionErgon`: Evaluates clinical distribution rules and fans out outbound notification tasks to dedicated egress queues.
- `AbstractProviderRegistryChangeErgon`: Base class for provider directory change processors (`PractitionerChangeErgon`, `OrganizationChangeErgon`, etc.) performing referential verification.

9.3 Workflow Orchestration Sequences (`energeia-praxis`)

`energeia-praxis` provides the sequence orchestration engine that chains ordered Ergon steps into auditable pipelines.

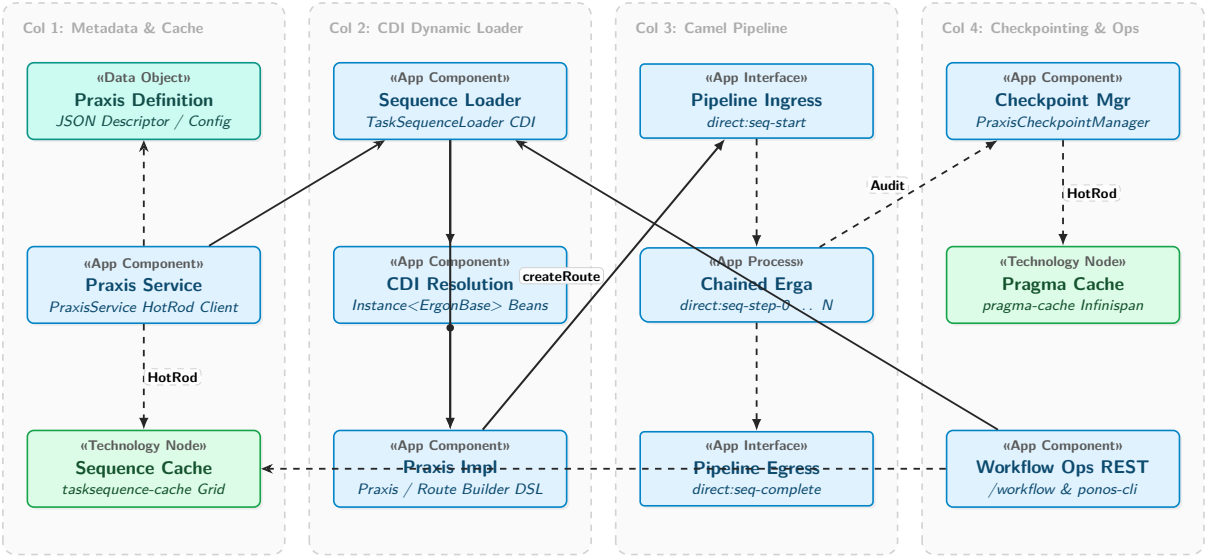


Figure 9.3: Praxis Dynamic Route Synthesis and Multi-Stage Pipeline Execution

9.3.1 Dynamic Route Synthesis & Checkpointing

Praxis constructs Camel routes dynamically using CDI bean discovery. At each Ergon boundary, Praxis creates an immutable `PragmaCheckpoint` capturing:

$$\langle \text{Checkpoint} \rangle = (\text{checkpointId}, \text{pragmaId}, \text{ergonId}, \text{stageName}, \\ \text{timestamp}, \text{status}, \text{statusMessage}, \text{stepIndex}) \tag{9.1}$$

Checkpoints are written to the `Mneme` cache grid, ensuring complete visibility and audit lineage for in-flight tasks.

Chapter 10

Presentation Services & User Interfaces (`Iris`)

The Iris subsystem is Harmonia's presentation tier, providing decoupled, modern web user interfaces and a high-performance Backend-For-Frontend (BEFE) gateway.

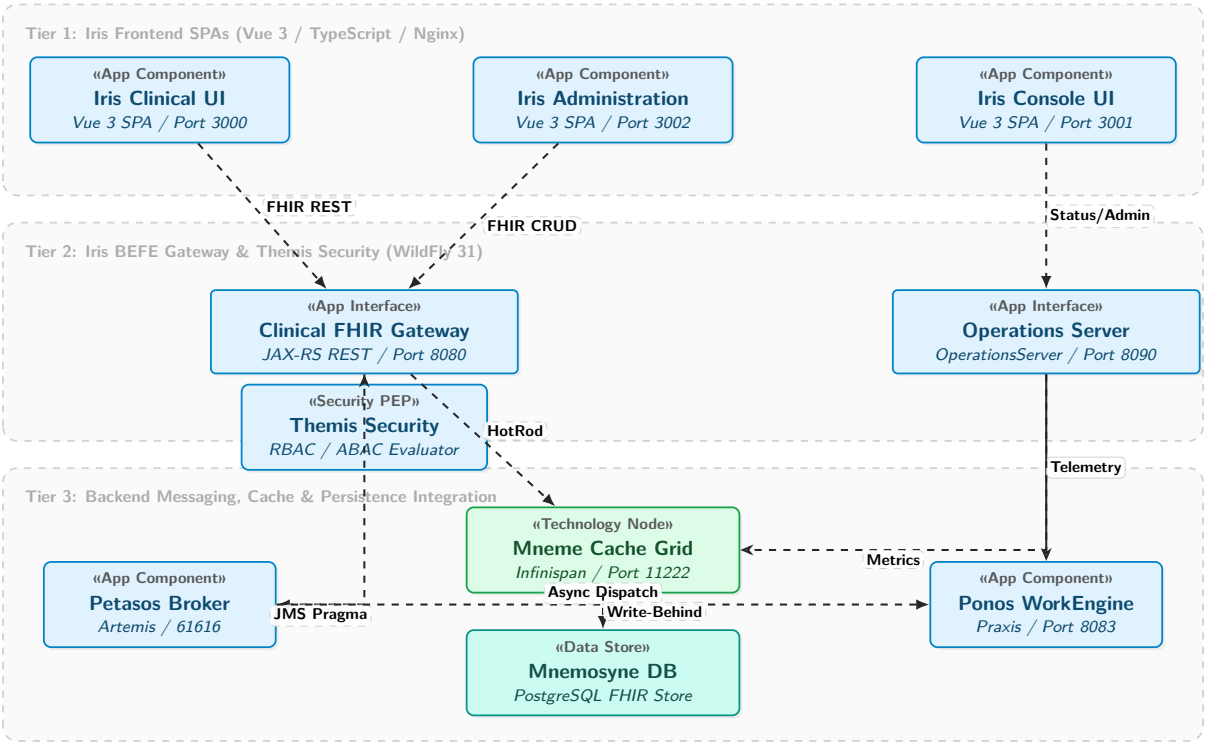


Figure 10.1: Iris Presentation Tier and BEFE Gateway Architecture

10.1 Presentation Single-Page Applications (SPAs)

Harmonia provides three purpose-built Vue 3 / TypeScript web applications served via Nginx:

10.1.1 1. `iris-clinical` (Clinical Portal)

The clinical portal provides care teams with fast, intuitive access to longitudinal patient histories, encounter summaries, lab observations, and diagnostic reports.

10.1.2 2. `iris-console` (Operations & Telemetry Portal)

The operations console provides Site Reliability Engineers (SREs), system administrators, and integration engineers with comprehensive operational telemetry, message flow tracing, and incident remediation capabilities.

Architectural Note

Terminology Reconciliation: Legacy documentation previously referred to this portal as `iris-monitor`. The canonical, repository-backed identifier is standardized across all modules and documentation as `iris-console`.

The operations console is structured around six primary operational perspectives and a diagnostic event flow:

1. **OVERVIEW (`/overview`)**: Answers *"Is Harmonia healthy? Is work moving? Are messages backing up?"* Features a concise, desktop-dense platform health strip, active alert summary ribbons with direct links to degraded subsystems, active work execution rates, and an area-by-area operational status summary. Giant dark KPI cards are eliminated in favor of restrained, readable light summary strips.
2. **SUBSYSTEMS (`/subsystems`)**: Answers *"What is the authoritative architectural status?"* Serves as the primary proving ground for the Iris Design System using a Split-Tree Master-Detail layout. The left tree panel presents Harmonia's six authoritative architectural areas (*Integration & Transport*, *Execution & Processing*, *Information & State*, *Security & Policy*, *Collaboration*, *Presentation*) with child subsystems, accessible status badges, and mandatory English subtitles alongside Greek names. The right master-detail panel provides `IrisSubsystemIdentity`, top-level action controls (`IrisToolbar`), and tabbed inspection of Runtime Instances, Operational Health & Dependencies, and Telemetry Statistics. Middleware runtimes (Artemis, Infinispan, PostgreSQL, Synapse) are subordinated under their consuming capabilities (Petasos, Mneme, Mnemosyne, Agora).
3. **INTERFACES (`/interfaces`)**: Answers *"Are protocol gateways listening and communicating?"* Compact operational table of Pylai inbound and outbound gateways (`pylai-mllp-in`, `pylai-mllp-out-his`, `pylai-mllp-out-lis`, `pylai-fhir-registry`) detailing direction, protocol, port, live status, and safety policy compliance (Invariant 4 Dual-Write AA ACK safety and Invariant 5 Destination Fan-Out). Where real-time message rates are pending live Camel telemetry, honest placeholders are rendered per `IRIS-API-GAP-001`.
4. **WORK (`/work`)**: Answers *"What tasks and workflows are currently executing?"* Displays Energeia Ponos task worker rates, Praxis workflow sequence definitions, and deep diagnostic drill-down into concrete Pragma execution envelopes and Ergon checkpoint timelines.
5. **MESSAGES (`/messages`)**: Answers *"How are messaging transport and queues performing?"* Petasos-centric messaging view subordinating Apache ActiveMQ Artemis cluster node health, queue depth, consumers, producers, enqueue/dequeue throughput rates, and Dead-Letter Queue (DLQ) accumulation surveillance.
6. **HEALTH & ALERTS (`/health`, `/alerts`)**: Answers *"What currently requires operator attention?"* Cross-subsystem operational health matrix, dependency latencies, actionable operational alerts classified by severity (Critical, Warning, Information) and status (Active, Acknowledged, Resolved) with structured remediation guidance ribbons.
7. **EVENTS (`/events`)**: Answers *"What happened to a particular interaction?"* Cross-subsystem diagnostic timeline tracing messages across gateways, brokers, workflow processors, and persistence using technical Correlation IDs, Causation IDs, Message IDs, and Pragma IDs with dual Flowchart and Table views.

Architectural Principle: Zero-PHI Presentation Boundary (Invariant 7)

In strict compliance with Invariant 7, the Iris Operations Console and Operations Backend enforce a complete Zero-PHI boundary. All diagnostic event timelines, queue inspections, and workflow details display only technical identifiers (`correlationId`, `pragmaId`, `messageId`, durations, and error reason codes). Clinical payloads, FHIR resource bodies, HL7 segments, and patient identifiers are strictly excluded from UI rendering and operations API responses.

10.1.3 3. `iris-administration` (Directory Administration Portal)

Provides directory stewards with UI forms for managing practitioners, roles, organizations, and endpoints.

Architectural Principle: Iris-Administration Role Invariant

In accordance with Invariant 3, `iris-administration` is strictly a frontend presentation client consuming FHIR REST APIs. It contains zero database drivers, no JPA/Hibernate entities, and does not execute Provider Registry state machines or referential integrity verification. All authoritative registry governance is executed server-side.

10.2 Backend-For-Frontend Gateway (`iris-befe`)

`iris-befe` is a WildFly 31.0.1 Jakarta EE microservice operating on dual HTTP ports:

- **Port 8080 (Clinical Gateway)**: Serves authenticated clinical FHIR REST endpoints to `iris-clinical`.
- **Port 8090 (Operations Gateway)**: Serves system telemetry, normalized subsystem models, queue statistics, workflow diagnostics, and alert remediation endpoints to `iris-console`.

10.2.1 Operations Backend Architecture (Port 8090)

The operations presentation tier communicates exclusively through `iris-befe` port 8090 via the `OperationsServerManager` Telemetry collection is orchestrated by the `OperationsAggregatorService` through a decoupled provider architecture:

- **Subsystem Health Provider SPI (`SubsystemHealthProvider`)**: Concrete provider implementations for each core subsystem (`PylaiHealthProvider`, `PetasosHealthProvider`, `EnergeiaHealthProvider`, `MnemeHealthProvider`, `MnemosyneHealthProvider`, `CalliopeHealthProvider`, `ThemisHealthProvider`, `AgoraHealthProvider`, and `IrisHealthProvider`) normalize disparate telemetry sources into uniform operational models.
- **Hybrid Telemetry Sourcing**: State is harvested primarily from Infinispan clustered caches (`modulestatus-cache`, `task-cache`, `messagequeue-cache`) supplemented by non-blocking subsystem actuator health probes and ActiveMQ Artemis broker management APIs.
- **Kubernetes Runtime Discovery (`KubernetesInstanceProvider`)**: Dynamically discovers live Pods in the `harmonia` namespace using in-cluster ServiceAccount credentials, mapping Pod lifecycle and resource metrics to domain-neutral instances with graceful fallback to Docker Compose.
- **Themis Default-Deny Governance**: Operations endpoints enforce strict RBAC checks using `themis-api` contracts, requiring `operations.viewer` or `operations.admin` authorities.
- **Partial Failure Resilience**: When downstream telemetry providers experience timeouts or temporary unavailability, the aggregator tags the corresponding subsystem as `UNKNOWN` or `UNAVAILABLE` with stale indicators, guaranteeing that platform summaries continue responding within bounded SLAs (< 200 ms).

10.3 The Iris Design System (`@harmonia/iris-befe`)

To establish visual cohesion, eliminate duplication, and enforce strict presentation boundaries across the Iris application family, `iris-befe/frontend` packages the authoritative Iris Design System as an internal workspace source package (`@harmonia/iris-befe`).

10.3.1 Architecture and Three-Tier Provenance

The design system enforces a strict three-tier provenance model:

1. **UI Primitives Layer**: PrimeVue 4 core components (`primevue@^4.3.0`) and the `@primevue/themes` foundation.
2. **Shared Iris Foundation Layer (`iris-befe/frontend`)**: Custom Aura Light theme preset defined via PrimeVue's `definePreset`, centralized design tokens (`tokens.css`), domain-neutral application shell components (`IrisApplicationShell`, `IrisHeader`, `IrisPrimaryNavigation`, `IrisToolbar`, `IrisBreadcrumbs`), and accessible presentation primitives (`IrisStatus`, `IrisSubsystemIdentity`, `IrisDataTable`, `IrisHierarchy`, `IrisEmptyState`).
3. **Application Layer**: Specialized SPAs (`iris-console`, and planned migrations for `iris-clinical` and `iris-administration`) consuming the shared package via Vite path aliases and file dependencies.

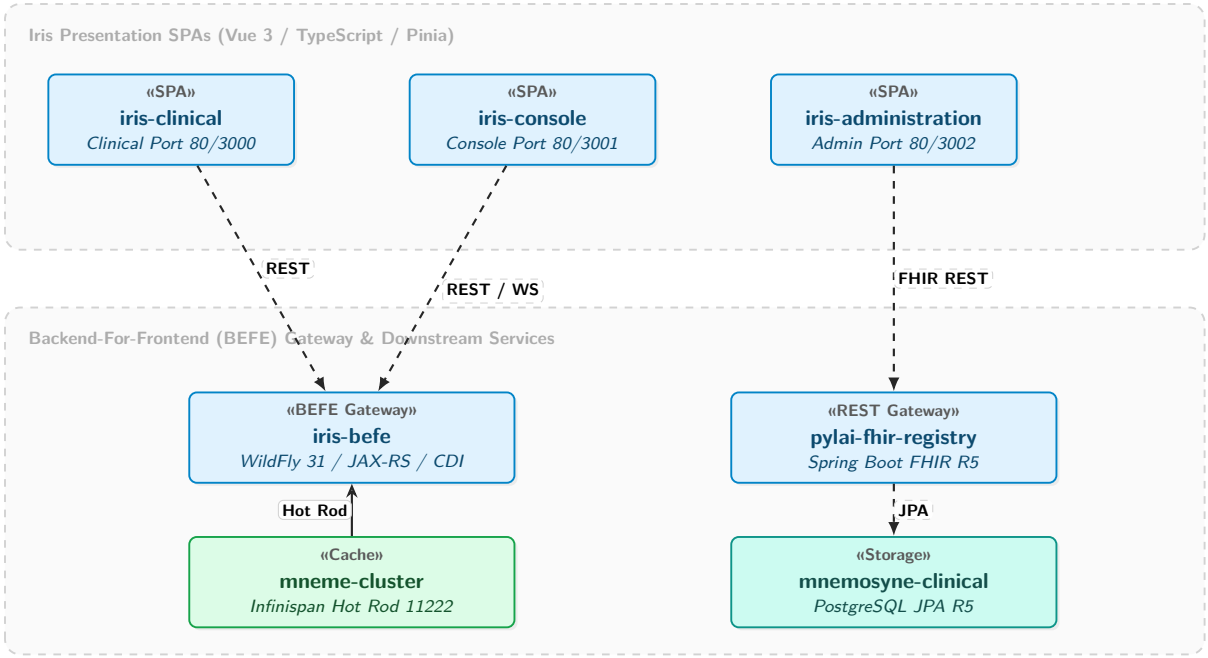


Figure 10.2: Iris UI Federation, Hot Rod Caching & REST Endpoint Integration

Architectural Principle: Unidirectional Presentation Dependency

The Iris Presentation Tier strictly enforces unidirectional dependency flow: `SPAs` → `iris-befe/frontend`. The shared design system package contains zero reverse dependencies on `iris-console`, `iris-clinical`, or `iris-administration`, and imports zero domain models or database drivers. This boundary is verified continuously by `IrisDecouplingArchitectureTest.java` and automated frontend test suites.

10.3.2 Design Tokens and Light Theme Baseline

The visual language replaces fragmented dark utility styling with an accessible, high-contrast Aura Light theme:

- **Surfaces**: Glare-free Slate-50 background (`#f8fafc`) paired with pure white cards and table surfaces (`#ffffff`).
- **Typography & Contrast**: Slate-900 primary text (`#0f172a`, 13:1 contrast ratio) and Slate-600 secondary metadata (`#475569`, 5.5:1 ratio), fully compliant with WCAG 2.1 AA benchmarks.
- **Accessible Status Semantics**: `IrisStatus` enforces color-independent status recognition by pairing distinct symbols (● Healthy, ▲ Degraded, × Unavailable, ? Unknown) with explicit textual labels and staleness indicators.

10.4 Future Migration Assessment: `iris-clinical` & `iris-administration`

While `iris-console` serves as the initial proving ground, the Iris Design System is engineered to enable seamless adoption across Harmonia's other portals:

- **`iris-clinical` Migration**: Can adopt `IrisApplicationShell` with `application="Clinical"`, `IrisPrimaryNavigation` across patient/encounter routes, `IrisToolbar` for FHIR search filtering, and `IrisDataTable` for clinical resource browsing. Estimated migration effort is Low-to-Medium (2–3 engineering days) with zero changes to existing Pinia stores or backend REST contracts.
- **`iris-administration` Migration**: Can replace legacy dark topbars with `IrisApplicationShell` and `IrisPrimaryNavigation`, standardize directory tables with `IrisDataTable`, and replace local status badges with `IrisStatus`. Estimated effort is Medium (3–4 engineering days), maintaining strict compliance with Invariant 3.

Chapter 11

Iris Presentation Tier and User Interfaces Specification & User Guide

Architectural Specification, Interface Catalog, and Operational Manual for the Iris Multi-Portal Ecosystem

11.1 Presentation Tier Overview & Architectural Principles

The **Iris Presentation Tier** provides the unified graphical and interface layer of the Harmonia Health Information Exchange (HIE). Operating as a decoupled, multi-portal presentation environment, Iris delivers tailored, role-specific user interfaces for clinicians, system operations engineers, provider directory administrators, and healthcare practitioners while mediating all backend communication through the **Iris Backend-For-Frontend (BEFE)** gateway.

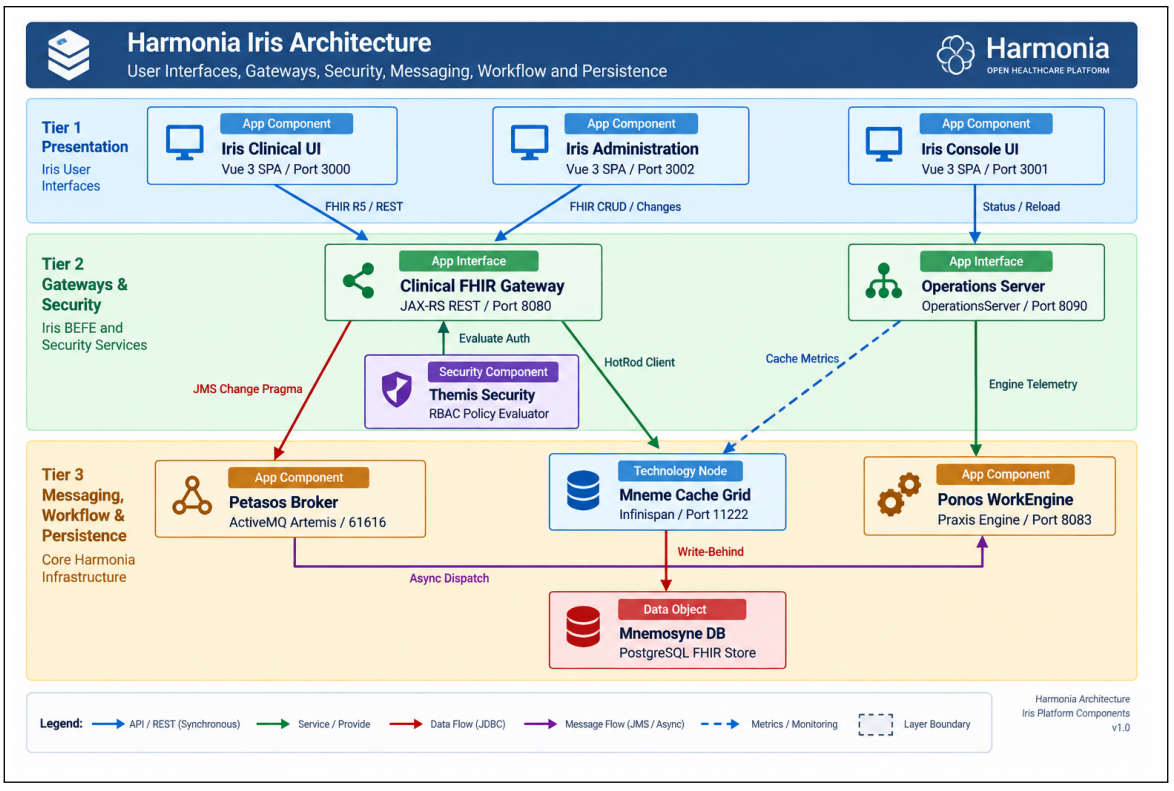


Figure 11.1: ArchiMate 3.2 Component and Interaction View for the Iris Presentation Tier and BEFE Gateway

11.1.1 Multi-Portal Topology and Separation of Concerns

To guarantee stringent security boundaries, eliminate cognitive clutter, and optimize performance for distinct operational personas, Iris is partitioned into three independent Single-Page Applications (SPAs) developed with TypeScript, Vue 3, and Pinia, complemented by the WildFly-hosted BEFE gateway:

11.1.2 Architectural Principles

The Iris presentation tier adheres to four core engineering principles:

Submodule	Application Type	Network Port	Target Persona & Core Capabilities
`iris-clinical`	Vue 3 SPA	Port 3000	**Clinicians & Care Coordinators**: Longitudinal patient demographics, provider search, location hierarchies, care teams, audit provenance, consent records, and document discovery.
`iris-console`	Vue 3 SPA	Port 3001	**Operations & Site Reliability Engineers**: Five canonical operational perspectives (Subsystems, Queues, Workflows, Events, Alerts), cross-subsystem event sequence tracing, instance detail inspections, and alert remediation.
`iris-administration`	Vue 3 Dual-Portal	Port 3002	**Practitioners & Directory Administrators**: Self-service profile change requests, role attestations, administrative work queue triage, directory CRUD, data quality audits, and RBAC governance.
`iris-befe`	Jakarta EE 10 / WildFly	Ports 8080 / 8090	**Integration Gateway**: Dual-port microprofile gateway providing synchronous FHIR R5 REST endpoints (Port 8080) and dedicated operations socket telemetry (Port 8090).

Table 11.1: Iris Presentation Tier Subsystem and Portal Topology Matrix

Architectural Principle: 1. Strict Domain and Network Port Isolation

Clinical FHIR transactions and administrative governance workflows operate over standard JAX-RS REST endpoints on Port 8080, while mission-critical systems operations, broker telemetry, cross-subsystem diagnostics, and alert governance execute over a dedicated socket server on Port 8090. Operational spikes or diagnostic queries can never degrade clinical query bandwidth.

Architectural Principle: 2. Governed Change Lifecycle over Direct Mutation

User interfaces never execute direct synchronous mutations against authoritative provider directory storage. All creations, updates, and attestations submitted via `iris-administration` are wrapped into canonical `ProviderRegistryChangePragma` envelopes, queued asynchronously via Petasos, and subjected to automated referential validation and administrative review.

Architectural Principle: 3. Zero-Trust Client-Side RBAC & Visual Security Badging

Every route transition in the Iris SPAs is intercepted by Themis-aware navigation guards. Views conditionally render visual `SecurityBadge` indicators displaying active security contexts (e.g. `CONFIDENTIALITY: RESTRICTED`, `AUTHORITY: provider.admin`), preventing unauthorized command dispatch before edge gateway evaluation.

Architectural Principle: 4. Reactive State Hydration with Pinia Stores

Component state is cleanly decoupled from presentation logic using modular Pinia stores. Stores manage asynchronous fetch lifecycles, optimistic caching, pagination cursors, and active search filters, ensuring instantaneous UI reactivity and resilient error boundary containment.

11.2 Iris BEFE Dual-Port Gateway Specification

The **Iris Backend-For-Frontend (BEFE)** module (`iris/iris-befe`) is a high-performance Jakarta EE 10 gateway hosted within the Harmonia WildFly application server runtime. It provides translation,

aggregation, security policy enforcement, and caching between the browser SPAs and the underlying Harmonia distributed tier.

11.2.1 Dual-Port Architecture: Port 8080 vs Port 8090

BEFE implements an architectural duality separating public clinical services from operational management:

- **Clinical FHIR REST Gateway (Port 8080)**: Standard Jakarta RESTful Web Services (JAX-RS 3.1) application activated via `net.fhirfactory.harmonia.befe.config.JaxRsActivator` at root path `/api/v1`. It mediates all clinical FHIR R5 resource requests, using the `FhirCacheService` and HotRod clients to query the `Mneme` Infinispan distributed cache grid.
- **Dedicated Operations HTTP Server (Port 8090)**: A lightweight, non-blocking socket server managed by `OperationsServerManager`. It isolates normalized subsystem, queue, workflow, event, and alert telemetry from JAX-RS connection pools and clinical request threads.

11.2.2 BEFE JAX-RS Resource Endpoints (Port 8080)

Table 11.2 catalogs the core clinical and provider directory endpoints hosted on Port 8080.

HTTP Verb	Path	Required Authority	Description & Response
`GET`	`/api/v1/Person`	`provider.read`	Search `Person` resources by name, identifier, or active status. Returns FHIR `Bundle`.
`GET`	`/api/v1/Practitioner`	`provider.read`	Query `Practitioner` directory entries with HPI-I/AHPRA filter parameters.
`GET`	`/api/v1/PractitionerRole`	`provider.read`	Traverse practitioner-to-organization affiliations, specialty codes, and locations.
`GET`	`/api/v1/Organization`	`provider.read`	List healthcare provider organizations, hospital trusts, and departmental units.
`GET`	`/api/v1/Location`	`provider.read`	Retrieve facility hierarchy nodes (jurisdiction, hospital, building, ward, bed).
`GET`	`/api/v1/HealthcareService`	`provider.read`	Retrieve specialized clinical service offerings and operating hours.
`GET`	`/api/v1/Group`	`provider.read`	Discover care teams, clinical specialty rosters, and multidisciplinary units.
`GET`	`/api/v1/Provenance`	`themis.audit`	Retrieve cryptographic audit provenance chains and digital signatures.
`GET`	`/api/v1/AuditEvent`	`themis.audit`	Query longitudinal security and data access event logs.
`GET`	`/api/v1/Consent`	`consent.read`	Inspect patient privacy directives, opt-in/opt-out status, and sharing provisions.
`GET`	`/api/v1/Task`	`provider.read`	Poll asynchronous governed change request processing status.
`POST`	`/api/v1/Task`	`provider.change.submit`	Submit new governed profile change request envelope. Returns `HTTP 202 Accepted`.
`GET`	`/api/v1/Communication`	`comm.read`	Retrieve clinical notification dispatches and secure message receipts.
`GET`	`/api/v1/DocumentReference`	`doc.read`	Query clinical document manifests and CDA/FHIR attachment pointers.

Table 11.2: Iris BEFE Clinical JAX-RS REST Endpoint Catalog (Port 8080)

11.2.3 CORS and Security Headers

BEFE enforces strict Cross-Origin Resource Sharing (CORS) rules through `net.fhirfactory.harmonia.befe.config.Cor`
The filter validates origin headers against configured whitelist patterns (`http://localhost:3000`, `http://localhost:3001`
`http://localhost:3002`, or production domain configurations), injecting the necessary response headers:

```
@Provider
@PreMatching
public class CorsFilter implements ContainerResponseFilter {
    @Override
    public void filter(ContainerRequestContext requestContext,
                       ContainerResponseContext responseContext) {
        responseContext.getHeaders().add("Access-Control-Allow-Origin",
            requestContext.getHeaderString("Origin") != null ?
            requestContext.getHeaderString("Origin") : "*");
        responseContext.getHeaders().add("Access-Control-Allow-Credentials", "true");
        responseContext.getHeaders().add("Access-Control-Allow-Headers",
            "origin, content-type, accept, authorization, x-correlation-id, x-
    requested-with");
        responseContext.getHeaders().add("Access-Control-Allow-Methods",
            "GET, POST, PUT, DELETE, OPTIONS, HEAD");
        responseContext.getHeaders().add("Access-Control-Max-Age", "1209600");
    }
}
```

Listing 11.1: BEFE CORS Header Management (CorsFilter.java)

11.3 Iris Clinical Portal (`iris-clinical`) Specification & User Guide

The **Iris Clinical Portal** (`iris/iris-clinical`) is designed for clinical practitioners, care coordinators, and medical ward clerks. Operating on Port 3000, it enables rapid search, exploration, and verification of patient records, provider registries, location trees, and clinical provenance trails.

11.3.1 Component Topology and Layout Architecture

The SPA is structured around a responsive viewport comprising:

1. **Top Navigation Bar**: Contains the global patient/provider quick search box, active facility context selector, system notification indicator, and current authenticated user badge with security clearance level.
2. **Collapsible Context Sidebar**: Provides direct navigation across clinical domains, grouping routes into *Identity & Demographics*, *Provider Directory*, *Governance & Consent*, and *Clinical Exchange*.
3. **Main Workspace Viewport**: Hosts the active Vue Router component, complete with responsive data tables, detail flyouts, and breadcrumb navigation.

11.3.2 Comprehensive Clinical View Catalog

`iris-clinical` provides 13 dedicated views mapped to Vue components in `src/views/`:

11.3.3 Pinia State Management: Clinical Store Pattern

The clinical portal utilizes Pinia stores to manage asynchronous fetch operations, pagination cursors, and local caching. The listing below illustrates the reactive store pattern utilized across all clinical views:

```
import { defineStore } from 'pinia';
import axios from 'axios';

const BEFE_API_URL = import.meta.env.VITE_BEFE_API_URL || 'http://localhost:8080/api/
    v1';

export const useClinicalStore = defineStore('clinical', {
  state: () => ({
    practitioners: [] as any[],
    selectedPractitioner: null as any | null,
    loading: false,
    error: null as string | null,
    searchFilter: {
      name: '',
```

View Name	Route Path	FHIR Resource	Clinical Workflow Function
`DashboardView`	`/`	Summary Metrics	Overview of platform activity, recent queries, urgent clinical notifications, and favorite practitioner shortcuts.
`PersonView`	`/persons`	`Person`	Search and review patient master index identities, merged records, and cross-system identifier links.
`PractitionerListView`	`/practitioners`	`Practitioner`	Search registered clinicians by name, identifier, specialty, and credential status with full profile inspection.
`OrganizationView`	`/organizations`	`Organization`	Explore hospital networks, clinics, and diagnostic labs with parent-child relationship visualizers.
`LocationHierarchyView`	`/locations`	`Location`	Navigate interactive tree structures of physical facilities from regional jurisdiction down to specific hospital beds.
`HealthcareServiceView`	`/services`	`HealthcareService`	Directory of clinical services (e.g. Cardiology, Radiology) with operating hours, eligibility, and referral paths.
`GroupManagementView`	`/groups`	`Group`	Inspect and manage multidisciplinary care teams, clinical committees, and on-call specialist groups.
`ProvenanceView`	`/provenance`	`Provenance`	Audit the origin, lifecycle transitions, and cryptographic signatures of modified clinical documents.
`AuditEventView`	`/audit`	`AuditEvent`	Review longitudinal access logs, security checks, and data export events for compliance auditing.
`ConsentView`	`/consent`	`Consent`	Review patient consent directives, privacy restrictions, and disclosure rules governing medical record access.
`TaskView`	`/tasks`	`Task`	Track clinical orders, lab fulfillments, and asynchronous workflow execution states.
`CommunicationView`	`/communication`	`Communication`	In-basket for urgent clinical alerts, lab panic results, and secure direct messages.
`DocumentReferenceView`	`/documents`	`DocumentReference`	Query and view clinical discharge summaries, lab reports, and CDA/PDF documents.

Table 11.3: Iris Clinical Portal View Matrix and FHIR Resource Mapping

```
      identifier: '',
      activeOnly: true
    }
  }),
  actions: {
    async fetchPractitioners() {
      this.loading = true;
      this.error = null;
      try {
        const params: any = {};
        if (this.searchFilter.name) params.name = this.searchFilter.name;
        if (this.searchFilter.identifier) params.identifier = this.searchFilter.
    identifier;
        if (this.searchFilter.activeOnly) params.active = 'true';

        const response = await axios.get(`${BEFE_API_URL}/Practitioner`, { params });
        this.practitioners = response.data.entry?.map((e: any) => e.resource) || [];
      } catch (err: any) {
        this.error = err.response?.data?.message || err.message;
      } finally {
        this.loading = false;
      }
    }
  }
});
```

Listing 11.2: TypeScript Pinia Store Pattern for Clinical Entities (stores/clinicalStore.ts)

11.4 Iris Operations Console (`iris-console`) Specification & User Guide

The **Iris Operations Console** (`iris/iris-console`) is the systems administration, telemetry surveillance, and workflow diagnostic dashboard operating on Port 3001. Built for Site Reliability Engineers (SREs), integration engineers, and DevOps personnel, it delivers deep real-time observability, message flow tracing, and incident remediation across the Harmonia integration platform.

11.4.1 Architectural Perspectives & Shell Topology

The console is architected around six canonical operational perspectives and a diagnostic event flow, unified by the shared `@harmonia/iris-befe` design system and Aura Light theme:

Architectural Note

Persistent Shell Architecture (`IrisApplicationShell.vue`):

- **Global Operations Header (`IrisHeader.vue`):** Displays active deployment environment (e.g. `PROD / microk8s-01`), cluster status, platform overall health badge (`HEALTHY`, `DEGRADED`, `UNAVAILABLE`), active subsystem count, critical and warning alert pill indicators, current timestamp, last refresh timestamp, and manual refresh trigger.
- **Persistent Top Navigation (`IrisPrimaryNavigation.vue`):** High-contrast top navigation bar with direct routes to the six canonical operational perspectives: `/overview`, `/subsystems`, `/interfaces`, `/work`, `/messages`, and `/health`.
- **Standardized Page Action Bar (`IrisToolbar.vue`):** Consistent page-level action bar supporting search filtering, time-window selectors (`15m`, `1h`, `6h`, `24h`), manual refresh with spin indicator, and action triggers.

Figure 11.2: Iris Operations Console Shell and Global Navigation Layout

11.4.2 The Six Canonical Operational Perspectives

1. **OVERVIEW Perspective (`/overview`):** Answers *"Is Harmonia healthy? Is work moving? Are messages backing up?"* Features a high-density platform health strip, active alert ribbons with direct

links to degraded subsystems, active work execution rates, and an area-by-area summary across all six Harmonia architectural areas.

2. **SUBSYSTEMS Perspective (`/subsystems`)**: Answers *"What is the authoritative architectural status?"* Operates as the primary proving ground for the Iris Design System via a Split-Tree Master-Detail layout:
   - **Architectural Tree Navigator (`SubsystemTreeNavigator.vue` / `IrisHierarchy`)**: Organizes Harmonia into the six authoritative architectural areas: *Integration & Transport* (Pylai, Petasos), *Execution & Processing* (Energeia: Ponos, Ergon, Praxis), *Information & State* (Calliope, Mneme, Mnemosyne), *Security & Policy* (Themis), *Collaboration* (Agora), and *Presentation* (Iris). Displays accessible status badges and mandatory English subtitles alongside Greek mythological names.
   - **Subsystem Header (`IrisSubsystemIdentity.vue`)**: Displays Greek name, English architectural description, version, status badge, and action buttons.
   - **Master-Detail Tabbed Inspection**: Features three tabs: *Runtime Instances* (using `IrisDataTable` with compact row density and slide-out Pod detail drawer), *Operational Health & Dependencies* (reporting P95 latency and dependency round-trip latencies), and *Telemetry Statistics* (reactive SVG micro-charts styled with Iris light theme tokens).
   - **Middleware Subordination**: Subordinates runtime infrastructure under consuming Harmonia capabilities (ActiveMQ Artemis under Petasos, Infinispan under Mneme, PostgreSQL under Mnemosyne, Synapse under Agora).
3. **INTERFACES Perspective (`/interfaces`)**: Answers *"Are protocol gateways listening and communicating?"* Compact operational table of Pylai inbound and outbound gateways (`pylai-mllp-in`, `pylai-mllp-out-his`, `pylai-mllp-out-lis`, `pylai-fhir-registry`) displaying direction, protocol, port, live status, and safety policy compliance (Invariant 4 Dual-Write AA ACK safety and Invariant 5 Destination Fan-Out). Where real-time throughput metrics are pending live Camel hooks, honest placeholders are rendered per `IRIS-API-GAP-001`.
4. **WORK Perspective (`/work`, formerly `/workflows`)**: Answers *"What tasks and workflows are currently executing?"* High-level indicators of active Praxis executions, Ponos task worker rates, queued/running/completed/failed work, and diagnostic drill-down into concrete Pragma execution envelopes and Ergon checkpoint timelines.
5. **MESSAGES Perspective (`/messages`, formerly `/queues`)**: Answers *"How are messaging transport and queues performing?"* Petasos-centric messaging view subordinating Apache ActiveMQ Artemis cluster node health, queue depth, consumers, producers, enqueue/dequeue throughput rates, and Dead-Letter Queue (DLQ) accumulation surveillance.
6. **HEALTH & ALERTS Perspective (`/health`, `/alerts`)**: Answers *"What currently requires operator attention?"* Cross-subsystem operational health matrix, dependency latencies, actionable operational alerts classified by severity (`CRITICAL`, `WARNING`, `INFORMATION`) and lifecycle status (`ACTIVE`, `ACKNOWLEDGED`, `RESOLVED`), with operator remediation guidance and one-click acknowledgment actions.
7. **EVENTS Diagnostic Flow (`/events`)**: Answers *"What happened to a particular interaction?"* Cross-subsystem diagnostic sequence tracing with dual Flowchart and Table views, correlation filtering, and zero-PHI technical metadata inspection.

11.4.3 Telemetry Gaps Register (`IRIS-API-GAP`)

Where backend JAX-RS endpoints on Port 8090 do not yet supply full operational telemetry, limitations are formalized in the API Gap Register:

- `IRIS-API-GAP-001`: Live Interface Throughput and Error Metrics (requires live Camel MBean hooks in `PylaiHealthProvider`).
- `IRIS-API-GAP-002`: Granular Synapse Matrix Collaboration Metrics (Matrix room counts and AS transaction rates from Agora).
- `IRIS-API-GAP-003`: Granular Interface Restart History and Wire Diagnostics.
- `IRIS-API-GAP-004`: Granular Container CPU/Memory Metrics Outside Kubernetes.
- `IRIS-API-GAP-005`: Durable Alert Acknowledgement Persistence.

11.4.4 Operations Backend REST API Catalog (Port 8090)

All operational telemetry is served via `iris-befe` on Port 8090 under the `/api/operations/*` resource path:

HTTP Endpoint	HTTP Method	Operational Telemetry Resource
`/api/operations/summary`	`GET`	Overall platform status, environment, cluster ID, subsystem counts, active alert counts.
`/api/operations/subsystems`	`GET`	Inventory of all 9 core subsystems and their operational states.
`/api/operations/subsystems/{id}`	`GET`	Single subsystem overview, description, instance count, and version.
`/api/operations/subsystems/{id}/instances`	`GET`	Runtime instances, Pod name, node, role, state, uptime, CPU %, and memory MB.
`/api/operations/subsystems/{id}/health`	`GET`	Availability %, failed ops, restart count, P95 latency, and dependency latencies.
`/api/operations/subsystems/{id}/statistics`	`GET`	Time-series statistics (events in/out, throughput, latency percentiles) for 15m/1h/6h/24h.
`/api/operations/queues`	`GET`	ActiveMQ Artemis queues, depths, consumer counts, rates, and DLQ metrics.
`/api/operations/queues/{id}`	`GET`	Single queue details and historical message depth trends.
`/api/operations/workflows`	`GET`	Praxis workflow sequence executions, throughput rates, and P95 latency.
`/api/operations/workflows/{id}`	`GET`	Single workflow sequence definition and active counts.
`/api/operations/pragmas/{id}`	`GET`	Pragma execution lifecycle details, retry count, and Ergon checkpoints.
`/api/operations/events`	`GET`	Cross-subsystem interaction diagnostic events with correlation filtering.
`/api/operations/events/{id}`	`GET`	Single diagnostic event details and correlated sequence metadata.
`/api/operations/alerts`	`GET`	Operational alerts filtered by severity, status, and subsystem.
`/api/operations/alerts/{id}/acknowledge`	`POST`	Acknowledge active operational alert with operator identity payload.

Table 11.4: Operations Backend REST Endpoint Catalog (iris-befe Port 8090)

11.5 Iris Provider Administration (`iris-administration`) Specification & User Guide

The **Iris Provider Administration** portal (`iris/iris-administration`) is a dual-portal application operating on Port 3002. It serves both individual healthcare providers through a **Self-Service Portal** and credentialing staff through a **Departmental Administration Portal**.

11.5.1 Dual-Portal Architecture and Route Guarding

The portal dynamically presents navigation menus and available actions based on authenticated Themis authorities:

11.5.2 The Governed Change Workflow Lifecycle

Modifications submitted by practitioners or administrators follow a deterministic asynchronous workflow:

1. **Submission (`RequestChangeView.vue`)**: The practitioner modifies contact details or adds a new practice location. Submitting the form generates a JSON change proposal and dispatches `POST /api/v1/Task` to BEFE.
2. **Pragma Envelope Creation**: BEFE constructs a `ProviderRegistryChangePragma` with state `RECEIVED`, assigns a unique UUID, and returns `HTTP 202 Accepted` with `Location: /Task/{id}`.
3. **Automated Pre-Validation**: Ponos executes the first phase of `seq-provider-registry-change-pipeline`, verifying identifier syntax, active status of referenced organizations, and lack of duplicate records. If valid, the task transitions to `PENDING_REVIEW`.
4. **Work Queue Triage (`WorkQueueView.vue`)**: A credentialing administrator opens the task, inspects the side-by-side visual diff (highlighting modified fields in green and deprecated fields in red), and evaluates supporting credential documents.
5. **Approval / Rejection**:
   - If **Approved**: The admin clicks *Approve Changes*. BEFE updates the Pragma to `APPROVED`. Ponos executes the commit phase, writing the updated resource to Mnemosyne PostgreSQL with an incremented `version_id`, and sets the Pragma to `COMPLETED`.
   - If **Rejected**: The admin enters a rejection rationale. The Pragma transitions to `REJECTED`, and a notification appears in the practitioner's *My Requests* view.

11.6 Themis Security & RBAC Integration

All Iris presentation applications integrate natively with the **Themis Policy Evaluation Engine**. Security is enforced through a multi-layered defense-in-depth model spanning the browser UI, route navigation guards, and backend BEFE interceptors.

11.6.1 Themis Authority Scopes

Access is governed by granular Themis authority strings:

- `provider.read`: Read-only query access to public practitioner, location, organization, and service directory records.
- `provider.search`: Advanced search and multi-criteria query capabilities across directory entities.
- `provider.change.submit`: Permission to author and submit change proposals via the self-service portal.
- `provider.admin`: Comprehensive directory administration authority, including work queue triage, direct entity editing, and data quality remediation.
- `themis.admin`: Administrative access to security policies, role-to-authority mappings, and user account provisioning.
- `themis.audit`: Access to non-redacted cryptographic audit logs, provenance chains, and system access histories.
- `ops.read` / `ops.admin`: Telemetry inspection and operational alert governance authority on Port 8090.

11.6.2 Client-Side Router Navigation Guards

Every SPA registers global Vue Router navigation guards to prevent unauthorized route activation:

```
router.beforeEach(async (to, from, next) => {
  const securityStore = useSecurityStore();

  // 1. Ensure security context is initialized
  if (!securityStore.initialized) {
```

Portal Mode	Route Path	Required Authority	Workflow Purpose
Provider Self-Service Portal			
My Details	`/self-service/my-details`	`provider.read`	Practitioner inspects own demographic profile, qualifications, identifiers, and languages.
My Roles	`/self-service/my-roles`	`provider.read`	Review organization affiliations, specialty roles, and clinical locations.
My Organizations	`/self-service/my-organizations`	`provider.read`	View managing organization hierarchy and departmental contacts.
My Locations	`/self-service/my-locations-services`	`provider.read`	Review physical facility assignments and clinical service offerings.
My Endpoints	`/self-service/my-endpoints`	`provider.read`	Verify electronic communication endpoints and secure messaging addresses.
My Requests	`/self-service/my-requests`	`provider.read`	Track submitted profile change requests, current review status, and reviewer notes.
Request Change	`/self-service/request-change`	`provider.change.submit`	Interactive form to submit updates to address, phone, qualifications, or roles.
Departmental Administration Portal			
Admin Dashboard	`/admin/dashboard`	`provider.admin`	High-level directory statistics, pending work queue counts, and data quality alerts.
Work Queue	`/admin/work-queue`	`provider.admin`	Triage, diff review, and approval/rejection of practitioner change requests.
Provider Search	`/admin/search`	`provider.search`	Advanced enterprise search across all practitioners, roles, and organizations.
Entity Admin	`/admin/practitioners`	`provider.admin`	Full administrative CRUD operations for `Practitioner` entities.
Role Admin	`/admin/roles`	`provider.admin`	Authoritative management of `PractitionerRole` associations.
Organization Admin	`/admin/organizations`	`provider.admin`	Directory management of hospi-

```
    await securityStore.initializeSecurity();
  }

  // 2. Check for required authority
  const requiredAuth = to.meta.requiredAuthority as string | undefined;
  if (requiredAuth && !securityStore.hasAuthority(requiredAuth)) {
    console.warn(`Access Denied: Missing authority '${requiredAuth}' for route '${to.path}'`);
    return next({ name: 'AccessDenied' });
  }

  // 3. Check for specific self-service requirements
  if (to.meta.requiresSelfService && !securityStore.isProviderAuthenticated) {
    return next({ name: 'AccessDenied' });
  }

  next();
});
```

Listing 11.3: Vue Router Navigation Guard with Themis Authorities (router/index.ts)

11.6.3 Visual Security Badging

To ensure complete transparency regarding the active security context, Iris components embed the `SecurityBadge.vue` component in page headers. The badge displays:

- Current active user identity and HPI-I identifier.
- Granted Themis role and authority pill tags.
- Confidentiality classification banner (e.g. `CONFIDENTIALITY: NORMAL` vs `CONFIDENTIALITY: RESTRICTED`).

11.7 Concrete Operational Examples & Step-by-Step Walkthroughs

This section provides three concrete, end-to-end operational scenarios demonstrating the real-world utility of the Iris user interfaces and BEFE gateway.

11.7.1 Example 1: Clinician Locating an On-Call Specialist and Endpoint

Scenario: Dr. Sarah Jenkins (Cardiologist) needs to locate the on-call specialist for Acute Pediatric Neurology at St. Jude Hospital and obtain their direct FHIR electronic messaging endpoint.

1. **User Interaction**: Dr. Jenkins opens `iris-clinical` (`http://localhost:3000`), clicks **Healthcare Services** in the sidebar, and filters by *Specialty: Pediatric Neurology* and *Location: St. Jude Hospital*.
2. **BEFE Request**: The portal executes a REST query to BEFE Port 8080:

```
GET /api/v1/HealthcareService?service-type=pediatric-neurology&organization=org-st-jude-01 HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJhbGciOi...
Accept: application/fhir+json
```

Listing 11.4: HTTP Request to Query Healthcare Services

3. **BEFE Response**: BEFE returns the matching `HealthcareService` and linked `PractitionerRole` and `Endpoint` references:

```
{
  "resourceType": "Bundle",
  "type": "searchset",
  "total": 1,
  "entry": [
    {
      "resource": {
        "resourceType": "HealthcareService",
        "id": "hcs-ped-neuro-01",
        "active": true,
        "providedBy": { "reference": "Organization/org-st-jude-01", "display": "St Jude Childrens Hospital" },
```

```
        "category": [ { "coding": [ { "code": "neurology", "display": "Pediatric
    Neurology" } ] } ],
        "location": [ { "reference": "Location/loc-st-jude-bld-b", "display": "
    Building B, Ward 4" } ],
        "endpoint": [ { "reference": "Endpoint/ep-secure-msg-09", "display": "
    Direct Secure Messaging REST" } ]
      }
    }
  ]
}
```

Listing 11.5: FHIR R5 Bundle Returned by BEFE

4. **UI Visualization**: The portal displays the active on-call physician (Dr. Robert Chen), the physical ward location, and a direct button to dispatch a clinical consultation communication.

11.7.2 Example 2: Operations Engineer Diagnosing Queue Backpressure and Workflow Stalls

Scenario: Systems Engineer Alex Rivera detects backpressure in the ActiveMQ Artemis queue `harmonia.mllp.inbound.v2` and uses the Operations Console to diagnose the root cause and identify stalled workflow executions.

1. **Queue Inspection**: Alex opens `iris-console` (`http://localhost:3001`), navigates to **Queues** (`/queues`), and inspects `harmonia.mllp.inbound.v2`. The queue detail drawer shows `depth: 412`, `consumerCount: 4`, and a historical depth sparkline indicating sustained growth over the past 15 minutes.
2. **Workflow Inspection**: Alex navigates to **Workflows** (`/workflows`) and selects the `seq-mllp-inbound-adt` workflow. The workflow summary shows `activeExecutions: 8`, `failedWork: 3`, and `P95 Duration: 2847ms`. Alex drills down into a failed Pragma instance and observes the failure reason code `ERR-FHIR-VALIDATION-FAILED` at the `FhirValidationErgon` checkpoint.
3. **Dependency Health Check**: Alex navigates to **Subsystems** (`/subsystems`), selects `Energeia`, and inspects the **Health & Dependencies** panel. The panel shows that the `Mnemosyne` dependency has a round-trip latency of 485ms (elevated from the baseline 14ms), indicating that FHIR validation is timing out due to downstream persistence latency.
4. **Alert Acknowledgement**: Alex navigates to **Alerts** (`/alerts`) and observes a `WARNING` alert: *"Energeia P95 latency elevated to 2847ms; Mnemosyne dependency latency 485ms."*. Alex clicks **Acknowledge** to mark the alert as reviewed and notes the operator guidance: *"Check Mnemosyne PostgreSQL connection pool saturation and consider scaling read replicas."*
5. **Remediation**: Alex contacts the database team to scale Mnemosyne read replicas. Within 5 minutes, the Mnemosyne dependency latency drops to 18ms, the queue depth begins draining, and the alert status transitions to `RESOLVED`.

11.7.3 Example 3: Practitioner Requesting Address Change and Admin Approval

Scenario: Dr. Marcus Vance moves his clinical practice to a new medical centre and submits a change request via the Self-Service Portal, which is subsequently approved by the directory administrator.

1. **Change Submission**: Dr. Vance logs into `iris-administration` (`http://localhost:3002`) as a self-service user. Under **Request Change**, he selects his `PractitionerRole/prr-vance-cardio`, enters the new clinic address (*450 Health Avenue, Suite 200*), and clicks **Submit Change Request**.
2. **Task Dispatch**: The SPA issues a `POST /api/v1/Task` request to BEFE:

```
{
  "resourceType": "Task",
  "status": "requested",
  "intent": "proposal",
  "code": { "coding": [ { "code": "update-practitioner-role" } ] },
  "focus": { "reference": "PractitionerRole/prr-vance-cardio" },
  "requester": { "reference": "Practitioner/prac-vance-01", "display": "Dr. Marcus
     Vance" },
  "input": [
    {
      "type": { "text": "Proposed Changes" },
      "valueString": "{\"address\": {\"line\": [\"450 Health Avenue, Suite 200\"],
     \"city\": \"Metro City\", \"postalCode\": \"3000\"}}"
```

```
      }
    ]
}
```

Listing 11.6: Governed Change Request Submission Payload

3. **Work Queue Triage**: Administrator Elena Rostova logs into `iris-administration` with `provider.admin` authority, opens the **Work Queue**, and selects Task `task-prr-change-8821`.
4. **Diff Review and Approval**: The portal displays the existing clinic address next to the proposed clinic address with highlighted diff indicators. Elena clicks **Approve and Commit**.
5. **Asynchronous Commit**: BEFE updates the Pragma state to `APPROVED`. Ponos commits the updated `PractitionerRole` resource to Mnemosyne PostgreSQL, incrementing its version from 2 to 3.
6. **Status Propagation**: Dr. Vance receives an approval alert in his **My Requests** view, and the new address is immediately searchable in `iris-clinical`.

Chapter 12

Provider Registry Core Services

Implementation and Functional Specification for FHIR R5 Provider Registry Core Services

12.1 Overview & Engineering Objectives

Within modern regional and national healthcare ecosystems, maintaining an authoritative, synchronized, and referentially sound healthcare provider directory is critical to patient safety, clinical routing, electronic service discovery, and secure messaging. The **Harmonia FHIR R5 Provider Registry** provides an enterprise-grade, distributed provider directory subsystem embedded natively within the Harmonia Health Information Exchange (HIE) platform.

This specification serves as the authoritative architectural blueprint and functional manual for the Provider Registry core services suite, detailing:

1. **Architectural Topology & 5-Tier Interaction Model**: The design separating synchronous read/search data paths from asynchronous governed change pipelines.
2. **FHIR R5 Resource Model & Relationship Topology**: First-class management and relationship graph traversal across seven core resources: `Practitioner`, `PractitionerRole`, `Organization`, `Location`, `HealthcareService`, `Endpoint`, and `Group`.
3. **REST API Gateway Contract & Interaction Matrix**: Exact HTTP verbs, request/response payloads, status codes, search parameters, dynamic `CapabilityStatement` generation (`GET /metadata`), and asynchronous status polling (`GET /Task/{id}`).
4. **Governed Change Processing Lifecycle & Pragma State Machine**: Event-driven write ingestion, state transitions (`RECEIVED` → `VALIDATING` → `APPROVED` → `COMMITTING` → `COMPLETED` or `REJECTED`), audit checkpoints, and structured `OperationOutcome` diagnostics.
5. **Per-Resource Ergon Processing Activities & Referential Integrity Engine**: The 7 dedicated per-resource Ergon activities, reference validation rules, duplicate identifier detection, and Apache Camel pipeline routing.
6. **Authoritative Persistence, Optimistic Concurrency & Versioning**: PostgreSQL relational document storage (`hie_fhir_resources`), monotonic version incrementing, and `If-Match`/`ETag` conflict protection.
7. **Security, RBAC & Resource-Level Authorization**: Granular permission scopes, token inspection, and security interceptor mechanics.
8. **Paradeigma Exemplar Generation & Automated Verification Testbed**: Synthetic graph generation, acceptance test suites, concurrency collision testing, and crash-recovery verification.

12.2 Architectural Topology & 5-Tier Interaction Model

Harmonia enforces a fundamental architectural dichotomy between **Synchronous Read/Search Pathways** and **Asynchronous Governed Write Pathways**:

Architectural Principle: The Governed Change Invariant

HTTP `POST` (create) and `PUT` (update) interactions on provider directory endpoints are treated strictly as **Requests for Change**, not immediate mutation instructions. Authoritative state is never mutated synchronously inside the HTTP request thread. All mutations must be verified, referentially checked, approved, and persisted asynchronously through the `Energeia` workflow engine.

12.2.1 Synchronous Read and Search Data Path

When an external clinical application, hospital EHR, or clinician user interface queries provider information (e.g. `GET /Practitioner/123` or `GET /PractitionerRole?organization=Org-1`):

1. **Edge Ingestion (`pylai-fhir-registry`)**: The request hits the `FhirRestGatewayController` on port `8089`.
2. **Authentication & Authorization**: The `FhirSecurityInterceptor` validates the bearer token/headers and delegates to `ThemisService` to evaluate `ProviderRegistryReadPolicy` (verifying caller has authority `provider.read` or role `PRV_RDR`).
3. **Direct Persistence Delegation**: The controller bypasses messaging queues and invokes `FhirStorageService` in `hestia/mnemosyne-clinical` directly.
4. **Relational Query Execution**: Indexed SQL lookups execute against `hie_fhir_resources`, returning the active FHIR JSON resource or search `Bundle` in sub-50 milliseconds with `HTTP 200 OK`.

12.2.2 Asynchronous Governed Change Path

When an external administration portal, credentialing service, or master file interface submits a modification (e.g. `POST /Practitioner` or `PUT /PractitionerRole/PRR-1`):

1. **Gateway Ingestion**: `FhirRestGatewayController` authenticates the caller and performs structural JSON parsing and schema validation.
2. **Pragma Envelope Creation**: `ChangeRequestSubmissionService` constructs a canonical `ProviderRegistryChang` with status `ACCEPTED`, recording the submitted resource, requester, source system, and `If-Match` version preconditions.
3. **Durable Queue Dispatch**: The gateway dispatches a `PetasosMessage` to Apache ActiveMQ Artemis queue `harmonia.provider.registry.change.request` and immediately returns `HTTP 202 Accepted` with header `Location: /Task/{pragmaId}`.
4. **Workflow Ingestion & Sequencing**: Ponos consumes the message and instantiates the Praxis sequence `seq-provider-registry-change-pipeline`.
5. **Ergon Verification**: The designated resource Ergon (e.g. `PractitionerRoleChangeErgon`) invokes `ProviderRegistryReferenceValidator` to verify that all referenced target entities exist and are active in Mnemosyne.
6. **Commit & Versioning**: Upon successful validation, the resource is committed to Mnemosyne PostgreSQL, incrementing its `version_id`. The underlying `Pragma` transitions to `COMPLETED`, and the updated resource becomes immediately visible to subsequent read/search requests.

12.3 FHIR R5 Provider Registry Resource Model & Relationship Topology

The Harmonia Provider Registry models the healthcare delivery enterprise across seven first-class FHIR R5 resources, adhering strictly to official HL7 FHIR Release 5 definitions:

12.3.1 Relationship Graph and Referential Topology

Figure 12.1 illustrates the interconnected relationship topology enforced across the seven directory resource types.

12.3.2 Endpoint as an Independent First-Class Entity

In accordance with architectural standards, `Endpoint` is modeled and persisted strictly as an independent FHIR resource rather than an embedded string property on PractitionerRole. This allows multiple organizations, facilities, and practitioner roles to share electronic endpoints, support certificate rotation, and manage technical communication parameters (e.g., encryption protocols, payload MIME types, TLS certificates) centrally without duplicating configuration across clinical records.

Resource Type	Core Identity	Domain Function & Structural Relationships
`Practitioner`	Individual	Individual healthcare practitioner (physician, surgeon, nurse, allied health). Holds legal names, professional qualifications, and deterministic identifiers (HPI-I, NPI, AHPRA).
`PractitionerRole`	Association Link	Core relationship binding a `Practitioner` to an `Organization`, operating at specific physical `Location`(s), providing specific `HealthcareService`(s), and reachable via technical `Endpoint`(s).
`Organization`	Legal Entity	Healthcare provider organization, hospital network, clinic, or diagnostic trust. Holds national identifiers (HPI-O), parent/child hierarchies, and administrative contacts.
`Location`	Physical Facility	Physical or virtual site of healthcare delivery (hospital ward, clinic suite, operating theatre). References managing `Organization` and communication `Endpoint`(s).
`HealthcareService`	Clinical Offering	Clinical service or specialty delivered by an organization (e.g. Inpatient Cardiology, Emergency Triage). References offering `Organization` and operating `Location`(s).
`Endpoint`	Electronic Channel	Independent technical communication destination (FHIR REST, secure messaging, MLLP, Direct). Holds connection type codings, address URLs, and managing `Organization`.
`Group`	Cohort / Specialty	Logical grouping of practitioners, organizations, or related healthcare entities for multidisciplinary teams, on-call rosters, or clinical specialties.

Table 12.1: FHIR R5 Provider Registry Managed Resource Model Catalog

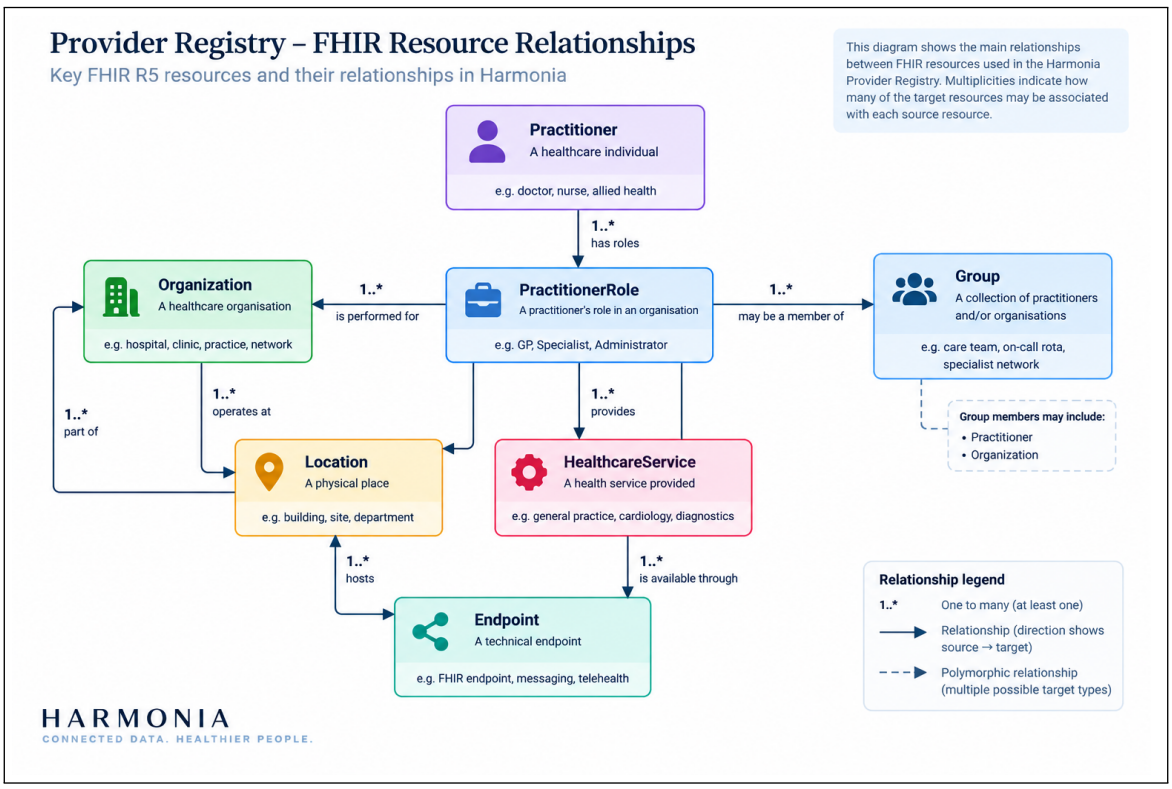


Figure 12.1: FHIR R5 Provider Registry Entity Relationship and Referential Dependency Graph

12.4 REST API Gateway Contract & Interaction Matrix

The `pylai-fhir-registry` gateway exposes standard FHIR R5 REST endpoints across all seven directory resources plus the `Task` and `metadata` operations:

Interaction	Verb	Path / URI	Mode	HTTP Response Codes
Read	`GET`	`/{Resource}/{id}`	Synchronous	`200 OK`, `404 Not Found`, `410 Gone`
Search	`GET`	`/{Resource}?{params}`	Synchronous	`200 OK` (`Bundle`)
Create Request	`POST`	`/{Resource}`	Asynchronous	`202 Accepted` (`Location: /Task/{id}`), `400 Bad Request`
Update Request	`PUT`	`/{Resource}/{id}`	Asynchronous	`202 Accepted` (`Location: /Task/{id}`), `412 Precondition Failed`
Task Polling	`GET`	`/Task/{id}`	Synchronous	`200 OK` (`Task`), `404 Not Found`
Metadata	`GET`	`/metadata`	Synchronous	`200 OK` (`CapabilityStatement`)

Table 12.2: FHIR R5 Provider Registry REST Interaction Matrix

12.4.1 Multi-Parameter Search Capabilities

The `FhirStorageService` in Mnemosyne executes multi-parameter search queries supporting standard FHIR search parameters:

- **`Practitioner`**: `_id`, `identifier` (HPI-I, NPI), `name` (family, given), `active`.
- **`PractitionerRole`**: `_id`, `identifier`, `practitioner` (reference), `organization` (reference), `location` (reference), `service` (reference), `active`.
- **`Organization`**: `_id`, `identifier` (HPI-O), `name`, `active`.
- **`Location`**: `_id`, `identifier`, `name`, `organization` (managing organization reference), `status`.
- **`HealthcareService`**: `_id`, `identifier`, `name`, `organization` (provided-by reference), `location` (operating site reference), `active`.
- **`Endpoint`**: `_id`, `identifier`, `name`, `organization` (managing organization reference), `status`, `connection-type`.
- **`Group`**: `_id`, `identifier`, `name`, `type`, `actual` (membership mode: `true`/`false`).

12.4.2 CapabilityStatement Specification (`GET /metadata`)

The `CapabilityStatementProvider` dynamically builds an official FHIR R5 `CapabilityStatement` describing:

- FHIR Release `5.0.0`.
- REST server mode with full resource descriptions for all 7 directory types plus `Task`.
- Advertised interactions (`read`, `search-type`, `create`, `update`).
- Exact supported search parameters per resource type.
- Security service configuration (`OAuth2` / `Bearer` token authentication).

12.4.3 Asynchronous Task Polling (`GET /Task/{id}`)

When a client polls `GET /Task/{id}`, the `ProviderRegistryTaskResourceProvider` maps internal `Pragma` state directly to standard FHIR `Task` elements:

- `Task.status`: Mapped from `PragmaStatus` (`requested` → `in-progress` → `completed` or `failed`).
- `Task.focus`: References the requested target resource (e.g. `PractitionerRole/PRR-101`).
- `Task.output`: Upon successful completion, embeds a reference to the persisted resource and resulting version (e.g. `Practitioner/P-100/_history/2`). On failure, embeds the diagnostic `OperationOutcome`.

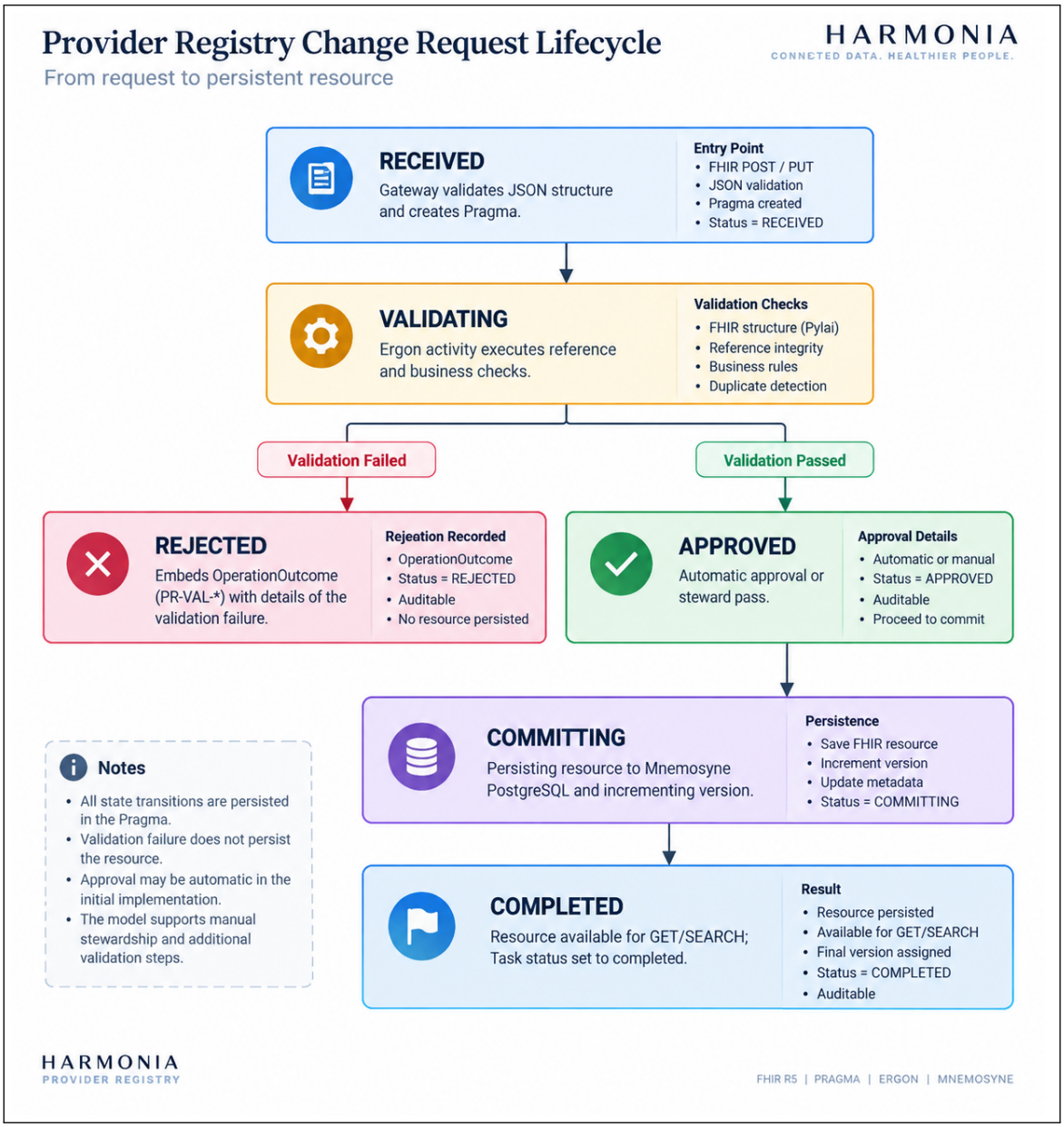


Figure 12.2: Provider Registry Change Request State Progression

12.5 Governed Change Processing Lifecycle & Pragma State Machine

Every asynchronous change request progresses through a deterministic persistent state machine tracked across `PragmaCheckpoint` records in the `Mneme` distributed cache and `Mnemosyne` relational database:

12.5.1 State Machine Transition Rules

`RECEIVED` → `VALIDATING` : Triggered when the Ponos WorkEngine consumes the change task from ActiveMQ Artemis and routes to the dedicated resource Ergon activity.

`VALIDATING` → `REJECTED` : Fired when the submitted resource references non-existent or inactive resources, contains duplicate deterministic identifiers, or fails business schema rules. An `OperationOutcome` is generated and attached to the task output.

`VALIDATING` → `APPROVED` : In the first iteration, change requests that pass all structural, referential, and uniqueness checks are automatically approved. The architecture includes extension hooks for human steward approval workflows.

`APPROVED` → `COMMITTING` : The Ergon persists the approved resource into `hie_fhir_resources`, incre-

menting the `version_id` and stamping `last_updated`.

`COMMITTING` → `COMPLETED` : Terminal success milestone. Checkpoints are synchronized, and the task output references the newly persisted resource version.

12.6 Per-Resource Ergon Processing Activities & Referential Integrity Engine

Harmonia implements dedicated, strongly-typed Ergon activities extending `ErgonBase` for each resource type:

Ergon Activity Class	Target Resource	Specialized Validation & Verification Responsibilities
`PractitionerChangeErgon`	`Practitioner`	Validates legal human name, gender, qualifications, and checks duplicate deterministic HPI-I / NPI identifiers in Mnemosyne.
`PractitionerRoleChangeErgon`	`PractitionerRole`	Performs deep referential verification ensuring target `Practitioner`, `Organization`, `Location`(s), `HealthcareService`(s), and `Endpoint`(s) exist and are active.
`OrganizationChangeErgon`	`Organization`	Validates organization name, active status, national identifier (HPI-O), and validates parent organization hierarchy references.
`LocationChangeErgon`	`Location`	Validates facility coordinates, addresses, operational status, and verifies managing organization reference existence.
`HealthcareServiceChangeErgon`	`HealthcareService`	Validates specialty codes, active status, and verifies provided-by organization and location references.
`EndpointChangeErgon`	`Endpoint`	Validates connection type coding, address URL structure, supported payload MIME types, and managing organization reference.
`GroupChangeErgon`	`Group`	Validates group membership type (definitional vs actual), member count, and verifies individual member entity references.

Table 12.3: Provider Registry Per-Resource Ergon Activity Responsibilities

12.6.1 `ProviderRegistryReferenceValidator` Engine

The `ProviderRegistryReferenceValidator` service in `mnemosyne-clinical` executes referential integrity inspections against `hie_fhir_resources` before approving changes:

```
1  public class ProviderRegistryReferenceValidator {
2      public ValidationResult validatePractitionerRoleReferences(PractitionerRole role)
       {
3          // 1. Validate mandatory Practitioner reference
4          if (role.hasPractitioner()) {
5              verifyTargetExistsAndActive("Practitioner", role.getPractitioner().
       getReference());
6          }
7          // 2. Validate mandatory Organization reference
8          if (role.hasOrganization()) {
9              verifyTargetExistsAndActive("Organization", role.getOrganization().
       getReference());
10         }
11         // 3. Validate Location references
12         for (Reference locRef : role.getLocation()) {
13             verifyTargetExistsAndActive("Location", locRef.getReference());
14         }
15         // 4. Validate HealthcareService references
16         for (Reference svcRef : role.getHealthcareService()) {
```

```
17                verifyTargetExistsAndActive("HealthcareService", svcRef.getReference());
18            }
19            // 5. Validate Endpoint references
20            for (Reference epRef : role.getEndpoint()) {
21                verifyTargetExistsAndActive("Endpoint", epRef.getReference());
22            }
23            return ValidationResult.valid();
24        }
25    }
```

Listing 12.1: Reference Validation Contract in ProviderRegistryReferenceValidator

If any referenced resource does not exist in Mnemosyne storage or is soft-deleted (`is_deleted = true`), the validator returns a structured rejection result with issue code `PR-VAL-004` (*Reference Not Found*) or `PR-VAL-005` (*Reference Inactive*), halting the pipeline before mutation.

12.7 Authoritative Persistence, Optimistic Concurrency & Versioning Model

Authoritative directory state is persisted in PostgreSQL table `hie_fhir_resources` (detailed in Section 14.6).

12.7.1 Monotonic Version Incrementing

When a resource is first created via `createResource()`, it is assigned:

$$\texttt{version_id} = 1, \quad \texttt{Meta.versionId} = \text{"1"}, \quad \texttt{Meta.lastUpdated} = \text{now}() \tag{12.1}$$

Subsequent approved updates executed via `updateResource()` increment the version monotonically ($v_{n+1} = v_n + 1$) and update the `last_updated` timestamp atomically.

12.7.2 Optimistic Concurrency Control (`If-Match` / `ETag`)

To prevent lost updates in concurrent multi-client environments, the Provider Registry supports optimistic concurrency locking:

1. When a client reads a resource, the gateway returns the current version in the HTTP header:

   `ETag: W/"3"`

2. When submitting a `PUT` update, the client supplies the precondition header:

   `If-Match: W/"3"`

3. `FhirStorageService` checks whether `entity.getVersionId() == expectedVersion`.
4. If the database version has advanced (e.g. version 4 exists due to an intervening update), the update is rejected with `HTTP 412 Precondition Failed` and diagnostic code `PR-VAL-006`.

12.8 Security, Themis Governance & Multi-Tier Authorization

The Provider Registry operates under the centralized governance of **Themis**, enforcing strict separation between identity verification (Authentication at Pylai) and deterministic capability evaluation (Authorization in Themis).

12.8.1 Mnemonic Roles and Granular Authority Matrix

Callers are assigned mnemonic roles (`HarmoniaRoleEnum`) which Themis policies map to granular authorities (`HarmoniaAuthorityEnum`):

12.8.2 `FhirSecurityInterceptor` & Defence-in-Depth Pipeline

The `FhirSecurityInterceptor` and downstream processing gates evaluate security deterministically:

1. Extracts caller credentials (`Authorization: Bearer <token>`, `X-User-Roles`, `X-Security-Scopes`) and derives `ThemisPrincipal`.

Role Code	Assigned Authorities	Allowed Operations & Target Healthcare Actor
PRV_RDR	provider.read, provider.search	Read and search individual directory resources; clinical clients, EMRs.
PRV_SUB	provider.change.submit	Submit asynchronous change requests (POST/PUT); medical registrars.
PRV_PROC	provider.change.process, provider.resource.*	Background Ergon task execution and persistence commit in Ponos.
PRV_APR	provider.change.approve, provider.read	Governance review and clinical change authorization.
PRV_ADM	provider.admin, provider.*	Full directory governance and administrative configuration.

Table 12.4: Provider Registry Mnemonic Role and Granular Authority Matrix

2. Invokes `ThemisService.authorize()` to evaluate default-deny policies (`ProviderRegistryReadPolicy` for `GET`, `ProviderRegistrySubmitPolicy` for `POST`/`PUT`).
3. Attaches immutable caller context to `ProviderRegistryChangePragma` and serializes claims into FHIR R5 Task extensions.
4. Ponos dispatch gate and Ergon persistence gates re-verify authorities independently, preventing privilege escalation.
5. Rejects unauthorized interactions immediately with `HTTP 403 Forbidden` (`ForbiddenOperationException`) and logs non-PHI audit records via `ThemisAuditService`.

12.9 Paradeigma Exemplar Generation & Verification Testbed

To guarantee end-to-end reliability and standard compliance, the **Paradeigma** simulation framework provides automated synthetic data generators and comprehensive test suites:

12.9.1 `SyntheticProviderRegistryGenerator`

Generates realistic, fully-interconnected healthcare provider graphs adhering to Australian and international directory standards:

- **Practitioners**: Synthetic physicians, specialists, and nurses with valid Australian HPI-I identifiers (prefix `800361...`), names, and specialties.
- **Organizations**: Hospital networks, community health clinics, and diagnostic trusts with valid HPI-O identifiers (prefix `800362...`).
- **Locations**: Acute care wards, outpatient suites, and pathology collection centres with physical addresses.
- **HealthcareServices**: Emergency, Cardiology, Radiology, and General Practice offerings.
- **Endpoints**: Secure FHIR REST, HL7 v2 MLLP, and Secure Messaging electronic endpoints.
- **PractitionerRoles**: Connected relationship graphs linking practitioners to their respective organizations, locations, services, and endpoints.

12.9.2 Automated Integration & Verification Test Suites

The platform includes four dedicated automated test suites in `paradeigma-test`:

1. `ProviderRegistryEndToEndWriteTest`: Validates the complete asynchronous change lifecycle: submitting `POST` requests for all 7 resources, receiving `202 Accepted`, polling `GET /Task/{id}` until `COMPLETED`, and executing subsequent `GET` read and search queries to confirm persisted state.
2. `ProviderRegistryReferentialRejectionTest`: Submits a `PractitionerRole` referencing a non-existent `Practitioner` ID, asserting that the request transitions to `REJECTED` with error code `PR-VAL-004` and that no orphan record is persisted.
3. `ProviderRegistryConcurrencyTest`: Simulates concurrent updates on the same resource, asserting that stale `If-Match` preconditions fail with `PR-VAL-006` concurrency conflict.
4. `ProviderRegistryRestartRecoveryTest`: Injects broker restarts during active in-flight change

processing, verifying that ActiveMQ Artemis and Ponos resume tasks safely without data loss or duplicate resource generation.

12.10 Summary & Engineering Checklist

Table 12.5 summarizes the key engineering invariants and operational checks for the Harmonia FHIR R5 Provider Registry.

Domain	Architectural Invariant	Verification Criteria
Gateway	Governed Write Dichotomy	Ensure `POST/PUT` returns `202 Accepted` with `Location: /Task/{id}`; direct synchronous mutation is strictly prohibited.
Discovery	Dynamic `CapabilityStatement`	Validate that `GET /metadata` accurately reflects all 7 resources, search parameters, and security schemes.
Validation	Deep Referential Integrity	Confirm `ProviderRegistryReferenceValidator` validates all target entity references before approving changes.
Storage	Hybrid Relational Document	Verify `hie_fhir_resources` indexes `resource_type`, `fhir_id`, `version_id`, and `is_deleted`.
Concurrency	Optimistic Locking	Enforce `If-Match` version validation on all `PUT` updates.
Resilience	Artemis Queue Durability	Verify change requests persist in `harmonia.provider.registry.change.request` across container restarts.

Table 12.5: Production Readiness Checklist for FHIR R5 Provider Registry Core Services

Part III

Information & Integration

Chapter 13

Information Architecture & Standard Message Envelopes

The Information Architecture layer defines the canonical domain models, the task state lifecycle, message transport envelopes, clinical mapping dictionaries, and the supported HL7 FHIR Release 5 (R5) resource schemas.

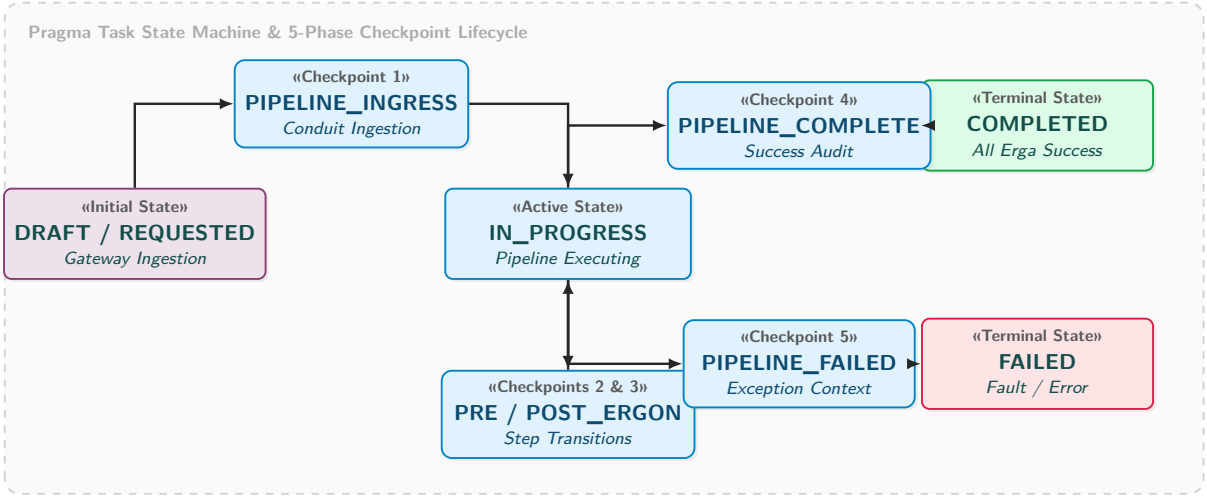


Figure 13.1: Canonical `Pragma` State Machine and Lifecycle Progression

13.1 Canonical Domain Models (`Calliope`)

Harmonia avoids coupling workflow logic to proprietary or concrete wire formats by establishing strongly-typed canonical domain entities in the `Calliope` library.

13.1.1 1. `Pragma` (Task State Entity)

`Pragma` represents the single authoritative state representation for an execution task:

```
public class Pragma implements Serializable {
    private String pragmaId;          // Unique UUID
    private String correlationId;     // End-to-end transaction thread
    private String causationId;       // Triggering event or message ID
    private String praxisId;          // Workflow sequence definition ID
    private PragmaStatus status;      // DRAFT, REQUESTED, IN_PROGRESS, COMPLETED,
    FAILED
    private Integer priority;         // 1 (Lowest) - 100 (Highest)
    private Date authoredOn;
    private Date lastModified;
    private String source;            // Originating gateway / interface
    private String destination;       // Target queue or service address
    private List<ErgonPayload> input = new ArrayList<>();
    private List<ErgonPayload> output = new ArrayList<>();
    private List<PragmaCheckpoint> checkpoints = new ArrayList<>();
    private Map<String, String> metadata = new LinkedHashMap<>();
}
```

Listing 13.1: Canonical Pragma Model Definition in Calliope

13.1.2 2. `PragmaCheckpoint` (Execution Audit Snapshot)

Captures an immutable audit milestone recorded at pipeline transitions:

```
public class PragmaCheckpoint implements Serializable {
    private String checkpointId;
    private String pragmaId;
    private String ergonId;          // Active Ergon activity
    private String stageName;        // PIPELINE_INGRESS, PRE_ERGON, POST_ERGON, etc.
    private Date timestamp;
    private PragmaStatus status;
    private String statusMessage;
    private Integer stepIndex;       // 0-based sequence step
}
```

Listing 13.2: PragmaCheckpoint Model Definition

13.1.3 3. `ProviderRegistryChangePragma` (Directory Change Wrapper)

Wraps and manages asynchronous change requests submitted via FHIR Provider Registry `POST`/`PUT` endpoints:

```
public class ProviderRegistryChangePragma implements Serializable {
    private final Pragma pragma;

    public String getPragmaId();
    public String getCorrelationId();
    public String getOperation();           // CREATE or UPDATE
    public String getResourceType();        // Practitioner, Organization, etc.
    public String getResourceId();          // Target FHIR logical ID
    public String getRequester();           // Submitting user / principal
    public String getSourceSystem();        // Source system identifier
    public String getIfMatch();             // Expected ETag / version header
    public String getValidationOutcome();   // Validation status / diagnostics
    public String getResultingResourceVersion();
    public Resource getRequestedResource(); // Parsed FHIR R5 resource payload
}
```

Listing 13.3: ProviderRegistryChangePragma Model Definition

13.1.4 4. Canonical Provider Registry Validation Codes

Harmonia standardizes error diagnostic reporting using canonical issue codes embedded within FHIR `OperationOutcome` responses:

Error Code	Issue Category	Architectural Description & Diagnostic Condition
`PR-VAL-001`	Structural Error	Malformed FHIR JSON, unparseable attributes, or schema violation.
`PR-VAL-002`	Missing Field	Required mandatory directory fields missing (e.g. Practitioner name).
`PR-VAL-003`	Duplicate Identifier	Duplicate deterministic provider identifier (HPI-I, HPI-O, NPI).
`PR-VAL-004`	Reference Not Found	Referenced target resource does not exist in Mnemosyne storage.
`PR-VAL-005`	Reference Inactive	Referenced target resource is marked inactive or soft-deleted.
`PR-VAL-006`	Concurrency Conflict	Stale update rejected due to `If-Match` / `ETag` version mismatch.
`PR-VAL-007`	Invalid Endpoint	Malformed technical endpoint URL, invalid payload type, or status.
`PR-VAL-008`	Invalid Group	Invalid group membership type or unresolved member references.
`PR-VAL-009`	Unsupported Resource	Submitted resource type is not supported by Provider Registry.
`PR-VAL-010`	Business Rule Failure	Custom business validation failure or governance policy violation.

Table 13.1: Canonical Provider Registry Validation and Error Diagnostic Codes

13.2 Canonical Outbound Models (`pylai-mllp-base`)

The Pylai gateway tier standardizes outbound MLLP interactions using canonical data structures:

```
public class OutboundMllpRequest implements Serializable {
    private String requestId;          // Unique dispatch UUID
    private String messageControlId;   // HL7 MSH-10 control identifier
```

```
    private String destinationId;       // Target endpoint identifier (e.g. HIS_NORTH)
    private String rawMessage;          // Raw HL7 v2 pipe-delimited message string
    private String targetQueue;         // Destination Petasos queue
    private Topic topic;                // Resolved clinical topic metadata
    private Date timestamp;             // Enqueue timestamp
}
```

Listing 13.4: OutboundMllpRequest Model Definition

13.3 Petasos Messaging Transport Envelopes (`petasos-api`)

Messages flowing through the messaging backbone are wrapped in a generic, opaque transport envelope:

```
public class PetasosMessage implements Serializable {
    private String messageId;           // Globally unique message UUID
    private String correlationId;       // Transaction correlation identifier
    private String topic;               // Logical clinical topic
    private String payloadType;         // MIME type (e.g. application/fhir+json,
    application/hl7-v2)
    private byte[] payload;             // Opaque binary or UTF-8 text payload
    private Map<String, String> headers = new HashMap<>();
    private long creationTimestamp;
}
```

Listing 13.5: PetasosMessage Envelope Definition

13.4 HL7 FHIR Release 5 (R5) Resource Schemas

Harmonia maps all clinical domain entities to 15 standardized HL7 FHIR Release 5 resource schemas:

1. **`Patient`**: Care recipient demographics, identifiers, and link registries.
2. **`Encounter`**: Inpatient admissions, outpatient visits, and emergency encounters.
3. **`Practitioner`**: Clinical practitioner records, qualifications, and credentials.
4. **`PractitionerRole`**: Practitioner appointments, roles, and facility assignments.
5. **`Organization`**: Health systems, hospitals, departments, and external laboratories.
6. **`Location`**: Physical wards, operating rooms, clinics, and beds.
7. **`HealthcareService`**: Clinical services offered at locations.
8. **`Endpoint`**: Technical connection details (MLLP host/port, FHIR base URL).
9. **`Group`**: Care teams and practitioner groups.
10. **`Observation`**: Laboratory results, vital signs, and diagnostic findings.
11. **`DiagnosticReport`**: Multi-component diagnostic summaries and radiology reports.
12. **`DocumentReference`**: Clinical notes, discharge summaries, and pathology attachments.
13. **`Task`**: Execution task representation mapped directly from `Pragma`.
14. **`AuditEvent`**: Zero-PHI audit trail capturing access, creation, and modification events.
15. **`Provenance`**: Data lineage, source system tracking, and digital signatures.

Chapter 14

Data & Information Architecture

The Data and Information Architecture layer defines the canonical information models, the task state lifecycle, message transport envelopes, clinical mapping dictionaries, and the 14 supported HL7 FHIR Release 5 (R5) resource schemas.

14.1 Canonical Domain Models (`Calliope`)

Harmonia avoids coupling workflow logic to proprietary or concrete wire formats by establishing strongly-typed canonical domain entities in the `Calliope` library.

14.1.1 1. `Pragma` (Task State Entity)

`Pragma` represents the single authoritative state representation for an execution task:

```
public class Pragma implements Serializable {
    private String pragmaId;          // Unique UUID
    private String correlationId;     // End-to-end transaction thread
    private String causationId;       // Triggering event or message ID
    private String praxisId;          // Workflow sequence definition ID
    private PragmaStatus status;      // DRAFT, REQUESTED, IN_PROGRESS, COMPLETED,
    FAILED
    private Integer priority;         // 1 (Lowest) - 100 (Highest)
    private Date authoredOn;
    private Date lastModified;
    private String source;            // Originating gateway / interface
    private String destination;       // Target queue or service address
    private List<ErgonPayload> input = new ArrayList<>();
    private List<ErgonPayload> output = new ArrayList<>();
    private List<PragmaCheckpoint> checkpoints = new ArrayList<>();
    private Map<String, String> metadata = new LinkedHashMap<>();
}
```

Listing 14.1: Canonical Pragma Model Definition in Calliope

14.1.2 2. `PragmaCheckpoint` (Execution Audit Snapshot)

Captures an immutable audit milestone recorded at pipeline transitions:

```
public class PragmaCheckpoint implements Serializable {
    private String checkpointId;
    private String pragmaId;
    private String ergonId;           // Active Ergon activity
    private String stageName;         // PIPELINE_INGRESS, PRE_ERGON, POST_ERGON, etc.
    private Date timestamp;
    private PragmaStatus status;
    private String statusMessage;
    private Integer stepIndex;        // 0-based sequence step
}
```

Listing 14.2: PragmaCheckpoint Model Definition

14.1.3 3. `ProviderRegistryChangePragma` (Directory Change Wrapper)

Wraps and manages asynchronous change requests submitted via FHIR Provider Registry `POST`/`PUT` endpoints:

```
public class ProviderRegistryChangePragma implements Serializable {
    private final Pragma pragma;

```

```
    public String getPragmaId();
    public String getCorrelationId();
    public String getOperation();           // CREATE or UPDATE
    public String getResourceType();        // Practitioner, Organization, etc.
    public String getResourceId();          // Target FHIR logical ID
    public String getRequester();           // Submitting user / principal
    public String getSourceSystem();        // Source system identifier
    public String getIfMatch();             // Expected ETag / version header
    public String getValidationOutcome();   // Validation status / diagnostics
    public String getResultingResourceVersion();
    public Resource getRequestedResource(); // Parsed FHIR R5 resource payload
}
```

Listing 14.3: ProviderRegistryChangePragma Model Definition

14.1.4 4. Canonical Provider Registry Validation Codes

Harmonia standardizes error diagnostic reporting using canonical issue codes embedded within FHIR `OperationOutcome` responses:

Error Code	Issue Category	Architectural Description & Diagnostic Condition
`PR-VAL-001`	Structural Error	Malformed FHIR JSON, unparseable attributes, or schema violation.
`PR-VAL-002`	Missing Field	Required mandatory directory fields missing (e.g. Practitioner name).
`PR-VAL-003`	Duplicate Identifier	Duplicate deterministic provider identifier (HPI-I, HPI-O, NPI).
`PR-VAL-004`	Reference Not Found	Referenced target resource does not exist in Mnemosyne storage.
`PR-VAL-005`	Reference Inactive	Referenced target resource is marked inactive or soft-deleted.
`PR-VAL-006`	Concurrency Conflict	Stale update rejected due to `If-Match` / `ETag` version mismatch.
`PR-VAL-007`	Invalid Endpoint	Malformed technical endpoint URL, invalid payload type, or status.
`PR-VAL-008`	Invalid Group	Invalid group membership type or unresolved member references.
`PR-VAL-009`	Unsupported Resource	Submitted resource type is not supported by Provider Registry.
`PR-VAL-010`	Business Rule Failure	Custom business validation failure or governance policy violation.

Table 14.1: Canonical Provider Registry Validation and Error Diagnostic Codes

14.2 Canonical Outbound Models (`pylai-mllp-base`)

The Pylai gateway tier standardizes outbound MLLP interactions using canonical data structures:

14.2.1 1. `OutboundMllpRequest`

Encapsulates an outbound MLLP dispatch payload with destination routing and clinical metadata:

```
public class OutboundMllpRequest implements Serializable {
    private String requestId;          // Unique dispatch UUID
    private String messageControlId;   // HL7 MSH-10 control identifier
    private String destinationId;      // Target endpoint identifier (e.g. HIS_NORTH)
    private String rawMessage;         // Raw HL7 v2 pipe-delimited message string
    private String targetQueue;        // Destination Petasos queue
    private Topic topic;               // Resolved clinical topic metadata
    private Date timestamp;            // Enqueue timestamp
    private int priority = 4;          // JMS priority
    private String taskId;             // Associated Pragma / Task resource ID
    private String facility;           // Clinical facility code
    private String correlationId;      // End-to-end trace identifier
}
```

Listing 14.4: OutboundMllpRequest Model Definition

14.2.2 2. `OutboundMllpResponse`

Encapsulates the synchronous result and acknowledgment returned from the external clinical endpoint:

```
public class OutboundMllpResponse implements Serializable {
    private String requestId;
    private String messageControlId;
    private String destinationId;
```

```
    private String ackCode;              // AA, AE, AR, CA, CE, CR
    private String ackText;              // Error description or text
    private String rawAckMessage;        // Raw HL7 ACK wire frame
    private boolean successful;          // True if AA or CA
    private long durationMs;             // Round-trip transmission latency
    private Date dispatchedAt;
    private Date acknowledgedAt;
    private String errorMessage;
    private String taskId;
    private String communicationId;      // Persisted FHIR Communication ID
    private String provenanceId;         // Persisted FHIR Provenance ID
}
```

Listing 14.5: OutboundMllpResponse Model Definition

14.2.3 3. `MllpDestinationConfig`

Defines connectivity and timeout parameters for remote clinical MLLP servers:

```
public class MllpDestinationConfig implements Serializable {
    private String destinationId;        // Unique target key (e.g. LIS_MAIN)
    private String name;                 // Human-readable destination label
    private String host;                 // Target IP or hostname
    private int port;                    // Target TCP port
    private int connectTimeoutMs = 5000;
    private int readTimeoutMs = 10000;
    private String charset = "UTF-8";
    private boolean keepAlive = true;
    private boolean autoAck = true;
    private String queueName;            // Dedicated Petasos queue override
    private String queuePrefix = "petasos.queue.mllp.outbound";
    private boolean sslEnabled = false;
    private boolean enabled = true;
    private String facility;
}
```

Listing 14.6: MllpDestinationConfig Model Definition

14.3 The 5-Phase Checkpoint Lifecycle State Flow

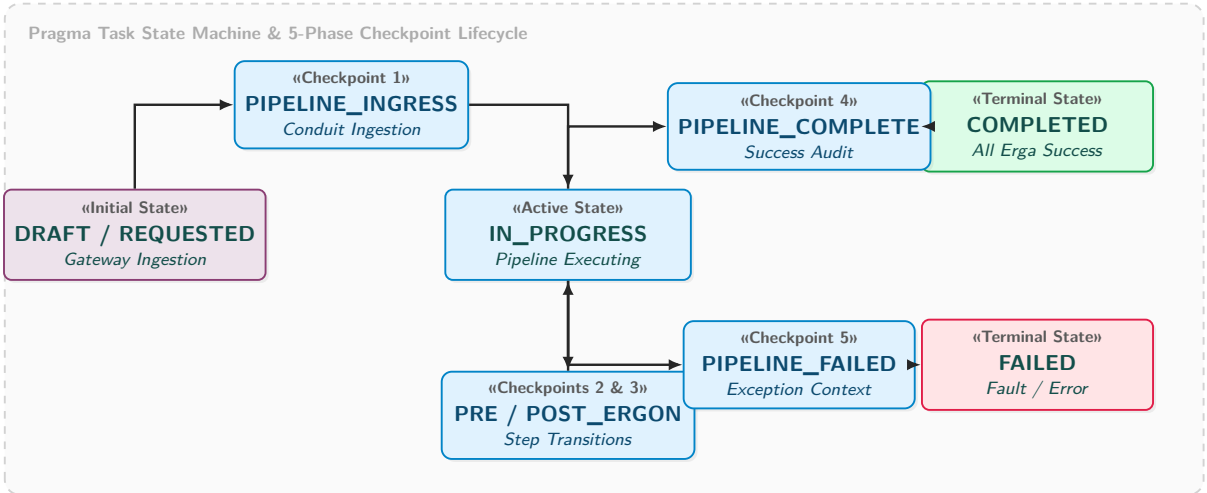


Figure 14.1: ArchiMate Pragma Task State Machine and 5-Phase Checkpoint Lifecycle

The `PraxisCheckpointManager` stamps and persists snapshots across five deterministic boundaries:

1. `PIPELINE_INGRESS`: Fired when `PetasosQueueToExchangeConduit` ingests a task from Artemis. Sets status to `IN_PROGRESS` and step to `0`.
2. `PRE_ERGON`: Fired immediately prior to invoking an individual `ErgonBase` activity, capturing input payload counts.

3. `POST_ERGON`: Fired upon successful return from an Ergon step, recording generated output counts and execution duration.
4. `PIPELINE_COMPLETE`: Fired when the entire Praxis sequence finishes successfully. Sets terminal status to `COMPLETED`.
5. `PIPELINE_FAILED`: Fired upon any uncaught exception or `ErgonException`. Records faulting step, exception class, and stack trace, setting terminal status to `FAILED`.

14.4 `PetasosMessage` Transport Envelope

All asynchronous transport operations across ActiveMQ Artemis use the standard immutable `PetasosMessage` envelope:

Field	Data Type	Transport Semantic & Function
`messageId`	`String`	Globally unique UUID.
`correlationId`	`String`	Tracks conversational thread or clinical episode.
`causationId`	`String`	Identifies immediate predecessor event.
`messageType`	`String`	Logical event type (e.g. `PatientAdmitEvent`).
`source` / `destination`	`String` / `Destination`	Originating module and target queue address.
`timestamp`	`Instant`	Creation timestamp.
`contentType`	`String`	MIME type (`application/json`, `application/fhir+json`).
`schemaIdentifier`	`String`	Canonical StructureDefinition URI.
`schemaVersion`	`String`	Model version (`5.0.0`, `R5`).
`payload`	`byte[]`	Opaque serialized binary/text payload (uninspected by transport).
`duplicateDetectionId`	`String`	Mapped to Artemis `_AMQ_DUPL_ID` for broker deduplication.
`durable`	`boolean`	Directs Artemis to write to persistent journal before commit.

Table 14.2: Petasos Standard Message Envelope Attribute Specification

14.5 Supported FHIR R5 Resource Schemas

Harmonia implements full lifecycle create, read, update, domain lifecycle state transitions, and search capabilities for 15 core FHIR R5 resources (ADR-020):

14.6 Relational Persistence Schema (`Mnemosyne-Clinical`)

Authoritative clinical and directory resources are persisted in PostgreSQL using a high-performance **Hybrid Relational-Document Pattern** in table `hie_fhir_resources`:

```
CREATE TABLE hie_fhir_resources (
    id            BIGSERIAL PRIMARY KEY,
    resource_type VARCHAR(64) NOT NULL,
    fhir_id       VARCHAR(128) NOT NULL,
    version_id    BIGINT NOT NULL DEFAULT 1,
    resource_json TEXT NOT NULL,
    is_deleted    BOOLEAN NOT NULL DEFAULT FALSE,
    last_updated  TIMESTAMP WITHOUT TIME ZONE NOT NULL DEFAULT NOW(),
    CONSTRAINT uk_resource_type_fhir_id UNIQUE (resource_type, fhir_id)
);

CREATE INDEX idx_resource_type_fhir_id ON hie_fhir_resources(resource_type, fhir_id);
CREATE INDEX idx_resource_type_deleted ON hie_fhir_resources(resource_type, is_deleted
    );
```

Listing 14.7: PostgreSQL Relational Schema for hie_fhir_resources

This hybrid storage strategy guarantees:

- **Lossless FHIR R5 Fidelity**: The complete resource structure, including complex extensions and contained resources, is stored in `resource_json`.

FHIR R5 Resource	Domain Classification	Harmonia Platform Role
`Patient`	Clinical Individual	Primary care recipient entity with consolidated identifiers.
`Person`	Demographic Master	Cross-facility master demographic identity.
`RelatedPerson`	Family / Carer	Emergency contacts, next-of-kin, and legal guardians.
`Practitioner`	Healthcare Provider	Medical professionals, physicians, and clinical staff.
`PractitionerRole`	Operational Role	Specialty, location, and organizational assignments for practitioners.
`Organization`	Facility / Authority	Hospitals, clinics, labs, and health network authorities.
`Location`	Physical Facility	Wards, rooms, clinics, and physical service delivery sites.
`HealthcareService`	Clinical Offering	Medical specialties and care services offered at locations.
`Endpoint`	Electronic Service	Secure messaging, FHIR REST, and MLLP communication endpoints.
`Group`	Cohort Entity	Patient cohorts, provider directories, and specialized care teams.
`Task`	Workflow Entity	Direct bidirectional projection of `Pragma` state instances and outbound transmission execution tracking.
`Communication`	Exchange Audit	Record of incoming and outgoing MLLP message transmissions, wire payloads, and ACK/NACK responses.
`DocumentReference`	Clinical Document	Metadata indexing for external clinical notes, CDA, and PDFs.
`Provenance`	Lineage Tracking	Source and destination attribution, creator identities, and transformation/egress delivery history.
`AuditEvent`	Security / Access Audit	Detailed audit records capturing all read, write, and failover events.

Table 14.3: Harmonia Supported FHIR R5 Resource Schema Catalog

- **High-Speed Indexed Search**: Hot lookup fields (`resource_type`, `fhir_id`, `is_deleted`) are indexed at the relational engine level.
- **Optimistic Concurrency & Versioning**: Each update atomically increments `version_id` and stamps `last_updated`, supporting `If-Match` / `ETag` precondition validation.
- **Audit-Preserving Lifecycle State (ADR-020)**: Governed persisted information is never physically deleted. Deactivated or retired resources maintain their historical records and Kleio audit evidence with `is_deleted = TRUE` via authoritative `UPDATE` operations.

Chapter 15

Implementation Guide for MLLP Ingress and Egress Services

15.1 Overview & Engineering Objectives

This implementation guide provides an exhaustive, production-grade technical manual detailing the architectural patterns, component blueprints, configuration parameters, state persistence workflows, and testing harnesses required to implement:

1. **A New MLLP Ingress Service**: Ingesting inbound HL7 v2 messages over Minimal Lower Layer Protocol (MLLP) TCP sockets, validating syntax, constructing canonical FHIR R5 `Communication` and `Task` resources, persisting records into the `Mneme` Infinispan cache grid and `Mnemosyne` PostgreSQL database, publishing asynchronous `ErgonEvent` notifications to the `Petasos` (ActiveMQ Artemis) message bus, and generating synchronous HL7 acknowledgements (`AA`, `AE`, `AR`).
2. **A New MLLP Egress Service**: Consuming outbound clinical dispatch tasks from dedicated `Petasos` destination queues, transforming canonical models into HL7 v2 payloads, dispatching over Apache Camel MLLP client routes with robust socket timeouts, deeply inspecting downstream HL7 `ACK`/`NACK` responses, managing full FHIR `Communication`, `Task`, and `Provenance` lifecycle state transitions in `Mneme`/`Mnemosyne`, and isolating destination failure domains through multi-instance container topologies.

15.2 Ingress Architectural Blueprint & Framing Protocols

The MLLP Ingress Gateway (`pylai-mllp-in`) acts as the secure, high-throughput perimeter gateway of Harmonia, terminating inbound TCP connections from external Patient Administration Systems (PAS), Laboratory Information Systems (LIS), Radiology Information Systems (RIS), and Electronic Medical Records (EMR).

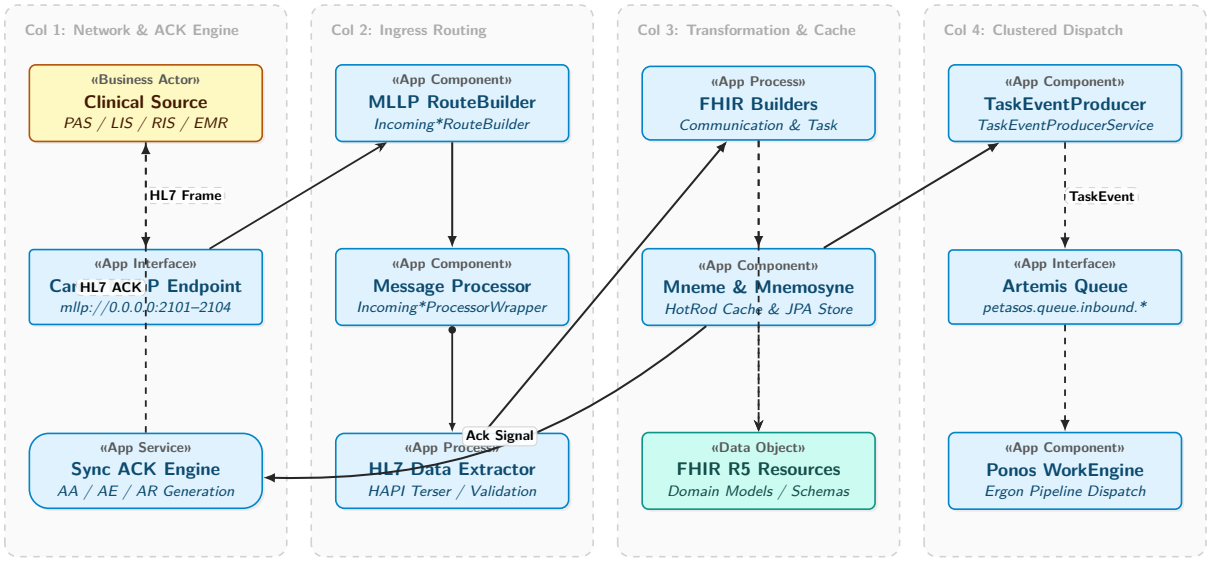


Figure 15.1: ArchiMate 3.2 Component Flow for the MLLP Ingress Service Processing Pipeline

15.2.1 MLLP Transport Framing

MLLP provides minimal framing over stream-oriented TCP sockets to delineate discrete message boundaries. All inbound MLLP ingress routes adhere to the standard framing specification:

⟨Inbound Frame⟩ ::= 0x0B (Start-of-Block / <VT>)+⟨HL7 v2.x Payload⟩+0x1C (End-of-Block / <FS>)+0x0D (Carriage Re (15.1)

Inbound frames are received over persistent or transient TCP sockets. Camel's Netty-based MLLP component buffers incoming bytes until the End-of-Block trailer (0x1C 0x0D) is detected, strips framing bytes, and passes the raw HL7 string to the downstream pipeline.

15.2.2 The 8-Phase Ingress Processing Lifecycle

Every inbound message traverses an immutable 8-phase execution pipeline:

1. **Socket Connection & TCP Frame Capture**: The client establishes a TCP connection to the configured port (e.g., 2101 for ADT, 2102 for Lab ORU, 2103 for Rad ORU, 2104 for ORM). Camel captures the framed byte stream.
2. **Frame Stripping & Charset Normalization**: Framing bytes (0x0B, 0x1C, 0x0D) are stripped; bytes are decoded into UTF-8 or ISO-8859-1 strings based on MSH-18 character set headers.
3. **Message Type & Trigger Event Validation**: The MSH segment is inspected to identify message structure (MSH-9.1), trigger event (MSH-9.2), and message control ID (MSH-10).
4. **HL7 v2 Data Extraction**: Specialized extractors leverage HAPI Terser to navigate segment hierarchies and extract demographic, clinical, and encounter fields.
5. **Canonical FHIR R5 Construction**: Canonical resource builders instantiate:
   - A FHIR R5 Communication resource preserving the complete raw HL7 message in Communication.payload.cont
   - A synthetic FHIR R5 Task resource (representing the executable Pragma) encapsulating workflow priority, routing metadata, and correlation IDs.
6. **Cache & Database Persistence**: The Communication and Task resources are persisted into the Mneme Infinispan data grid via TaskCacheService and write-behind synced to Mnemosyne JPA stores.
7. **Petasos Event Dispatch**: A lightweight ErgonEvent notification referencing the persistent Task.id and Communication.id is published to the target ActiveMQ Artemis inbound queue (e.g., petasos.queue.inbound.adt).
8. **Synchronous HL7 Acknowledgement Generation**: A standards-compliant HL7 ACK containing matching MSA-2 and appropriate MSA-1 code (AA, AE, AR) is framed in MLLP and flushed over the TCP socket before closing the connection.

15.3 Ingress Component Implementation

15.3.1 Camel RouteBuilder Configuration

Ingress routes are defined using Apache Camel's Java DSL, configured as CDI @Dependent beans. The route registers an mllp:// endpoint, sets connection options, and wires incoming exchanges to dedicated processor wrappers.

```
@Dependent
public class IncomingAdtMessageMllpRouteBuilder extends RouteBuilder {

    private static final Logger log = LoggerFactory.getLogger(
    IncomingAdtMessageMllpRouteBuilder.class);

    @Inject
    private MllpConfig mllpConfig;

    @Inject
    private IncomingAdtMessageProcessorWrapper incomingAdtMessageProcessorWrapper;

    @Override
    public void configure() throws Exception {
        if (mllpConfig == null) {
            mllpConfig = new MllpConfig();
```

```
        }

        String host = mllpConfig.getHost();
        int port = mllpConfig.getAdtPort(); // Port 2101
        boolean autoAck = mllpConfig.isAutoAck(); // false: explicit ACK control
        int bindTimeout = mllpConfig.getBindTimeout();
        int readTimeout = mllpConfig.getReadTimeout();

        // MLLP Ingress Listener Route
        from(String.format("mllp://%s:%d?autoAck=%b&bindTimeout=%d&readTimeout=%d",
                host, port, autoAck, bindTimeout, readTimeout))
            .routeId("hl7-mllp-adt-receiver")
            .log(LoggingLevel.INFO, log, "MLLP Inbound ADT message received on port "
    + port)
            .process(incomingAdtMessageProcessorWrapper)
            .log(LoggingLevel.INFO, log, "MLLP Inbound ADT processing completed; ACK
    ready");

        // Internal Direct Route for In-Process Triggers
        from("direct:adt-events")
            .routeId("direct-adt-processor")
            .process(incomingAdtMessageProcessorWrapper);
    }
}
```

Listing 15.1: IncomingAdtMessageMllpRouteBuilder Implementation Pattern

15.3.2 Processor Wrapper & Context Management

The `IncomingAdtMessageProcessorWrapper` acts as the boundary adapter between Apache Camel's exchange model and Harmonia's HL7 domain processor.

```
@ApplicationScoped
public class IncomingAdtMessageProcessorWrapper implements Processor {

    private static final Logger log = LoggerFactory.getLogger(
    IncomingAdtMessageProcessorWrapper.class);

    @Inject
    private IncomingAdtMessageProcessor incomingAdtMessageProcessor;

    @Override
    public void process(Exchange exchange) throws Exception {
        String hl7Message = exchange.getIn().getBody(String.class);
        log.debug("Processing inbound HL7 payload of length: {}", hl7Message.length())
    ;

        // Delegate to domain processor
        AdtProcessingResult result = incomingAdtMessageProcessor.process(hl7Message);

        // Populate exchange headers and set synchronous ACK body
        exchange.getMessage().setHeader("HIE_MESSAGE_CONTROL_ID", result.
    getMessageControlId());
        exchange.getMessage().setHeader("HIE_ACK_CODE", result.getAckCode());
        exchange.getMessage().setBody(result.getAckMessage());
    }
}
```

Listing 15.2: IncomingAdtMessageProcessorWrapper Blueprint

15.3.3 HL7 v2 Data Extraction with HAPI Terser

To ensure resilience against optional or missing segments across varied HL7 versions (2.3, 2.4, 2.5), extractors employ the HAPI HL7 Terser with defensive navigation wrappers:

```
public class AdtMessageExtractor {

    private final Terser terser;

    public AdtMessageExtractor(Message message) {
        this.terser = new Terser(message);
    }
```

```
8
9       public String extractMessageControlId() {
10          return getSafeValue("/.MSH-10");
11      }
12
13      public String extractTriggerEvent() {
14          return getSafeValue("/.MSH-9-2");
15      }
16
17      public String extractPatientId() {
18          String pid3 = getSafeValue("/.PID-3-1");
19          return (pid3 != null && !pid3.isEmpty()) ? pid3 : getSafeValue("/.PID-2-1");
20      }
21
22      public String extractFamilyName() {
23          return getSafeValue("/.PID-5-1");
24      }
25
26      public String extractGivenName() {
27          return getSafeValue("/.PID-5-2");
28      }
29
30      public String extractDateOfBirth() {
31          return getSafeValue("/.PID-7-1");
32      }
33
34      public String extractAdministrativeGender() {
35          return getSafeValue("/.PID-8");
36      }
37
38      public String extractVisitNumber() {
39          return getSafeValue("/.PV1-19-1");
40      }
41
42      public String extractPatientClass() {
43          return getSafeValue("/.PV1-2");
44      }
45
46      public String extractAssignedLocation() {
47          String poc = getSafeValue("/.PV1-3-1"); // Point of care
48          String room = getSafeValue("/.PV1-3-2");
49          String bed = getSafeValue("/.PV1-3-3");
50          return String.format("%s/%s/%s",
51              poc != null ? poc : "",
52              room != null ? room : "",
53              bed != null ? bed : "");
54      }
55
56      private String getSafeValue(String terserPath) {
57          try {
58              String value = terser.get(terserPath);
59              return (value != null && !value.trim().isEmpty()) ? value.trim() : null;
60          } catch (HL7Exception e) {
61              return null; // Defensive fallback on missing segment/field
62          }
63      }
64  }
```

Listing 15.3: AdtMessageExtractor Implementation Blueprint

15.4 Canonical FHIR Model Mapping & Resource Builders

Harmonia normalizes all inbound HL7 messages into dual canonical FHIR R5 entities:

1. **FHIR R5 `Communication`**: Holds message provenance, sender/recipient metadata, and the raw immutable HL7 v2 payload.
2. **FHIR R5 `Task`**: Models the executable units of work (`Pragma`) consumed by the `Ponos` workflow engine.

15.4.1 FHIR Communication Resource Construction

```java
public class AdtCommunicationResourceBuilder {

    public Communication buildCommunication(AdtMessageExtractor extractor, String rawHl7) {
        Communication comm = new Communication();
        comm.setId(UUID.randomUUID().toString());
        comm.setStatus(Communication.CommunicationStatus.COMPLETED);

        // Identifiers
        Identifier msh10Id = new Identifier();
        msh10Id.setSystem("http://harmonia.hie/identifiers/msh-10");
        msh10Id.setValue(extractor.extractMessageControlId());
        comm.addIdentifier(msh10Id);

        // Category
        CodeableConcept category = new CodeableConcept();
        category.addCoding().setSystem("http://terminology.hl7.org/CodeSystem/communication-category")
                            .setCode("notification")
                            .setDisplay("HL7 Inbound Notification");
        comm.addCategory(category);

        // Sender & Recipient
        comm.setSender(new Reference().setDisplay(extractor.getSendingFacility()));
        comm.addRecipient(new Reference().setDisplay("Harmonia-HIE-Ingress"));

        // Payload Attachment (Raw HL7 Message)
        Communication.CommunicationPayloadComponent payload = new Communication.CommunicationPayloadComponent();
        Attachment attachment = new Attachment();
        attachment.setContentType("text/plain");
        attachment.setData(rawHl7.getBytes(StandardCharsets.UTF_8));
        attachment.setCreation(new Date());
        payload.setContent(attachment);
        comm.addPayload(payload);

        comm.setSent(new Date());
        return comm;
    }
}
```

Listing 15.4: AdtCommunicationResourceBuilder Blueprint

15.4.2 FHIR Task (Pragma) Resource Construction

The synthetic `Task` represents the executable pipeline instance. Table 15.1 outlines the canonical field mappings.

HL7 v2 Source Field	FHIR R5 Task Property	Datatype	Transformation Logic
`MSH-10`	`Task.identifier[msh10]`	`Identifier`	Copied verbatim for correlation
`MSH-9-2` (Trigger)	`Task.businessStatus`	`CodeableConcept`	Mapped to clinical event type (e.g. `A01` → `ADMISSION`)
`PV1-2` (Patient Class)	`Task.priority`	`TaskPriority`	`E` → `stat`, `I` → `urgent`, `O` → `routine`
`PID-3-1` (MRN)	`Task.for`	`Reference`	Reference to `Patient/MRN-{id}`
`PV1-19-1` (Visit Num)	`Task.encounter`	`Reference`	Reference to `Encounter/VIS-{num}`
Auto-generated UUID	`Task.id`	`IdType`	Unique Pragma identifier
Literal `"requested"`	`Task.status`	`TaskStatus`	Initial state for Ponos engine
Literal `"order"`	`Task.intent`	`TaskIntent`	Required FHIR workflow intent

Table 15.1: HL7 v2 to FHIR R5 Task Canonical Mapping Specifications

15.5 Persistence, Event Dispatch & Synchronous Acknowledgement

15.5.1 Persistence Integration with Mneme & Mnemosyne

The ingress processor writes canonical entities into the `Mneme` Infinispan data grid via `TaskCacheService` and `CommunicationService`:

```
public AdtProcessingResult process(String rawHl7) {
    try {
        Message hl7Msg = hapiParser.parse(rawHl7);
        AdtMessageExtractor extractor = new AdtMessageExtractor(hl7Msg);

        // 1. Build Canonical Resources
        Communication comm = commBuilder.buildCommunication(extractor, rawHl7);
        Task task = taskBuilder.buildTask(extractor, comm.getId());

        // 2. Persist to Mneme Cache Grid / Mnemosyne Store
        communicationService.save(comm);
        taskService.save(task);

        // 3. Publish Asynchronous Notification to Petasos Message Bus
        ErgonEvent event = new ErgonEvent();
        event.setEventId(UUID.randomUUID().toString());
        event.setTaskId(task.getId());
        event.setCommunicationId(comm.getId());
        event.setMessageType("ADT");
        event.setTriggerEvent(extractor.extractTriggerEvent());
        event.setTimestamp(System.currentTimeMillis());

        taskEventProducerService.publishInboundEvent("petasos.queue.inbound.adt",
    event);

        // 4. Return Synchronous Application Accept (AA)
        String ackHl7 = hl7AckGenerator.generateAck(hl7Msg, "AA", "Message Accepted");
        return new AdtProcessingResult(extractor.extractMessageControlId(), "AA",
    ackHl7);

    } catch (HL7Exception e) {
        log.error("HL7 parse error: {}", e.getMessage());
        String nackHl7 = hl7AckGenerator.generateNack(rawHl7, "AR", "Structural Parse
    Failure: " + e.getMessage());
        return new AdtProcessingResult(null, "AR", nackHl7);
    } catch (Exception e) {
        log.error("Internal processing error: {}", e.getMessage());
        String nackHl7 = hl7AckGenerator.generateNack(rawHl7, "AE", "Internal Storage
    Failure: " + e.getMessage());
        return new AdtProcessingResult(null, "AE", nackHl7);
    }
}
```

Listing 15.5: Ingress Persistence & Event Dispatch Implementation

15.5.2 Synchronous Acknowledgement Mapping Rules

The ingress service guarantees deterministic acknowledgement status codes as summarized in Table 15.2.

ACK Code	Status Name	Triggering Condition	Client Action
`AA`	Application Accept	Syntactically valid HL7; successfully written to Mneme cache and Petasos queue.	Message considered durably accepted; do not retry.
`AE`	Application Error	Transient infrastructure failure (Infinispan timeout, database disconnect, Artemis broker unreachable).	Re-queue and retry with exponential backoff.
`AR`	Application Reject	Malformed HL7 frame, missing required MSH segments, unsupported major version.	Route to operator dead-letter queue; do not retry.

Table 15.2: Synchronous HL7 Acknowledgement Response Contracts

15.5.3 REST Management & CDI Deployment Configuration

Every ingress gateway exposes optional REST endpoints for health checks, administrative ping, and direct HTTP ingestion:

```
@Path("/mllp/adt")
@ApplicationScoped
public class AdtResource {

    @Inject
    private IncomingAdtMessageProcessor processor;

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public Response ingestAdtMessage(String rawHl7) {
        AdtProcessingResult result = processor.process(rawHl7);
        if ("AA".equals(result.getAckCode())) {
            return Response.ok(result.getAckMessage()).build();
        } else if ("AE".equals(result.getAckCode())) {
            return Response.status(Response.Status.SERVICE_UNAVAILABLE).entity(result.
    getAckMessage()).build();
        } else {
            return Response.status(Response.Status.BAD_REQUEST).entity(result.
    getAckMessage()).build();
        }
    }
}
```

Listing 15.6: JAX-RS Direct Ingestion Endpoint (AdtResource)

15.6 Multi-Instance Egress Architecture & Queue Partitioning

In contrast to ingress gateways that bind to inbound listening ports, the MLLP Egress Gateway (`pylai-mllp-out`) functions as a dynamic TCP client, connecting to downstream clinical endpoints (such as hospital EMR, LMS, and RIS-PACS systems).

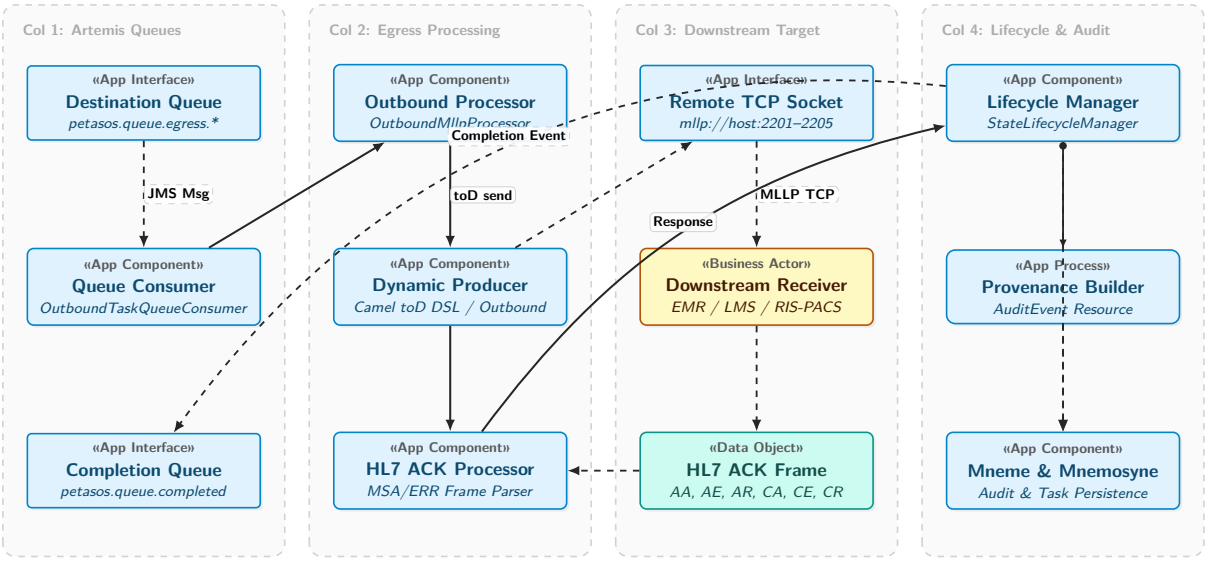


Figure 15.2: ArchiMate 3.2 Component Flow for the MLLP Egress Multi-Instance Dispatch Pipeline

15.6.1 Head-of-Line Blocking Mitigation & Destination Isolation

In healthcare integration environments, downstream clinical systems exhibit disparate network latency, maintenance windows, and failure modes. If a single outbound gateway serviced all destinations from a shared queue:

1. A transient network outage or TCP socket stall at one hospital endpoint (e.g., LMS analyzer reboot) would block outbound transmissions to healthy endpoints (e.g., EMR, RIS-PACS).

2. Retry storms and queue head-of-line blocking would cascade across the entire healthcare exchange.

To eliminate head-of-line blocking, Harmonia enforces **destination-partitioned multi-instance isolation**:

- Each downstream destination is allocated a dedicated, persistent ActiveMQ Artemis queue:

$$\text{Queue Pattern} ::= \texttt{petasos.queue.egress.}\langle\text{destinationId}\rangle\texttt{.}\langle\text{messageType}\rangle \tag{15.2}$$

  Examples: `petasos.queue.egress.emr.adt`, `petasos.queue.egress.lms.orm`, `petasos.queue.egress.rispac.`
- Each destination runs as an isolated, containerized `pylai-mllp-out` microservice instance consuming exclusively from its dedicated queue.
- Failures, exponential backoff retries, and socket timeouts in one container never impact message throughput or delivery SLAs of other destinations.

15.6.2 Outbound Configuration & Destination Registry

Configuration is managed via `MllpOutboundConfig` and `MllpDestinationRegistry`, populated via microprofile environment variables at container launch.

Environment Variable	Config Property	Default	Description
`MLLP_OUTBOUND_DESTINATION_ID`	`destinationId`	`"DEFAULT"`	Unique destin (e.g. `EMR`, `LMS`
`MLLP_OUTBOUND_HOST`	`host`	`"localhost"`	Target TCP h address
`MLLP_OUTBOUND_PORT`	`port`	2575	Target MLLP port
`MLLP_OUTBOUND_READ_TIMEOUT_MS`	`readTimeoutMs`	10000	Socket read t liseconds
`MLLP_OUTBOUND_CONNECT_TIMEOUT_MS`	`connectTimeoutMs`	5000	TCP handsha milliseconds
`MLLP_OUTBOUND_AUTO_ACK`	`autoAck`	`false`	Requires ex stream ACK
`MLLP_OUTBOUND_QUEUE_NAME`	`dedicatedEventQueueName`	*Derived*	Dedicated Ar tion queue
`MLLP_OUTBOUND_COMPLETION_QUEUE_NAME`	`completionEventQueueName`	`"petasos.queue.egress.completed"`	Completion t

Table 15.3: MLLP Outbound Gateway Environment Configuration Parameters

15.6.3 Outbound Data Transfer Models

The egress pipeline coordinates transmissions using two standard DTOs:

- **`OutboundMllpRequest`**: Encapsulates `taskId`, `communicationId`, `destinationId`, `messageControlId`, `targetHost`, `targetPort`, `rawHl7Payload`, and retry metadata.
- **`OutboundMllpResponse`**: Encapsulates delivery status (`isSuccessful`), `ackCode` (`AA`, `AE`, `AR`, `CA`, `CE`, `CR`), `rawAckMessage`, `errorMessage`, `durationMs`, and `acknowledgedAt` timestamp.

15.7 Outbound Queue Consumer & Camel Dynamic MLLP Routes

15.7.1 Dedicated Task Queue Consumer

The `OutboundTaskQueueConsumer` implements JMS `MessageListener`, pulling outbound tasks from the dedicated Artemis queue and dispatching them through Apache Camel.

```
1 @ApplicationScoped
2 public class OutboundTaskQueueConsumer implements MessageListener {
3
4     private static final Logger log = LoggerFactory.getLogger(
      OutboundTaskQueueConsumer.class);
5
6     @Inject
7     private MllpOutboundConfig outboundConfig;
8
```

```
    @Inject
    private OutboundStateLifecycleManager lifecycleManager;

    private ProducerTemplate producerTemplate;

    @Override
    public void onMessage(jakarta.jms.Message message) {
        try {
            if (message instanceof TextMessage) {
                String jsonPayload = ((TextMessage) message).getText();
                OutboundMllpRequest request = JsonUtils.fromJson(jsonPayload,
    OutboundMllpRequest.class);

                // Dispatch via Camel ProducerTemplate
                OutboundMllpResponse response = producerTemplate.requestBody(
                    "direct:mllp-outbound-send", request, OutboundMllpResponse.class);

                // Update FHIR Task, Communication and Provenance states
                lifecycleManager.handleOutboundResponse(request, response);
                message.acknowledge(); // Acknowledge JMS message upon lifecycle
    recording
            }
        } catch (Exception e) {
            log.error("Fatal exception in outbound queue consumer: {}", e.getMessage()
    , e);
            // Non-acknowledged message triggers Artemis redelivery policy
        }
    }
}
```

Listing 15.7: OutboundTaskQueueConsumer Implementation Pattern

15.7.2 Camel Dynamic Outbound Route Builder

The `OutboundMllpRouteBuilder` configures dynamic dispatch to external TCP sockets using Camel's `toD` dynamic URI expression, setting connection timeouts and auto-acknowledgment rules.

```
@ApplicationScoped
public class OutboundMllpRouteBuilder extends RouteBuilder {

    private static final Logger log = LoggerFactory.getLogger(OutboundMllpRouteBuilder
    .class);

    @Inject
    private OutboundMllpProcessor outboundMllpProcessor;

    @Inject
    private Hl7AckProcessor hl7AckProcessor;

    @Override
    public void configure() throws Exception {

        // Global Exception Handler for Socket / Network Failures
        onException(Exception.class)
            .handled(true)
            .setHeader("HIE_TRANSMISSION_SUCCESS", constant(false))
            .setHeader("HIE_ACK_CODE", constant("AE"))
            .setHeader("HIE_ERROR_MESSAGE", simple("${exception.message}"))
            .process(exchange -> {
                Exception cause = exchange.getProperty(Exchange.EXCEPTION_CAUGHT,
    Exception.class);
                String dest = exchange.getIn().getHeader("HIE_DESTINATION_ID", String.
    class);
                String msh10 = exchange.getIn().getHeader("HIE_EXPECTED_CONTROL_ID",
    String.class);
                long duration = System.currentTimeMillis() - exchange.getProperty("
    HIE_START_TIME", Long.class);

                OutboundMllpResponse failure = OutboundMllpResponse.failure(
                    msh10, "Socket connection error to " + dest + ": " + cause.
    getMessage(), duration);
                failure.setDestinationId(dest);
                exchange.getMessage().setBody(failure);
```

```
            });

        // Outbound Dynamic MLLP Route
        from("direct:mllp-outbound-send")
            .routeId("mllp-outbound-dynamic-sender")
            .process(outboundMllpProcessor) // Validates request and populates Camel headers
            .setProperty("HIE_START_TIME", simple("${date:now:getTime()}"))
            .log(LoggingLevel.INFO, log, "Dispatching MLLP frame to ${header.HIE_DEST_HOST}:${header.HIE_DEST_PORT}")

            // Dynamic MLLP Producer
            .toD("mllp://${header.HIE_DEST_HOST}:${header.HIE_DEST_PORT}?"
                + "autoAck=${header.HIE_AUTO_ACK}"
                + "&readTimeout=${header.HIE_READ_TIMEOUT}"
                + "&connectTimeout=${header.HIE_CONNECT_TIMEOUT}"
                + "&charsetName=UTF-8")

            .process(hl7AckProcessor) // Deep inspection of received ACK/NACK frame
            .log(LoggingLevel.INFO, log, "Outbound MLLP dispatch complete: ACK=${header.HIE_ACK_CODE}, Success=${header.HIE_TRANSMISSION_SUCCESS}");
    }
}
```

Listing 15.8: OutboundMllpRouteBuilder Dynamic Dispatch DSL

15.7.3 Outbound MLLP Processor Header Injection

The `OutboundMllpProcessor` validates the inbound DTO, resolves target connection parameters from the `MllpDestinationRegistry`, and initializes exchange headers:

```
@ApplicationScoped
public class OutboundMllpProcessor implements Processor {

    @Inject
    private MllpOutboundConfig config;

    @Inject
    private MllpDestinationRegistry destinationRegistry;

    @Override
    public void process(Exchange exchange) throws Exception {
        OutboundMllpRequest request = exchange.getIn().getBody(OutboundMllpRequest.class);
        if (request == null || StringUtils.isBlank(request.getRawHl7Payload())) {
            throw new IllegalArgumentException("OutboundMllpRequest payload must not be null or blank");
        }

        String destinationId = StringUtils.isNotBlank(request.getDestinationId())
            ? request.getDestinationId() : config.getDestinationId();

        MllpDestinationConfig destConfig = destinationRegistry.getDestination(destinationId)
            .orElse(config.toDestinationConfig());

        // Configure Exchange Headers for Dynamic toD URI
        exchange.getMessage().setHeader("HIE_DESTINATION_ID", destinationId);
        exchange.getMessage().setHeader("HIE_DEST_HOST", destConfig.getHost());
        exchange.getMessage().setHeader("HIE_DEST_PORT", destConfig.getPort());
        exchange.getMessage().setHeader("HIE_READ_TIMEOUT", destConfig.getReadTimeoutMs());
        exchange.getMessage().setHeader("HIE_CONNECT_TIMEOUT", destConfig.getConnectTimeoutMs());
        exchange.getMessage().setHeader("HIE_AUTO_ACK", false);
        exchange.getMessage().setHeader("HIE_EXPECTED_CONTROL_ID", request.getMessageControlId());

        // Set Outbound Body to Raw HL7 Message String
        exchange.getMessage().setBody(request.getRawHl7Payload());
    }
}
```

Listing 15.9: OutboundMllpProcessor Header Initialization

15.8 Downstream HL7 ACK/NACK Deep Inspection

A critical requirement in clinical integration is preventing false-positive transmission confirmations. Receiving a TCP `FIN` or `ACK` at the transport layer does not imply clinical acceptance by the receiving hospital system. The `Hl7AckProcessor` performs deep structural inspection on the received HL7 acknowledgement frame.

15.8.1 MSA & ERR Segment Parsing Specification

The processor evaluates the `MSA` (Message Acknowledgement) and `ERR` (Error) segments:

```
public OutboundMllpResponse parseAck(String rawAck, String expectedControlId, String destinationId, long durationMs) {
    if (StringUtils.isBlank(rawAck)) {
        return OutboundMllpResponse.failure(expectedControlId, "Empty ACK response received from " + destinationId, durationMs);
    }

    Matcher matcher = MSA_PATTERN.matcher(rawAck);
    if (!matcher.find()) {
        OutboundMllpResponse resp = OutboundMllpResponse.failure(
            expectedControlId, "Malformed ACK: missing MSA segment", durationMs);
        resp.setRawAckMessage(rawAck);
        return resp;
    }

    String ackCode = matcher.group(1).trim().toUpperCase(); // MSA-1: AA, AE, AR, CA, CE, CR
    String msaControlId = matcher.group(2);                  // MSA-2: Referenced MSH-10
    String ackText = matcher.group(3);                       // MSA-3: Text explanation

    // Strict Correlation Validation
    boolean controlIdMatches = true;
    if (StringUtils.isNotBlank(expectedControlId) && StringUtils.isNotBlank(msaControlId)) {
        if (!StringUtils.equals(expectedControlId.trim(), msaControlId.trim())) {
            controlIdMatches = false;
            log.warn("MSA-2 control ID mismatch: expected '{}' but received '{}'", expectedControlId, msaControlId);
        }
    }

    OutboundMllpResponse response = new OutboundMllpResponse();
    response.setMessageControlId(msaControlId != null ? msaControlId : expectedControlId);
    response.setDestinationId(destinationId);
    response.setAckCode(ackCode);
    response.setAckText(ackText);
    response.setRawAckMessage(rawAck);
    response.setDurationMs(durationMs);
    response.setAcknowledgedAt(new Date());

    boolean isAccept = "AA".equals(ackCode) || "CA".equals(ackCode);
    if (isAccept && controlIdMatches) {
        response.setSuccessful(true);
    } else if (isAccept && !controlIdMatches) {
        response.setSuccessful(false);
        response.setErrorMessage("Control ID mismatch: expected " + expectedControlId + " but received " + msaControlId);
    } else {
        response.setSuccessful(false);
        response.setErrorMessage(StringUtils.isNotBlank(ackText) ? ackText : "HL7 Application NACK (" + ackCode + ")");
    }

    return response;
```

```
48 }
```

Listing 15.10: HI7AckProcessor MSA Evaluation Logic

15.8.2 Downstream ACK Code Decision Matrix

Table 15.4 defines the egress gateway behavior across all downstream ACK codes and error conditions.

Response Code	Semantics	Success?	State Transition	Retry Strategy
`AA` / `CA`	Application Accept	**YES**	`Task.status` → `COMPLETED`	None
`AE` / `CE`	Application Error	**NO**	`Task.status` → `IN_PROGRESS`	Exponential Backoff
`AR` / `CR`	Application Reject	**NO**	`Task.status` → `FAILED`	Terminal (DLQ)
`TIMEOUT`	Socket Read Timeout	**NO**	`Task.status` → `IN_PROGRESS`	Immediate Reconnect
`CONN_REFUSED`	Port Closed	**NO**	`Task.status` → `IN_PROGRESS`	Linear Backoff
`MSH-10 MISMATCH`	Correlation Breach	**NO**	`Task.status` → `FAILED`	Operator Review

Table 15.4: Downstream HL7 Acknowledgement Code Decision Matrix

15.9 State, Task, and Provenance Lifecycle Management

15.9.1 Outbound State Lifecycle Synchronization

Upon receiving the `OutboundMllpResponse`, the `OutboundStateLifecycleManager` updates the associated canonical FHIR resources in `Mneme` and `Mnemosyne`, constructs an immutable `Provenance` record, and emits an audit event to Petasos:

```
1  @ApplicationScoped
2  public class OutboundStateLifecycleManager {
3
4      @Inject
5      private TaskService taskService;
6
7      @Inject
8      private CommunicationService communicationService;
9
10     @Inject
11     private ProvenanceService provenanceService;
12
13     @Inject
14     private OutboundProvenanceResourceBuilder provenanceBuilder;
15
16     @Inject
17     private TaskEventProducerService taskEventProducerService;
18
19     public void handleOutboundResponse(OutboundMllpRequest request,
       OutboundMllpResponse response) {
20         // 1. Update Synthetic Task (Pragma) Status
21         if (StringUtils.isNotBlank(request.getTaskId())) {
22             Task task = taskService.getTask(request.getTaskId());
23             if (task != null) {
24                 if (response.isSuccessful()) {
25                     task.setStatus(Task.TaskStatus.COMPLETED);
26                     task.setBusinessStatus(new CodeableConcept().setText("DELIVERED_AA
       "));
27                 } else if ("AE".equals(response.getAckCode())) {
28                     task.setStatus(Task.TaskStatus.INPROGRESS);
29                     task.setBusinessStatus(new CodeableConcept().setText("
       RETRY_PENDING_AE"));
30                 } else {
31                     task.setStatus(Task.TaskStatus.FAILED);
32                     task.setBusinessStatus(new CodeableConcept().setText("REJECTED_AR"
       ));
33                 }
34                 task.setLastModified(new Date());
35                 taskService.update(task);
```

```
            }
        }

        // 2. Build and Persist Immutable Provenance Record
        Provenance provenance = provenanceBuilder.buildProvenance(request, response);
        provenanceService.save(provenance);

        // 3. Publish Egress Completion Event to Petasos
        ErgonEvent completionEvent = new ErgonEvent();
        completionEvent.setEventId(UUID.randomUUID().toString());
        completionEvent.setTaskId(request.getTaskId());
        completionEvent.setDestinationId(response.getDestinationId());
        completionEvent.setStatus(response.isSuccessful() ? "SUCCESS" : "FAILURE");
        completionEvent.setDurationMs(response.getDurationMs());

        taskEventProducerService.publishOutboundCompletionEvent("petasos.queue.egress.completed", completionEvent);
    }
}
```

Listing 15.11: OutboundStateLifecycleManager Implementation

15.9.2 FHIR Provenance Resource Construction

The `OutboundProvenanceResourceBuilder` captures complete transmission audit trails:

```
public class OutboundProvenanceResourceBuilder {

    public Provenance buildProvenance(OutboundMllpRequest request, OutboundMllpResponse response) {
        Provenance prov = new Provenance();
        prov.setId(UUID.randomUUID().toString());
        prov.setRecorded(new Date());

        // Target References (Task & Communication)
        if (StringUtils.isNotBlank(request.getTaskId())) {
            prov.addTarget(new Reference("Task/" + request.getTaskId()));
        }
        if (StringUtils.isNotBlank(request.getCommunicationId())) {
            prov.addTarget(new Reference("Communication/" + request.getCommunicationId()));
        }

        // Activity: Data Transmission
        CodeableConcept activity = new CodeableConcept();
        activity.addCoding().setSystem("http://terminology.hl7.org/CodeSystem/v3-DataOperation")
                            .setCode("TRANSMIT")
                            .setDisplay("MLLP Outbound Transmission");
        prov.setActivity(activity);

        // Agent: Transmitter Gateway
        Provenance.ProvenanceAgentComponent agent = new Provenance.ProvenanceAgentComponent();
        agent.getType().addCoding().setSystem("http://terminology.hl7.org/CodeSystem/provenance-participant-type")
                                   .setCode("transmitter")
                                   .setDisplay("Harmonia MLLP Egress Gateway");
        agent.setWho(new Reference().setDisplay("pylai-mllp-out-" + response.getDestinationId()));
        prov.addAgent(agent);

        return prov;
    }
}
```

Listing 15.12: OutboundProvenanceResourceBuilder Blueprint

15.10 Testing, Verification Harnesses & Containerization

To guarantee high availability and fault resilience prior to hospital production rollout, new MLLP ingress and egress services must be thoroughly validated across four testing tiers.

15.10.1 Tier 1: Unit Testing with HAPI HL7 v2 Parsers

Unit tests validate segment parsing, field extraction, and boundary conditions without network dependencies:

```
public class AdtMessageExtractorTest {

    private final PipeParser parser = new PipeParser();

    @Test
    public void testSuccessfulAdtExtraction() throws Exception {
        String hl7 = "MSH|^~\\&|PAS|HOSPITAL|HARMONIA|HIE|20260916080000||ADT^A01|MSG-100421|P|2.4\r"
                   + "PID|1||MRN-90210^^^HOSP||DOE^JANE^^^^||19850412|F\r"
                   + "PV1|1|I|WARD-3A^RM-12^BED-02||||1234^SMITH^JOHN^^DR||||||||||VIS-88001";

        Message msg = parser.parse(hl7);
        AdtMessageExtractor extractor = new AdtMessageExtractor(msg);

        assertEquals("MSG-100421", extractor.extractMessageControlId());
        assertEquals("A01", extractor.extractTriggerEvent());
        assertEquals("MRN-90210", extractor.extractPatientId());
        assertEquals("DOE", extractor.extractFamilyName());
        assertEquals("JANE", extractor.extractGivenName());
        assertEquals("VIS-88001", extractor.extractVisitNumber());
    }
}
```

Listing 15.13: Unit Test Suite for ADT Message Extraction

15.10.2 Tier 2: Camel Route Harness Testing

Camel routes are validated using `CamelTestSupport` and `MockEndpoint` assertions:

```
public class OutboundMllpRouteTest extends CamelTestSupport {

    @EndpointInject("mock:mllp-target")
    private MockEndpoint mockTarget;

    @Test
    public void testSuccessfulOutboundDispatch() throws Exception {
        mockTarget.expectedMessageCount(1);
        mockTarget.whenAnyExchangeReceived(exchange -> {
            exchange.getMessage().setBody("MSH|^~\\&|||||20260916080001||ACK^A01|ACK-01|P|2.4\rMSA|AA|MSG-100421");
        });

        OutboundMllpRequest req = new OutboundMllpRequest();
        req.setMessageControlId("MSG-100421");
        req.setDestinationId("EMR");
        req.setRawHl7Payload("MSH|^~\\&|HARMONIA||EMR||20260916080000||ADT^A01|MSG-100421|P|2.4\rPID|1||MRN-90210");

        OutboundMllpResponse resp = template.requestBody("direct:mllp-outbound-send", req, OutboundMllpResponse.class);

        assertTrue(resp.isSuccessful());
        assertEquals("AA", resp.getAckCode());
        mockTarget.assertIsSatisfied();
    }
}
```

Listing 15.14: Camel Route Mock Test Harness

15.10.3 Tier 3: Embedded Mock MLLP Server Integration Testing

Integration tests launch embedded TCP servers (via Paradeigma's `MllpServer`) to simulate real network conditions, latency, and connection resets:

```
@Test
public void testTimeoutAndApplicationRejectHandling() throws Exception {
    // Launch embedded mock MLLP receiver returning AR Reject
    MllpServer mockHospitalServer = new MllpServer(2204, (req) -> {
        return "MSH|^~\\&|||||20260916080002||ACK^O01|ACK-99|P|2.4\rMSA|AR|ORD-5501|
    Patient MRN Not Found";
    });
    mockHospitalServer.start();

    try {
        OutboundMllpRequest req = new OutboundMllpRequest();
        req.setMessageControlId("ORD-5501");
        req.setDestinationId("LMS");
        req.setTargetPort(2204);
        req.setRawHl7Payload("MSH|^~\\&|HARMONIA||LMS||...|ORM^O01|ORD-5501|P|2.4\r...
    ");

        OutboundMllpResponse resp = template.requestBody("direct:mllp-outbound-send",
    req, OutboundMllpResponse.class);

        assertFalse(resp.isSuccessful());
        assertEquals("AR", resp.getAckCode());
        assertTrue(resp.getErrorMessage().contains("Patient MRN Not Found"));
    } finally {
        mockHospitalServer.stop();
    }
}
```

Listing 15.15: Embedded MLLP Integration Test with Fault Simulation

15.10.4 Tier 4: Multi-Instance Docker Compose Deployment Topology

Production deployments isolate egress gateways into independent containers with resource quotas and dedicated queues:

```
version: '3.8'

services:
  # Ingress Gateway (Inbound MLLP Listeners)
  pylai-mllp-in:
    image: harmonia/pylai-mllp-in:1.0.0
    ports:
      - "2101:2101" # ADT Inbound
      - "2102:2102" # Lab ORU Inbound
      - "2103:2103" # Rad ORU Inbound
      - "2104:2104" # ORM Inbound
    environment:
      - MLLP_BIND_HOST=0.0.0.0
      - ARTEMIS_BROKER_URL=tcp://petasos-broker:61616
      - INFINISPAN_HOTROD_SERVERS=mneme-cluster:11222

  # Egress Gateway Instance 1: Dedicated EMR Gateway
  pylai-mllp-out-emr:
    image: harmonia/pylai-mllp-out:1.0.0
    environment:
      - MLLP_OUTBOUND_DESTINATION_ID=EMR
      - MLLP_OUTBOUND_HOST=emr.hospital.internal
      - MLLP_OUTBOUND_PORT=2201
      - MLLP_OUTBOUND_QUEUE_NAME=petasos.queue.egress.emr.adt
      - MLLP_OUTBOUND_READ_TIMEOUT_MS=8000
      - ARTEMIS_BROKER_URL=tcp://petasos-broker:61616

  # Egress Gateway Instance 2: Dedicated LMS Gateway
  pylai-mllp-out-lms:
    image: harmonia/pylai-mllp-out:1.0.0
    environment:
      - MLLP_OUTBOUND_DESTINATION_ID=LMS
      - MLLP_OUTBOUND_HOST=lms.lab.hospital.internal
```

```
      - MLLP_OUTBOUND_PORT=2204
      - MLLP_OUTBOUND_QUEUE_NAME=petasos.queue.egress.lms.orm
      - MLLP_OUTBOUND_READ_TIMEOUT_MS=15000
      - ARTEMIS_BROKER_URL=tcp://petasos-broker:61616

  # Egress Gateway Instance 3: Dedicated RIS-PACS Gateway
  pylai-mllp-out-rispac:
    image: harmonia/pylai-mllp-out:1.0.0
    environment:
      - MLLP_OUTBOUND_DESTINATION_ID=RISPAC
      - MLLP_OUTBOUND_HOST=rispac.radiology.hospital.internal
      - MLLP_OUTBOUND_PORT=2205
      - MLLP_OUTBOUND_QUEUE_NAME=petasos.queue.egress.rispac.orm
      - MLLP_OUTBOUND_READ_TIMEOUT_MS=20000
      - ARTEMIS_BROKER_URL=tcp://petasos-broker:61616
```

Listing 15.16: Docker Compose Multi-Instance Egress Architecture

Part IV

Middleware Architecture

Chapter 16

Technology & Infrastructure Architecture

The Technology Architecture layer defines the underlying system software runtimes, execution nodes, communication paths, network protocols, and the master platform port allocation specification.

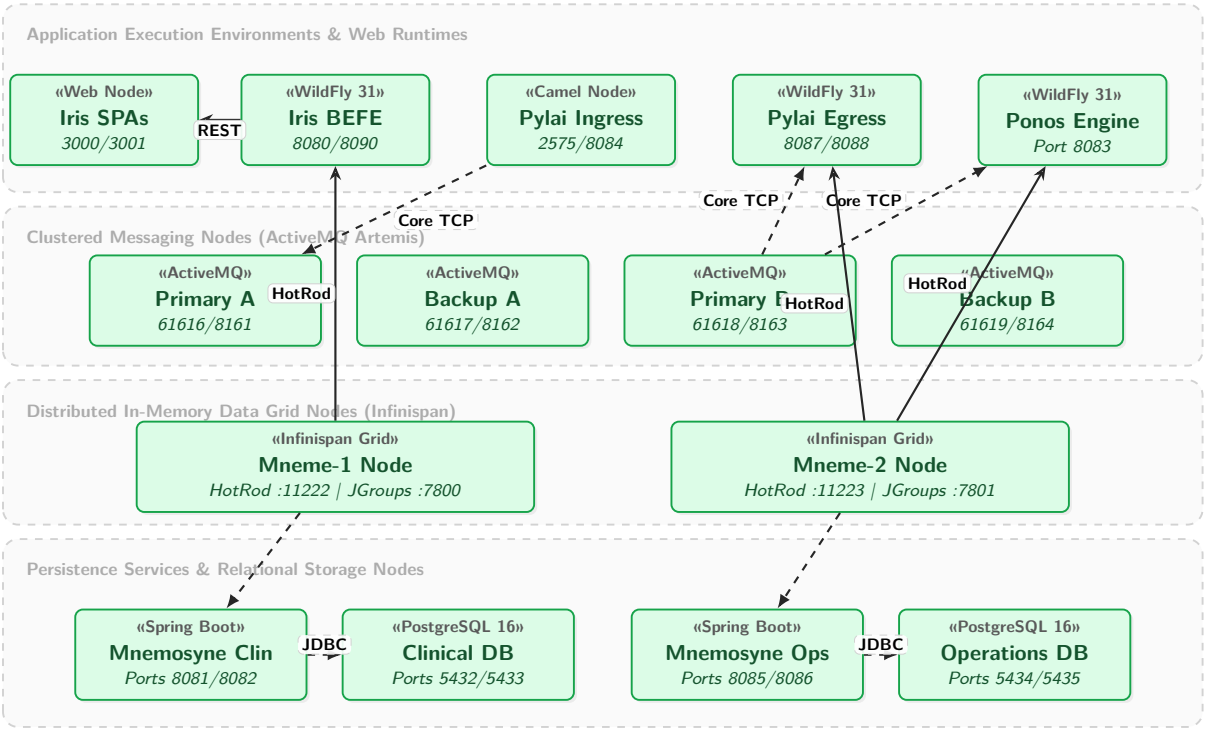


Figure 16.1: ArchiMate Technology Infrastructure, System Software & Communication Paths

16.1 System Software Runtimes

Harmonia standardizes on production-grade, enterprise open-source software runtimes:

WildFly Jakarta EE 10 / Java 21 Provides the enterprise container runtime for `iris-befe`, `ponos`, `pylai-mllp-in`, and `pylai-mllp-out`, delivering CDI dependency injection, JAX-RS REST endpoints, Apache Camel routes, and managed thread pools.

Themis Security Subsystem / Java 21 Provides deterministic policy evaluation (`themis-core`), role-to-authority mapping, data security classification tagging, and structured non-PHI security decision audit streaming (`themis-audit`).

Apache ActiveMQ Artemis 2.38+ High-performance, non-blocking asynchronous message broker providing asynchronous NIO journal durability, server-side clustering, and real-time replication.

Infinispan 15+ Distributed Cache Grid In-memory data grid providing partitioned and replicated L1 caching, non-blocking write-behind SPI stores, and HotRod binary protocol access.

Spring Boot 3.x / HAPI FHIR JPA Serves as the persistence and service framework for `mnemosyne-clinical`, `mnemosyne-operations`, and the `pylai-fhir-registry` REST gateway, encapsulating JPA 3.1 entity mapping, Liquibase migrations, and HAPI FHIR storage engines.

PostgreSQL 16+ Relational Database ACID-compliant relational persistence backend for clinical resources, provider directory records, and operational task histories.

Node.js / Vite / Vue 3 Presentation runtime for the Iris Clinical and Console Single Page Applications.

16.2 In-Memory Cache Grid & Persistence SPI (`Hestia`)

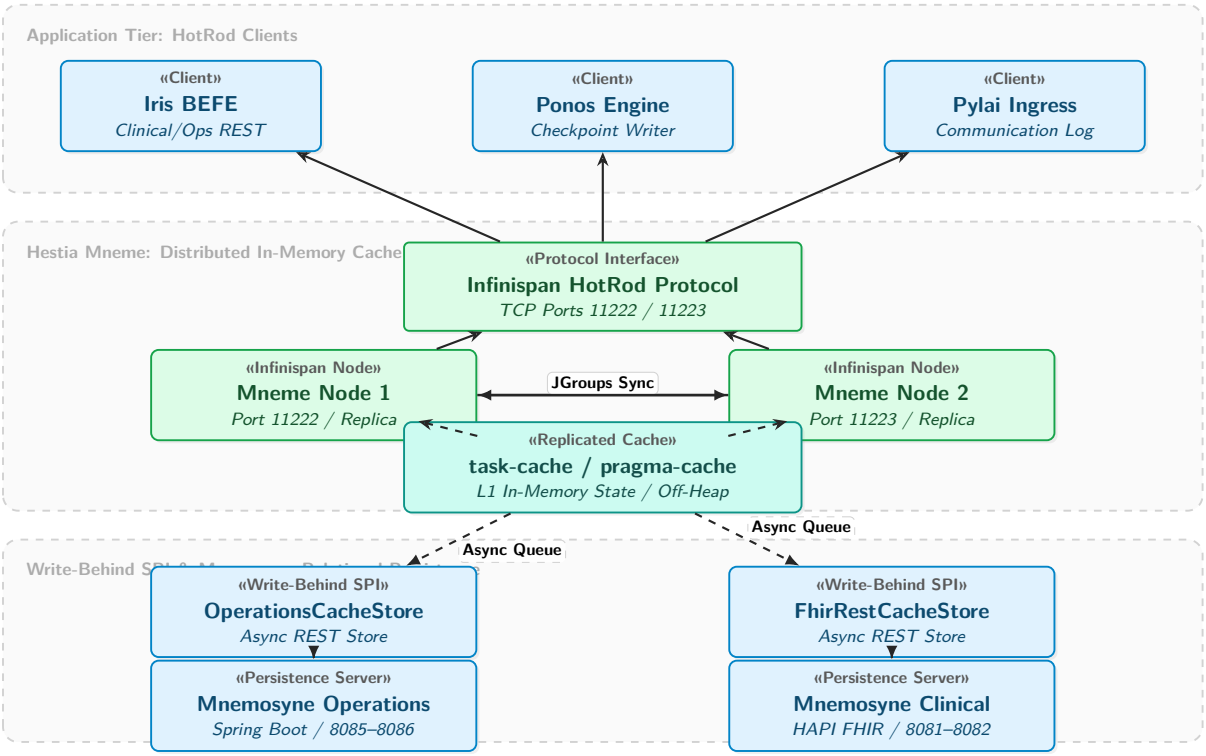


Figure 16.2: ArchiMate Hestia In-Memory Data Grid & Write-Behind Persistence SPI Architecture

The `Hestia` data tier separates fast in-memory execution from relational database I/O:

- **HotRod Binary Protocol**: Applications connect to Infinispan nodes over TCP ports `11222` and `11223` using near-zero latency HotRod serialization.
- **Asynchronous Write-Behind Queue**: When a `Pragma` checkpoint or `Communication` is put to the cache, Infinispan immediately acknowledges the write in RAM and places the entry on an asynchronous write-behind persistence queue.
- **`NonBlockingStore` SPI**: Custom SPI handlers (`OperationsRestCacheStore`) drain the queue and issue batched HTTP `PUT`/`POST` requests to the Mnemosyne REST APIs, preventing pipeline stalls.

16.3 Security Governance, Authorization & Audit Infrastructure (`Themis`)

Themis establishes architectural security governance across runtime execution boundaries, message queues, and persistence stores:

16.3.1 Data Classification & Security Tagging (`HarmoniaSecurityLabelEnum`)

Healthcare entities and operational payloads carry explicit security labels mapped to the FHIR R5 `Resource.meta.security` coding system (`http://harmonia.net/security-label`):

- `PROVIDER_REGISTRY`: Authoritative healthcare practitioner and organization directory records.
- `INTERNAL`: Internal queue envelopes, intermediate workflow states, and system metadata.
- `CLINICAL`: Protected Health Information (PHI), diagnostic observations, and patient identity records.
- `DIAGNOSTIC`: Laboratory, pathology, and radiology imaging reports.
- `ADMINISTRATIVE`: Encounter scheduling, billing, and organizational metadata.
- `AUDIT`: Security decision audit logs, evaluation traces, and compliance records.
- `RESTRICTED`: High-sensitivity clinical or security records requiring explicit elevated authority.

16.3.2 Identity Context Transport & Serialization

Security contexts established at edge gateways are propagated downstream without credential leakage:

- **Ingress Translation**: HTTP headers (`X-User-Roles`, `X-Security-Scopes`) are decoded into `ThemisPrincipal` and `ThemisAuthority` objects at Pylai.
- **Pragma Extension Binding**: Caller identities and granted authorities are immutably serialized into standardized FHIR R5 Task extensions (`http://harmonia.net/fhir/StructureDefinition/security-*`).
- **No Credential Leakage**: Authentication secrets (bearer tokens, passwords, session keys) are stripped at the gateway boundary and never serialized into message queues, caches, or database tables.

16.3.3 Non-PHI Decision Audit Trail Architecture (`ThemisAuditService`)

Every security evaluation emits a structured `ThemisAuditEvent` carrying:

- **Decision Metadata**: `decisionId`, `decision` (`ALLOW`/`DENY`), `reason` (`ThemisDecisionReason`), and `policyId`.
- **Principal & Action Lineage**: `principalId`, `principalType`, `action`, `resourceType`, and `resourceId`.
- **Distributed Correlation**: `correlationId` and `causationId` for cross-subsystem trace alignment.
- **Zero PHI Invariant**: Clinical payload bodies and sensitive patient attributes are strictly excluded from audit events.

16.3.4 Fail-Closed Determinism & Reason Code Diagnostics

In the event of an evaluation error, missing policy, or unparseable context, Themis deterministically fails closed to `DENY` and logs the diagnostic code:

- `ALLOWED_BY_POLICY`: Active policy explicitly granted access.
- `EXPLICIT_DENY`: Explicit negative policy rule triggered.
- `DEFAULT_DENY` / `POLICY_NOT_FOUND`: No matching allow rule; default-deny triggered.
- `AUTHORITY_MISSING` / `EXECUTION_AUTHORITY_MISSING`: Principal or activity lacked required capabilities.
- `CONTEXT_MALFORMED` / `SECURITY_SERVICE_ERROR`: Malformed context or internal evaluation failure.

16.4 PHI-Aware Diagnostic Logging Runtime & Appender Isolation

Harmonia implements a centralized runtime framework for governed clinical logging that guarantees zero PHI leakage into operational log streams while enabling rich diagnostic capabilities for engineering troubleshooting.

16.4.1 Dual-Logger Runtime Framework (`Calliope` / `net.fhirfactory.harmonia.logging`)

Every clinical processing module instantiates two distinct loggers:

- **Standard SLF4J Logger**: Emits operational milestones (`INFO`, `WARN`, `ERROR`) carrying non-PHI correlation IDs, transaction status codes, and latency metrics.
- **Harmonia `PhiLogger`**: Emits diagnostic messages exclusively at `DEBUG` and `TRACE` severity. Compile-time enforcement prevents operational logging via `PhiLogger`.

16.4.2 Dual-Gate Security Evaluation Mechanics

Log evaluation through `PhiLogger` is governed by a dual-gate security check:

1. **Gate 1 (Global Configuration Switch)**: Verifies that `harmonia.logging.phi-enabled=true` (or `HARMONIA_LOGGING_PHI_ENABLED=true`). When `false` (the default), all PHI log statements evaluate to strict no-ops.
2. **Gate 2 (Active Severity Level)**: Verifies that the underlying logger is actively configured for `DEBUG` or `TRACE`.

Supplier lambdas (`Supplier<?>`) provide lazy evaluation, ensuring zero serialization overhead or heap allocations when the gates are closed.

16.4.3 SLF4J Namespace Routing & Non-Additive Appender Isolation

When both gates pass, `PhiLogger` routes events to the dedicated SLF4J logger namespace `org.harmonia.phi` with an attached `PHI` marker. Logback configuration enforces `additivity="false"` on this logger, isolating diagnostic logs to encrypted, restricted-access storage volumes (`phi-diagnostic.log`) with strict 7-day retention windows. Complete developer blueprints, API signatures, XML configurations, and testing strategies are cataloged in Appendix 40.

16.5 Master Platform Port Allocation Specification

To ensure complete isolation and zero port collisions across development, test, and containerized production environments, Harmonia enforces the following deterministic port allocation matrix:

Port	Protocol	Target Component	Functional Purpose
2575	MLLP / TCP	`pylai-mllp-in`	Inbound HL7 v2.x clinical trigger event stream.
3000	HTTP	`iris-clinical`	Vue 3 Clinical Exploration User Interface.
3001	HTTP	`iris-console`	Vue 3 Operations & Monitoring Console.
5432	PostgreSQL	`clinical-db-primary`	Primary PostgreSQL clinical database.
5433	PostgreSQL	`clinical-db-replica`	Streaming replica PostgreSQL clinical database.
5434	PostgreSQL	`operations-db-primary`	Primary PostgreSQL operations/task database.
5435	PostgreSQL	`operations-db-replica`	Streaming replica PostgreSQL operations database.
61616	Artemis Core	`artemis-primary-a`	Artemis HA Primary broker A (Messaging).
61617	Artemis Core	`artemis-backup-a`	Artemis HA Backup broker A (Replication).
61618	Artemis Core	`artemis-primary-b`	Artemis HA Primary broker B (Messaging).
61619	Artemis Core	`artemis-backup-b`	Artemis HA Backup broker B (Replication).
8080	HTTP REST	`iris-befe`	Clinical FHIR R5 REST API.
8081	HTTP REST	`mnemosyne-clinical-1`	Clinical JPA persistence server (Primary).
8082	HTTP REST	`mnemosyne-clinical-2`	Clinical JPA persistence server (Secondary).
8083	HTTP REST	`ponos`	Ponos WorkEngine management & telemetry.
8084	HTTP REST	`pylai-mllp-in`	Pylai MLLP gateway health & administration.
8085	HTTP REST	`mnemosyne-operations-1`	Operations JPA persistence server (Primary).
8086	HTTP REST	`mnemosyne-operations-2`	Operations JPA persistence server (Secondary).
8087	HTTP REST	`pylai-mllp-out-his`	Outbound MLLP gateway (HIS instance) REST dispatch & telemetry.
8088	HTTP REST	`pylai-mllp-out-lis`	Outbound MLLP gateway (LIS instance) REST dispatch & telemetry.
8089	HTTP REST	`pylai-fhir-registry`	FHIR R5 Provider Registry REST Gateway API & Task Polling.
8090	HTTP REST	`iris-befe`	Operations and task sequence management REST API.
8161	HTTP Web	`artemis-primary-a`	Artemis Primary A web administration console.
8162	HTTP Web	`artemis-backup-a`	Artemis Backup A web administration console.
8163	HTTP Web	`artemis-primary-b`	Artemis Primary B web administration console.
8164	HTTP Web	`artemis-backup-b`	Artemis Backup B web administration console.
11222	HotRod / TCP	`mneme-node-1`	Infinispan data grid cluster node 1.
11223	HotRod / TCP	`mneme-node-2`	Infinispan data grid cluster node 2.

Table 16.1: Harmonia Master Technology Infrastructure Port Allocation Matrix

Chapter 17

Apache ActiveMQ Artemis Topology

The messaging backbone of Harmonia relies on Apache ActiveMQ Artemis 2.33.0 (packaged using container image `apache/activemq-artemis:2.33.0`). It provides guaranteed message delivery, high-availability journal replication, and dynamic load redistribution across active broker instances.

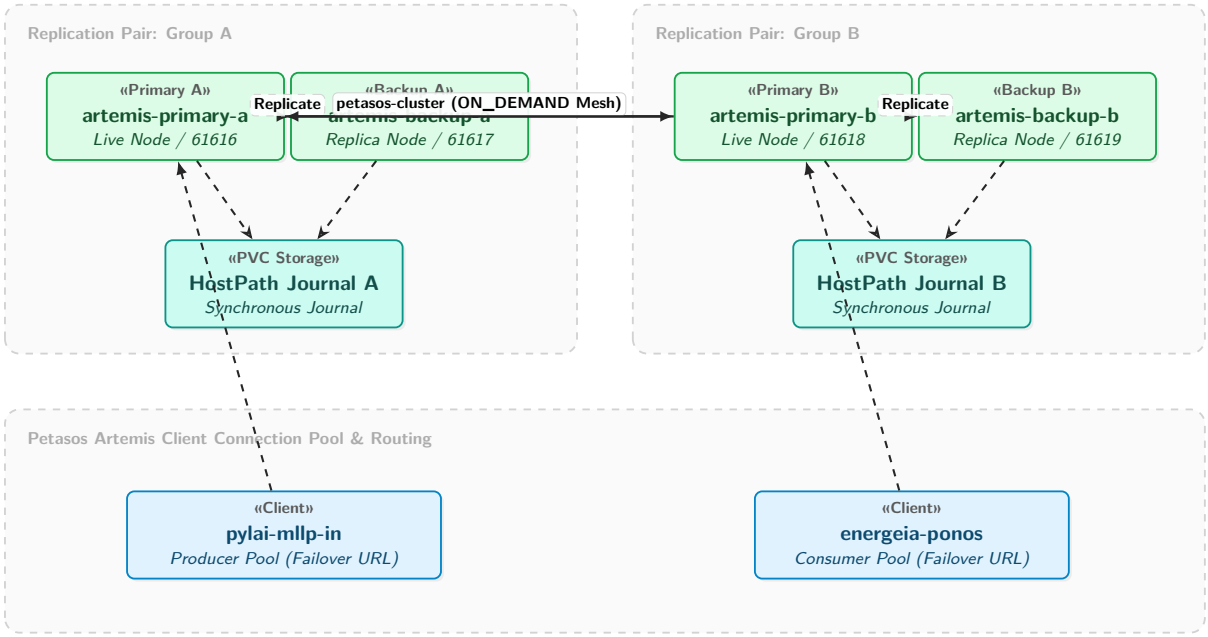


Figure 17.1: ActiveMQ Artemis 2.33.0 4-Broker Clustered HA Messaging Topology and Journal Replication

17.1 4-Broker HA Clustering Topology

Harmonia deploys four broker StatefulSets arranged into two replicated high-availability pairs:

- **Group A**: `artemis-primary-a` (Live Primary) and `artemis-backup-a` (Replicating Backup).
- **Group B**: `artemis-primary-b` (Live Primary) and `artemis-backup-b` (Replicating Backup).

All broker communication—including client connections, synchronous journal replication, and cluster discovery—operates over a unified Netty transport port (**Port 61616**); separate external replication ports are not utilized within the Kubernetes pod networking namespace.

17.1.1 Symmetric Server-Side Clustering (`petasos-cluster`)

Primary brokers establish a symmetric cluster connection mesh (`petasos-cluster`) configured with `ON_DEMAND` message load balancing and instantaneous message redistribution (`redistribution-delay: 0`). When consumer demand fluctuates, messages queued on Primary A are automatically migrated across the cluster bridge to Primary B to maintain balanced processing throughput.

17.1.2 Synchronous Journal Replication

Within each group, the primary broker continuously replicates its journal transactions over a dedicated TCP channel to the passive backup broker. In the event of primary pod termination or node loss, the backup broker automatically assumes the live role within 2 seconds with zero message loss.

17.2 Address & Queue Settings (broker.xml)

Broker addresses and queues are governed by standardized XML configuration in broker.xml:

- **Address Pattern Match**: match="#" applies global delivery policies across all Petasos addresses.
- **Dead-Letter Queue Address**: DLQ (messages exceeding maximum delivery attempts are routed to DLQ).
- **Expiry Address**: ExpiryQueue (expired messages exceeding time-to-live are routed to ExpiryQueue).
- **Redelivery Policy**: Initial redelivery-delay is 1000 ms with a multiplier of 1.5.
- **Maximum Delivery Attempts**: Bounded at max-delivery-attempts="3" before dead-letter routing.
- **Paging Policy**: address-full-policy="PAGE" activates when queue size exceeds 100 MB (104857600 bytes), paging in 10 MB (10485760 byte) chunks.

```
<address-settings>
   <address-setting match="#">
      <dead-letter-address>DLQ</dead-letter-address>
      <expiry-address>ExpiryQueue</expiry-address>
      <redelivery-delay>1000</redelivery-delay>
      <redelivery-delay-multiplier>1.5</redelivery-delay-multiplier>
      <max-delivery-attempts>3</max-delivery-attempts>
      <max-size-bytes>104857600</max-size-bytes>
      <page-size-bytes>10485760</page-size-bytes>
      <address-full-policy>PAGE</address-full-policy>
      <redistribution-delay>0</redistribution-delay>
   </address-setting>
</address-settings>
```

Listing 17.1: Authoritative ActiveMQ Artemis Address Settings in broker.xml

17.3 Durable Journal Persistence & Storage Requirements

Each broker pod attaches a dedicated persistent volume:

- **PVC Claim Size**: 2 GiB per broker (8 GiB total across all 4 brokers).
- **Storage Class**: microk8s-hostpath (backed by local SSD storage).
- **Directory Paths**: ./data/journal, ./data/bindings, ./data/paging, ./data/large-messages.

17.4 Broker-Level Message Deduplication

Artemis natively enforces message deduplication via the _AMQ_DUPL_ID header cache. When Petasos producers publish messages, they attach a deterministic unique ID. Artemis validates incoming IDs against its internal ring cache (default size: 20,000 entries), silently discarding duplicates and returning success.

Chapter 18

Infinispan Distributed Cache Grid

The Mneme caching tier (`hestia/mneme-cluster`) is deployed as a clustered stateful data grid running Infinispan 15.0.3. The runtime container is packaged as the custom repository image `localhost:32000/harmonia/mneme-cluster:1.0.0-SNAPSHOT` (built from `hestia/mneme-cluster/Dockerfile` using upstream base `quay.io/infinispan/server:15.0.3.Final`).

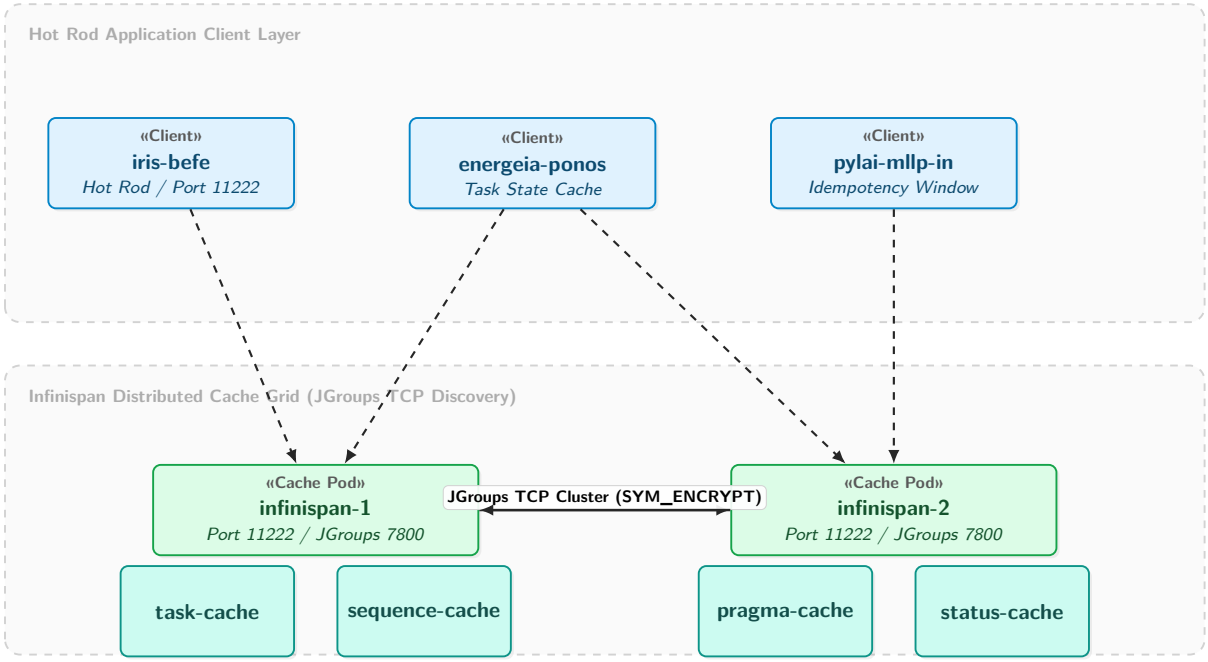


Figure 18.1: Infinispan 15.0.3 Distributed Cache Grid Topology and Hot Rod Integration

18.1 Clustered Grid Topology & Protocols

Harmonia deploys a two-pod StatefulSet (`infinispan-1`, `infinispan-2`) exposed via the Kubernetes service `infinispan-service`:

- **Hot Rod Protocol (Port 11222)**: High-speed binary RPC protocol used by `iris-befe`, `ponos-task-processor`, and `pylai-mllp-in` for sub-millisecond key-value operations and near-cache invalidation.
- **JGroups Cluster Discovery (Port 7800)**: TCP-based peer discovery and symmetric encryption mesh maintaining cluster membership and cache partition state.

18.2 Named Cache Configurations & Semantics

Mneme configures exactly 17 replicated caches in `infinispan.xml`. All 17 caches are configured with synchronous replication (`REPL_SYNC`), `text/plain` key/value encoding, and asynchronous write-behind persistence stores:

18.2.1 Write-Behind Store Architecture (`mneme-persistence`)

To isolate low-latency in-memory cache operations from relational database latency, Mneme implements Infinispan's `NonBlockingStore` SPI:

Cache Name	Mode	Encoding	Store Binding	Target Entity / Operational Role
person-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Person resources (clinical demographics)
relatedperson-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR RelatedPerson resources (kin / contacts)
practitioner-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Practitioner master entities (Provider Registry)
practitionerrole-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR PractitionerRole role bindings
organization-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Organization healthcare facilities
location-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Location wards, beds, and facilities
healthcareservice-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR HealthcareService clinical offerings
group-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Group multidisciplinary care teams
provenance-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Provenance cryptographic lifecycle lineage
auditevent-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR AuditEvent security access events
consent-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Consent privacy directives
task-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Task active workflow tasks
communication-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR Communication collaboration messages
documentreference-cache	REPL_SYNC	text/plain	FhirRestCacheStore	FHIR DocumentReference clinical documents
tasksequence-cache	REPL_SYNC	text/plain	OperationsRestCacheStore	Praxis TaskSequence workflow definitions
messagequeue-cache	REPL_SYNC	text/plain	OperationsRestCacheStore	Petasos message queue state tracking
modulestatus-cache	REPL_SYNC	text/plain	OperationsRestCacheStore	Module health, liveness, and telemetry

Table 18.1: Infinispan 15.0.3 Replicated Named Cache Definitions and Write-Behind Store Bindings

- **Asynchronous Flushing**: Cache mutations are queued in a write-behind buffer (`modification-queue-size="1024"`
- **Dual Store Dispatchers**:
  - `FhirRestCacheStore`: Flushes clinical cache modifications asynchronously via HTTP REST to `http://mnemosyne-clinical-1:8080/fhir/r5`.
  - `OperationsRestCacheStore`: Flushes operational telemetry and sequence updates to `http://mnemosyne-opera`
- **Replication Guarantee**: Because caches use `REPL_SYNC`, every grid node maintains an identical copy in memory, providing instant failover if either node restarts.

18.3 Cache Classification Invariant

Architectural Principle: Operational Cache vs Durable Storage

Mneme is classified strictly as an **operational in-memory cache grid**, not durable authoritative storage. While data is replicated across 2 owners in memory to survive single-pod restarts, long-term clinical durability and compliance audit records are owned exclusively by Mnemosyne PostgreSQL databases.

Chapter 19

PostgreSQL & HAPI FHIR R5 Persistence Architecture

The Mnemosyne subsystem (`hestia/mnemosyne-clinical` and `hestia/mnemosyne-operations`) provides long-term durable persistence, structured querying, and regulatory compliance storage for the Harmonia platform.

19.1 Database Separation: Clinical vs Operations vs Collaboration

Harmonia strictly separates clinical health records from operational telemetry, audit logs, and collaboration state by maintaining three independent PostgreSQL 16 database clusters (image `postgres:16-alpine`):

19.1.1 1. Clinical Database (`postgres-1` & `postgres-2`)

- **Internal Service Port**: 5432
- **Schema Name**: `hie_fhir_resources`
- **Owning Server**: `mnemosyne-clinical` (Spring Boot HAPI FHIR R5 JPA)
- **Stored Entities**: Authoritative FHIR R5 resources (`Patient`, `Practitioner`, `Organization`, `Observation`, `Encounter`, etc.) with complete version histories and search indexes.

19.1.2 2. Operations Database (`postgres-ops-1` & `postgres-ops-2`)

- **Internal Service Port**: 5434
- **Schema Name**: `hie_operations_resources`
- **Owning Server**: `mnemosyne-operations` (Spring Boot Operations JPA)
- **Stored Entities**: Non-clinical `Task` histories, `AuditEvent` logs, `Provenance` lineage, and system configuration records.

19.1.3 3. Collaboration Database (`postgres-synapse`)

- **Internal Service Port**: 5436 (mapped to 5432 in pod)
- **Database Name**: `synapse_db`
- **Owning Services**: Matrix Synapse Homeserver and Agora Core JPA
- **Stored Entities**: Agora resource ID mappings (`agora_resource_mappings`), Application Service transactions (`agora_as_transactions`), and Synapse rooms, state events, and membership rosters.

19.2 JPA Server Deployment Architecture

In the Kubernetes base manifests, the JPA servers are deployed as stateless application Deployments interfacing with the database StatefulSets:

- **Clinical Deployments**: `hapi-fhir-jpa-server-1` and `hapi-fhir-jpa-server-2` (Ports 8080/8081).
- **Operations Deployments**: `hie-operations-jpa-server-1` and `hie-operations-jpa-server-2` (Ports 8080/8085).

19.3 Optimistic Concurrency & Referential Integrity

All FHIR R5 directory entities managed by `mnemosyne-clinical` enforce:

- **Optimistic Locking**: Enforced via integer version IDs (`version_id`) and HTTP `ETag` headers. Stale updates that fail the `If-Match` check are rejected with `PR-VAL-006`.
- **Referential Validation**: Foreign key links (such as `PractitionerRole.practitioner` or `PractitionerRole.organization`) are checked by server-side validators before commit.

Chapter 20

Matrix Synapse Homeserver & Collaboration Middleware

The Agora collaboration gateway interfaces directly with Matrix Synapse 1.120.0 (packaged using container image `matrixdotorg/synapse:v1.120.0`). Synapse serves as Harmonia's internal collaboration homeserver, delivering standardized Matrix Client-Server v3, Administration v2, and Application Service (AS) push protocols.

20.1 Homeserver Clustered Topology & Network Architecture

Synapse is deployed as a single-node stateful workload in the `harmonia` Kubernetes namespace:

- **Container Image**: `matrixdotorg/synapse:v1.120.0`.
- **Workload Type**: Kubernetes StatefulSet (`synapse`) with 1 replica.
- **HTTP Listener Port**: Port **8008** exposing both Client-Server and Application Service APIs.
- **Relational Database**: PostgreSQL 16 database running in StatefulSet `postgres-synapse` on Port **5436** (mapped from container port 5432).
- **Persistent Storage**: 5 GiB PersistentVolumeClaim (`synapse-data`) mounted at `/data` for homeserver media, signing keys, and logs.

20.2 Application Service Registration (`appservice-agora.yaml`)

Synapse is integrated with the Agora gateway via declarative Application Service registration:

```
id: harmonia-agora
url: "http://agora.harmonia.svc.cluster.local:8092"
as_token: "agora_as_token_secret_replace_in_vault"
hs_token: "synapse_hs_token_secret_replace_in_vault"
sender_localpart: "_harmonia_bot"
namespaces:
  users:
    - exclusive: true
      regex: "@_harmonia_.*"
  aliases:
    - exclusive: true
      regex: "#_harmonia_.*"
  rooms:
    - exclusive: false
      regex: ".*"
```

Listing 20.1: Agora Matrix Application Service Registration Manifest

20.2.1 Registration Invariants

1. **Exclusive Namespaces**: Agora asserts exclusive ownership over the `@_harmonia_.*` user namespace and `#_harmonia_.*` room alias namespace. Synapse prevents non-AS users from claiming matching identifiers.
2. **Push Transaction Endpoint**: Synapse dispatches incoming room events to Agora via HTTP `PUT/_matrix/app/v1/transactions/{txnId}` on Port 8092 using the shared `hs_token`.
3. **Service Identity**: System events (such as room invitations and state updates) are enacted using the privileged bot identity `@_harmonia_bot:synapse`.

20.3 Spaces Hierarchy & Zero-PHI Metadata Invariant

Agora manages a strict two-level room hierarchy within Synapse:

- **Parent Space**: A Matrix Space (`creation_content:{type:"m.space"}`) representing an encounter or care team context.
- **Child Rooms**: Exactly four typed child rooms linked via `m.space.child` and `m.space.parent` state events:
  1. **Statistics Room**: Longitudinal metrics, vitals, and trajectory telemetry.
  2. **Tasks Room**: Real-time clinical action items and workflow assignments.
  3. **Discussion Room**: Multidisciplinary care team communication.
  4. **Diagnostics Room**: Pathology and radiology result notifications.

In strict adherence to AGORA-ADR-005, room aliases and names are constructed exclusively using synthetic identifiers and opaque UUIDs (e.g. `#encounter_9b1deb4d3b7d4b89:synapse`). Patient names, national identifiers, and medical record numbers are prohibited from room metadata.

20.4 Ephemeral Retention & Database Archival

In accordance with AGORA-ADR-004:

Architectural Principle: Ephemeral Matrix Retention Invariant

Synapse enforces an automated 90-day retention cleanup policy on all room history. Matrix is treated as a transient collaboration surface; all clinical decisions, care orders, and trajectory summaries must be committed to Mnemosyne FHIR storage before retention purge.

Part V

Runtime Architecture

Chapter 21

Container Packaging & Runtime Profiles

Harmonia standardizes its execution environments using lightweight, containerized runtime profiles running on Eclipse Temurin OpenJDK 21, WildFly 31.0.1, and Spring Boot 3.2.5.

21.1 Container Image Hierarchy & Registry Taxonomy

All container images are built and pushed to the local MicroK8s registry using canonical tags:

`localhost:32000/harmonia/<artifact-name>:1.0.0-SNAPSHOT`

21.1.1 1. Upstream Middleware Base Images

- **ActiveMQ Artemis**: `apache/activemq-artemis:2.33.0`
- **PostgreSQL**: `postgres:16-alpine`
- **Infinispan**: Built from `hestia/mneme-cluster/Dockerfile` using base image `quay.io/infinispan/server:15.0.`

21.1.2 2. Java Application Runtimes

- **WildFly Jakarta EE WAR Runtime**: Used by `iris-befe`, `pylai-mllp-in`, `pylai-mllp-out`, and `ponos-task-processor`; each cited Dockerfile deploys a WAR to WildFly and starts `standalone.sh`.
- **Spring Boot 3.2.5 Runtimes**: Used by `pylai-fhir-registry`, `mnemosyne-clinical`, and `mnemosyne-operations`; the Mnemosyne Dockerfiles use the Eclipse Temurin 21 JRE Jammy image.
- **Nginx Static SPAs**: Alpine-based Nginx images serving compiled Vue 3 assets for `iris-clinical`, `iris-console`, and `iris-administration`.

21.2 JVM Resource Profiling & Memory Ergonomics

Containers configure deterministic memory constraints:

$$\text{Container Memory Limit} = \text{JVM Heap (MaxRAMPercentage=75.0)} + \text{Metaspace} + \text{Native Buffers} \tag{21.1}$$

Chapter 22

MicroK8s Pod Scheduling, Services & Ingress Routing

Harmonia's containerized workloads are orchestrated on Ubuntu MicroK8s using declarative Kustomize base manifests (`deployment/kubernetes/base`) and environment overlays (`deployment/kubernetes/environments/microk8s`

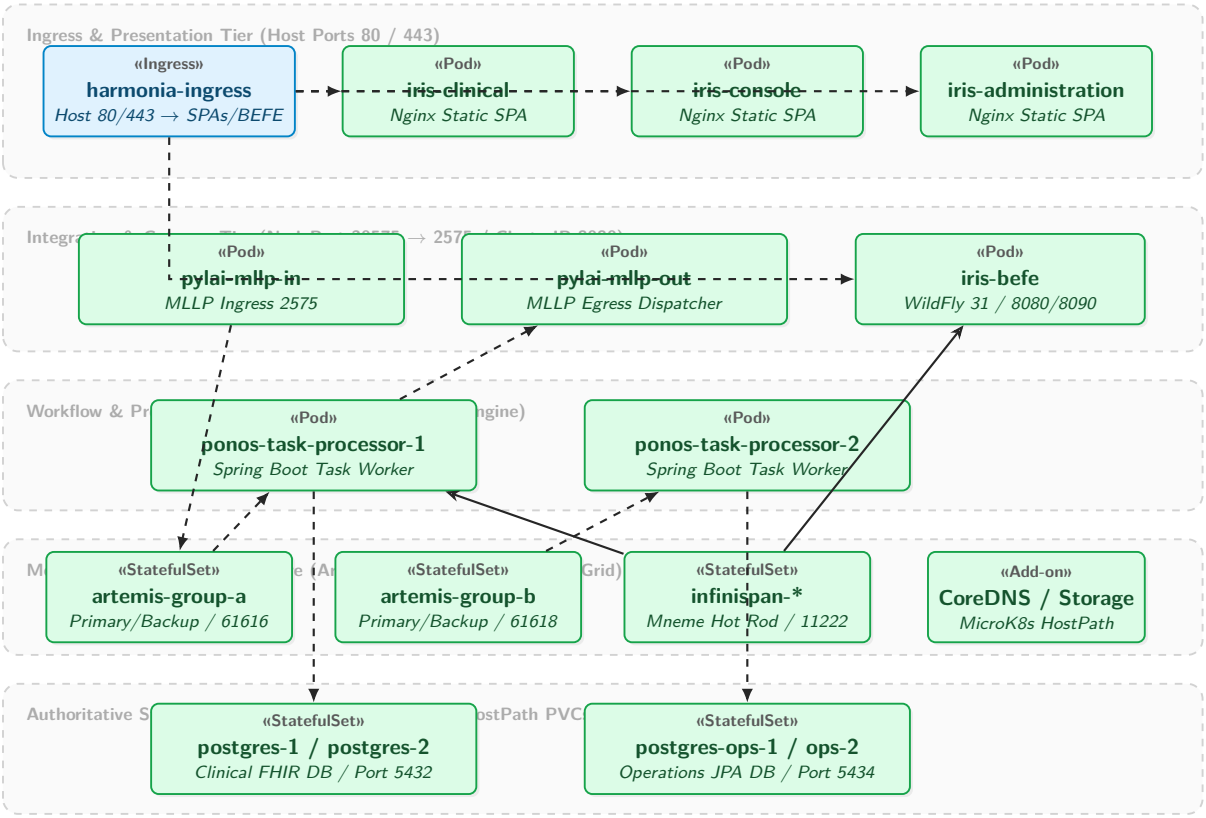


Figure 22.1: MicroK8s 5-Tier Deployment Topology and Add-on Integration

22.1 Service Discovery & Internal DNS Mesh

All internal microservice communication is mediated through Kubernetes ClusterIP Services and CoreDNS resolution:

- **Artemis Services**: `petasos-artemis-discovery`, `artemis-primary-a`, `artemis-backup-a`, `artemis-primary-b`, `artemis-backup-b` (Ports 61616–61619).
- **Infinispan Services**: `infinispan-service`, `infinispan-1`, `infinispan-2` (Hot Rod Port 11222, JGroups Port 7800).
- **PostgreSQL Services**: `postgres-1`, `postgres-2` (Port 5432) and `postgres-ops-1`, `postgres-ops-2` (Port 5434).
- **Application Services**: `mllp-gateway`, `mllp-outbound-his`, `mllp-outbound-lis`, `task-processor`, `hapi-fhir-jpa-server-1/2`, `hie-operations-jpa-server-1/2`, `befe`.

22.2 Ingress Routing & NodePort Mapping

External traffic enters the cluster via two ingress pathways:

1. **HTTP/HTTPS Ingress (Ports 80 / 443)**: The Nginx Ingress Controller routes requests by virtual host and path:
   - `http://clinical.harmonia.local/`: Routes to `iris-clinical` SPA (Port 80).
   - `http://console.harmonia.local/`: Routes to `iris-console` SPA (Port 80).
   - `http://admin.harmonia.local/`: Routes to `iris-administration` SPA (Port 80).
   - `http://api.harmonia.local/clinical`: Routes to `befe` gateway (Port 8080).
   - `http://api.harmonia.local/operations`: Routes to `befe` gateway (Port 8090).
   - `http://gateway.harmonia.local/`: Routes to `mllp-gateway` Actuator (Port 8080).
2. **MLLP / TCP Ingress (NodePort 30575 → 2575)**: External hospital systems send HL7 v2 frames directly to the host's NodePort 30575, forwarding to `mllp-gateway`.

Chapter 23

Single-Node Resilience vs Host-Level Availability

Understanding the operational boundaries of single-node MicroK8s is vital for production planning and disaster recovery engineering.

23.1 Resilience Semantics on Single-Node MicroK8s

In a single-node reference deployment, MicroK8s provides **workload and process-level resilience**:

- **Process Self-Healing**: Kubernetes kubelet monitors container liveness and restarts failed or unresponsive microservice processes automatically.
- **Rolling Updates**: Application deployments update pods sequentially with zero service interruption.
- **Memory & CPU Fencing**: Cgroup resource requests and limits prevent memory leaks in one container from crashing neighboring services.
- **Middleware Failover Testing**: The multi-pod middleware configurations (4 Artemis brokers, 2 Infinispan pods, 2 Postgres instances) validate clustering logic, replication channels, and network reconnect loops in a realistic containerized topology.

23.2 Host-Level Failure Domain Boundaries

Architectural Note

Host Failure Limitation: A single physical Ubuntu server represents a single physical failure domain. If the host machine suffers power loss, kernel panic, or physical SSD failure, all platform pods and services become unavailable simultaneously. Single-node MicroK8s does **not** protect against host hardware failure.

23.3 Production Multi-Node Scaling Pathway

To achieve true host-level High Availability (HA) in production environments:

1. Form a multi-node MicroK8s cluster (minimum 3 control-plane nodes).
2. Distribute Artemis Primary/Backup pairs across distinct physical nodes using Kubernetes `podAntiAffinity`.
3. Replace local `microk8s-hostpath` storage classes with distributed block storage (e.g., Ceph RBD, Rook, or enterprise SAN CSI drivers).

Chapter 24

Physical & High Availability Deployment

The Physical and Deployment Architecture layer specifies the high-availability infrastructure topology, Apache ActiveMQ Artemis multi-broker replication groups, Infinispan cache grid clustering, Docker Compose deployment configurations, and failover/recovery mechanics.

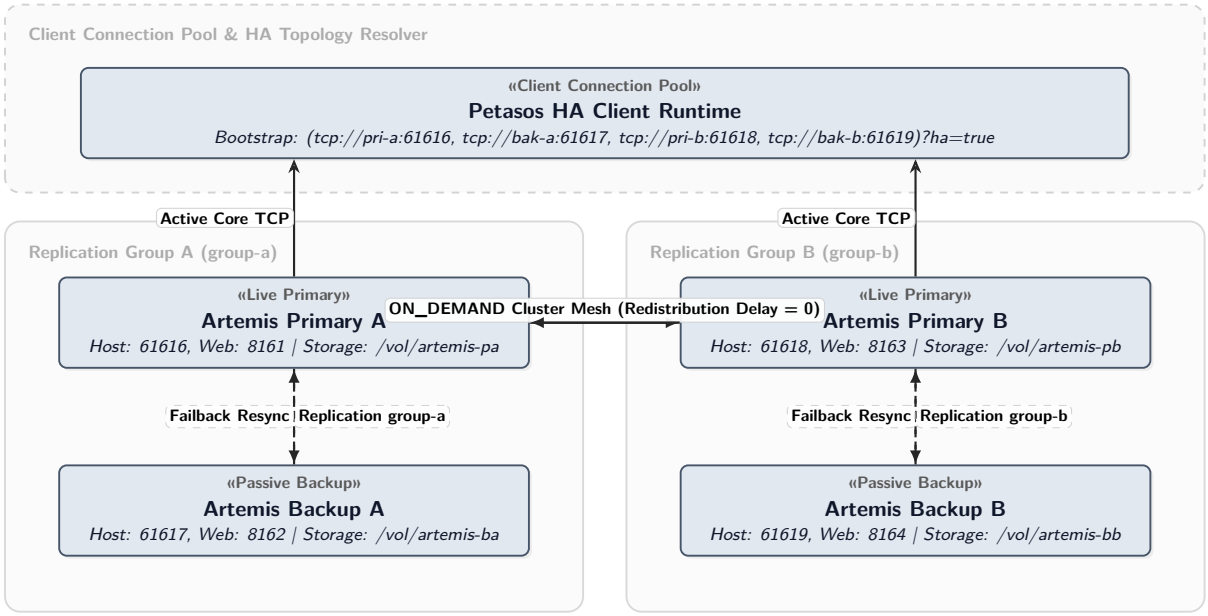


Figure 24.1: ArchiMate Physical Deployment & Multi-Broker ActiveMQ Artemis HA Architecture

24.1 Clustering vs. High Availability (HA) Replication

Harmonia establishes a clear architectural distinction between horizontal clustering and instance-level replication:

24.1.1 1. Server-Side Clustering (`petasos-cluster`)

- **Active-Active Mesh**: Unites multiple live brokers (`Primary A` and `Primary B`) into a distributed message routing fabric.
- **Dynamic Load Distribution (`ON_DEMAND`)**: Messages published to `Primary A` are dynamically forwarded across the cluster bridge to `Primary B` if consumers exist on `Primary B` and not on `Primary A`.
- **Immediate Message Redistribution (`redistribution-delay = 0`)**: When all consumers on one broker disconnect, unconsumed messages are immediately redistributed to alternative nodes with active consumers, preventing queue starvation.
- **Horizontal Scalability**: Additional Primary/Backup pairs can be introduced dynamically without disrupting existing message flows.

24.1.2 2. Journal Replication Groups (`group-a`, `group-b`)

- **NIO Journal Streaming**: An active Primary continuously streams file journal updates over TCP to its paired passive Backup replica.

- **Activation on Failure**: If `Primary A` terminates or loses network connectivity, `Backup A` detects connection loss within 10 seconds and promotes itself to become the live active broker for `group-a`.
- **Automatic Failback (`allow-failback = true`)**: When the failed Primary recovers, it connects to the active Backup, resynchronizes any journal deltas written during downtime, and gracefully resumes the Primary role.

24.2 Isolated Persistent Storage Ownership

To eliminate single points of failure and storage lock contention, multiple brokers **never share journal directories or filesystems**. Each broker instance owns an independent persistent Docker volume:

Instance Name	HA Role	Docker Volume	Isolated Storage Path
`artemis-primary-a`	Live Primary A	`artemis_primary_a_data`	`/var/lib/artemis/data`
`artemis-backup-a`	Replicating Backup A	`artemis_backup_a_data`	`/var/lib/artemis/data`
`artemis-primary-b`	Live Primary B	`artemis_primary_b_data`	`/var/lib/artemis/data`
`artemis-backup-b`	Replicating Backup B	`artemis_backup_b_data`	`/var/lib/artemis/data`

Table 24.1: ActiveMQ Artemis Broker Storage Isolation and Persistent Volume Mapping

24.3 Client Reconnection & Failover Configuration

All Harmonia modules connect to Petasos using a multi-endpoint high-availability bootstrap URL:

```
(tcp://artemis-primary-a:61616,tcp://artemis-backup-a:61617,tcp://artemis-primary-b
    :61618,tcp://artemis-backup-b:61619)?ha=true&reconnectAttempts=-1&retryInterval
    =500&maxRetryInterval=2000&retryIntervalMultiplier=1.5&connectionTTL=60000&
    clientFailureCheckPeriod=10000
```

Listing 24.1: Petasos Multi-Broker HA Client Bootstrap URL

- `ha=true`: Enables automatic cluster topology discovery. The client continuously receives live node lists from the brokers.
- `reconnectAttempts=-1`: Directs the client to attempt reconnection indefinitely during transient network partitions.
- `retryInterval=500` & `maxRetryInterval=2000`: Exponential backoff with jitter prevents thundering herd reconnection storms on broker reboot.
- `failoverListener`: The `PetasosArtemisAdapter` registers failover lifecycle listeners to record reconnect telemetry and ensure in-flight transaction integrity.

24.4 In-Memory Cache Grid HA Topology (`Mneme`)

The `Mneme` distributed cache grid deploys across two clustered Infinispan nodes:

- **Replication Strategy**: Caches are configured with `REPL_SYNC` mode across `mneme-node-1` (port `11222`) and `mneme-node-2` (port `11223`).
- **HotRod Client Smart Routing**: Clients maintain TCP connections to both nodes, automatically routing read/write operations to the nearest available replica.
- **Node Crash Resilience**: If either node crashes, the surviving node serves read and write traffic without data loss or interruption.

24.5 Docker Compose Multi-Container Deployment

The complete Harmonia platform stack is orchestrated via a unified `docker-compose.yml` containing:

- 4 PostgreSQL 16 relational database containers (2 Clinical nodes, 2 Operations nodes) with persistent isolated volumes.
- 4 Spring Boot Mnemosyne JPA persistence servers (2 Clinical HAPI FHIR nodes, 2 Operations nodes).

- 2 Infinispan 15+ distributed cache grid nodes (`infinispan-1`, `infinispan-2`).
- 6 Gateway and workflow application containers (`pylai-mllp-in`, `pylai-mllp-out-his`, `pylai-mllp-out-lis`, `pylai-fhir-registry`, `ponos`, `iris-befe`).
- 2 Vue 3 / Nginx frontend web containers (`iris-clinical`, `iris-console`).

Health checks with configurable start periods ensure that dependent containers only boot once their prerequisite messaging, caching, and database services report operational readiness.

24.5.1 Provider Registry High Availability & Crash Recovery

The FHIR Provider Registry subsystem leverages the clustered Petasos messaging fabric and durable Mnemosyne relational storage to guarantee crash resilience:

- **Durable Queue Storage**: Change requests published to `harmonia.provider.registry.change.request` are written to the ActiveMQ Artemis persistent journal before `HTTP 202 Accepted` is returned to the client.
- **In-Flight Process Crash Recovery**: If the Ponos container restarts or terminates during Ergon execution, Artemis retains unacknowledged change requests. Upon broker reconnect, the task is redelivered and re-executed idempotently without generating duplicate resources.
- **Stateless REST Gateways**: The `pylai-fhir-registry` gateway operates completely statelessly. Multiple instances can be deployed behind a round-robin load balancer for seamless scale-out and zero-downtime rolling upgrades.

24.5.2 Egress Failure Domain Isolation

The multi-instance outbound architecture guarantees complete failure domain isolation across external clinical systems. Each egress instance (`mllp-outbound-his`, `mllp-outbound-lis`) consumes from its own dedicated Petasos queue (e.g. `petasos.queue.mllp.outbound.his`). If a destination hospital system experiences downtime, network unreachability, or socket timeouts:

- Egress queue backlog is strictly contained within that destination's specific queue without affecting inbound processing or alternate outbound destinations.
- Thread pools and socket resources in the faulting gateway instance remain isolated, preventing thread starvation or memory leaks from propagating to other services.
- Ponos task sequences can continue dispatching messages to healthy clinical destinations concurrently.

Chapter 25

Themis Policy Evaluation & Security Boundaries

The Themis security subsystem (`themis/themis-core` and `themis/themis-api`) provides centralized Attribute- and Role-Based Access Control (ABAC/RBAC) across all Harmonia ingress interfaces, queue dispatchers, and persistence layers.

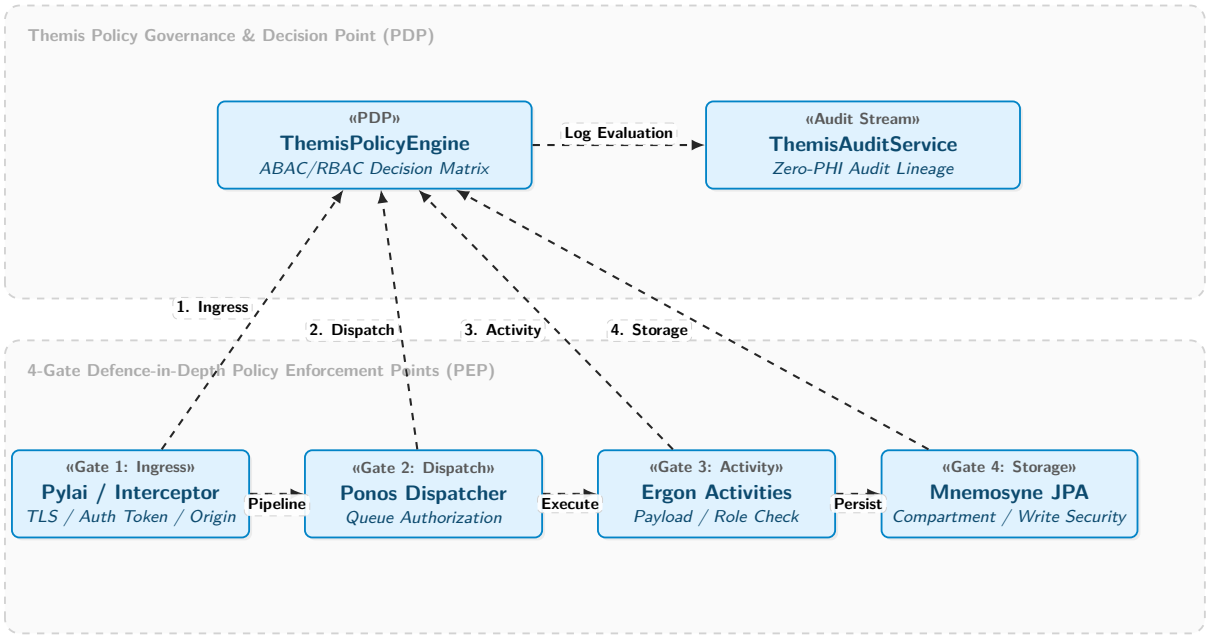


Figure 25.1: Themis 4-Gate Defence-in-Depth Authorization Pipeline and PDP Evaluation

25.1 Default-Deny Policy Invariant

Architectural Principle: Default-Deny Security Enforcement

In accordance with Invariant 6, all integration endpoints, task processors, and persistence operations default to `ThemisDecision.DENY`. Access is granted if and only if an explicit Themis policy rule evaluates to `PERMIT` for the authenticated security context.

25.2 The 4-Gate Defence-in-Depth Pipeline

Security enforcement is applied at four successive gates:

1. **Gate 1 — Ingress Screening**: `FhirSecurityInterceptor` validates TLS certificates, JWT bearer tokens, and client IP origins.
2. **Gate 2 — Dispatch Authorization**: Ponos validates whether the authenticated principal has permission to publish to or consume from target Petasos queues.
3. **Gate 3 — Activity Execution**: Individual Ergon activities evaluate payload-specific security labels and user scopes.

4. **Gate 4 — Storage Compartment Enforcement**: Mnemosyne JPA mutators verify that the write operation complies with resource compartment and tenancy rules.

Chapter 26

PHI-Sanitized Logging & Audit Lineage Tracking

Safeguarding Protected Health Information (PHI) while maintaining actionable diagnostic observability requires a strict architectural separation between operational logs and diagnostic audit streams.

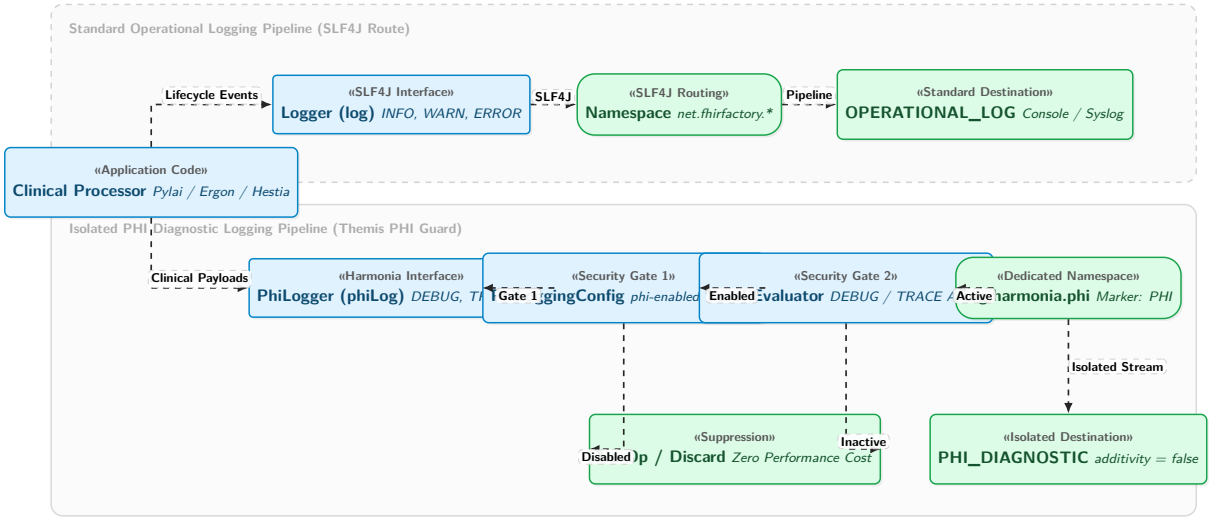


Figure 26.1: Dual-Gate PHI-Aware Logging Architecture and Isolated Logback Routing

26.1 Zero-PHI Logging Invariant

Architectural Principle: Zero-PHI Operational Log Invariant

In accordance with Invariant 7:

1. Standard application log streams (`INFO`, `WARN`, `ERROR`) must **never** contain unmasked Protected Health Information (patient names, street addresses, dates of birth, clinical notes, or observation values).
2. Standard log statements are restricted to technical identifiers (message UUIDs, transaction correlation IDs, anonymized MRNs) and operational timing metrics.

26.2 Dual-Logger Pattern & Non-Additive Routing

When deep diagnostic tracing is required:

- Components utilize the dedicated `PhiLogger` facade.
- Log events containing clinical payloads are routed strictly to the isolated `org.harmonia.phi` logger namespace.
- Logback configuration sets `additivity="false"`, routing diagnostic payloads exclusively to encrypted, access-controlled audit files and preventing leakage into standard console streams.

Part VI

Normal Deployment

Chapter 27

Declarative Configuration Hierarchy

Harmonia implements a 5-stage declarative configuration hierarchy ensuring clean separation between code, infrastructure settings, and sensitive cryptographic secrets.

27.1 The 5-Stage Configuration Hierarchy

Configuration properties are resolved according to strict precedence rules:

1. **Stage 1: Application Base Defaults**: Hardcoded baseline fallbacks embedded within Java `application.properties` or WildFly `standalone.xml`.
2. **Stage 2: Declarative ConfigMaps**: Environment-specific parameters (e.g., hostnames, port bindings, timeouts, queue names) declared in Kubernetes ConfigMaps.
3. **Stage 3: Secret Injection & Vault**: Sensitive credentials (database passwords, Artemis security credentials, TLS private keys) mounted as Kubernetes Secrets or populated from HashiCorp Vault / Ansible Vault.
4. **Stage 4: Container Environment Variables**: Explicit environment variables injected into container pods by Kubernetes manifest overlays.
5. **Stage 5: Command-Line Overrides**: Ephemeral CLI flags and JVM system properties (`-Dprop=value`) used for localized diagnostic sessions.

27.2 Secret Management & Zero-Plaintext Policy

Harmonia strictly forbids plaintext passwords or API keys in source code or unencrypted Git repositories:

- In development/testing, secrets are managed via sealed Kubernetes Secret manifests.
- In production, secrets are dynamically fetched at startup using external secret operators or Ansible Vault injection during deployment.

Chapter 28

Configuration Register & Property Cross-Reference

This chapter details the primary application and infrastructure configuration parameters across all Harmonia subsystems.

28.1 Core Platform Properties Matrix

Subsystem	Property Name	Default Value	Description & Scope
Petasos	`petasos.artemis.broker-url`	`(tcp://...)?ha=true`	Artemis failover connection URL for client pools.
Petasos	`petasos.dedup.window-minutes`	`30`	In-memory deduplication sliding window duration.
Mneme	`infinispan.client.hotrod.server_list`	`infinispan-service:11222`	Hot Rod server endpoint list.
Mneme	`infinispan.cache.task.owners`	`2`	Number of cluster owners for in-flight task cache.
Mnemosyne	`spring.datasource.url`	`jdbc:postgresql://...:5432`	PostgreSQL JDBC connection string.
Mnemosyne	`spring.datasource.hikari.maximum-pool-size`	`20`	Maximum JDBC connection pool limit.
Pylai	`pylai.mllp.port`	`2575`	Inbound MLLP TCP listening port.
Pylai	`pylai.mllp.auto-ack`	`false`	Disables Camel auto-ACK to ensure dual-write safety.
Iris	`iris.befe.clinical-port`	`8080`	BEFE JAX-RS clinical endpoint port.
Iris	`iris.befe.ops-port`	`8090`	BEFE operational telemetry endpoint port.

Table 28.1: Primary Platform Configuration Parameters Cross-Reference

Chapter 29

Ubuntu Host Preparation & MicroK8s Add-ons

A production-grade Harmonia installation on Ubuntu Linux requires baseline host system tuning, package prerequisites, and specific MicroK8s add-on configurations.

29.1 Hardware Prerequisites & OS Requirements

- **Operating System**: Ubuntu 22.04 LTS or Ubuntu 24.04 LTS (x86_64 / 64-bit).
- **CPU / Compute**: Minimum 8 vCPUs (16 vCPUs recommended for multi-tenant deployments).
- **Memory (RAM)**: Minimum 32 GiB RAM (64 GiB recommended).
- **Disk Storage**: Minimum 100 GiB NVMe SSD mounted on `/var/snap/microk8s/common/default-storage`.

29.2 Kernel Parameter & Sysctl Tuning

High-throughput networking and Infinispan memory-mapped files require host sysctl adjustments in `/etc/sysctl.d/99-harmonia.conf`:

```
# Allow Infinispan memory-mapped files
vm.max_map_count=262144

# Increase file descriptor limits
fs.file-max=2097152

# TCP socket buffer tuning for high-throughput MLLP
net.core.somaxconn=4096
net.ipv4.tcp_max_syn_backlog=4096
```

Listing 29.1: Harmonia Host Kernel Tuning Parameters

29.3 MicroK8s Installation & Required Add-ons

Harmonia requires MicroK8s 1.28+ with five essential add-ons enabled:

```
# Install MicroK8s Snap
sudo snap install microk8s --classic --channel=1.28/stable

# Add user to microk8s group
sudo usermod -a -G microk8s $USER
newgrp microk8s

# Enable mandatory Harmonia add-ons
microk8s enable dns storage ingress registry:size=30Gi metrics-server
```

Listing 29.2: MicroK8s Setup and Add-on Activation Commands

Chapter 30

Normal Workload Deployment & Verification Runbook

This chapter provides the step-by-step engineering runbook for deploying and verifying a normal production Harmonia instance on MicroK8s.

30.1 Phase 1: Build & Container Image Push

Build all subprojects and push images to the local MicroK8s registry:

```
# Execute full Maven build and container packaging
mvn clean package -DskipTests -Pcontainer-build

# Push tagged images to MicroK8s registry
docker push localhost:32000/harmonia/pylai-mllp-in:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/pylai-mllp-out:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/ponos-task-processor:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/mneme-cluster:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/mnemosyne-clinical:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/mnemosyne-operations:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/iris-befe:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/iris-clinical:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/iris-console:1.0.0-SNAPSHOT
docker push localhost:32000/harmonia/iris-administration:1.0.0-SNAPSHOT
```

Listing 30.1: Maven Build and Container Image Generation

30.2 Phase 2: Apply Kustomize Overlay Manifests

Deploy the complete namespace, middleware StatefulSets, and application pods:

```
# Apply MicroK8s environment overlay via Kustomize
kubectl apply -k deployment/kubernetes/environments/microk8s

# Monitor pod rollout across namespace
kubectl rollout status statefulset/artemis-primary-a -n harmonia
kubectl rollout status statefulset/infinispan-1 -n harmonia
kubectl rollout status deployment/befe -n harmonia
```

Listing 30.2: Applying Harmonia MicroK8s Manifests

30.3 Phase 3: Verification & Health Validation Runbook

Execute the following verification sequence to confirm cluster readiness:

```
# 1. Check all pods are in Running state
kubectl get pods -n harmonia

# 2. Verify Ingress endpoints
curl -k https://localhost/health

# 3. Verify MLLP Inbound Port is listening
nc -zv localhost 30575

# 4. Verify Iris BEFE Actuator health
curl http://localhost:8080/actuator/health
```

Listing 30.3: Cluster Readiness and Health Probe Verification

Chapter 31

Safe Undeployment & Storage Retention Mechanics

To prevent accidental data loss during maintenance, cluster upgrades, or redeployments, Harmonia implements a strict two-stage teardown protocol separating workload termination from persistent volume reclamation.

31.1 Workload Termination (Data-Preserving)

Executing standard manifest deletion removes all stateless Deployments and stateful pods while deliberately leaving PersistentVolumeClaims (PVCs) intact:

```
# Delete workloads and services without destroying PVCs
kubectl delete -k deployment/kubernetes/environments/microk8s
```

Listing 31.1: Data-Preserving Workload Teardown

Following this command:

- Artemis broker journal files on `microk8s-hostpath` remain preserved.
- PostgreSQL database data directories remain intact on disk.
- Subsequent re-execution of `kubectl apply -k` reconnects newly scheduled pods to existing volumes, resuming operations without data loss.

31.2 Authoritative Storage Wipe Protocol

When an intentional, permanent teardown of the cluster (including all clinical databases and journals) is required:

```
# 1. Delete all PVCs in the harmonia namespace
kubectl delete pvc --all -n harmonia

# 2. Delete namespace
kubectl delete namespace harmonia

# 3. Clean host storage directory
sudo rm -rf /var/snap/microk8s/common/default-storage/harmonia-*
```

Listing 31.2: Permanent Storage Deletion Commands

Chapter 32

Platform Infrastructure, MicroK8s & Middleware Configuration

Authoritative Platform Specification, MicroK8s Add-on Architecture, Declarative Middleware-configuration, Middleware Parameters, and Operational Automation Runbooks

32.1 Platform Overview & Infrastructure Separation Strategy

The **Harmonia Health Information Exchange (HIE)** platform is designed to operate seamlessly across bare-metal servers, virtualized instances, and enterprise cloud infrastructure. To achieve high operational predictability, rapid fault isolation, and reproducible deployments with zero configuration drift, Harmonia enforces a strict five-stage architectural separation of concerns between runtime application logic and infrastructure orchestration:

32.1.1 Architectural Separation of Concerns

The separation model guarantees that software artifacts remain completely environment-agnostic while enabling automated provisioning across development, testing, staging, and production environments:

1. **Harmonia Application Logic (Java 21 / Spring Boot 3 / WildFly 31 / Vue 3)**: All business logic, FHIR resource modeling, transactional state machines, and cryptographic authorization routines are contained in upstream source modules. Code contains zero hardcoded hostnames, distribution-specific hooks, or environment-conditional branching (e.g. `if (microk8s)`).
2. **OCI Container Images**: Standard Open Container Initiative (OCI) images are packaged using Maven container profiles (`-Pcontainer-build`, `-Pcontainer-publish`) with standardized entrypoints, unprivileged service execution accounts, and parameterization via POSIX environment variables.
3. **Authoritative Kubernetes Manifests (Kustomize Base)**: Located in `deployment/kubernetes/base/`, these declarative YAML resources define the cluster blueprint: Deployments, StatefulSets, ClusterIP Services, Ingress objects, NetworkPolicies, and PersistentVolumeClaims.
4. **Environment Overlays (MicroK8s / Multi-Node)**: Located in `deployment/kubernetes/environments/`, declarative Kustomize overlays apply environment-specific patches (such as the `microk8s-hostpath` storage class or node port assignments) without modifying base manifests.
5. **Ansible Automation Engine**: Located in `deployment/ansible/`, automated playbooks orchestrate host-level sysctl kernel tuning, snap package lifecycle, add-on activation, secret injection from encrypted vaults, and rollout verification loops.

Architectural Principle: 1. Declarative Infrastructure as Code (IaC) Invariant

No deployment, configuration change, or volume binding may be executed manually via ad-hoc terminal interventions. All cluster state must originate from declarative Kustomize manifests and Ansible playbooks under strict Git version control.

32.1.2 Single-Node MicroK8s Resilience Semantics vs Multi-Node Scaling

Harmonia designates single-node **MicroK8s on Ubuntu Linux** as its authoritative baseline deployment target. In this topology:

- **Pod & Process Restart Resilience**: The Kubernetes kubelet and containerd runtime automatically recover crashed application containers, microservices, and database processes with sub-second restart loops.

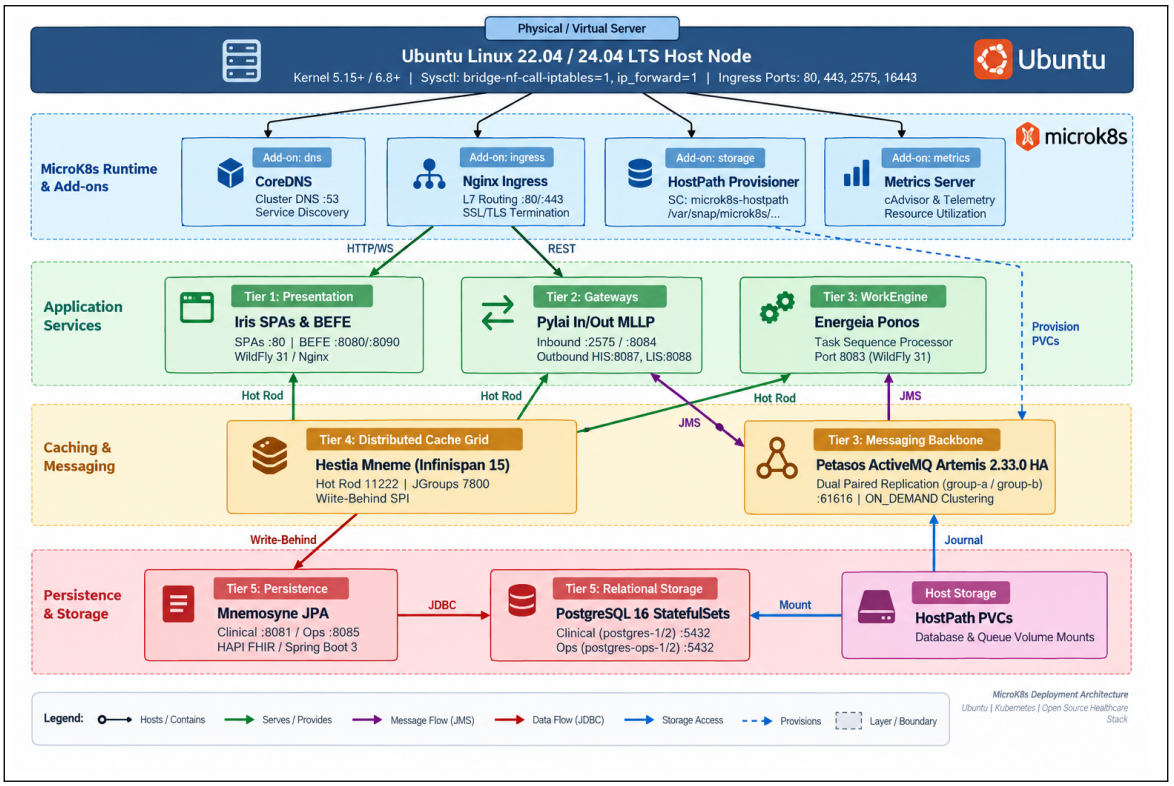


Figure 32.1: ArchiMate 3.2 Physical Infrastructure, MicroK8s Add-ons, and 5-Tier Workload Deployment Topology

- **State Persistence across Node Reboots**: StatefulSets (Artemis messaging journals, PostgreSQL database tables, and Infinispan cache partitions) mount persistent volumes through the local `microk8s-hostpath` provisioner, preserving transactional integrity across operating system restarts.
- **Zero-Friction Multi-Node Evolution**: Because manifests in `deployment/kubernetes/base/` use standard Kubernetes API primitives, transitioning to multi-node clusters requires only substituting the `microk8s-hostpath` StorageClass with a CSI network provisioner (e.g. Ceph RBD, AWS EBS, NFS) and adding `podAntiAffinity` scheduling constraints.

32.2 Platform & Host Infrastructure Prerequisites

32.2.1 Supported Operating Systems & Architecture Support

Harmonia is tested and certified on canonical enterprise Linux distributions:

- **Ubuntu Linux 22.04 LTS (Jammy Jellyfish)**: 64-bit x86 (`x86_64` / `amd64`) and 64-bit ARM (`aarch64` / `arm64`).
- **Ubuntu Linux 24.04 LTS (Noble Numbat)**: 64-bit x86 (`x86_64` / `amd64`) and 64-bit ARM (`aarch64` / `arm64`).

Automated host initialization via `deployment/ansible/roles/microk8s/tasks/prerequisites.yml` ensures the installation of core system utilities: `snapd`, `iptables`, `curl`, `conntrack`, `socat`, and `ca-certificates`.

32.2.2 Hardware Sizing Profiles & Resource Allocation Matrix

Harmonia provides three standardized sizing profiles calibrated for varying computational demands:

32.2.3 Linux Kernel Networking & Sysctl Configuration

Kubernetes container networking and high-volume clinical messaging require specific Linux kernel parameters to prevent packet drops, connection tracking exhaustion, and file descriptor starvation:

Sizing Profile	vCPU Cores	RAM	Storage (SSD/NVMe)	Operational Scope & Use Case
Minimum Reference	4 vCPU	8 GB	40 GB	**CI/CD & Functional Verification**: Single-node functional smoke testing, integration test pipelines, and developer environments.
Recommended Dev/Test	8 vCPU	16 GB	100 GB	**Integration & Load Simulation**: End-to-end multi-module testing, MLLP message simulation, and cache clustering verification.
Production Target	16+ vCPU	32+ GB	250+ GB (Fast NVMe)	**High-Throughput Clinical HIE**: Sustained HL7/FHIR ingest streams, dual Artemis HA journal replication, and write-behind persistence bursts.

Table 32.1: Harmonia Host Platform Sizing Profiles and Resource Allocation Matrix

32.2.4 Host Firewall, Ingress Routing & Port Exposure Matrix

Table 32.3 defines the network ingress and inter-service port allocations that must be open on the host operating system firewall (e.g. `ufw` or cloud security groups).

32.3 MicroK8s Component Architecture & Add-on Specification

32.3.1 Snap Packaging, Confinement & Permission Management

MicroK8s is deployed via Canonical's `snapd` package manager using classic confinement:

- **Snap Channels**: Harmonia supports the `1.28/stable` baseline channel (configured in `deployment/ansible/roles/m` and `1.30/stable` target channel.
- **Classic Confinement (`-classic`)**: Grants MicroK8s full access to host storage devices, network interfaces, and kernel cgroups necessary for unhindered container orchestration.
- **Group Permissions**: The operational user (default `ubuntu`) is assigned to the `microk8s` POSIX group, granting unprivileged access to the `microk8s` CLI without requiring `sudo` elevation.
- **Kubeconfig Synchronization**: The playbook automatically initializes `~/.kube/config` with client certificates extracted from `/var/snap/microk8s/current/credentials/client.config`.

32.3.2 Authoritative Add-on Matrix

Harmonia requires four fundamental MicroK8s add-ons, systematically validated and activated by `deployment/ansible/roles/microk8s/tasks/addons.yml`:

32.3.3 CoreDNS Add-on (`dns`) Configuration

CoreDNS runs as a cluster-internal deployment in the `kube-system` namespace. It provides authoritative DNS resolution for the `cluster.local` domain while forwarding unhandled upstream domain lookups to the host system's resolver defined in `/etc/resolv.conf`.

```
# Internal Resolution Examples:
infinispan-1.harmonia.svc.cluster.local       -> 10.152.183.42:11222
postgres-1.harmonia.svc.cluster.local         -> 10.152.183.88:5432
```

Kernel Parameter (sysctl)	Value	Subsystem Impact	Technical Justification
net.bridge.bridge-nf-call-iptables	1	Bridge Netfilter	Forces bridged IPv4 traffic through host iptables chains for Kubernetes CNI packet filtering.
net.bridge.bridge-nf-call-ip6tables	1	Bridge Netfilter IPv6	Forces bridged IPv6 packets through ip6tables filtering rules.
net.ipv4.ip_forward	1	IPv4 Forwarding	Enables IP packet forwarding between the physical host network adapter and virtual CNI pod network interfaces.
fs.inotify.max_user_watches	524288	Inotify Subsystem	Prevents filesystem event watcher exhaustion across containerized WildFly deployments and Hot Rod log listeners.
vm.max_map_count	262144	Virtual Memory Mappings	Ensures sufficient memory map areas for Infinispan 15 data grid off-heap memory allocations and JVM thread stacks.
vm.swappiness	10	Memory Swapping Policy	Minimizes aggressive swap activity to preserve deterministic low latency across Petasos Artemis and Mneme cache nodes.

Table 32.2: Authoritative Linux Kernel Parameters and Sysctl Network Tuning

Port / Protocol	Exposure	Target Component	Functional Description
80/tcp	External	Nginx Ingress	Plaintext HTTP ingress routing for Iris SPAs and BEFE APIs.
443/tcp	External	Nginx Ingress	TLS-terminated HTTPS ingress for production browser interfaces.
2575/tcp	External	Pylai Inbound MLLP	Minimal Lower Layer Protocol (MLLP) clinical HL7 v2 ingest gateway.
16443/tcp	Bastion/LAN	MicroK8s Kube-API	MicroK8s Kubernetes control plane API server for kubectl and Ansible.
10.1.0.0/16	Cluster Internal	Calico / Host CNI	Internal Pod network overlay CIDR for inter-pod routing.
10.152.183.0/24	Cluster Internal	Kube-Proxy / Services	Internal ClusterIP service virtual IP network range.

Table 32.3: Host Network Port Matrix and Firewall Exposure Rules

Add-on Name	Underlying Technology	Cluster Address / Target	Role & Platform Justification
`dns`	CoreDNS 1.10+	`10.152.183.10:53` (UDP/TCP)	Provides dynamic, resilient service discovery for internal microservice resolution (e.g. `infinispan-1.harmonia.svc.cluster.local`).
`ingress`	Ingress-Nginx 1.9+	Host Ports `80/tcp`, `443/tcp`	Delivers centralized Layer 7 ingress routing, proxy buffer management, and SSL/TLS termination for SPAs and REST endpoints.
`hostpath-storage`	Dynamic HostPath Provisioner	`/var/snap/microk8s/common/default-storage/`	Satisfies PersistentVolumeClaims (PVCs) by provisioning dedicated local host directories under StorageClass `microk8s-hostpath`.
`metrics-server`	Kubernetes Metrics Server	Kubelet cAdvisor Telemetry	Exposes real-time CPU and memory utilization metrics to `kubectl top` and automated deployment verification gates.

Table 32.4: Authoritative MicroK8s Add-on Specification and Capabilities

```
task-processor.harmonia.svc.cluster.local      -> 10.152.183.95:61616
befe.harmonia.svc.cluster.local                -> 10.152.183.110:8080
```

Listing 32.1: CoreDNS Upstream Forwarding and Service Resolution Scheme

32.3.4 Nginx Ingress Controller (`ingress`) Specification

The Nginx Ingress Controller routes external traffic to the 5-tier workloads. Harmonia's base ingress resource (`deployment/kubernetes/base/ingress.yaml`) configures specific proxy buffer and timeout annotations to accommodate clinical throughput:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: harmonia-ingress
  namespace: harmonia
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "64m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "120"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "120"
spec:
  rules:
    - host: clinical.harmonia.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: iris-clinical, port: { number: 80 } }
    - host: console.harmonia.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: iris-console, port: { number: 80 } }
    - host: admin.harmonia.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: iris-administration, port: { number: 80 } }
    - host: api.harmonia.local
      http:
        paths:
```

```
          - path: /clinical
            pathType: Prefix
            backend:
              service: { name: befe, port: { number: 8080 } }
          - path: /operations
            pathType: Prefix
            backend:
              service: { name: befe, port: { number: 8090 } }
    - host: gateway.harmonia.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: mllp-gateway, port: { number: 8080 } }
```

Listing 32.2: Ingress Annotations and Host Routing Rules

32.3.5 Dynamic Storage Provisioner (`hostpath-storage`) Specification

The `hostpath-storage` add-on registers the `microk8s-hostpath` StorageClass as the default dynamic provisioner:

- **StorageClass Name**: `microk8s-hostpath`
- **Provisioner Type**: `microk8s.io/hostpath`
- **Host Directory**: `/var/snap/microk8s/common/default-storage/`
- **Volume Binding Mode**: `Immediate`
- **Reclaim Policy**: `Delete` (governed by Kubernetes PersistentVolume lifecycle).

Every StatefulSet in the platform dynamically generates a dedicated subdirectory on the host:

- PostgreSQL Clinical Database 1: `default-storage/harmonia-postgres-data-1-postgres-1-0/`
- PostgreSQL Operations Database 1: `default-storage/harmonia-postgres-ops-data-1-postgres-ops-1-0/`
- ActiveMQ Artemis Primary A: `default-storage/harmonia-artemis-primary-a-data-artemis-primary-a-0/`
- ActiveMQ Artemis Primary B: `default-storage/harmonia-artemis-primary-b-data-artemis-primary-b-0/`
- Infinispan Mneme Cluster Node 1: `default-storage/harmonia-mneme-data-1-infinispan-1-0/`

32.3.6 Resource Telemetry Add-on (`metrics-server`) Specification

The `metrics-server` add-on scrapes real-time resource utilization from the kubelet cAdvisor interfaces on every pod. It provides authoritative metrics for validating sizing envelopes and executes automated capacity sanity checks during the rollout verification phase:

```
# Inspect cluster-wide node and pod resource consumption
microk8s kubectl top nodes
microk8s kubectl top pods -n harmonia
```

Listing 32.3: Resource Telemetry Verification Command

32.4 Kubernetes Middleconfiguration & Cluster Architecture

32.4.1 Declarative Kustomize Base and Overlay Hierarchy

Harmonia organizes cluster middleconfiguration using a hierarchical Kustomize architecture. This decouples core application definitions from platform-specific infrastructure parameters:

```
deployment/kubernetes/
|-- base/                                # Target-Agnostic Declarative Base
|   |-- kustomization.yaml               # Master Resource Aggregator
|   |-- namespace.yaml                   # "harmonia" Namespace Isolation
|   |-- network-policy.yaml              # Multi-Tier Network Ingress/Egress Isolation
|   |-- ingress.yaml                     # Nginx Ingress Rules & Buffer Annotations
|   |-- secrets.yaml                     # Base Secret Key Signatures & Defaults
|   |-- petasos/                         # Artemis HA StatefulSets & ConfigMaps
|   |-- hestia/                          # Infinispan, JPA & PostgreSQL StatefulSets
|   |-- energeia/                        # Ponos WorkEngine Deployment & Services
```

```
|   |-- pylai/                          # Inbound & Outbound MLLP Deployments
|   `-- iris/                           # BEFE Gateway & Vue 3 SPAs
`-- environments/
    `-- microk8s/                       # MicroK8s Platform Specific Overlay
        |-- kustomization.yaml          # Overlay Aggregator & Patch Rules
        `-- patches/
            `-- storage-class-patch.yaml # StorageClassName: microk8s-hostpath
```

Listing 32.4: Kustomize Manifest Directory Structure and Composition

The MicroK8s environment overlay (`deployment/kubernetes/environments/microk8s/kustomization.yaml`) references the declarative base and applies JSON6902/Strategic-Merge patches:

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: harmonia
resources:
  - ../../base
patches:
  - target:
      kind: StatefulSet
    patch: |-
      - op: replace
        path: /spec/volumeClaimTemplates/0/spec/storageClassName
        value: microk8s-hostpath
```

Listing 32.5: MicroK8s StorageClass Strategic Patch

32.4.2 Namespace Governance & Multi-Tier NetworkPolicies

All platform workloads execute within the isolated `harmonia` Kubernetes namespace. Inter-service traffic boundaries are governed by the authoritative NetworkPolicy manifest (`deployment/kubernetes/base/network-policy.ya`

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: harmonia-default-network-policy
  namespace: harmonia
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # 1. Allow full intra-namespace communication across all Harmonia tiers
    - from:
        - podSelector: {}
    # 2. Allow ingress controller traffic from ingress-nginx namespace
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress
        - namespaceSelector:
            matchLabels:
              name: ingress
  egress:
    # 1. Allow full intra-namespace communication
    - to:
        - podSelector: {}
    # 2. Allow cluster DNS resolution (CoreDNS UDP/TCP 53)
    - to:
        - namespaceSelector: {}
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    # 3. Allow controlled external egress for outbound MLLP dispatch
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
            except:
              - 10.0.0.0/8
```

```
41                 - 172.16.0.0/12
42                 - 192.168.0.0/16
```

Listing 32.6: Authoritative Harmonia NetworkPolicy Definition

32.4.3 Secret Management, Sensitive Credentials & Ansible Vault

Harmonia strictly isolates non-sensitive static runtime configuration from sensitive credentials:

- **Static ConfigMaps**: Server configurations, broker topology definitions (`broker.xml`), JGroups cluster discovery properties, and Camel routing rules are stored unencrypted in Git under `deployment/kubernetes/base/`
- **Ansible Vault**: Database passwords, ActiveMQ cluster keys, and TLS private keys are stored in encrypted form in `deployment/ansible/inventory/group_vars/vault.yml`.
- **Execution Invariant (`no_log: true`)**: All Ansible playbook tasks that template, render, or inject Kubernetes Secret resources execute with `no_log: true` enforced to prevent credential leakage into CI/CD build outputs and terminal logs.

32.4.4 Container Health Probes & Resource Allocation Matrix

Every container in the Harmonia ecosystem defines rigorous Kubernetes health probes and resource limits to ensure reliable container lifecycle management and automated recovery:

Workload Component	CPU Req	CPU Lim	Mem Req	Mem Lim	Probe Port/Type	Initial (s)	Period (s)
iris-clinical	50m	200m	64Mi	128Mi	80 / HTTP	10	10
iris-console	50m	200m	64Mi	128Mi	80 / HTTP	10	10
iris-administration	50m	200m	64Mi	128Mi	80 / HTTP	10	10
iris-befe	250m	1000m	512Mi	1536Mi	8080 / TCP	30	15
pylai-mllp-in	250m	1000m	512Mi	1536Mi	8080 / TCP	30	15
pylai-mllp-out-his	250m	1000m	512Mi	1536Mi	8080 / TCP	30	15
pylai-mllp-out-lis	250m	1000m	512Mi	1536Mi	8080 / TCP	30	15
ponos-task-processor	250m	1000m	512Mi	1536Mi	8080 / TCP	30	15
artemis-primary-a/b	200m	1000m	512Mi	1024Mi	61616 / TCP	20	10
artemis-backup-a/b	200m	1000m	512Mi	1024Mi	61616 / TCP	20	10
infinispan-1/2	250m	1000m	512Mi	1536Mi	11222 / TCP	30	15
mnemosyne-clinical-1/2	250m	1000m	512Mi	1536Mi	8080 / HTTP	30	15
mnemosyne-operations-1/2	250m	1000m	512Mi	1536Mi	8080 / HTTP	30	15
postgres-1/2	100m	500m	256Mi	1024Mi	5432 / TCP	15	10
postgres-ops-1/2	100m	500m	256Mi	1024Mi	5432 / TCP	15	10

Table 32.5: Kubernetes Container Probe Specifications and Compute Resource Allocations

32.5 Tier-by-Tier Subsystem Middleware Configuration Catalog

32.5.1 Petasos Messaging: Apache ActiveMQ Artemis 2.33.0 HA Cluster

Petasos utilizes an enterprise-grade 4-node dual paired replication topology:

- **Replication Group A**: Primary A (`artemis-primary-a`) replicating asynchronously to Backup A (`artemis-backup-a`) over port 61616.
- **Replication Group B**: Primary B (`artemis-primary-b`) replicating asynchronously to Backup B (`artemis-backup-b`) over port 61616.
- **Symmetric Server-Side Clustering**: Formed via the `petasos-cluster` configuration mesh linking Primary A and Primary B with `ON_DEMAND` load balancing and `redistribution-delay: 0` for zero-latency queue redistribution upon consumer migration.
- **Client Connection URL**: JMS clients utilize client-side failover strings ensuring continuous connectivity during node maintenance or failover events:

```
1 failover:(tcp://artemis-primary-a:61616,tcp://artemis-backup-a:61616,tcp://artemis-
      primary-b:61616,tcp://artemis-backup-b:61616)?randomize=false&priorityBackup=true&
      maxReconnectAttempts=-1&reconnectDelay=500
```

Listing 32.7: Petasos Artemis Dual Paired Replication Failover URI

32.5.2 Hestia Mneme: Infinispan 15 Distributed In-Memory Cache Grid

The Mneme cache tier provides sub-millisecond in-memory data access and write-behind persistence coordination:

- **Deployment Model**: Two Clustered StatefulSets (`infinispan-1` and `infinispan-2`) communicating via JGroups TCP discovery on port 7800.
- **Client Access Protocol**: Hot Rod binary protocol on port 11222 (`infinispan-1`) and port 11223 (`infinispan-2`).
- **Custom Write-Behind SPI (`mneme-persistence`)**:
  - `FhirRestCacheStore`: Custom implementation of Infinispan's `NonBlockingStore` SPI that intercepts cache write mutations for 14 FHIR R5 resource types and asynchronously streams them to the Mnemosyne Clinical JPA REST servers.
  - `OperationsRestCacheStore`: Intercepts non-FHIR operational and task sequence cache updates, streaming them to the Mnemosyne Operations JPA REST servers.
  - **Asynchronous Queue Parameters**: Flush thread pool size of 8, batch size of 50 modifications, and in-memory modification queue capacity of 10,000 entries.

32.5.3 Hestia Mnemosyne & PostgreSQL 16 Relational Persistence

The persistence tier guarantees durable relational disk storage for clinical records and operational metadata:

- **JPA REST Application Servers**:
  - `mnemosyne-clinical`: Spring Boot 3 / HAPI FHIR JPA servers (Ports 8081 and 8082) managing clinical FHIR R5 relational tables.
  - `mnemosyne-operations`: Spring Boot 3 JPA servers (Ports 8085 and 8086) managing workflow task sequences, operational metrics, and audit logs.
- **Connection Pool Tuning (HikariCP)**:
  - `maximum-pool-size`: 20 connections per JPA node.
  - `minimum-idle`: 5 connections.
  - `idle-timeout`: 300,000 ms (5 minutes).
  - `connection-timeout`: 20,000 ms (20 seconds).
  - `max-lifetime`: 1,200,000 ms (20 minutes).
- **PostgreSQL 16 StatefulSets**: StatefulSets `postgres-1`, `postgres-2`, `postgres-ops-1`, and `postgres-ops-2` running official PostgreSQL 16 OCI images mounting isolated persistent volumes at `/var/lib/postgresql/data`.

32.5.4 Energeia Ponos: WildFly 31 Workflow Execution Engine

The Ponos workflow engine executes complex task sequences and clinical transformations:

- **Runtime Environment**: WildFly 31 Jakarta EE 10 application server deployed on port 8083 (HTTP) and port 9990 (Management).
- **JMS Queue Consumers**: Consumes Ergon tasks and Petasos `TaskEvent` envelopes from ActiveMQ Artemis queues via integrated Camel routes.
- **Cache Synchronization**: Maintains active Hot Rod client connections to the Mneme Infinispan cluster on `infinispan-1:11222` to synchronize task instance states (`Pragma`).

32.5.5 Pylai Gateways: Inbound & Outbound MLLP Interfaces

Pylai manages the external protocol boundary for legacy HL7 v2 clinical data exchanges:

- **Inbound Gateway (`pylai-mllp-in`)**: Listens on MLLP port 2575 and REST management port 8084. Parses HL7 v2.4/v2.5 ADT/MFN messages, generates FHIR `Communication` records in the Mneme cache, and produces `ErgonEvent` envelopes onto the Petasos broker.
- **Outbound Gateways (`pylai-mllp-out`)**: Dedicated container instances per destination:
  - **HIS Instance (`mllp-outbound-his`)**: REST management port 8087, consuming from dedicated queue `petasos.queue.mllp.outbound.HIS_NORTH`.

  - **LIS Instance (`mllp-outbound-lis`)**: REST management port 8088, consuming from dedicated queue `petasos.queue.mllp.outbound.LIS_MAIN`.
- **Synchronous HL7 ACK/NACK Validation**: Outbound gateways wait synchronously for remote MLLP ACK responses before committing Artemis JMS queue transactions, preventing message loss during remote system outages.

32.5.6 Iris Presentation: BEFE Gateway & Vue 3 SPAs

The Iris presentation tier provides secure, responsive user portals:

- **Iris BEFE Gateway (`iris-befe`)**: Hosted on WildFly 31, providing dual-port REST interfaces:
  - **Port 8080 (`/api/fhir/`)**: Synchronous JAX-RS clinical FHIR R5 endpoints backed by Hot Rod cache queries.
  - **Port 8090 (`/api/operations/`)**: Non-blocking socket server for queue telemetry, cache topology inspection, and dynamic sequence reload triggers.
- **Vue 3 Single-Page Applications**: Pre-compiled TypeScript SPAs served via lightweight Nginx Alpine containers listening on internal port 80:
  - `iris-clinical`: Clinical resource explorer and longitudinal record viewer.
  - `iris-console`: Systems operations, Artemis queue monitoring, and Infinispan cache telemetry dashboard.
  - `iris-administration`: Practitioner directory management, credential attestation, and governed change triage portal.

32.6 Ansible Deployment Automation, Storage Lifecycle & Operational Runbooks

32.6.1 Automated Playbook Architecture & Workflow Pipeline

Harmonia provides four end-to-end Ansible playbooks under `deployment/ansible/` providing deterministic cluster provisioning, zero-downtime rolling upgrades, non-destructive teardowns, and disaster recovery:

32.6.2 Persistent Storage Lifecycle Governance: Safe Teardown vs Destructive Purge

A primary design principle of the Harmonia deployment engine is the strict safeguarding of clinical and audit persistence:

```
# 1. Default Safe Undeployment (Retains all PostgreSQL DBs, Artemis Queues & Cache)
ansible-playbook -i inventory/hosts.ini undeploy-harmonia.yml

# 2. Destructive Data Purge (Requires explicit opt-in confirmation variables)
ansible-playbook -i inventory/hosts.ini undeploy-harmonia.yml \
  -e harmonia_purge_data=true \
  -e harmonia_purge_data_confirm=true
```

Listing 32.8: Undeployment Invariants and Opt-In Purge Syntax

Architectural Principle: 2. Clinical Data Preservation Invariant

By default, `undeploy-harmonia.yml` never deletes PersistentVolumeClaims or hostpath directories. Workloads may be stopped, patched, upgraded, or redeployed without risk of data loss. Destructive purges require two separate explicit boolean variables passed at runtime.

32.6.3 Operational Health Check Runbooks & Diagnostic Commands

Operators and site reliability engineers (SREs) can evaluate the real-time operational status of the cluster using standardized diagnostic CLI commands:

```
# 1. Verify all 17 workload pods are Running and 1/1 Ready
microk8s kubectl get pods -n harmonia -o wide
```

Ansible Playbook	Target Role	Lifecycle Phase & Automated Operations
`prepare-microk8s.yml`	`roles/microk8s`	**Host Preparation & Add-on Bootstrap**: Validates Ubuntu host OS/vCPU/RAM prerequisites, configures sysctl kernel networking parameters, installs MicroK8s via Snap with classic confinement, adds operator to `microk8s` group, synchronizes `~/.kube/config`, enables the 4 core add-ons (`dns`, `ingress`, `hostpath-storage`, `metrics-server`), and blocks until cluster status reports ready.
`deploy-harmonia.yml`	`roles/harmonia-deploy`	**Workload Deployment & Rollout**: Injects sensitive secrets from Ansible Vault with `no_log: true`, generates runtime ConfigMaps, applies the MicroK8s Kustomize overlay (`deployment/kubernetes/environments/microk8s/`), and executes automated rollout polling across all 17 StatefulSet and Deployment workloads until 100% healthy.
`undeploy-harmonia.yml`	`roles/harmonia-undeploy`	**Safe Workload Teardown**: Removes all Deployments, StatefulSets, Services, and Ingress resources while **strictly preserving all PersistentVolumeClaims and persistent disk storage**.
`decommission-microk8s.yml`	`roles/microk8s-decom`	**Complete Node Purge**: Uninstalls MicroK8s snap packages, reverts sysctl kernel modifications, and purges hostpath storage directories.

Table 32.6: Harmonia Ansible Lifecycle Playbooks and Operational Capabilities

```
3
4  # 2. Inspect Ingress routing status and SSL certificates
5  microk8s kubectl get ingress -n harmonia
6
7  # 3. Verify PersistentVolumeClaim binding status
8  microk8s kubectl get pvc -n harmonia
9
10 # 4. Query Petasos ActiveMQ Artemis queue statistics and message depth
11 microk8s kubectl exec -it artemis-primary-a-0 -n harmonia -- \
12   /var/lib/artemis/bin/artemis queue stat --user admin --password admin
13
14 # 5. Validate Infinispan JGroups 2-node cluster membership view
15 microk8s kubectl logs infinispan-1-0 -n harmonia | grep -E "ISPN000094|received new
       view"
16
17 # 6. Check Mnemosyne JPA database connection pool metrics
18 microk8s kubectl logs deployment/mnemosyne-clinical-1 -n harmonia | grep "HikariPool"
```

Listing 32.9: Cluster Operational Health Check Runbook

32.6.4 Disaster Recovery & Single-Node Node Restoration Procedures

In the event of an abrupt host crash, hardware fault, or hypervisor reboot:

1. **Operating System Boot**: `snapd` initializes and automatically starts the `snap.microk8s.daemon-kubelite` service.
2. **Volume Reattachment**: Kubelet mounts existing subdirectories from `/var/snap/microk8s/common/default-stor` to their respective StatefulSets.
3. **Journal Recovery & Sequence Replay**:
   - PostgreSQL processes transaction write-ahead logs (WAL) to ensure ACID consistency.
   - ActiveMQ Artemis inspects its AIO/NIO journal files, reloads unacknowledged messages, and resynchronizes the dual paired replication mesh.

- Infinispan 15 forms the cluster ring via JGroups discovery and hydrates in-memory caches from PostgreSQL via read-through `FhirRestCacheStore` loading.

4. **Automated Ingress Restoration**: Nginx Ingress routes external HTTP/HTTPS and MLLP traffic back into the freshly recovered pods.

Part VII

Paradeigma Simulation

Chapter 33

Synthetic Clinical Generators & Scenario Framework

The Paradeigma subsystem provides a deterministic synthetic clinical simulation and testing framework. It generates realistic HL7 v2 and FHIR R5 event streams to drive end-to-end integration scenarios, stress testing, and failure recovery validation.

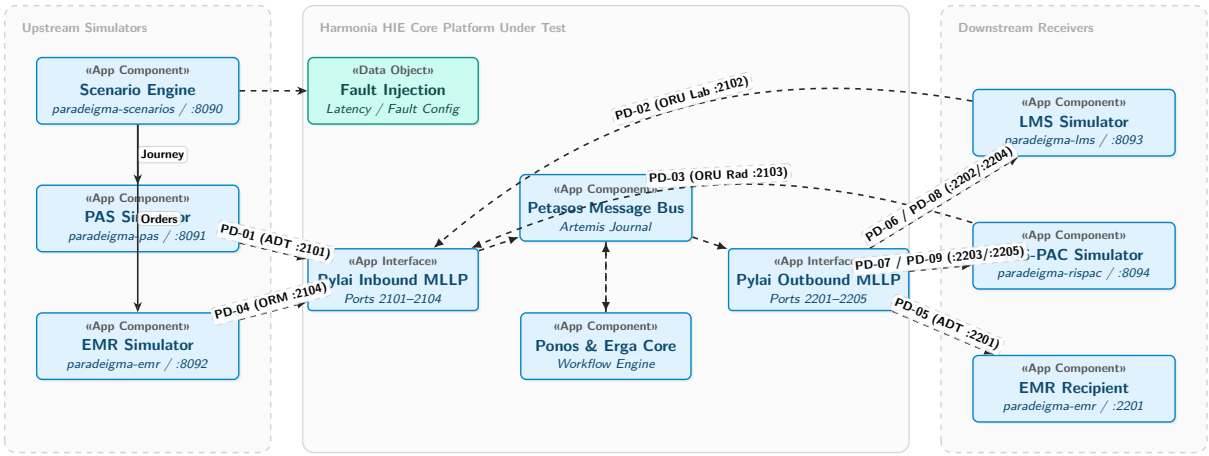


Figure 33.1: Paradeigma Simulation Architecture: Generators, Personas & Scenario Engine

33.1 Synthetic Actor Simulators & Module Inventory

Paradeigma comprises seven leaf modules under `paradeigma/`:

1. **`paradeigma-common`**: Pure domain library providing deterministic personas, Australian identifier generators (HPI-I with Luhn check, HPI-O, AHPRA, Medicare), clinical observation catalogues, and test fixtures.
2. **`paradeigma-pas`**: Simulates hospital Patient Administration Systems (`PasApplication`, `PasRestController`, `PasService`, `PasScheduler`, `PasPatientLifecycleManager`), generating HL7 v2 ADT feeds (A01, A03, A08, A40) with synthetic demographic personas.
3. **`paradeigma-emr`**: Simulates Electronic Medical Records (`EmrApplication`, `EmrRestController`, `EmrService`, `EmrPatientManager`, `EmrScheduler`, `EmrMllpListener`), generating encounter summaries, clinical notes, and receiving fan-out ADT feeds on Port 2201.
4. **`paradeigma-lms`**: Simulates Laboratory Information Systems (`LmsApplication`, `LmsRestController`, `LmsService`, `LmsResultWorker`, `LmsMllpListener`), generating HL7 ORU observation results and pathology reports, listening on Ports 2202 and 2204.
5. **`paradeigma-rispac`**: Simulates Radiology Information Systems and PACS (`RispacApplication`, `RispacRestController`, `RispacService`, `RispacResultWorker`, `RispacMllpListener`), generating imaging orders and diagnostic reports, listening on Ports 2203 and 2205.
6. **`paradeigma-scenarios`**: The scenario orchestration engine (`ScenariosApplication`, `ScenariosRestController`, `ParadeigmaScenarioEngine`, `PatientJourneyScenario`) coordinating multi-actor Patient Journeys and automated failure injection tests.
7. **`paradeigma-test`**: Architecture and regression test harness containing the 24 ArchUnit rules verifying strict compile-time, package, and runtime isolation.

33.2 Deterministic Patient Journeys & Scenario Engine

The scenario engine executes reproducible, timestamp-sequenced clinical pathways (e.g., Emergency Admission → Lab Order → Radiology Examination → Transfer → Discharge), verifying that Harmonia correctly correlates disparate trigger events into a unified longitudinal record.

33.2.1 REST Scenario Orchestration API

The scenario engine exposes REST endpoints for automated test execution:

- **`POST /api/scenarios/journey`**: Triggers an immediate single 9-step or 12-step Patient Journey. Requires no request body; parameters default to the standard deterministic journey seed.
- **`POST /api/scenarios/concurrent`**: Initiates N parallel journey executions via `ParadeigmaScenarioEngine.runConcurrentJourneys(intcount,ExecutionProfileprofile)`.
- **`GET /api/scenarios/status`**: Queries active and completed scenario run metrics, event success counts, and latency percentiles.

Chapter 34

Paradeigma Workload Deployment & Isolation Guardrails

The deployment of synthetic simulation workloads is governed by strict architectural isolation invariants to prevent simulation code or test artifacts from leaking into production builds.

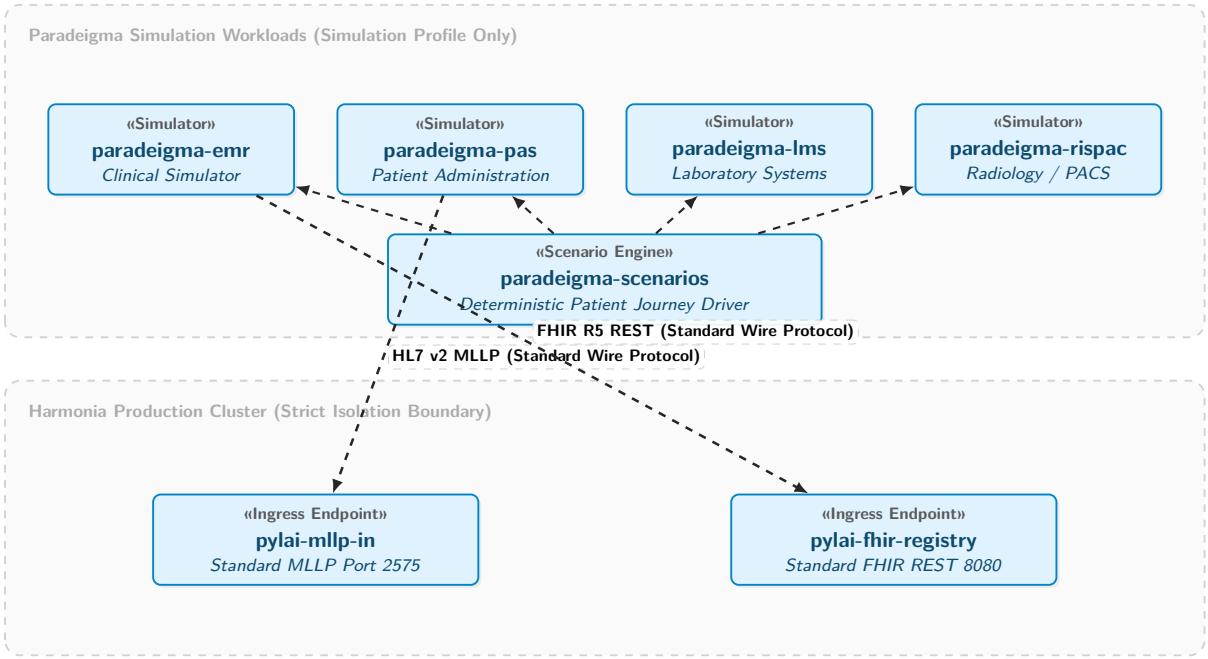


Figure 34.1: Normal Production vs Paradeigma Simulation Deployment Isolation Boundary

34.1 Strict Production Isolation Invariants

Architectural Principle: Paradeigma Production Isolation Invariant

In accordance with Invariant 1:

1. Production subprojects (`calliope`, `themis`, `hestia`, `petasos`, `energeia`, `pylai`, `iris`, `agora`) must **never** import or declare Maven dependencies on `net.fhirfactory.harmonia.paradeigma.*`.
2. Production classes must not contain simulation toggle flags (e.g. `paradeigmaMode`, `simulationMode`, `syntheticRequest`, `isParadeigmaGenerated`).
3. Production container images must not package simulator classes or scenario runners.

34.2 Automated ArchUnit Verification

This isolation rule is enforced continuously by the automated ArchUnit test suite (`ParadeigmaIsolationArchitectureTest` and `AgoraIsolationArchitectureTest`) in the CI/CD pipeline:

```
mvn test -pl paradeigma/paradeigma-test -am -Dtest="*ArchitectureTest" -Dsurefire.
    failIfNoSpecifiedTests=false
```

Listing 34.1: Executing ArchUnit Isolation Test Suite

34.3 Normal Production vs Paradeigma Simulation Deployment Matrix

Table 34.1 outlines the authoritative structural differences between normal production and simulation deployments:

Dimension	Normal Production	Simulation Deployment	Isolation Invariant
Workload Count	**25 Workloads** (12 StatefulSets + 13 Deployments)	**30 Workloads** (25 core + 5 simulator pods)	Simulators run as external client pods.
Persistent Storage	**12 PVCs (25 GB total)**	**12 PVCs (25 GB total)**	Simulators are stateless; request 0 PVCs.
Network Listeners	**18 Platform Listeners**	**18 Platform + 5 Simulator Listeners**	Core listeners operate identically.
Clinical Data	Real patient demographics & orders	Deterministic synthetic personas	Zero real PHI enters simulation.
Failure Injection	Disabled	Programmable network/ACK faults	Isolated to simulation overlay.

Table 34.1: Authoritative Normal Production vs Paradeigma Simulation Deployment Matrix

34.4 Paradeigma Simulation Profile Deployment

When testing in a dedicated staging or simulation environment, Paradeigma workloads are deployed as external client containers via the simulation overlay:

```
kubectl apply -k deployment/kubernetes/environments/simulation
```

Listing 34.2: Deploying Paradeigma Simulation Workloads

Chapter 35

Paradeigma Clinical Simulation Subsystem

35.1 Subsystem Overview & Architectural Principles

The `Paradeigma` subsystem constitutes Harmonia's dedicated clinical simulation environment, reference testbed, and resilience verification framework. In enterprise healthcare integration, testing against live clinical systems (such as hospital PAS, EMR, LMS, and RIS-PACS platforms) introduces substantial clinical risk, privacy constraints under health data regulations, and non-deterministic environment behavior. `Paradeigma` resolves these constraints by providing deterministic, stateful, and autonomous microservice emulators for each major hospital actor.

35.1.1 Core Design Principles

1. **High-Fidelity HL7 v2 and MLLP Emulation**: Each simulator operates as an authentic network actor over Minimal Lower Layer Protocol (MLLP) over TCP/IP, adhering to the standard `<VT>...<FS><CR>` framing protocol and standard HL7 v2.4/v2.5 message structures.
2. **Deterministic Synthetic Generation**: All clinical identities, patient demographics, accession numbers, lab observations, and imaging findings are generated using pseudorandom seed algorithms (`SeedRandom`). Given the same seed, the generator produces identical, reproducible clinical records across test runs.
3. **Synchronous Acknowledgement Contracts**: Inbound and outbound interfaces strictly enforce HL7 acknowledgment contracts (`ACK^R01`, `ACK^O01`, `ACK^A01`), requiring MSA-2 acknowledgment codes (`AA`, `AE`, `AR`) to reference the originating MSH-10 Message Control ID.
4. **Autonomous and Orchestrated Operating Modes**: Simulators can execute either autonomously on configurable background timer schedules or deterministically driven via the `ParadeigmaScenarioEngine` REST API.
5. **Integrated Fault Injection**: Every simulator embeds the `FailureSimulator` engine, permitting runtime injection of connection drops, acknowledgment timeouts, artificial latency, and application rejections for automated resilience verification.

35.2 ArchiMate Application & Interface Landscape

Figure 35.1 illustrates the ArchiMate 3.2 application component landscape for `Paradeigma` and its integration with Harmonia's `Pylai` interface gateway and `Petasos` messaging bus.

The architecture partitions into five microservices:

- **`paradeigma-pas`**: Simulates the Patient Administration System (PAS), initiating patient registration, admission, transfer, demographic updates, and discharge events over interface `PD-01`.
- **`paradeigma-emr`**: Simulates the Electronic Medical Record (EMR), receiving ADT notifications over `PD-05` and originating laboratory and diagnostic imaging orders over `PD-04`.
- **`paradeigma-lms`**: Simulates the Laboratory Information System (LMS), receiving ADT updates (`PD-06`) and pathology orders (`PD-08`), and transmitting structured LOINC pathology results over `PD-02`.
- **`paradeigma-rispac`**: Simulates the Radiology Information System & PACS, receiving ADT updates (`PD-07`) and radiology orders (`PD-09`), and returning DICOM-linked diagnostic imaging reports over `PD-03`.
- **`paradeigma-scenarios`**: Orchestrates multi-system clinical workflows (Patient Journeys) and controls fault injection parameters across the simulation fleet via REST APIs.

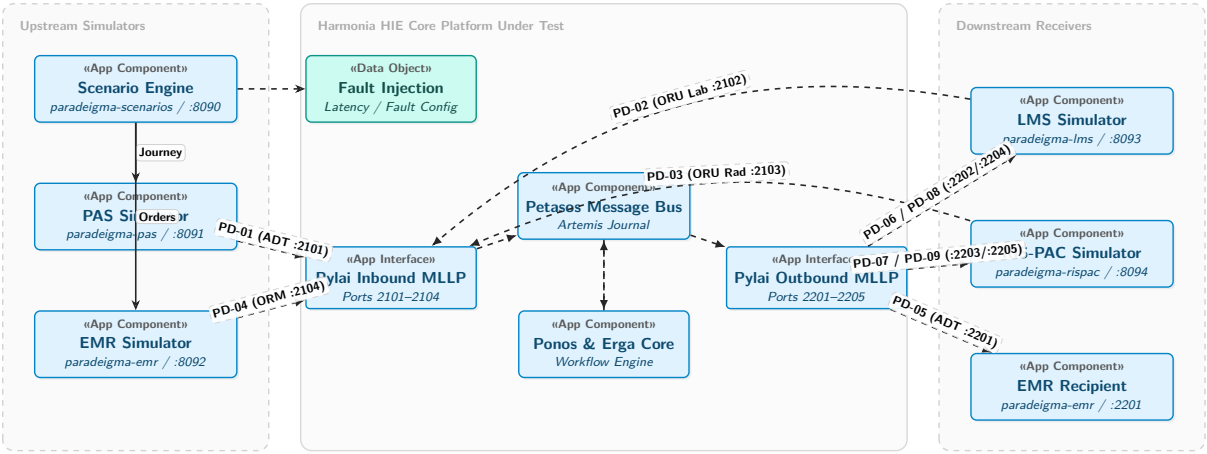


Figure 35.1: ArchiMate 3.2 Component Landscape for the Paradeigma Simulation Subsystem

35.3 Formal Interface Catalogue (PD-01 through PD-09)

Table 35.1 specifies the formal interface catalogue spanning ingress, egress, and routing channels across the Paradeigma simulation fleet and Harmonia.

ID	Source Actor	Target Actor	HL7 Message	Port	Trigger / Purpose
PD-01	paradeigma-pas	Harmonia Inbound	ADT^A01..A08	2101	Patient Lifecycle Events
PD-02	paradeigma-lms	Harmonia Inbound	ORU^R01	2102	LOINC Lab Results
PD-03	paradeigma-rispac	Harmonia Inbound	ORU^R01	2103	Imaging Diagnostic Reports
PD-04	paradeigma-emr	Harmonia Inbound	ORM^O01	2104	Lab & Imaging Orders
PD-05	Harmonia Outbound	paradeigma-emr	ADT^A01..A08	2201	EMR Demographics Fan-Out
PD-06	Harmonia Outbound	paradeigma-lms	ADT^A01..A08	2202	LMS Demographics Fan-Out
PD-07	Harmonia Outbound	paradeigma-rispac	ADT^A01..A08	2203	RIS Demographics Fan-Out
PD-08	Harmonia Outbound	paradeigma-lms	ORM^O01	2204	Pathology Order Routing
PD-09	Harmonia Outbound	paradeigma-rispac	ORM^O01	2205	Radiology Order Routing

Table 35.1: Paradeigma Interface Catalogue and Port Mappings

35.3.1 MLLP Framing & Transport Protocol

All network interfaces (PD-01 through PD-09) communicate using the Minimal Lower Layer Protocol (MLLP) over dedicated persistent TCP/IP sockets. An MLLP payload is framed strictly as:

$$\langle \text{MLLP Frame} \rangle ::= \texttt{0x0B}\ (\text{Start-of-Block / } \texttt{<VT>}) + \langle \text{HL7 v2 Payload} \rangle + \texttt{0x1C}\ (\text{End-of-Block / } \texttt{<FS>}) + \texttt{0x0D}\ (\text{Trailer / } \texttt{<CR>}) \tag{35.1}$$

35.3.2 Synchronous Acknowledgement Contract

Every transmitted HL7 v2 message requires an immediate, synchronous HL7 acknowledgement (`ACK`) over the established TCP socket prior to socket closure. The acknowledgment status in `MSA-1` determines downstream transmission handling:

- **`AA` (Application Accept)**: Indicates successful receipt, structural validation, and internal queue persistence. The sending client marks the transmission as successfully completed.

- `AE` **(Application Error)**: Indicates a recoverable or transient downstream processing condition. Triggering clients engage exponential backoff retry.
- `AR` **(Application Reject)**: Indicates an unrecoverable structural or schema violation (such as missing required `PID-3` or malformed segment delimiters). The message is immediately routed to the Dead Letter Queue without automatic retry.

35.4 Hospital Simulator Microservice Specifications

35.4.1 Patient Administration System Simulator (`paradeigma-pas`)

The `paradeigma-pas` service models an acute care hospital Patient Administration System responsible for master patient identity and encounter tracking.

- **Internal State Engine**: Maintains an active in-memory patient directory transitioning through state phases: UNREGISTERED → REGISTERED → ADMITTED → TRANSFERRED → UPDATED → DISCHARGED.
- **Autonomous Scheduling Modes**:
  - `FIXED`: Emits patient events at a deterministic interval (default 5000 ms).
  - `RANDOM_RANGE`: Emits events at random intervals within a bounded window (e.g., 2000–10000 ms).
  - `OFF`: Background timers disabled; events triggered exclusively via REST endpoints.
- **REST Management Interface** (Port `8091`):
  - `GET /api/pas/status`: Returns active configuration, simulated patient count, and message telemetry.
  - `POST /api/pas/config`: Updates generation mode, delay intervals, and target MLLP endpoints.
  - `POST /api/pas/register`, `POST /api/pas/admit`, `POST /api/pas/transfer`, `POST /api/pas/update`, `POST /api/pas/discharge`: Triggers discrete patient state transitions.
  - `POST /api/pas/trigger-next`: Advances the next patient in the demographic queue through their lifecycle.

35.4.2 Electronic Medical Record Simulator (`paradeigma-emr`)

The `paradeigma-emr` service simulates clinician interaction within an Electronic Medical Record:

- **Inbound Ingestion** (Port `2201`, `PD-05`): Receives inbound `ADT^A01`–`A08` broadcasts fanned out from Harmonia, synchronizing local patient and encounter tables.
- **Order Placement Engine** (Outbound Port `2104`, `PD-04`): Generates synthetic `ORM^O01` orders containing clinical indications, placer order numbers, priority flags (`STAT` vs. `ROUTINE`), and attending physician metadata.
- **REST API** (Port `8092`):
  - `GET /api/emr/status`: Simulator health and received ADT counters.
  - `POST /api/emr/orders/laboratory`: Originates pathology orders for registered patients.
  - `POST /api/emr/orders/imaging`: Originates diagnostic imaging orders for registered patients.
  - `GET /api/emr/patients`: Inspects locally synchronized patient registries.

35.4.3 Laboratory Information System Simulator (`paradeigma-lms`)

The `paradeigma-lms` service emulates clinical pathology and laboratory analyzer workflows:

- **Inbound Ingestion**: Receives patient admissions on Port `2202` (`PD-06`) and lab orders on Port `2204` (`PD-08`).
- **Pathology Result Generation**: Synthesizes `ORU^R01` observation messages containing multi-segment `OBX` records mapped to standard LOINC coding systems.
- **Clinical Pathology Catalog**: Supports standard diagnostic panels detailed in Table 35.2.
- **REST API** (Port `8093`):
  - `GET /api/lms/status`: Simulator status and received order queues.

  - `POST /api/lms/results/laboratory`: Triggers immediate generation and transmission of completed lab results to Harmonia Port `2102` (`PD-02`).

Panel Code	Panel Description	LOINC Code	Observation Name	Reference Range
`CBC`	Full Blood Count	`718-7`	Haemoglobin	130–180 g/L
		`6690-2`	White Blood Cell Count	4.0–11.0 $\times 10^9$/L
		`777-3`	Platelet Count	150–400 $\times 10^9$/L
		`4544-3`	Haematocrit	0.40–0.52 L/L
		`789-8`	Red Blood Cell Count	4.5–6.5 $\times 10^{12}$/L
`ELEC`	Electrolytes & Urea	`2951-2`	Sodium	135–145 mmol/L
		`2823-3`	Potassium	3.5–5.0 mmol/L
		`2075-0`	Chloride	95–105 mmol/L
		`2028-9`	Bicarbonate	22–29 mmol/L
		`2160-0`	Creatinine	60–110 μmol/L
		`3094-0`	Urea	2.5–7.8 mmol/L
`LFT`	Liver Function Tests	`1751-7`	Albumin	35–50 g/L
		`1975-2`	Total Bilirubin	3–20 μmol/L
		`1742-6`	Alanine Aminotransferase (ALT)	10–40 U/L
		`1920-8`	Aspartate Aminotransferase (AST)	10–40 U/L
		`6768-6`	Alkaline Phosphatase (ALP)	30–120 U/L
`GLU`	Fasting Blood Glucose	`2345-7`	Glucose	3.9–6.0 mmol/L

Table 35.2: Paradeigma Synthetic Pathology LOINC Catalog

35.4.4 Radiology Information System & PACS Simulator (`paradeigma-rispac`)

The `paradeigma-rispac` service simulates diagnostic imaging modalities and PACS archiving:

- **Inbound Ingestion**: Receives patient demographic feeds on Port `2203` (`PD-07`) and radiology examination orders on Port `2205` (`PD-09`).
- **Diagnostic Report Synthesis**: Formulates structured textual radiology impressions and narratives (`OBX` value type `TX`) with associated performing radiologist credentials.
- **Imaging Modality Catalog**:
  - `XR_CHEST`: 2-View Chest Radiography (evaluating infiltrate, consolidation, cardiomegaly).
  - `CT_HEAD`: Non-contrast Cranial Computed Tomography (acute hemorrhage and midline shift assessment).
  - `CT_ABDOMEN`: Contrast-enhanced Abdominal/Pelvic CT (parenchymal organ integrity).
  - `MRI_BRAIN`: High-resolution Magnetic Resonance Neuroimaging.
  - `US_ABDOMEN`: Abdominal Diagnostic Ultrasonography.
- **REST API** (Port `8094`):
  - `GET /api/rispac/status`: Service health and pending examination list.
  - `POST /api/rispac/results/imaging`: Generates and transmits finalized imaging report `ORU^R01` messages to Harmonia Port `2103` (`PD-03`).

35.5 Synthetic Data Models & Deterministic Generators

35.5.1 Canonical Domain Entities

The `paradeigma-common` module models realistic clinical workflows using five core profile structures:

1. `PatientProfile`: Represents master patient demographics.
   - `patientId` (MRN, e.g., `MRN-100421`), `nationalIdentifier` (e.g., Medicare/NHS identifier).
   - Demographic attributes: `firstName`, `lastName`, `dateOfBirth`, `gender` (`M`, `F`, `O`).
   - Contact & Geo: `addressLine`, `city`, `state`, `postalCode`, `phoneNumber`, `email`.
   - Next of Kin: `nextOfKinName`, `nextOfKinRelationship`, `nextOfKinPhone`.

2. **VisitProfile**: Represents encounter and admission context.
   - visitNumber (FIN/Account ID, e.g., VIS-900421).
   - Admission & Discharge timestamps, visitType (INPATIENT, OUTPATIENT, EMERGENCY).
   - Location context: pointOfCare (e.g., WARD-3A), room (RM-12), bed (BED-02), facility (HOSP-MAIN).
   - Care Team: attendingDoctorId, attendingDoctorName, admittingDiagnosis.
3. **OrderProfile**: Captures clinical orders.
   - Identifiers: placerOrderNumber (ORD-300001), fillerOrderNumber.
   - Context: orderType (PATHOLOGY, DIAGNOSTIC_IMAGING), universalServiceId, universalServiceText.
   - Clinical attributes: priority (R for Routine, S for Stat), orderDateTime, clinicalIndication.
4. **ResultProfile**: Encapsulates diagnostic report outputs.
   - Order correlation: placerOrderNumber, fillerOrderNumber, resultStatus (F for Final, P for Preliminary, C for Corrected).
   - Narrative report text: diagnosticReportNarrative.
   - Observations: List<ObservationItem>.
5. **ObservationItem**: Represents granular test observations mapped to HL7 OBX segments.
   - setSubId: Sequential identifier (OBX-1).
   - valueType: NM (Numeric), ST (String), TX (Text), or CE (Coded).
   - observationId: Standard LOINC or internal test code (OBX-3.1).
   - observationText: Observation description (OBX-3.2).
   - codingSystem: Coding system identifier, defaults to LN (OBX-3.3).
   - value: Quantitative or qualitative test result value (OBX-5).
   - units: Measurement units (OBX-6).
   - referenceRange: Reference normal boundary (OBX-7).
   - abnormalFlags: N (Normal), H (High), L (Low), A (Abnormal) (OBX-8).
   - resultStatus: Observation status flag (OBX-11).

35.5.2 Seed-Based Deterministic Generation Engine

The SeedRandom utility wraps Java's pseudorandom generation algorithms with strict seed encapsulation. This guarantees that given an identical seed (e.g., patient seed 100421L), generators produce identical clinical datasets across distributed test runners:

```
public class SyntheticPatientGenerator {
    private final SeedRandom random;

    public SyntheticPatientGenerator(long seed) {
        this.random = new SeedRandom(seed);
    }

    public PatientProfile generatePatient(String patientId) {
        Gender gender = random.nextBoolean(0.5) ? Gender.FEMALE : Gender.MALE;
        String firstName = random.pick(gender == Gender.FEMALE ? FEMALE_NAMES :
    MALE_NAMES);
        String lastName = random.pick(LAST_NAMES);
        LocalDate dob = LocalDate.now().minusYears(random.nextInt(18, 85))
                                       .minusDays(random.nextInt(1, 365));
        return new PatientProfile(patientId, firstName, lastName, dob, gender);
    }
}
```

Listing 35.1: SeedRandom Deterministic Generation Pattern in Paradeigma

35.6 Scenario Orchestration Engine & Patient Journey Workflows

The `ParadeigmaScenarioEngine` (microservice `paradeigma-scenarios`, Port `8090`) orchestrates end-to-end, multi-actor clinical workflows across the simulation testbed.

35.6.1 9-Step Canonical Patient Journey Sequence

A complete synthetic encounter exercises all ingress, egress, routing, and transformation pathways across Harmonia in a deterministic 9-step progression:

1. **Step 1: Patient Registration (`PAS` → `ADT^A04`)**: `paradeigma-pas` registers a new synthetic patient with full demographic and Next-of-Kin details, transmitting over `PD-01` (:2101).
2. **Step 2: Inpatient Admission (`PAS` → `ADT^A01`)**: `paradeigma-pas` assigns the patient to an acute inpatient ward (`WARD-3A`, `RM-12`, `BED-02`) and emits an admission event. Harmonia fans out `ADT^A01` to `EMR` (`PD-05`), `LMS` (`PD-06`), and `RIS-PAC` (`PD-07`).
3. **Step 3: Pathology Order Placement (`EMR` → `ORM^O01`)**: `paradeigma-emr` generates a Full Blood Count (`CBC`) lab order, transmitting over `PD-04` (:2104). Harmonia routes this order to `paradeigma-lms` over `PD-08` (:2204).
4. **Step 4: Pathology Result Ingestion (`LMS` → `ORU^R01`)**: `paradeigma-lms` analyzes the specimen and produces a multi-line LOINC pathology result (`718-7`, `6690-2`, `777-3`), transmitting to Harmonia over `PD-02` (:2102).
5. **Step 5: Diagnostic Imaging Order Placement (`EMR` → `ORM^O01`)**: `paradeigma-emr` requests a 2-view Chest X-Ray (`XR_CHEST`) over `PD-04` (:2104). Harmonia routes the radiology order to `paradeigma-rispac` over `PD-09` (:2205).
6. **Step 6: Diagnostic Imaging Result Ingestion (`RIS-PAC` → `ORU^R01`)**: `paradeigma-rispac` returns a structured diagnostic report with impression and findings narrative over `PD-03` (:2103).
7. **Step 7: Inpatient Transfer (`PAS` → `ADT^A02`)**: `paradeigma-pas` transfers the patient to a step-down ward (`WARD-4B`, `RM-401`) and broadcasts `ADT^A02` across all clinical destinations.
8. **Step 8: Patient Demographics Update (`PAS` → `ADT^A08`)**: Updates patient contact information and address, triggering fan-out reconciliation across EMR, LMS, and RIS-PAC.
9. **Step 9: Patient Discharge (`PAS` → `ADT^A03`)**: Emits encounter close and discharge disposition over `PD-01`, concluding the patient episode.

35.6.2 Execution Profiles

The scenario engine supports three operational execution profiles:

- `TEST`: Deterministic execution with minimal step pacing (10–50 ms delay). Optimized for continuous integration pipelines and automated regression test suites.
- `DEMO`: Human-observable pacing with 2000–10000 ms intervals between steps, designed for live demonstrations and interactive operational console monitoring.
- `LOAD`: Concurrent execution across configurable worker thread pools without synthetic pacing delays, evaluating platform throughput, queue depth, and memory footprint.

35.6.3 Scenario Control REST API

Table 35.3 documents the REST control endpoints exposed on port `8090`.

35.7 Failure Simulation, Fault Injection & Automated Recovery

35.7.1 Fault Injection Modes (`FailureSimulator`)

The `FailureSimulator` embeds within each simulator's MLLP pipeline, permitting dynamic simulation of network anomalies and application failures without modifying underlying platform infrastructure. Table 35.4 enumerates the supported failure modes configured via `FaultInjectionConfig`.

35.7.2 Automated Recovery Acceptance Test Suite

The integration test suite in `paradeigma-test` (`RecoveryAcceptanceTest.java`) automates formal recovery verification across six critical failure scenarios:

HTTP Method	Endpoint Path	Operational Function
POST	/api/scenarios/journey	Triggers an immediate single 9-step Patient Journey.
POST	/api/scenarios/concurrent	Initiates N parallel journey executions under specified profile.
POST	/api/scenarios/continuous	Starts sustained continuous journey background generation.
POST	/api/scenarios/stop	Gracefully halts active background scenarios and thread pools.
GET	/api/scenarios/status	Queries execution telemetry, success rates, and active threads.

Table 35.3: Scenario Engine REST Control Endpoints

Fault Injection Parameter	Range / Type	Simulated Physical Failure Condition
dropConnectionProbability	$0.0 - 1.0$	TCP connection RST / network partition mid-frame.
noAckProbability	$0.0 - 1.0$	Destination unresponsiveness causing socket read timeout.
ackDelayMs	≥ 0 ms	Network congestion or downstream processing latency.
applicationErrorProbability	$0.0 - 1.0$	Returns MSA-1=AE simulating transient system error.
applicationRejectProbability	$0.0 - 1.0$	Returns MSA-1=AR simulating schema/rule validation error.
malformedSegmentProbability	$0.0 - 1.0$	Corrupts HL7 segment delimiters or required fields.
duplicateMsh10Probability	$0.0 - 1.0$	Replays previous MSH-10 to evaluate deduplication filters.

Table 35.4: FailureSimulator Fault Injection Capabilities

1. **Test 1 — Broker Journal Recovery**: Verifies that unacknowledged in-flight messages persist across complete ActiveMQ Artemis broker crashes and restarts, replaying cleanly to consumers upon broker recovery without data loss.
2. **Test 2 — Ponos Worker Crash Recovery**: Simulates worker termination during Camel pipeline execution. Validates that uncommitted Artemis transactions redeliver to healthy worker instances, advancing the associated Pragma to COMPLETED.
3. **Test 3 — Outbound Destination Outage & Backoff Recovery**: Simulates an LMS outage during ORM order transmission. Verifies that Pylai engages exponential retry backoff, retaining the message until LMS recovers and returns ACK^AA.
4. **Test 4 — Duplicate HL7 MSH-10 Idempotency**: Transmits identical MSH-10 control IDs twice. Validates that Petasos deduplication filters intercept the re-transmission, returning an idempotent AA acknowledgment without triggering duplicate task executions.
5. **Test 5 — Partial ADT Fan-Out Isolation**: Simulates an ADT fan-out where EMR and LMS succeed but RIS-PAC fails. Verifies that upon RIS-PAC restoration, only the pending RIS-PAC delivery is retried, avoiding duplicate broadcasts to EMR or LMS.
6. **Test 6 — Platform Cold Restart Recovery**: Queues multiple clinical transactions and performs a hard platform stop and restart. Verifies that all persistent journal messages and cache state reconstruct cleanly to terminal completion.

35.8 Deployment Topology & Docker Orchestration

The Paradeigma simulation fleet deploys seamlessly alongside Harmonia using Docker Compose (docker-compose-paradeig
Table 35.5 outlines the container topology, port bindings, and health checks.

35.9 Provider Registry Simulation & Synthetic FHIR R5 Generators

With the introduction of the Harmonia FHIR R5 Provider Registry core services (Appendix 12), Paradeigma expands beyond legacy HL7 v2 messaging to provide high-fidelity, deterministic synthetic

Container Service	Base Image	REST Port	MLLP Ports	Health Check Dependency
paradeigma-pas	OpenJDK 17	8091	—	http://localhost:8091/api/pas/status
paradeigma-emr	OpenJDK 17	8092	2201	http://localhost:8092/api/emr/status
paradeigma-lms	OpenJDK 17	8093	2202, 2204	http://localhost:8093/api/lms/status
paradeigma-rispac	OpenJDK 17	8094	2203, 2205	http://localhost:8094/api/rispac/status
paradeigma-scenarios	OpenJDK 17	8090	—	Depends on PAS, EMR, LMS, RIS-PAC healthy

Table 35.5: Paradeigma Docker Compose Deployment Configuration

generation of HL7 FHIR Release 5 directory resources.

35.9.1 Standardized Australian Digital Health Identifiers

All synthetic directory entities adhere strictly to canonical Australian digital health identifier schemas:

- **Healthcare Provider Identifier – Individual (HPI-I)**: Formatted under system URI `http://ns.electronicheal` with standard prefix `800361` followed by 10 deterministic pseudorandom digits and a Luhn check digit (e.g., `8003610000000042`).
- **Healthcare Provider Identifier – Organisation (HPI-O)**: Formatted under system URI `http://ns.electronichealth.net.au/id/hi/hpio/1.0` with prefix `800362` (e.g., `8003620000000042`).
- **AHPRA Registration Numbers**: Australian Health Practitioner Regulation Agency practitioner licenses (system `http://hl7.org.au/id/ahpra-registration-number`, e.g., `MED0001234567`).
- **SNOMED CT Specialty Codes**: Standardized Australian clinical specialties and role classifications (system `http://snomed.info/sct`, e.g., `309988001` for General Medical Practice).

35.9.2 Dedicated Synthetic Generators Catalog

The package `net.fhirfactory.harmonia.paradeigma.common.generator` provides seven dedicated, seed-controlled generators corresponding to each managed FHIR R5 resource type:

35.9.3 Interconnected Graph Builders (`SyntheticProviderRegistryGenerator`)

To test complex relational queries and referential integrity enforcement without manual fixture construction, `SyntheticProviderRegistryGenerator` provides five canonical graph topology builders:

1. **Solo Practitioner Graph (`generateSoloPractitioner(seed)`)**: Constructs an isolated practitioner bound to a single healthcare organization and primary practice location via a single `PractitionerRole`.
2. **Multi-Role Specialist Graph (`generatePractitionerWithMultipleRoles(seed)`)**: Constructs a medical specialist operating across multiple distinct hospital networks and consulting suites, generating multiple `PractitionerRole` entities referencing independent organizations.
3. **Healthcare Organization Graph (`generateOrganizationWithLocations(seed)`)**: Builds a comprehensive enterprise hospital network comprising headquarters, regional campuses, specialized wings, and technical communication endpoints.
4. **Connected Provider Network (`generateProviderNetwork(seed)`)**: Constructs an exhaustive clinical ecosystem linking all seven resource types: `Organization`, `Location`, `Endpoint`, `HealthcareService`, `Practitioner`, `PractitionerRole`, and multidisciplinary clinical `Group`.
5. **Invalid Reference Graph (`generateInvalidReferenceGraph(seed)`)**: Deliberately constructs resources containing broken and dangling logical references (such as non-existent target `Organization/org-missing-9` for automated negative testing of referential validation and `OperationOutcome` rejection pipelines.

```
// Build deterministic complete provider network topology
long seed = 42L;
SyntheticProviderRegistryGenerator generator = new SyntheticProviderRegistryGenerator(
    seed);
SyntheticProviderRegistryGenerator.ProviderRegistryGraph network = generator.
    generateProviderNetwork(seed);

// Access strongly typed generated resources
```

Generator Class	Target Resource	Key Synthetic Capabilities & Variations
`PractitionerGenerator`	`Practitioner`	Deterministic human names, HPI-I, AHPRA credentials, qualification codings, telecom endpoints, address history, version updates, and duplicate identifier conflict models.
`PractitionerRoleGenerator`	`PractitionerRole`	Specialty SNOMED codes, role classifications, multi-target referential linkages (`Organization`, `Location`, `HealthcareService`, `Endpoint`), and availability schedules.
`OrganizationGenerator`	`Organization`	Hospital network names, HPI-O identifiers, Australian Business Numbers (ABN), facility types, physical campuses, and postal routing addresses.
`LocationGenerator`	`Location`	Physical hospital wards, clinical wings, operating suites, physical type codes (`bu`, `wi`, `ro`), and managing organization references.
`HealthcareServiceGenerator`	`HealthcareService`	Clinical offerings, consultation specialties, operational hours, referral requirements, and parent organization linkages.
`EndpointGenerator`	`Endpoint`	Technical communication channels across FHIR REST (`hl7-fhir-rest`), HL7 v2 MLLP (`direct-project`), and secure messaging endpoints with payload mime-types.
`GroupGenerator`	`Group`	Practitioner specialist panels, clinical governance committees, on-call rosters, and enumerated clinical team memberships.

Table 35.6: Dedicated FHIR R5 Synthetic Resource Generators

```
Organization org = network.getOrganization();
Practitioner pract = network.getPractitioner();
PractitionerRole role = network.getPractitionerRole();
Location location = network.getLocation();
HealthcareService service = network.getHealthcareService();
Endpoint endpoint = network.getEndpoint();
Group group = network.getGroup();
```

Listing 35.2: Constructing Topological Graphs with SyntheticProviderRegistryGenerator

35.10 Themis Security Simulation & Authorization Fixtures

Harmonia's **Themis** security engine governs role-based access control (RBAC), fine-grained attribute-based access control (ABAC), and dual-authority asynchronous change verification. `Paradeigma` provides standardized security actor fixtures and context factories in `net.fhirfactory.harmonia.paradeigma.common.security` to facilitate comprehensive positive and negative authorization testing.

35.10.1 Standard Security Actors Catalog

The `ParadeigmaSecurityActors` catalog defines seven standard principals, mapped roles, granted authorities, and operational boundaries, as detailed in Table 35.7.

35.10.2 Security Context Orchestration (`SecurityScenarioContext`)

Test scenarios utilize `SecurityScenarioContext` to inject validated security credentials, caller correlation identifiers, and client identifiers directly into HTTP requests and Camel exchange headers:

```
// Create Provider Steward context with auto-generated correlation ID
SecurityScenarioContext stewardContext = SecurityScenarioContext.providerSteward();

// Create Clinician context with explicit correlation ID
SecurityScenarioContext clinicianContext = SecurityScenarioContext.clinician("corr-
    audit-901");
```

Actor Name	Harmonia Roles	Granted Authorities	Permitted Functional Boundary
Provider Steward	`PRV_ADM`, `PRV_SUB`, `PRV_APR`, `PRV_RDR`	`provider.change.submit`, `provider.change.approve`, `provider.read`, `provider.search`, `provider.admin`	Full directory governance: synchronous read/search, change request submission, and change approval across all directory resources.
Clinician	`PRV_RDR`	`provider.read`, `provider.search`	Clinical query operations: synchronous read and search. Write or update submissions are rejected with HTTP 403 Forbidden.
System Administrator	`SYS_ADM`, `PRV_ADM`	`system.admin`, `provider.admin`	Infrastructure management, pipeline monitoring, security label maintenance, and administrative override.
Integration Service	`SYS_INT`, `PRV_PROC`	`system.integration`, `provider.change.process`	Automated messaging queues, Ponos Ergon asynchronous execution, and inter-service dispatch.
Read-Only User	`PRV_RDR`	`provider.read`	Resource inspection and single-resource retrieval; search querying restricted.
Unauthorized User	*None*	*None*	Authenticated identity without directory permissions; all protected API calls evaluate to `DENY`.
Unauthenticated	*None*	*None*	Anonymous identity; rejected at gateway ingress with HTTP 401 Unauthorized.

Table 35.7: Paradeigma Themis Security Actors Reference Catalog

```
6
7  // Evaluate authorization against Themis policy evaluator
8  ThemisResource resource = ThemisResource.of(
9      "Practitioner",
10     "pract-101",
11     "PROVIDER_REGISTRY",
12     Set.of(ThemisSecurityLabel.of("PROVIDER_REGISTRY"))
13 );
14
15 ThemisAuthorizationRequest authRequest = ThemisAuthorizationRequest.builder()
16     .principal(stewardContext.principal())
17     .authorities(stewardContext.grantedAuthorities())
18     .action(ThemisAction.CREATE)
19     .target(resource)
20     .context(stewardContext.securityContext())
21     .build();
22
23 ThemisAuthorizationDecision decision = evaluator.authorize(authRequest);
24 assertThat(decision.isAllowed()).isTrue();
```

Listing 35.3: Orchestrating Security Scenario Contexts and Themis Authorization

35.10.3 Negative Testing Authorization Seams

To verify defensive platform behavior when encountering security anomalies, `ParadeigmaSecurityActors` exposes context factories that inject specific security faults:

- **Expired Context (`ParadeigmaSecurityActors.expiredContext()`)**: Simulates an expired bearer token or session context where `expirationTime` is in the past, validating that gateway interceptors reject the interaction before routing.
- **Revoked Authority (`ParadeigmaSecurityActors.revokedAuthorityContext(...)`)**: Simulates a steward whose `provider.change.submit` authority has been revoked mid-session, verifying policy rejection.
- **Missing Role (`ParadeigmaSecurityActors.missingRoleContext(...)`)**: Simulates a principal with partial roles missing mandatory execution authorities (`PRV_SUB`), confirming granular RBAC boundary enforcement.

35.11 PHI-Aware Logging Simulation & Credential Protection

In accordance with Harmonia's dual-gate PHI logging policy (Appendix 40), operational logs (`INFO`, `WARN`, `ERROR`) must remain strictly devoid of clinical payloads, patient identifiers, and credentials under all circumstances.

35.11.1 Dual-Gate Verification Probe (`PhiLogTestProbe`)

The `PhiLogTestProbe` provides an in-memory, auto-closeable Logback appender probe designed for automated assertion of logging compliance during scenario execution:

```
1  try (PhiLogTestProbe probe = PhiLogTestProbe.startCapture()) {
2      // 1. Execute system operation under test
3      gatewayController.createResource("Practitioner", fhirJson, correlationId, "pas", "
       steward", null);
4
5      // 2. Assert that no PHI attributes leaked into operational logs
6      probe.assertNoPhiInOperationalLogs("Dr. Elena Vance", "8003610000000042");
7
8      // 3. Assert that PHI is routed to diagnostic destination when dual-gate is active
9      probe.assertPhiPresentInDiagnosticLogs("Dr. Elena Vance");
10     probe.assertPhiMarkerAttachedToDiagnosticLogs();
11
12     // 4. Assert zero leakage of credentials or sensitive tokens across ALL log
       destinations
13     probe.assertNoSecretsInAnyLog(SecretLeakageAssertion.getStandardSyntheticSecrets()
       );
14 }
```

Listing 35.4: Dual-Gate Assertion Flow using PhiLogTestProbe

35.11.2 Synthetic Secret Leakage Protection (`SecretLeakageAssertion`)

Authentication credentials, connection strings, and crypto keys must never appear in any log sink, regardless of whether diagnostic logging is active. `SecretLeakageAssertion` generates comprehensive synthetic sentinel values to validate zero-leakage constraints across:

- Synthetic JWT Bearer and OAuth tokens (`eyJhbGciOiJIUzI1Ni...`).
- Platform API secret keys (`harmonia_sec_live_...`).
- Database credentials and PostgreSQL connection strings.
- ActiveMQ Artemis message broker passwords.
- RSA and PKCS#8 private key blocks (`---BEGIN PRIVATE KEY---`).

35.11.3 Integrated Multi-Capability Verification Listing

Listing 35.5 provides a complete, worked exemplar demonstrating the coordinated usage of synthetic generators, security contexts, governed gateway interactions, and PHI logging probes.

```
@Test
@DisplayName("Integrated Verification: Seeded Generation + Themis Ingress + Governed Workflow + PHI Isolation")
void testCompleteGovernedLifecycleWithSecurityAndLogging() throws Exception {
    long seed = 42L;
    String correlationId = "corr-exemplar-042";

    // Step 1: Initialize deterministic synthetic generator
    PractitionerGenerator generator = new PractitionerGenerator(seed);
    Practitioner practitioner = generator.generateValid("pract-dr-vance-42");
    String familyName = practitioner.getNameFirstRep().getFamily();
    String hpii = practitioner.getIdentifierFirstRep().getValue();
    String fhirJson = FhirContext.forR5().newJsonParser().encodeResourceToString(practitioner);

    // Step 2: Establish Provider Steward Security Context
    SecurityScenarioContext steward = SecurityScenarioContext.providerSteward();
    ThemisResource resource = ThemisResource.of(
        "Practitioner", "pract-dr-vance-42", "PROVIDER_REGISTRY",
        Set.of(ThemisSecurityLabel.of("PROVIDER_REGISTRY"))
    );

    // Step 3: Verify Themis Ingress Policy Authorization
    ThemisAuthorizationRequest authReq = ThemisAuthorizationRequest.builder()
        .principal(steward.principal())
        .authorities(steward.grantedAuthorities())
        .action(ThemisAction.CREATE)
        .target(resource)
        .context(steward.securityContext())
        .build();
    ThemisAuthorizationDecision decision = policyEvaluator.authorize(authReq);
    assertThat(decision.isAllowed()).isTrue();

    // Step 4: Attach PHI Test Probe & Execute Governed Ingress
    PhiLoggingConfig.setPhiEnabled(true);
    try (PhiLogTestProbe probe = PhiLogTestProbe.startCapture()) {
        ResponseEntity<String> postResp = gatewayController.createResource(
            "Practitioner", fhirJson, correlationId, "pas",
            steward.principal().principalId(), null
        );
        assertThat(postResp.getStatusCode().value()).isEqualTo(202);

        // Step 5: Process Governed Workflow via Ponos Ergon
        String location = postResp.getHeaders().getLocation().toString();
        String pragmaId = location.substring(location.lastIndexOf('/') + 1);
        Pragma pragma = pragmaCacheService.getPragma(pragmaId).orElseThrow();
        practitionerErgon.processErgon(pragma, camelContext.createProducerTemplate().getDefaultEndpoint().createExchange());
        assertThat(pragma.getStatus()).isEqualTo(PragmaStatus.COMPLETED);

        // Step 6: Validate Synchronous Read via Mnemosyne
        ResponseEntity<String> getResp = gatewayController.readResource("Practitioner", "pract-dr-vance-42", null);
        assertThat(getResp.getStatusCode().value()).isEqualTo(200);
```

```
51
52          // Step 7: Assert Strict Dual-Gate Logging Policy & Secret Protection
53          probe.assertNoPhiInOperationalLogs(familyName, hpii);
54          probe.assertPhiPresentInDiagnosticLogs(familyName, hpii);
55          probe.assertPhiMarkerAttachedToDiagnosticLogs();
56          probe.assertNoSecretsInAnyLog(SecretLeakageAssertion.
        getStandardSyntheticSecrets());
57      }
58  }
```

Listing 35.5: Multi-Capability Integration Test: Registry + Security + PHI Logging

35.12 Comprehensive Configuration Reference Matrix

The Paradeigma simulation suite and its integrated security/logging harnesses are controlled through standard Spring Boot properties, JVM system properties, and container environment variables.

35.12.1 Simulator Service Properties & Port Allocations

Table 35.8 lists the core runtime parameters across the simulation microservices.

Configuration Property	Environment Variable	Default	Description
`server.port` (PAS)	`SERVER_PORT`	`8091`	REST management port for `paradeigma-pas`.
`server.port` (EMR)	`SERVER_PORT`	`8092`	REST management port for `paradeigma-emr`.
`server.port` (LMS)	`SERVER_PORT`	`8093`	REST management port for `paradeigma-lms`.
`server.port` (RIS-PAC)	`SERVER_PORT`	`8094`	REST management port for `paradeigma-rispac`.
`server.port` (Scenarios)	`SERVER_PORT`	`8090`	REST orchestration port for `paradeigma-scenarios`.
`server.port` (FHIR Gateway)	`SERVER_PORT`	`8089`	REST FHIR R5 Provider Registry gateway port.
`paradeigma.pas.generation-mode`	`PAS_GEN_MODE`	`OFF`	Scheduling mode: `OFF`, `FIXED`, or `RANDOM_RANGE`.
`paradeigma.pas.interval-ms`	`PAS_INTERVAL_MS`	`5000`	Pacing interval in milliseconds when operating in `FIXED` mode.
`paradeigma.pas.target-host`	`PAS_TARGET_HOST`	`localhost`	Inbound Harmonia Pylai MLLP bridge hostname.
`paradeigma.pas.target-port`	`PAS_TARGET_PORT`	`2101`	Inbound Harmonia Pylai MLLP bridge port for `PD-01`.
`paradeigma.emr.mllp-port`	`EMR_MLLP_PORT`	`2201`	Inbound MLLP port for `PD-05` ADT demographics ingestion.
`paradeigma.lms.mllp-port-adt`	`LMS_MLLP_PORT_ADT`	`2202`	Inbound MLLP port for `PD-06` ADT demographics ingestion.
`paradeigma.lms.mllp-port-orm`	`LMS_MLLP_PORT_ORM`	`2204`	Inbound MLLP port for `PD-08` pathology order routing.
`paradeigma.rispac.mllp-port-adt`	`RISPAC_MLLP_PORT_ADT`	`2203`	Inbound MLLP port for `PD-07` ADT demographics ingestion.
`paradeigma.rispac.mllp-port-orm`	`RISPAC_MLLP_PORT_ORM`	`2205`	Inbound MLLP port for `PD-09` radiology order routing.

Table 35.8: Paradeigma Simulator Microservice Configuration Properties

35.12.2 Security & PHI Logging Governance Configuration

Table 35.9 details the governance switches controlling Themis policy evaluation and PHI log routing.

Configuration Property	Environment Variable	Default	Description
`harmonia.logging.phi-enabled`	`HARMONIA_LOGGING_PHI_ENABLED`	`false`	Global security gate. When `true`, permits PHI diagnostic logs to route to `org.harmonia.phi`.
`themis.security.enabled`	`THEMIS_SECURITY_ENABLED`	`true`	Enables declarative Themis RBAC/ABAC policy enforcement across gateway ingress controllers.
`themis.audit.trail.enabled`	`THEMIS_AUDIT_TRAIL_ENABLED`	`true`	Emits tamper-evident security audit records for all authorization evaluations and policy decisions.

Table 35.9: Security & PHI Logging Governance Properties

35.12.3 FailureSimulator Fault Injection Knobs

Table 35.10 summarizes the fault injection switches configurable dynamically via `FaultInjectionConfig`.

Property Key	Type / Range	Default	Simulated Fault Behavior
`paradeigma.fault.drop-connection-prob`	$0.0 - 1.0$	`0.0`	Probability of resetting active TCP connection before transmitting frame.
`paradeigma.fault.no-ack-prob`	$0.0 - 1.0$	`0.0`	Probability of dropping response to trigger socket read timeout in caller.
`paradeigma.fault.ack-delay-ms`	≥ 0 ms	`0`	Artificial latency injected prior to returning acknowledgment frame.
`paradeigma.fault.application-error-prob`	$0.0 - 1.0$	`0.0`	Probability of returning `MSA-1=AE` (transient error).
`paradeigma.fault.application-reject-prob`	$0.0 - 1.0$	`0.0`	Probability of returning `MSA-1=AR` (schema/validation rejection).
`paradeigma.fault.malformed-segment-prob`	$0.0 - 1.0$	`0.0`	Probability of corrupting message framing or segment delimiters.
`paradeigma.fault.duplicate-msh10-prob`	$0.0 - 1.0$	`0.0`	Probability of duplicating `MSH-10` to test idempotency filters.

Table 35.10: FailureSimulator Fault Injection Configuration Knobs

35.13 Step-by-Step CLI & REST Execution Instructions

This section provides concrete, copy-pasteable instructions for building, running, and orchestrating the Paradeigma simulation suite across diverse operating environments.

35.13.1 Building the Solution via Maven

To compile all core platform and Paradeigma simulator modules:

```
# Build entire repository without running tests
mvn clean install -DskipTests

```

```
# Build only the Paradeigma simulation subsystem
mvn clean install -pl paradeigma -am -DskipTests

# Run all unit and component tests across Paradeigma
mvn test -pl paradeigma/paradeigma-common,paradeigma/paradeigma-scenarios
```

Listing 35.6: Building Harmonia and Paradeigma Modules

35.13.2 Running Simulators as Standalone Microservices

Each simulator microservice can execute locally as an independent Spring Boot process across separate terminal windows:

```
# Terminal 1: Launch Patient Administration System Simulator (Port 8091)
mvn spring-boot:run -pl paradeigma/paradeigma-pas

# Terminal 2: Launch Electronic Medical Record Simulator (Port 8092, MLLP 2201)
mvn spring-boot:run -pl paradeigma/paradeigma-emr

# Terminal 3: Launch Laboratory Information System Simulator (Port 8093, MLLP
    2202/2204)
mvn spring-boot:run -pl paradeigma/paradeigma-lms

# Terminal 4: Launch Radiology Information System & PACS (Port 8094, MLLP 2203/2205)
mvn spring-boot:run -pl paradeigma/paradeigma-rispac

# Terminal 5: Launch Scenario Conductor Engine (Port 8090)
mvn spring-boot:run -pl paradeigma/paradeigma-scenarios
```

Listing 35.7: Launching Standalone Simulator Microservices

35.13.3 Docker Compose Topology Orchestration

To deploy the unified simulation and platform environment using Docker Compose:

```
# Navigate to the Paradeigma deployment directory
cd paradeigma/deployment

# Build images and start all containers in background
docker compose -f docker-compose-paradeigma.yml up --build -d

# Check runtime health and container port status
docker compose -f docker-compose-paradeigma.yml ps

# Tail aggregated logs across the simulation fleet
docker compose -f docker-compose-paradeigma.yml logs -f

# Gracefully stop and tear down containers
docker compose -f docker-compose-paradeigma.yml down -v
```

Listing 35.8: Docker Compose Lifecycle Management

35.13.4 REST API Scenario Orchestration (cURL Snippets)

The `paradeigma-scenarios` engine and individual simulator management APIs can be controlled via standard HTTP cURL commands:

```
# 1. Trigger an immediate single 9-step Patient Journey
curl -i -X POST http://localhost:8090/api/scenarios/journey

# 2. Trigger concurrent journeys under the high-speed TEST profile
curl -i -X POST "http://localhost:8090/api/scenarios/concurrent?count=10&profile=TEST"

# 3. Trigger concurrent journeys under human-paced DEMO profile
curl -i -X POST "http://localhost:8090/api/scenarios/concurrent?count=3&profile=DEMO"

# 4. Query scenario conductor status and execution counters
curl -s http://localhost:8090/api/scenarios/status | jq .

# 5. Halt all active background scenarios
curl -i -X POST http://localhost:8090/api/scenarios/stop
```

```
15
16 # 6. Trigger discrete PAS hospital registration
17 curl -i -X POST http://localhost:8091/api/pas/register \
18   -H "Content-Type: application/json" \
19   -d '{"patientId":"MRN-90901","firstName":"Jane","lastName":"Doe","gender":"F"}'
20
21 # 7. Ingress FHIR R5 Practitioner Create via Pylai Gateway (Port 8089)
22 curl -i -X POST http://localhost:8089/Practitioner \
23   -H "Content-Type: application/fhir+json" \
24   -H "X-Correlation-ID: corr-curl-001" \
25   -H "X-Source-System: pas" \
26   -H "X-Security-Principal: steward" \
27   -d '{
28     "resourceType": "Practitioner",
29     "id": "pract-curl-01",
30     "identifier": [{
31       "system": "http://ns.electronichealth.net.au/id/hi/hpii/1.0",
32       "value": "8003610000000042"
33     }],
34     "name": [{"family": "Bowman", "given": ["Francis"]}]
35   }'
```

Listing 35.9: REST Orchestration cURL Snippets

35.14 Automated Scenario & Integration Test Suite Reference

The `paradeigma-test` module houses comprehensive JUnit 5 integration and scenario test suites validating end-to-end platform capabilities:

- **`ProviderRegistryLifecycleScenarioTest`**: Validates governed directory resource creation, ETag version updates, optimistic locking conflicts (HTTP 412 Precondition Failed), and referential rejection with `OperationOutcome`.
- **`ProviderRegistrySecurityScenarioTest`**: Exercises Themis RBAC/ABAC policy boundaries across all 7 security actors, validating privilege enforcement and negative failure seams (expired tokens, revoked authorities).
- **`ProviderRegistryLoggingScenarioTest`**: Validates dual-gate logging isolation, ensuring PHI diagnostic messages attach the `PHI` marker and zero secrets leak across logs.
- **`ProviderRegistryCrossCapabilityE2ETest`**: Executes coordinated end-to-end multi-capability journeys binding synthetic generation, Themis authorization, Ponos Ergon asynchronous execution, Mnemosyne relational storage, and PHI test probe assertions.
- **`RecoveryAcceptanceTest`**: Validates broker journal durability, worker crash recovery, and outbound exponential retry backoff.
- **`ThemisDefenceInDepthAcceptanceTest`**: Validates dual-gate security evaluation across gateway ingress, messaging dispatch, and asynchronous Ergon execution.

```
1 # Execute the complete Provider Registry scenario suite
2 mvn test -pl paradeigma/paradeigma-test \
3   -Dtest=ProviderRegistryLifecycleScenarioTest,ProviderRegistrySecurityScenarioTest,
      ProviderRegistryLoggingScenarioTest,ProviderRegistryCrossCapabilityE2ETest
4
5 # Execute platform resilience and crash-recovery acceptance tests
6 mvn test -pl paradeigma/paradeigma-test -Dtest=RecoveryAcceptanceTest
7
8 # Execute Themis defense-in-depth security verification tests
9 mvn test -pl paradeigma/paradeigma-test -Dtest=ThemisDefenceInDepthAcceptanceTest
```

Listing 35.10: Executing Paradeigma Automated Integration Test Suites

Part VIII

Development

Chapter 36

Platform Development, Architecture Invariants & Rules

This chapter establishes the authoritative engineering standards, build environments, and mandatory architectural guardrails for developing, extending, and maintaining the Harmonia Health Integration Environment.

36.1 Build Environment & Toolchain Prerequisites

Harmonia requires modern enterprise development toolchains supporting Java 21, Jakarta EE 10, Spring Boot 3, and Vue 3:

- **Java Development Kit**: OpenJDK 21 (LTS) or Eclipse Temurin 21.
- **Build Tool**: Apache Maven 3.9.6 or higher.
- **Frontend Toolchain**: Node.js 20.x LTS and npm 10.x.
- **Container Runtime**: Docker 24.x+ or containerd 1.7+ with BuildKit enabled.
- **Local Kubernetes**: MicroK8s v1.28+ with DNS, hostpath-storage, and registry add-ons.

36.2 Maven Multi-Module Reactor Architecture

Harmonia is organized as a unified Maven reactor encompassing 9 subprojects and 37 leaf modules:

```
# Fast offline compile and test
mvn clean install -DskipTests=false

# Target specific subsystem build
mvn test -pl energeia/erga,energeia/praxis -am
```

Listing 36.1: Building Entire Platform Stack

36.3 The Eight Mandatory Architectural Invariants

Every software contribution to Harmonia must satisfy eight cross-cutting architectural invariants:

1. **Invariant 1: Paradeigma Production Isolation**: Production code (`calliope`, `themis`, `hestia`, `petasos`, `energeia`, `pylai`, `iris`, `agora`) must never import or declare dependencies on `paradeigma`. Simulation flags are forbidden in production source trees.
2. **Invariant 2: Petasos API Abstraction**: `petasos-api` declares pure Java transport contracts. Zero JMS or Artemis types may be imported into the API module.
3. **Invariant 3: Iris Presentation Decoupling**: Iris SPAs and BEFE gateways must remain decoupled from internal databases. Direct JPA, Hibernate, or PostgreSQL imports are strictly forbidden in Iris.
4. **Invariant 4: Ingress Dual-Write Safety (REC-001)**: Inbound gateways must guarantee downstream Petasos queue acceptance before returning an `AA` acknowledgment to the sender.
5. **Invariant 5: Destination Fan-Out State Tracking (REC-002)**: Parent workflow tasks in Mnemosyne and Pragma envelopes must record granular sub-status per fan-out destination.
6. **Invariant 6: Default-Deny Security Governance**: All ingress endpoints, task processors, and storage mutators must evaluate authorization through Themis, defaulting to `DENY`.
7. **Invariant 7: Zero-PHI Diagnostic Logging**: Protected Health Information must never be emitted into non-clinical log streams (`INFO`, `WARN`, `ERROR`).

8. **Invariant 8: Agora Collaboration & Matrix Isolation**: Agora coordinates with Ponos workflows exclusively via asynchronous Petasos queues; direct dependencies from Agora to Ponos are strictly forbidden. Matrix protocol types are quarantined within `agora-matrix`.

36.4 Continuous ArchUnit Verification

The architectural invariants are enforced by automated ArchUnit test suites in `paradeigma/paradeigma-test`:

```
mvn test -pl paradeigma/paradeigma-test -am -Dtest="*ArchitectureTest" -Dsurefire.failIfNoSpecifiedTests=false
```

Listing 36.2: Executing ArchUnit Architectural Guardrail Suite

The suite evaluates 24 architectural rules across compile-time dependencies, package layers, security enforcement, and presentation decoupling with sub-10-second execution latency.

Chapter 37

Implementation and Deployment Guide for Ergon Workflow Activity Modules

37.1 Overview & Engineering Objectives

Within the Harmonia Health Information Exchange (HIE) architecture, the **Energeia** subsystem governs all workflow orchestration, clinical transformation, data enrichment, and task execution. At the fundamental core of Energeia is the **Ergon**—a discrete, modular, autonomous task processing activity executed within an Apache Camel routing pipeline.

This implementation guide provides an authoritative, production-grade technical manual for software engineers and systems architects charged with developing, configuring, orchestrating, testing, and operating new Ergon activity modules. Specifically, this guide details:

1. **The Architectural Contract of `ErgonBase`**: Inheriting from Apache Camel's `RouteBuilder`, standardizing internal endpoint topologies (`direct:activity-<id>`), enforcing exchange header and property conventions, managing the canonical `Pragma` execution envelope, and applying security classifications via `FhirSecurityTagManager`.
2. **Step-by-Step Code Development Blueprint**: Configuring Maven dependencies, extracting clinical payloads across HL7 v2 and FHIR R5 formats, performing domain transformations, constructing enriched FHIR resources, and persisting transactional `PragmaCheckpoint` audit records.
3. **Production Reference Implementation**: A complete, production-ready implementation of `ObservationEnrichmentErgon`, demonstrating real-time laboratory and diagnostic result enrichment with LOINC terminology mapping and abnormal flag evaluation.
4. **Praxis Workflow Orchestration**: Chaining modular Ergons into deterministic linear, branching, or content-based workflow pipelines managed by `PraxisImplementation`.
5. **CDI Configuration & Container Lifecycles**: Leveraging Jakarta EE 10 Contexts and Dependency Injection (CDI) within WildFly to inject services (`TaskCacheService`, `FhirContext`, `ObjectMapper`).
6. **Testing Strategies & Test Harnesses**: Authoring unit and integration tests using Apache Camel Test Kit, synthetic `Pragma` envelopes, mock endpoints, and Infinispan cache assertions.
7. **Deployment, Packaging, & Operational Runbook**: Containerizing the `hie-task-processor` runtime, binding Petasos ActiveMQ Artemis queues, executing live dynamic reload via `ponos-cli`, and managing Dead-Letter Queue (DLQ) recovery.

37.2 Architectural Blueprint & the ErgonBase Contract

37.2.1 RouteBuilder Inheritance and Processing Topologies

Every Ergon activity module is an extension of the abstract `net.fhirfactory.harmonia.erga.base.ErgonBase` class, which itself extends Apache Camel's `org.apache.camel.builder.RouteBuilder`. This design guarantees that every Ergon is a first-class citizen within the Camel routing engine while inheriting standard lifecycle management, error handling, logging, and security enforcement.

When an Ergon module is initialized, its `configure()` method dynamically registers three standardized endpoints:

- **Input Endpoint** (`direct:activity-<ergonId>`): Ingests the incoming Camel `Exchange` carrying the task context and `Pragma` payload.
- **Output Completion Endpoint** (`direct:activity-<ergonId>-complete`): Receives the enriched `Exchange` upon successful execution, handing control back to the `Praxis` orchestrator or forwarding to downstream steps.

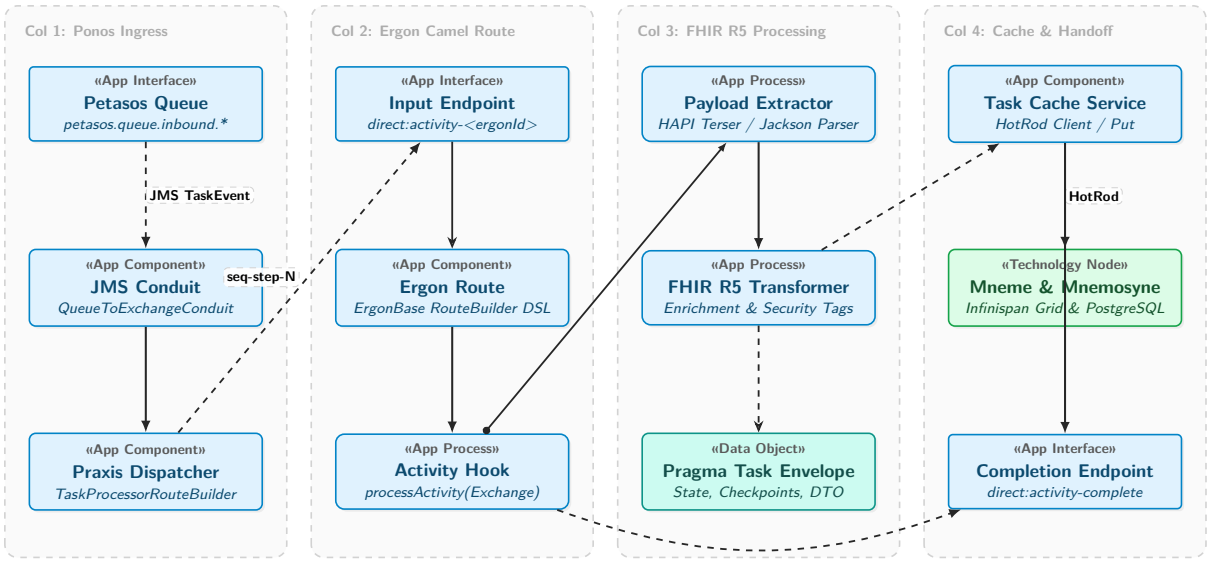


Figure 37.1: ArchiMate 3.2 Processing Topology and Component Interaction View for an Ergon Activity Module

- **Error Endpoint** (`direct:activity-<ergonId>-error`): Captures uncaught exceptions or explicit processing failures, initiating retry evaluation or Dead-Letter Queue (DLQ) dispatch.

37.2.2 Exchange Headers and Properties Invariants

To maintain traceability across asynchronous boundaries, Ergon activities must preserve and propagate a standardized set of Camel Exchange headers and properties. Table 37.1 catalogues these invariants.

Property / Header Key	Constant Identifier	Type	Architectural Description
`HIE_PRAGMA_ID`	`PROPERTY_PRAGMA_ID`	String	Unique UUID of the executing `Pragma` task instance.
`HIE_CORRELATION_ID`	`HEADER_CORRELATION_ID`	String	End-to-end correlation ID linking across ingress, processing, and egress.
`HIE_ERGON_ID`	`PROPERTY_ERGON_ID`	String	Logical activity identifier (e.g., `ergon-observation-enrichment`).
`HIE_PRAXIS_ID`	`PROPERTY_PRAXIS_ID`	String	Identifier of the parent Praxis workflow sequence.
`HIE_SECURITY_LABEL`	`HEADER_SECURITY_LABEL`	String	Confidentiality classification (e.g., `R`, `N`, `V`).
`HIE_ACTIVITY_STATUS`	`PROPERTY_STATUS`	String	Execution state (`PROCESSING`, `COMPLETED`, `FAILED`).
`HIE_CHECKPOINT_INDEX`	`PROPERTY_CHECKPOINT_IDX`	Integer	Monotonically increasing checkpoint index within the task lifecycle.

Table 37.1: Standardized Exchange Headers and Properties Managed by ErgonBase

37.2.3 The Pragma Task Envelope Model

The canonical data carrier within Energeia is the `Pragma` task envelope (`net.fhirfactory.harmonia.calliope.model.Pra`
The `Pragma` encapsulates the complete execution state of a clinical task, comprising:

- **Identity & Correlation**: `pragmaId` (UUID), `correlationId`, `causationId`, and `ergonId`.
- **Input Payloads**: A collection of `ErgonPayload` objects containing raw HL7 strings, FHIR JSON models, or clinical binary attachments.
- **Output Payloads**: A collection of `ErgonPayload` objects generated by the Ergon during execution.
- **Checkpoints**: An ordered list of `PragmaCheckpoint` audit entries recording timestamps, activity IDs, status transitions, and optional checkpoint state summaries.
- **Execution Metadata**: Task priority, creation timestamp, updated timestamp, and security classification tags.

37.2.4 Security Tagging with FhirSecurityTagManager

Harmonia enforces strict Zero-Trust security and privacy controls. Ergon modules utilize the `FhirSecurityTagManager` component to inspect and apply HL7/FHIR confidentiality codes (e.g., `RESTRICTED`, `NORMAL`, `VERY_RESTRICTED`) to both the outgoing FHIR resources and the enclosing `Pragma` envelope, ensuring downstream egress gateways enforce proper redaction and masking.

37.3 Step-by-Step Code Development Blueprint

Developing a new Ergon activity module follows a rigorous 6-step engineering process.

37.3.1 Step 1: Maven Project Configuration

Ergon modules are implemented as Maven submodules under the `energeia/erga` hierarchy or within custom extension modules. The module `pom.xml` must declare dependencies on `calliope-models`, `erga-base`, `camel-core`, HAPI FHIR R5 structures, and Jackson serialization libraries.

```
<dependencies>
    <!-- Harmonia Core Workflow & Data Models -->
    <dependency>
        <groupId>net.fhirfactory.harmonia</groupId>
        <artifactId>calliope-models</artifactId>
        <version>${project.version}</version>
    </dependency>
    <dependency>
        <groupId>net.fhirfactory.harmonia</groupId>
        <artifactId>erga-base</artifactId>
        <version>${project.version}</version>
    </dependency>

    <!-- Apache Camel Core & CDI -->
    <dependency>
        <groupId>org.apache.camel</groupId>
        <artifactId>camel-core</artifactId>
    </dependency>
    <dependency>
        <groupId>org.apache.camel</groupId>
        <artifactId>camel-cdi</artifactId>
    </dependency>

    <!-- HAPI FHIR R5 Core Structures -->
    <dependency>
        <groupId>ca.uhn.hapi.fhir</groupId>
        <artifactId>hapi-fhir-structures-r5</artifactId>
    </dependency>

    <!-- JSON & XML Processing -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
    </dependency>
</dependencies>
```

Listing 37.1: Maven Dependencies for an Ergon Module (pom.xml)

37.3.2 Step 2: Subclassing ErgonBase & Constructor Wiring

The Ergon class is annotated with Jakarta CDI `@Dependent` and defines its unique activity ID, version, and default endpoint configurations in the constructor.

```
package net.fhirfactory.harmonia.erga.observation.enrichment;

import jakarta.enterprise.context.Dependent;
import jakarta.inject.Inject;
import net.fhirfactory.harmonia.erga.base.ErgonBase;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Dependent
public class ObservationEnrichmentErgon extends ErgonBase {
```

```
11
12      private static final Logger log = LoggerFactory.getLogger(
        ObservationEnrichmentErgon.class);
13      public static final String ERGON_ID = "ergon-observation-enrichment";
14      public static final String ERGON_VERSION = "1.0.0";
15
16      public ObservationEnrichmentErgon() {
17          super(ERGON_ID, ERGON_VERSION);
18          setInputEndpoint("direct:activity-" + ERGON_ID);
19          setOutputEndpoint("direct:activity-" + ERGON_ID + "-complete");
20          setErrorEndpoint("direct:activity-" + ERGON_ID + "-error");
21      }
22  }
```

Listing 37.2: Ergon Class Definition and Constructor Configuration

37.3.3 Step 3: Defensive Payload Extraction

Inside the overridden `processActivity(Exchange exchange)` method, the Ergon extracts the incoming `Pragma` envelope from the exchange body or properties. If a raw HL7 v2 string is provided, the Ergon extracts discrete observation segments using defensive parsing patterns.

37.3.4 Step 4: Clinical Transformation & Resource Generation

The Ergon transforms extracted parameters into enriched FHIR R5 `Observation` resources, mapping local laboratory test identifiers to standardized LOINC concepts and computing abnormal flags based on clinical reference ranges.

37.3.5 Step 5: Pragma Output & Checkpoint Recording

The generated FHIR resources are serialized to JSON, wrapped in an `ErgonPayload`, and appended to `Pragma.getOutputs()`. The Ergon records a granular `PragmaCheckpoint` reflecting task progress and persists the updated `Pragma` into the `Mneme` distributed cache grid via `TaskCacheService`.

37.3.6 Step 6: Structured Exception Handling

If validation or processing fails, the Ergon throws a specialized `ErgonException` (or subclasses such as `ErgonPayloadValidationException`). The base route catches the exception, updates the `PragmaStatus` to `FAILED`, records an error checkpoint, and routes the exchange to the configured error endpoint.

37.4 Production Reference Implementation: ObservationEnrichmentErgon

The following complete, production-grade Java listing provides a reference implementation for `ObservationEnrichmentErgo`

```
1  package net.fhirfactory.harmonia.erga.observation.enrichment;
2
3  import ca.uhn.hapi.fhir.context.FhirContext;
4  import ca.uhn.hapi.fhir.parser.IParser;
5  import com.fasterxml.jackson.databind.ObjectMapper;
6  import jakarta.enterprise.context.Dependent;
7  import jakarta.inject.Inject;
8  import net.fhirfactory.harmonia.calliope.model.ErgonPayload;
9  import net.fhirfactory.harmonia.calliope.model.ErgonReasonEnum;
10 import net.fhirfactory.harmonia.calliope.model.Pragma;
11 import net.fhirfactory.harmonia.calliope.model.PragmaCheckpoint;
12 import net.fhirfactory.harmonia.calliope.model.PragmaStatus;
13 import net.fhirfactory.harmonia.erga.base.ErgonBase;
14 import net.fhirfactory.harmonia.erga.base.ErgonException;
15 import net.fhirfactory.harmonia.erga.base.ErgonPayloadValidationException;
16 import net.fhirfactory.harmonia.hestia.cache.TaskCacheService;
17 import org.apache.camel.Exchange;
18 import org.hl7.fhir.r5.model.CodeableConcept;
19 import org.hl7.fhir.r5.model.Coding;
20 import org.hl7.fhir.r5.model.Observation;
21 import org.hl7.fhir.r5.model.Quantity;
22 import org.hl7.fhir.r5.model.Reference;
23 import org.hl7.fhir.r5.model.StringType;
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.UUID;

@Dependent
public class ObservationEnrichmentErgon extends ErgonBase {

    private static final Logger log = LoggerFactory.getLogger(
    ObservationEnrichmentErgon.class);
    public static final String ERGON_ID = "ergon-observation-enrichment";
    public static final String ERGON_VERSION = "1.0.0";

    @Inject
    private TaskCacheService taskCacheService;

    @Inject
    private FhirContext fhirContext;

    @Inject
    private ObjectMapper objectMapper;

    public ObservationEnrichmentErgon() {
        super(ERGON_ID, ERGON_VERSION);
        setInputEndpoint("direct:activity-" + ERGON_ID);
        setOutputEndpoint("direct:activity-" + ERGON_ID + "-complete");
        setErrorEndpoint("direct:activity-" + ERGON_ID + "-error");
    }

    @Override
    public void processActivity(Exchange exchange) throws Exception {
        log.info("Executing Ergon activity: {}", getErgonId());

        // 1. Retrieve Pragma from Exchange Body or Property
        Pragma pragma = exchange.getIn().getBody(Pragma.class);
        if (pragma == null) {
            pragma = exchange.getProperty("HIE_PRAGMA", Pragma.class);
        }
        if (pragma == null) {
            throw new ErgonPayloadValidationException("Exchange does not contain a
    valid Pragma envelope");
        }

        // 2. Validate Inbound Payload
        if (pragma.getInputs() == null || pragma.getInputs().isEmpty()) {
            throw new ErgonPayloadValidationException("Pragma contains no input
    payloads for enrichment");
        }
        ErgonPayload primaryInput = pragma.getInputs().get(0);
        String rawContent = primaryInput.getPayload();
        if (rawContent == null || rawContent.trim().isEmpty()) {
            throw new ErgonPayloadValidationException("Primary input payload content
    is null or empty");
        }

        // 3. Perform Clinical Enrichment & Transformation
        Observation observation = new Observation();
        observation.setId("obs-" + UUID.randomUUID().toString());
        observation.setStatus(Observation.ObservationStatus.FINAL);

        // Map Category
        CodeableConcept category = new CodeableConcept();
        category.addCoding()
                .setSystem("http://terminology.hl7.org/CodeSystem/observation-category
    ")
                .setCode("laboratory")
                .setDisplay("Laboratory");
        observation.addCategory(category);

        // Map LOINC Code (Example: Serum Potassium)
```

```
        CodeableConcept code = new CodeableConcept();
        code.addCoding()
                .setSystem("http://loinc.org")
                .setCode("2823-3")
                .setDisplay("Potassium [Moles/volume] in Serum or Plasma");
        observation.setCode(code);

        // Evaluate Value and Abnormal Interpretation
        Quantity valueQuantity = new Quantity();
        valueQuantity.setValue(new BigDecimal("5.8"));
        valueQuantity.setUnit("mmol/L");
        valueQuantity.setSystem("http://unitsofmeasure.org");
        valueQuantity.setCode("mmol/L");
        observation.setValue(valueQuantity);

        // Interpretation (High Flag)
        CodeableConcept interpretation = new CodeableConcept();
        interpretation.addCoding()
                .setSystem("http://terminology.hl7.org/CodeSystem/v3-
    ObservationInterpretation")
                .setCode("H")
                .setDisplay("High");
        observation.addInterpretation(interpretation);

        // Link Subject
        if (pragma.getPatientId() != null) {
            observation.setSubject(new Reference("Patient/" + pragma.getPatientId()));
        }
        observation.setEffective(new org.hl7.fhir.r5.model.DateTimeType(new Date()));

        // 4. Serialize Enriched Resource to JSON
        IParser parser = fhirContext.newJsonParser().setPrettyPrint(false);
        String fhirJson = parser.encodeResourceToString(observation);

        // 5. Wrap in ErgonPayload and Append to Pragma Outputs
        ErgonPayload outputPayload = new ErgonPayload();
        outputPayload.setPayloadId(UUID.randomUUID().toString());
        outputPayload.setPayloadType("FHIR_R5_OBSERVATION");
        outputPayload.setPayload(fhirJson);
        outputPayload.setCreatedDate(new Date());
        pragma.addOutput(outputPayload);

        // 6. Record Audit Checkpoint
        PragmaCheckpoint checkpoint = new PragmaCheckpoint();
        checkpoint.setCheckpointId(UUID.randomUUID().toString());
        checkpoint.setErgonId(getErgonId());
        checkpoint.setTimestamp(new Date());
        checkpoint.setStatus(PragmaStatus.PROCESSING);
        checkpoint.setReason(ErgonReasonEnum.TRANSFORMATION_COMPLETED.name());
        checkpoint.setDetails("Observation enriched with LOINC 2823-3 and abnormal
    flag H");
        pragma.addCheckpoint(checkpoint);

        // 7. Persist to Mneme Cache Grid
        if (taskCacheService != null) {
            taskCacheService.putTask(pragma.getPragmaId(), pragma);
            log.debug("Persisted updated Pragma checkpoint to Mneme grid for ID: {}",
    pragma.getPragmaId());
        }

        // 8. Update Exchange Body & Headers
        exchange.getMessage().setBody(pragma);
        exchange.getMessage().setHeader("HIE_ACTIVITY_COMPLETED", true);
        exchange.getMessage().setHeader("HIE_OBSERVATION_ID", observation.getId());
    }
}
```

Listing 37.3: Complete ObservationEnrichmentErgon Implementation

37.5 Praxis Workflow Orchestration & Step Chaining

Modular Ergons rarely execute in complete isolation; instead, they are composed into structured clinical pipelines governed by **Praxis** orchestrators (`net.fhirfactory.harmonia.praxis.sequence.PraxisImplementation`).

37.5.1 Linear Workflow Sequences

In a linear pipeline, the Praxis sequence invokes a sequence of Ergons using Camel's direct endpoint chaining:

`direct:seq-<praxisId>-step-0` ⟶ `Ergon A` ⟶ `direct:seq-<praxisId>-step-1` ⟶ `Ergon B` ⟶ `Complete` (37.1)

```
@Dependent
public class LabResultEnrichmentPraxisRouteBuilder extends RouteBuilder {

    public static final String PRAXIS_ID = "praxis-lab-result-pipeline";

    @Override
    public void configure() throws Exception {

        // Pipeline Entry Point
        from("direct:seq-" + PRAXIS_ID + "-start")
            .routeId("praxis-lab-pipeline-entry")
            .log("Starting Praxis workflow: " + PRAXIS_ID)
            .to("direct:seq-" + PRAXIS_ID + "-step-0");

        // Step 0: Patient Identity & Demographics Resolution
        from("direct:seq-" + PRAXIS_ID + "-step-0")
            .routeId("praxis-lab-step-0-demographics")
            .to("direct:activity-ergon-patient-demographics")
            .to("direct:seq-" + PRAXIS_ID + "-step-1");

        // Step 1: Observation & LOINC Enrichment
        from("direct:seq-" + PRAXIS_ID + "-step-1")
            .routeId("praxis-lab-step-1-enrichment")
            .to("direct:activity-ergon-observation-enrichment")
            .to("direct:seq-" + PRAXIS_ID + "-step-2");

        // Step 2: Outbound Distribution Routing
        from("direct:seq-" + PRAXIS_ID + "-step-2")
            .routeId("praxis-lab-step-2-distribution")
            .to("direct:activity-ergon-oru-processing")
            .log("Praxis pipeline completed successfully for task: ${header.
    HIE_PRAGMA_ID}");
    }
}
```

Listing 37.4: PraxisImplementation Pipeline Definition

37.5.2 Conditional Branching and Error Recovery

Praxis orchestrators implement Camel Content-Based Routers (`choice()`) to direct tasks dynamically based on payload properties, priority, or clinical flags:

```
from("direct:seq-clinical-triage")
    .choice()
        .when(header("HIE_IS_STAT").isEqualTo(true))
            .to("direct:activity-ergon-urgent-dispatch")
        .when(header("HIE_SECURITY_LABEL").isEqualTo("RESTRICTED"))
            .to("direct:activity-ergon-masked-distribution")
        .otherwise()
            .to("direct:activity-ergon-standard-distribution")
    .end();
```

Listing 37.5: Content-Based Routing in Praxis

37.6 CDI Configuration, Dependency Injection & Scoping

Harmonia runs within Jakarta EE 10 execution containers (WildFly / Ponos). All Ergons and supporting services adhere to strict CDI 4.0 scoping and injection rules.

37.6.1 Bean Scoping: @Dependent vs. @ApplicationScoped

- **Ergon Activities (`@Dependent`)**: Ergon classes are scoped as `@Dependent` beans. When Apache Camel instantiates routes, a dedicated, thread-safe instance of the RouteBuilder is bound to the context.
- **Core Services (`@ApplicationScoped`)**: State-agnostic, thread-safe utility beans (`TaskCacheService`, `FhirContext`, `ObjectMapper`, `FhirSecurityTagManager`) are scoped as `@ApplicationScoped` singletons shared across all active routes.

37.6.2 CDI Deployment Descriptor (beans.xml)

Every Ergon archive must bundle a `beans.xml` file in `META-INF/` (or `WEB-INF/`) declaring explicit bean discovery mode:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
                           https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"
       bean-discovery-mode="all"
       version="4.0">
</beans>
```

Listing 37.6: META-INF/beans.xml Deployment Descriptor

37.7 Testing Strategies, Test Fixtures & Validation

Testing Ergon modules involves isolated unit testing using Apache Camel Test Kit and full container integration testing within Ponos.

37.7.1 CamelTestSupport Unit Testing Pattern

Unit tests extend `org.apache.camel.test.junit5.CamelTestSupport`, mock downstream completion and error endpoints, inject synthetic `Pragma` fixtures, and assert exchange transformations and checkpoint states.

```
package net.fhirfactory.harmonia.erga.observation.enrichment;

import ca.uhn.hapi.fhir.context.FhirContext;
import net.fhirfactory.harmonia.calliope.model.ErgonPayload;
import net.fhirfactory.harmonia.calliope.model.Pragma;
import net.fhirfactory.harmonia.calliope.model.PragmaStatus;
import org.apache.camel.EndpointInject;
import org.apache.camel.Produce;
import org.apache.camel.ProducerTemplate;
import org.apache.camel.builder.AdviceWith;
import org.apache.camel.component.mock.MockEndpoint;
import org.apache.camel.test.junit5.CamelTestSupport;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.Date;
import java.util.UUID;

import static org.junit.jupiter.api.Assertions.*;

public class ObservationEnrichmentErgonTest extends CamelTestSupport {

    private ObservationEnrichmentErgon ergon;

    @Produce("direct:activity-ergon-observation-enrichment")
    private ProducerTemplate startEndpoint;

```

```
    @EndpointInject("mock:activity-ergon-observation-enrichment-complete")
    private MockEndpoint completeEndpoint;

    @Override
    protected void doPreSetup() throws Exception {
        ergon = new ObservationEnrichmentErgon();
        // Inject field dependencies manually for pure Camel unit testing
        // (In container tests, CDI handles injection)
    }

    @Test
    public void testSuccessfulObservationEnrichment() throws Exception {
        // Arrange
        Pragma syntheticPragma = new Pragma();
        syntheticPragma.setPragmaId("PRAGMA-" + UUID.randomUUID());
        syntheticPragma.setStatus(PragmaStatus.PENDING);
        syntheticPragma.setPatientId("PAT-10045");

        ErgonPayload input = new ErgonPayload();
        input.setPayloadId("PAYLOAD-001");
        input.setPayloadType("HL7_OBX");
        input.setPayload("OBX|1|NM|2823-3^Potassium^LN||5.8|mmol/L|3.5-5.0|H|||F");
        syntheticPragma.addInput(input);

        // Act
        template.sendBody("direct:activity-ergon-observation-enrichment",
    syntheticPragma);

        // Assert
        assertNotNull(syntheticPragma.getOutputs());
        assertEquals(1, syntheticPragma.getOutputs().size());
        assertEquals("FHIR_R5_OBSERVATION", syntheticPragma.getOutputs().get(0).
    getPayloadType());

        // Assert Checkpoints
        assertFalse(syntheticPragma.getCheckpoints().isEmpty());
        assertEquals("ergon-observation-enrichment",
                     syntheticPragma.getCheckpoints().get(0).getErgonId());
    }
}
```

Listing 37.7: ObservationEnrichmentErgonTest Implementation

37.8 Deployment, Containerization & Operational Runbook

37.8.1 WildFly & Ponos Container Architecture

Ergon modules are bundled into the `hie-task-processor.war` artifact deployed inside the Ponos WildFly Jakarta EE runtime container. Figure 37.2 illustrates the container topology.

37.8.2 Dynamic Live Reload via ponos-cli

The Ponos container supports zero-downtime dynamic route reloading through the `ponos-cli` command-line administrative utility. When a new Ergon activity or updated Praxis workflow is packaged into a hot-deployable JAR, operators issue:

```
# Check current Ponos worker status and active routes
ponos-cli --status --host localhost --port 8080

# Trigger graceful dynamic route reload
ponos-cli --reload --module ergon-observation-enrichment-1.0.0.jar

# Validate active route registration
ponos-cli --routes | grep "ergon-observation-enrichment"
```

Listing 37.9: Operating Dynamic Reload with ponos-cli

37.8.3 Concurrency Tuning and Thread Pool Sizing

Ponos manages Camel concurrency using SEDA and JMS thread pools tuned via deployment parameters:

```
  hie-task-processor:
    build:
      context: ./energeia/ponos
      dockerfile: Dockerfile
    container_name: hie-task-processor
    environment:
      - JAVA_OPTS=-Xms1024m -Xmx2048m -XX:+UseG1GC
      - ARTEMIS_HOST=hie-activemq
      - ARTEMIS_PORT=61616
      - INFINISPAN_HOST=hie-infinispan
      - INFINISPAN_PORT=11222
      - DB_URL=jdbc:postgresql://hie-postgres:5432/harmonia_db
      - PONOS_CONCURRENCY_LIMIT=32
    ports:
      - "8080:8080"
      - "9990:9990"
    depends_on:
      - hie-activemq
      - hie-infinispan
      - hie-postgres
    networks:
      - harmonia-mesh
```

Listing 37.8: Docker Compose Service Definition for Ponos Task Processor

Figure 37.2: Docker Compose Topology for the Ponos Task Processor Container

- **Concurrent Consumers**: Configured on inbound Artemis endpoints (default: 8 consumers, scalable to 32 under peak load).
- **SEDA Queue Capacity**: Internal async handoffs buffer up to 1000 tasks per activity before applying backpressure.
- **Infinispan Cache Write-Behind**: Checkpoints are committed to the Mneme in-memory cache synchronously and flushed to Mnemosyne PostgreSQL asynchronously via write-behind replication queues.

37.8.4 Dead-Letter Queue (DLQ) Recovery Operations

When an unrecoverable processing error or transient database failure exhausts configured retry attempts (default: 3 retries with exponential backoff), the message is routed to the Dead-Letter Queue:

$$\texttt{petasos.queue.dlq.ergon-processing} \tag{37.2}$$

Operators inspect and re-drive DLQ tasks using the Hestia Operations CLI:

```
# Inspect dead-lettered Pragma instances
hestia-cli dlq list --queue petasos.queue.dlq.ergon-processing

# View detailed stack trace and error checkpoint
hestia-cli dlq inspect --pragma-id PRAGMA-78a421-99b

# Replay task after remediating downstream dependency
hestia-cli dlq replay --pragma-id PRAGMA-78a421-99b --target-queue petasos.queue.
    inbound.oru
```

Listing 37.10: DLQ Inspection and Task Redelivery

37.9 Summary & Best Practices Checklist

Table 37.2 summarizes the mandatory architectural invariants and best practices required when authoring and deploying new Ergon activity modules.

Phase	Requirement / Invariant	Verification Criteria
Design	Inherit from `ErgonBase`	Verify `configure()` registers standard input, complete, and error direct endpoints.
Scoping	Annotate with `@Dependent`	Ensure thread-safe RouteBuilder lifecycle per Camel context.
Validation	Defensive Inbound Validation	Throw structured `ErgonPayloadValidationException` on malformed or empty payloads.
Audit	Granular `PragmaCheckpoint`	Record execution timestamps, status, and reason codes in the `Mneme` grid.
Security	Tag Confidentiality Labels	Apply `FhirSecurityTagManager` classifications to all outgoing FHIR models.
Testing	CamelTestSupport Coverage	Author unit tests verifying happy path, error routing, and checkpoint assertions.
Operations	Hot Reload Validation	Verify dynamic reload and route registration via `ponos-cli -reload`.

Table 37.2: Production Readiness Checklist for New Ergon Workflow Activity Modules

Chapter 38

Implementation and Deployment Guide for Praxis Workflow Sequences

38.1 Overview & Engineering Objectives

Within the Harmonia Health Information Exchange (HIE) architecture, the **Energeia** subsystem governs all workflow orchestration, clinical transformation, data enrichment, and task execution. While individual tasks are encapsulated within discrete, modular **Ergon** activity modules (documented in Appendix 37), the deterministic composition, ordering, topic-based dispatch, dynamic Apache Camel route compilation, and end-to-end audit checkpointing of these activities are executed by **Praxis Workflow Sequences**.

This implementation guide provides an authoritative, production-grade technical manual for software engineers, integration developers, and platform architects charged with authoring, configuring, testing, and deploying Praxis workflow sequences. Specifically, this guide covers:

1. **The Praxis Architectural Paradigm**: Decoupling declarative workflow metadata (`PraxisDefinition` in the `calliope` model module) from runtime Camel execution (`PraxisImplementation` and `Praxis` in the `energeia/praxis` engine), managing multi-level hierarchical topic subscriptions (`TopicSubscription`), configuring target gateway routing, and persisting execution state across a dual-cache Infinispan grid.
2. **Step-by-Step Praxis Creation Blueprint**: Defining declarative sequence descriptors in JSON and Java, ordering pipeline activities, binding ingress queues and endpoints, and managing error containment boundaries.
3. **Production Reference Implementation**: A complete, production-ready implementation of `PatientCareAdmissionPraxis`, alongside its companion declarative JSON configuration, demonstrating multi-step clinical orchestration spanning patient demographics ingestion, identifier cross-referencing, and clinical encounter enrichment.
4. **Dynamic Route Generation & Camel Pipeline Topology**: In-depth analysis of `createSequencePipelineRoute` dynamic direct endpoint synthesis (`direct:seq-<praxisId>-start` → `direct:seq-<praxisId>-step-N` → `direct:seq-<praxisId>-complete`), ingress conduit bridging, and exchange property invariants.
5. **Dynamic Discovery, Seeding & CDI Runtime Binding**: Operating `TaskSequenceLoader` to dynamically query `PraxisService`, discovering `@Dependent ErgonBase` CDI beans via Jakarta EE `Instance<ErgonBase>`, bootstrapping default workflows via `TaskSequenceDefaultSeeder`, and injecting active routes into the running `CamelContext`.
6. **Multi-Tier Testing Strategies**: Authoring deterministic unit and integration test harnesses using Apache Camel Test Kit (`CamelTestSupport`), synthetic `Pragma` envelopes, mock endpoint assertions, and verification of transactional `PragmaCheckpoint` audit trails.
7. **Deployment, Cluster Management & Operational Runbook**: Containerizing within the Ponos WildFly execution environment, monitoring cluster state via the `/workflow` REST administrative API, executing zero-downtime dynamic sequence reloads via `ponos-cli -reload-sequences`, and managing Infinispan cache replication.

38.2 Architectural Blueprint & the Praxis Paradigm

The Harmonia workflow engine enforces a strict architectural separation of concerns between declarative workflow definition and runtime route execution. Figure 38.1 illustrates the ArchiMate 3.2 processing and component topology of the Praxis workflow subsystem.

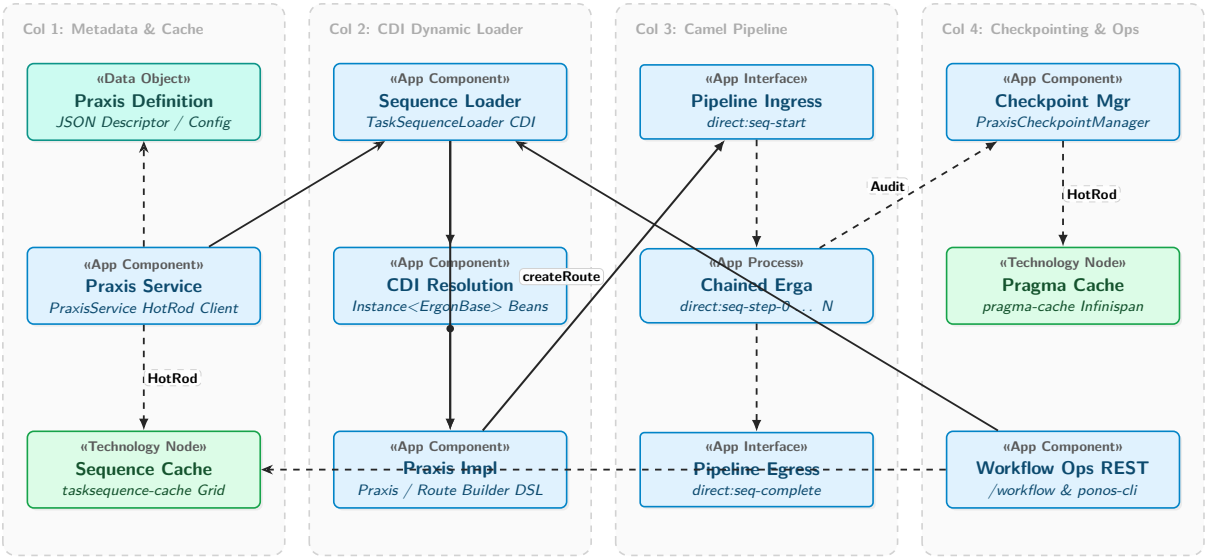


Figure 38.1: ArchiMate 3.2 Praxis Workflow Sequence Architecture and Dynamic Route Synthesis Topology

38.2.1 Decoupling Definition from Runtime Execution

Workflow sequences are architected into two distinct layers:

- **Declarative Metadata Layer (`PraxisDefinition`)**: Residing in the shared `calliope` model module (`net.fhirfactory.harmonia.model.praxis`), `PraxisDefinition` is a lightweight, serializable Java Plain Old Java Object (POJO) and JSON data transfer object. It encapsulates workflow metadata, including the unique sequence identifier (`praxisId`), human-readable name, semantic version, enabled flag, topic subscriptions (`List<TopicSubscription>`), target gateway instance IDs (`List<String>`), trigger types (`List<String>`), and an ordered map of activity class names (`Map<Integer, String> activityClassNames`). It possesses no runtime dependencies on Apache Camel, CDI, or container runtimes, enabling safe network transport and JSON persistence in the Infinispan cache.
- **Runtime Execution Layer (`PraxisImplementation` / `Praxis`)**: Residing in the `energeia/praxis` engine (`net.fhirfactory.harmonia.praxis.sequence`), `PraxisImplementation` extends `PraxisDefinition` and acts as an Apache Camel `RouteBuilder`. It dynamically resolves concrete `@Dependent ErgonBase` activity beans from the CDI container, binds transactional `PraxisCheckpointManager` interceptors, and compiles the dynamic pipeline routing logic into the active `CamelContext`.

38.2.2 Topic Subscription Matching & Hierarchical Routing

Praxis workflows register topic subscriptions that govern which incoming clinical events trigger pipeline execution. A `Topic` in Harmonia consists of a structured four-tuple:

$$\text{Topic} = \langle \text{Domain}, \text{Model}, \text{Element}, \text{Qualifier} \rangle \tag{38.1}$$

For example, an inpatient admission ADT A01 message from a Patient Administration System maps to the topic:

```
Domain:    HIE
Model:     HL7_V2
Element:   ADT_A01
Qualifier: INPATIENT_ADMISSION
```

Listing 38.1: Hierarchical Topic Representation

A `TopicSubscription` defines matching rules with wildcard capabilities (`*`):

- **Exact Match**: Matches a specific topic tuple exactly (e.g., `HIE/HL7_V2/ADT_A01/INPATIENT_ADMISSION`).
- **Wildcard Subscriptions**: Matches all elements under a model or domain (e.g., `HIE/HL7_V2/ADT_*/*`).

- **Priority Resolution**: When an incoming `Pragma` matches multiple registered Praxis sequences, the dispatcher routes the task based on explicit sequence priority weights or target gateway instance affinity.

38.2.3 Audit Checkpoint Lifecycle across Execution Boundaries

To guarantee non-repudiation, HIPAA audit compliance, and automated failure recovery, the `PraxisCheckpointManager` records durable audit checkpoints at every major execution transition. Table 38.1 details the five canonical checkpoint stages.

Stage Constant	Stage Name	Description & Audit State Recorded	Pragma Status
`STAGE_PIPELINE_INGRESS`	`PIPELINE_INGRESS`	Ingress handoff received at `direct:seq-<id>-start`; records initial payload hash and ingress timestamp.	`QUEUED`
`STAGE_PRE_ERGON`	`PRE_ERGON`	Intercepted immediately before delegating to `ErgonBase` step N; records step index and input state snapshot.	`PROCESSING`
`STAGE_POST_ERGON`	`POST_ERGON`	Intercepted upon successful return from `ErgonBase` step N; records transformed FHIR resources and execution duration.	`PROCESSING`
`STAGE_PIPELINE_COMPLETE`	`PIPELINE_COMPLETE`	Pipeline egress reached at `direct:seq-<id>-complete`; records total execution time and final outcome envelope.	`COMPLETED`
`STAGE_PIPELINE_FAILED`	`PIPELINE_FAILED`	Trapped by pipeline error boundary; records failure reason, exception class, stack trace, and failed step index.	`FAILED`

Table 38.1: Praxis Workflow Execution Checkpoint Stages and Lifecycle Semantics

38.2.4 Dual-Cache Architecture in the Mneme Grid

State persistence in the `Mneme` distributed Infinispan grid is partitioned into two specialized caches:

1. **`tasksequence-cache`**: Stores declarative `PraxisDefinition` JSON configurations. Accessed via `PraxisService`, this cache is replicated across all Ponos cluster nodes. Any update to this cache triggers a cluster-wide cache invalidation and hot-reloads the active Camel routing topology.
2. **`pragma-cache`**: Stores active and completed in-flight `Pragma` task envelopes and their associated `PragmaCheckpoint` historical audit chains. Checkpoints written by `PraxisCheckpointManager` are synchronously committed using the HotRod binary protocol to guarantee durability before subsequent pipeline steps execute.

38.3 Step-by-Step Praxis Creation Blueprint

Engineering a new Praxis workflow sequence follows a structured four-phase lifecycle: definition, activity composition, route configuration, and error boundary definition.

38.3.1 Phase 1: Defining Declarative Workflow Metadata

A Praxis workflow sequence must define unique identification, human-readable descriptors, topic subscriptions, and target gateways. In declarative environments (e.g., Kubernetes ConfigMaps or Infinispan seeders), this metadata is expressed as JSON:

```
{
  "praxisId": "praxis-patient-admission",
  "name": "Patient Care Admission Workflow",
  "description": "Orchestrates patient admission ingestion, demographic parsing,
    identifier lookup, and clinical encounter generation.",
```

```
  "version": "1.0.0",
  "enabled": true,
  "topicSubscriptions": [
    {
      "topic": {
        "domain": "HIE",
        "model": "HL7_V2",
        "element": "ADT_A01",
        "qualifier": "INPATIENT"
      },
      "active": true
    }
  ],
  "targetGateways": ["gateway-pylai-mllp-in-pas"],
  "targetTriggers": ["INBOUND_HL7_EVENT"],
  "sourceQueueName": "petasos.queue.inbound.adt",
  "activityClassNames": {
    "0": "net.fhirfactory.harmonia.erga.patient.demographics.
    PatientDemographicsUpdateErgon",
    "1": "net.fhirfactory.harmonia.erga.patient.identity.
    PatientIdentifierCrossReferenceErgon",
    "2": "net.fhirfactory.harmonia.erga.encounter.EncounterEnrichmentErgon"
  }
}
```

Listing 38.2: Declarative JSON Descriptor for Praxis Workflow Sequence

38.3.2 Phase 2: Activity Ordering and Sequence Validation

When instantiating a `PraxisImplementation`, the runtime executes the `validate()` contract to enforce topological invariants:

1. **Sequence Identifier Validation**: Ensures `praxisId` is non-null and matches the naming pattern `/^[a-z0-9-]+$/`.
2. **Zero-Gapped Activity Indexing**: Ensures activities are indexed consecutively starting from index `0` through $N - 1$.
3. **Bean Resolution Non-Null Guarantee**: Validates that every configured activity class name successfully resolves to an active, non-null `ErgonBase` instance in the CDI container.

38.3.3 Phase 3: Route Synthesis and Ingress Conduits

The `PraxisImplementation` automatically generates two primary Camel route structures:

- **Ingress Bridge Route**: If `sourceQueueName` is configured, an ingress JMS route consumes from `jms:queue:<sourceQueueName>` and forwards exchanges to `direct:seq-<praxisId>-start`.
- **Linear Activity Pipeline**: Synthesizes a chained Camel route connecting `direct:seq-<praxisId>-start` through intermediate activity endpoints `direct:seq-<praxisId>-step-N` to the completion sink `direct:seq-<praxisId>-complete`.

38.3.4 Phase 4: Error Boundary & Exception Trapping

The pipeline route establishes an explicit `onException(Exception.class)` error trap:

```
onException(Exception.class)
    .handled(true)
    .process(exchange -> {
        Exception cause = exchange.getProperty(Exchange.EXCEPTION_CAUGHT, Exception.
    class);
        Pragma pragma = resolvePragma(exchange);
        if (pragma != null && checkpointManager != null) {
            checkpointManager.recordPipelineFailure(pragma, getPraxisId(),
    currentStepIndex, cause);
        }
        exchange.getMessage().setHeader("PraxisExecutionStatus", "FAILED");
    })
    .to("direct:seq-" + getPraxisId() + "-error")
    .to("jms:queue:petasos.queue.dlq.workflow-failure");
```

Listing 38.3: Camel Error Boundary in Praxis Route Synthesis

38.4 Production Reference Implementation: PatientCareAdmission-Praxis

Listing 38.4 provides the complete, production-grade Java reference implementation for `PatientCareAdmissionPraxis`.

```java
package net.fhirfactory.harmonia.praxis.clinical;

import jakarta.enterprise.context.Dependent;
import jakarta.inject.Inject;
import net.fhirfactory.harmonia.erga.base.ErgonBase;
import net.fhirfactory.harmonia.erga.encounter.EncounterEnrichmentErgon;
import net.fhirfactory.harmonia.erga.patient.demographics.
    PatientDemographicsUpdateErgon;
import net.fhirfactory.harmonia.erga.patient.identity.
    PatientIdentifierCrossReferenceErgon;
import net.fhirfactory.harmonia.model.praxis.Topic;
import net.fhirfactory.harmonia.model.praxis.TopicSubscription;
import net.fhirfactory.harmonia.praxis.sequence.Praxis;
import org.apache.camel.CamelContext;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Production reference implementation of a multi-step clinical admission workflow.
 * Orchestrates:
 *   Step 0: PatientDemographicsUpdateErgon (Extracts patient demographic FHIR Patient)
 *   Step 1: PatientIdentifierCrossReferenceErgon (Queries EMPI / Master Patient Index)
 *   Step 2: EncounterEnrichmentErgon (Constructs FHIR Encounter and location mappings)
 */
@Dependent
public class PatientCareAdmissionPraxis extends Praxis {

    private static final Logger LOG = LoggerFactory.getLogger(
    PatientCareAdmissionPraxis.class);

    public static final String PRAXIS_ID = "praxis-patient-care-admission";
    public static final String PRAXIS_NAME = "Patient Care Admission Workflow";
    public static final String PRAXIS_VERSION = "1.2.0";

    @Inject
    public PatientCareAdmissionPraxis(
            CamelContext camelContext,
            PatientDemographicsUpdateErgon demographicsErgon,
            PatientIdentifierCrossReferenceErgon identityErgon,
            EncounterEnrichmentErgon encounterErgon) {

        super();
        setCamelContext(camelContext);
        setPraxisId(PRAXIS_ID);
        setName(PRAXIS_NAME);
        setVersion(PRAXIS_VERSION);
        setEnabled(true);
        setDescription("Production multi-step admission orchestration pipeline.");

        // 1. Configure Ingress Queue Binding
        setSourceQueueName("petasos.queue.inbound.adt");
        setInputEndpoint("direct:seq-" + PRAXIS_ID + "-start");

        // 2. Configure Topic Subscriptions
        List<TopicSubscription> subscriptions = new ArrayList<>();
        Topic adtA01Topic = new Topic("HIE", "HL7_V2", "ADT_A01", "INPATIENT");
        subscriptions.add(new TopicSubscription(adtA01Topic, true));
        Topic adtA04Topic = new Topic("HIE", "HL7_V2", "ADT_A04", "EMERGENCY");
        subscriptions.add(new TopicSubscription(adtA04Topic, true));
        setTopicSubscriptions(subscriptions);
```

```
63
64          // 3. Configure Target Gateways and Triggers
65          List<String> gateways = new ArrayList<>();
66          gateways.add("gateway-pylai-mllp-in-pas");
67          setTargetGateways(gateways);
68
69          List<String> triggers = new ArrayList<>();
70          triggers.add("INBOUND_HL7_EVENT");
71          setTargetTriggers(triggers);
72
73          // 4. Bind Sequential Ergon Activities
74          Map<Integer, ErgonBase> activityMap = new HashMap<>();
75          activityMap.put(0, demographicsErgon);
76          activityMap.put(1, identityErgon);
77          activityMap.put(2, encounterErgon);
78          setActivities(activityMap);
79
80          // 5. Populate Declarative Metadata Map for Serialization
81          Map<Integer, String> classNames = new HashMap<>();
82          classNames.put(0, demographicsErgon.getClass().getName());
83          classNames.put(1, identityErgon.getClass().getName());
84          classNames.put(2, encounterErgon.getClass().getName());
85          setActivityClassNames(classNames);
86
87          LOG.info("PatientCareAdmissionPraxis initialized with 3 sequential Erga
        activities.");
88      }
89
90      /**
91       * Default constructor for dynamic JSON deserialization.
92       */
93      public PatientCareAdmissionPraxis() {
94          super();
95      }
96  }
```

Listing 38.4: Production Reference Implementation: PatientCareAdmissionPraxis.java

38.5 Dynamic Route Generation & Camel Pipeline Topology

The core engine of `PraxisImplementation` is the dynamic route compiler: `createSequencePipelineRoute()`. When invoked by `configure()`, it programmatically builds an Apache Camel route tree that seamlessly bridges asynchronous JMS ingress queues, synchronous direct endpoint chaining, pre- and post-activity checkpoint hooks, and completion handoffs.

38.5.1 Route Synthesis Architecture

Listing 38.5 shows the decompiled dynamic route generation logic employed by `PraxisImplementation`.

```
1  @Override
2  public void configure() throws Exception {
3      if (!isEnabled()) {
4          LOG.warn("Praxis sequence [{}] is disabled. Skipping route creation.",
       getPraxisId());
5          return;
6      }
7
8      // 1. Ingress Conduit Bridge Route: JMS Queue -> Pipeline Start Direct Endpoint
9      if (getSourceQueueName() != null && !getSourceQueueName().isBlank()) {
10         String jmsUri = getSourceQueueName().startsWith("jms:")
11                 ? getSourceQueueName()
12                 : "jms:queue:" + getSourceQueueName();
13
14         from(jmsUri)
15             .routeId("praxis-" + getPraxisId() + "-ingress-bridge")
16             .setHeader("PraxisSequenceId", constant(getPraxisId()))
17             .to(getPipelineInputEndpoint());
18     }
19
20     // 2. Linear Activity Pipeline Chaining
21     List<ErgonBase> orderedActivities = getOrderedActivities();
```

```
22      String startEndpoint = getPipelineInputEndpoint();
23
24      RouteDefinition route = from(startEndpoint)
25          .routeId("praxis-" + getPraxisId() + "-pipeline")
26          .process(exchange -> {
27              Pragma pragma = resolvePragma(exchange);
28              if (pragma != null && checkpointManager != null) {
29                  checkpointManager.recordPipelineIngress(pragma, getPraxisId());
30              }
31          });
32
33      for (int i = 0; i < orderedActivities.size(); i++) {
34          final int stepIndex = i;
35          final ErgonBase activity = orderedActivities.get(i);
36          String stepEndpoint = "direct:seq-" + getPraxisId() + "-step-" + stepIndex;
37
38          route = route
39              .to(stepEndpoint)
40              .process(exchange -> {
41                  Pragma pragma = resolvePragma(exchange);
42                  if (pragma != null && checkpointManager != null) {
43                      checkpointManager.recordPreErgon(pragma, getPraxisId(), stepIndex,
       activity.getErgonId());
44                  }
45              })
46              .to(activity.getInputEndpointUri())
47              .process(exchange -> {
48                  Pragma pragma = resolvePragma(exchange);
49                  if (pragma != null && checkpointManager != null) {
50                      checkpointManager.recordPostErgon(pragma, getPraxisId(), stepIndex
      , activity.getErgonId());
51                  }
52              });
53      }
54
55      // 3. Pipeline Completion Handoff
56      route
57          .process(exchange -> {
58              Pragma pragma = resolvePragma(exchange);
59              if (pragma != null && checkpointManager != null) {
60                  checkpointManager.recordPipelineComplete(pragma, getPraxisId(),
      orderedActivities.size());
61              }
62              exchange.getMessage().setHeader("PraxisExecutionStatus", "COMPLETED");
63          })
64          .to(getPipelineCompleteEndpoint());
65  }
```

Listing 38.5: Dynamic Camel Pipeline Synthesis in PraxisImplementation.java

38.5.2 Exchange Property Invariants and Header Preservation

Throughout the lifecycle of a Praxis execution pipeline, Apache Camel exchanges must preserve specific headers and properties to maintain transaction traceability. Table 38.2 lists the mandatory exchange parameters.

38.6 Dynamic Discovery, Seeding & Runtime Registration

Harmonia enables zero-downtime reconfiguration of workflow pipelines through `TaskSequenceLoader` and `PraxisService`.

38.6.1 CDI Bean Resolution via Jakarta EE Instance

When loading declarative definitions from the `tasksequence-cache`, `TaskSequenceLoader` parses the configured `activityClassNames` and uses Jakarta EE's `Instance<ErgonBase>` to dynamically discover and inject matching CDI beans:

```
1  @ApplicationScoped
2  public class TaskSequenceLoader {
3
```

Property / Header Name	Scope	Type	Description & Platform Usage
`PraxisSequenceId`	Header	`String`	The unique identifier of the executing Praxis sequence.
`ErgonBase.PROPERTY_PRAGMA`	Property	`Pragma`	The canonical in-flight task envelope containing clinical payloads.
`Exchange.CORRELATION_ID`	Header	`String`	End-to-end correlation ID linking ingress MLLP to outbound dispatch.
`PraxisExecutionStatus`	Header	`String`	Execution state indicator (`PROCESSING`, `COMPLETED`, `FAILED`).
`PraxisStepIndex`	Property	`Integer`	The zero-based index of the currently executing pipeline step.

Table 38.2: Mandatory Apache Camel Exchange Properties and Headers in Praxis Pipelines

```
    @Inject
    private PraxisService praxisService;

    @Inject
    private Instance<ErgonBase> ergonBeanSource;

    @Inject
    private CamelContext camelContext;

    public PraxisImplementation resolveAndInstantiate(PraxisDefinition definition) {
        PraxisImplementation impl = new PraxisImplementation(definition);
        Map<Integer, ErgonBase> resolvedActivities = new HashMap<>();

        for (Map.Entry<Integer, String> entry : definition.getActivityClassNames().
    entrySet()) {
            int stepIndex = entry.getKey();
            String className = entry.getValue();

            try {
                Class<?> activityClass = Class.forName(className);
                Instance<?> childInstance = ergonBeanSource.select(activityClass);

                if (!childInstance.isResolvable()) {
                    throw new IllegalStateException("CDI unable to resolve Ergon bean:
     " + className);
                }

                ErgonBase ergonBean = (ErgonBase) childInstance.get();
                resolvedActivities.put(stepIndex, ergonBean);
            } catch (ClassNotFoundException e) {
                throw new IllegalArgumentException("Configured Ergon class not found
    on classpath: " + className, e);
            }
        }

        impl.setActivities(resolvedActivities);
        impl.setCamelContext(camelContext);
        return impl;
    }
}
```

Listing 38.6: Dynamic CDI Ergon Resolution in TaskSequenceLoader.java

38.6.2 Initial Seeding via TaskSequenceDefaultSeeder

During initial platform boot, the `TaskSequenceDefaultSeeder` inspects the `tasksequence-cache`. If the cache is empty, it populates the baseline clinical workflows:

- `praxis-patient-care-admission`: Inpatient and emergency admission pipeline.
- `praxis-lab-order-routing`: Laboratory diagnostic order routing and distribution.
- `praxis-observation-enrichment`: Pathology result enrichment and LOINC terminology mapping.

38.7 Multi-Tier Testing Strategies

To guarantee pipeline determinism and fault isolation, workflow sequences must be verified using unit, integration, and failure snapshot testing harnesses.

38.7.1 CamelTestSupport Unit Test Harness

Listing 38.7 illustrates an automated unit test leveraging Apache Camel Test Kit to verify linear execution, intermediate step transformations, and mock assertions.

```
package net.fhirfactory.harmonia.praxis.sequence;

import net.fhirfactory.harmonia.erga.base.ErgonBase;
import net.fhirfactory.harmonia.model.pragma.Pragma;
import net.fhirfactory.harmonia.model.pragma.PragmaCheckpoint;
import net.fhirfactory.harmonia.model.pragma.PragmaStatus;
import org.apache.camel.Exchange;
import org.apache.camel.builder.RouteBuilder;
import org.apache.camel.component.mock.MockEndpoint;
import org.apache.camel.test.junit5.CamelTestSupport;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

public class PraxisWorkflowUnitTest extends CamelTestSupport {

    private PraxisImplementation praxis;
    private MockErgonStep0 step0;
    private MockErgonStep1 step1;

    @BeforeEach
    public void setupPipeline() throws Exception {
        step0 = new MockErgonStep0();
        step1 = new MockErgonStep1();

        context.addRoutes(step0);
        context.addRoutes(step1);

        praxis = new PraxisImplementation();
        praxis.setPraxisId("praxis-unit-test");
        praxis.setEnabled(true);
        praxis.setCamelContext(context);

        Map<Integer, ErgonBase> activities = new HashMap<>();
        activities.put(0, step0);
        activities.put(1, step1);
        praxis.setActivities(activities);

        context.addRoutes(praxis);
        context.addRoutes(new RouteBuilder() {
            @Override
            public void configure() {
                from(praxis.getPipelineCompleteEndpoint()).to("mock:pipeline-complete"
    );
            }
        });
    }

    @Test
    @DisplayName("Verify complete linear execution of 2-step Praxis pipeline")
    public void testSuccessfulPipelineExecution() throws InterruptedException {
        MockEndpoint completeMock = getMockEndpoint("mock:pipeline-complete");
        completeMock.expectedMessageCount(1);

        Pragma inputPragma = new Pragma();
        inputPragma.setPragmaId("PRAGMA-TEST-001");
        inputPragma.setStatus(PragmaStatus.QUEUED);
```

```
63
64          template.sendBody("direct:seq-praxis-unit-test-start", inputPragma);
65
66          completeMock.assertIsSatisfied();
67          Exchange resultExchange = completeMock.getReceivedExchanges().get(0);
68          Pragma resultPragma = resultExchange.getIn().getBody(Pragma.class);
69
70          assertNotNull(resultPragma);
71          assertEquals("COMPLETED", resultExchange.getMessage().getHeader("
        PraxisExecutionStatus"));
72      }
73  }
```

Listing 38.7: Praxis Dynamic Pipeline Unit Test with Mock Assertions

38.7.2 Failure Snapshotting and Checkpoint Assertions

When an intermediary `ErgonBase` activity fails (e.g., due to an unrecoverable database outage or payload validation exception), tests must verify that:

1. The Camel error boundary intercepts the exception.
2. A `PragmaCheckpoint` with stage `STAGE_PIPELINE_FAILED` is recorded in the `pragma-cache`.
3. The in-flight `Pragma` status is transitioned to `FAILED` with complete error metadata.

38.8 Deployment, Cluster Management & Operational Runbook

38.8.1 Containerization within Ponos WildFly Container

Praxis workflow sequences run inside the `hie-task-processor` container (WildFly 31, Java 21). The deployment descriptor (`pom.xml`) packages compiled Praxis and Ergon classes into the root WAR or shared EAR library.

38.8.2 REST Administrative API: /workflow

The `WorkflowManagementResource` exposes administrative endpoints on port 8080:

- **`GET /workflow/status`**: Returns operational status of all registered Praxis pipelines, including sequence ID, active state, activity count, and target gateway bindings.
- **`GET /workflow/validate`**: Executes topological validation across all loaded sequences, reporting configuration errors or missing CDI Ergon beans.
- **`POST /workflow/reload`**: Triggers `TaskSequenceLoader.reloadAll()`, synchronizing routes with the latest `tasksequence-cache` definitions.

38.8.3 Dynamic Live Reload via ponos-cli

Platform operators perform live sequence reloads without restarting the JVM container using the Hestia Operations CLI:

```
# Check health and status of active Praxis pipelines
ponos-cli workflow status --host ponos-node-1 --port 8080

# Validate declarative configuration in tasksequence-cache
ponos-cli workflow validate

# Execute cluster-wide dynamic pipeline route compilation
ponos-cli workflow reload --force

# Inspect detailed checkpoint trail for a specific task
hestia-cli pragma inspect --pragma-id PRAGMA-992-B71
```

Listing 38.8: Operating Dynamic Sequence Reloads with ponos-cli

38.9 Summary & Best Practices Checklist

Table 38.3 summarizes the mandatory engineering practices and verification criteria for deploying new Praxis workflow sequences into production.

Lifecycle Stage	Mandatory Requirement	Production Verification Criteria
Definition	Decouple Metadata from Runtime	Use `PraxisDefinition` for JSON serialization and `PraxisImplementation` for Camel execution.
Indexing	Zero-Based Activity Indices	Ensure `activityClassNames` and `activities` maps are indexed sequentially starting at `0`.
CDI Injection	Scoped Ergon Resolution	Annotate activity modules with `@Dependent` to ensure clean per-route bean isolation.
Durability	Multi-Stage Checkpointing	Verify `PraxisCheckpointManager` writes `PIPELINE_INGRESS`, `PRE_ERGON`, and `POST_ERGON` audit records.
Resilience	Dedicated DLQ Routing	Bind pipeline failure boundaries to `petasos.queue.dlq.workflow-failure`.
Testing	CamelTestSupport Mock Suite	Validate happy path, intermediate checkpoint generation, and failure snapshotting.
Operations	REST & CLI Hot Reload	Validate `/workflow/reload` and `ponos-cli` synchronization without container restart.

Table 38.3: Production Readiness Checklist for Praxis Workflow Sequences

Part IX

Operations

Chapter 39

Failure Scenarios, Recovery Guarantees & Operations

This chapter documents the failure matrix, recovery guarantees, and health monitoring mechanisms implemented across the Harmonia platform.

39.1 Platform Failure Scenario Matrix

Failure Event	Affected Subsystem	Automated Detection	Platform Recovery Guarantee
Artemis Primary Pod Crash	Petasos Broker A	Synchronous replication heartbeat loss	Backup broker assumes live role within 2s; zero message loss.
Infinispan Node Loss	Mneme Cache Grid	JGroups cluster view change event	Data preserved via 2-owner replication on surviving pod.
Ponos Worker Termination	Energeia Workflow	Unacknowledged JMS consumer timeout	ActiveMQ Artemis redelivers unacknowledged task to surviving worker.
Downstream Egress Failure	Pylai MLLP Outbound	Socket timeout / NACK response	Task retried via exponential backoff; moved to DLQ on max retries.
PostgreSQL Instance Restart	Mnemosyne Clinical	HikariCP connection test query failure	HikariCP pool automatically reconnects on DB pod readiness.

Table 39.1: Platform Failure Scenario Matrix and Automated Recovery Guarantees

39.2 Dead-Letter Queue (DLQ) Governance

Tasks that exceed maximum redelivery attempts (default: 5) are routed automatically to the centralized dead-letter queue:

```
petasos.dlq
```

Engineers can inspect, repair, and replay poisoned messages via `iris-console` or administrative CLI utilities.

39.3 Kubernetes Health & Readiness Probes

All application pods expose Spring Boot Actuator health endpoints:

- **Liveness Probe**: `GET /actuator/health/liveness` (Period: 10s, Timeout: 3s). Restarts unresponsive containers.

- **Readiness Probe**: `GET /actuator/health/readiness` (Period: 5s, Timeout: 3s). Validates Artemis, Infinispan, and DB connectivity before routing traffic.

Chapter 40

PHI-Aware Logging Architecture & Developer Implementation Guide

40.1 Overview & Regulatory Objectives

The Harmonia Health Information Exchange (HIE) platform processes high-velocity, distributed healthcare interactions encompassing HL7 v2.x clinical messages, HL7 FHIR R5 resources, and proprietary electronic health record (EHR) interchanges across heterogeneous clinical boundaries. In modern clinical environments, operational diagnostics must coexist with strict privacy, confidentiality, and data protection mandates.

To fulfill regulatory requirements across multiple jurisdictions—including the United States Health Insurance Portability and Accountability Act (HIPAA Security Rule §164.312(b)), the European Union General Data Protection Regulation (GDPR Article 32), and the Australian Privacy Principles (APP 11)—Harmonia establishes a centralized, architectural control mechanism for logging Protected Health Information (PHI).

The core architectural invariant of Harmonia logging is:

Architectural Principle: Governed PHI Logging

Operational log streams (`INFO`, `WARN`, `ERROR`) must remain strictly devoid of Protected Health Information (PHI), personally identifiable demographics, and security credentials under all circumstances. Clinical and diagnostic payload data is permitted exclusively at `DEBUG` and `TRACE` severity levels, conditionally governed by an active platform security gate and routed to isolated, encrypted diagnostic storage.

40.1.1 The Logging Policy Matrix

Table 40.1 specifies the governance rules across all standard logging severity levels.

Log Level	PHI Policy	Architectural Description & Governance Rule
`TRACE`	**Conditional**	Deep wire and protocol traces (e.g., raw HL7 v2 ER7 segments, low-level socket payloads). Permitted only when the global PHI gate is enabled **AND** logger level is set to `TRACE`.
`DEBUG`	**Conditional**	Granular clinical diagnostics (e.g., extracted patient demographics, unmasked FHIR resource JSON/XML structures). Permitted only when the global PHI gate is enabled **AND** logger level is set to `DEBUG`.
`INFO`	**Prohibited**	High-level lifecycle milestones, task state transitions, routing decisions, and throughput metrics. Strictly prohibited from containing PHI.
`WARN`	**Prohibited**	Recoverable runtime anomalies, circuit breaker activations, and transient network retry events. Strictly prohibited from containing PHI.
`ERROR`	**Prohibited**	Unhandled system faults, parsing exceptions, and infrastructure failures. Must never contain clinical payloads, patient identifiers, or unsanitized stack traces.

Table 40.1: Harmonia Platform Logging Policy Matrix

Architectural Note

Definition of Conditional PHI Emission: Diagnostic clinical logs are evaluated and output **only** when both of the following prerequisites are simultaneously met:

1. Global configuration flag `harmonia.logging.phi-enabled=true` (or environment variable `HARMONIA_LOGGING_PHI_ENABLED=true`) is set.
2. The underlying SLF4J / Logback logger category level is explicitly configured to `DEBUG` or `TRACE`.

If either prerequisite is unsatisfied, PHI log statements are treated as strict no-ops without evaluating argument parameters or executing lambda suppliers.

40.2 Architectural Model & Security Gates

Harmonia implements a decoupled, defence-in-depth logging architecture that enforces compile-time safety, runtime gate evaluation, and physical log destination isolation. Figure 40.1 illustrates the end-to-end processing topology from component code down to physical Logback appenders.

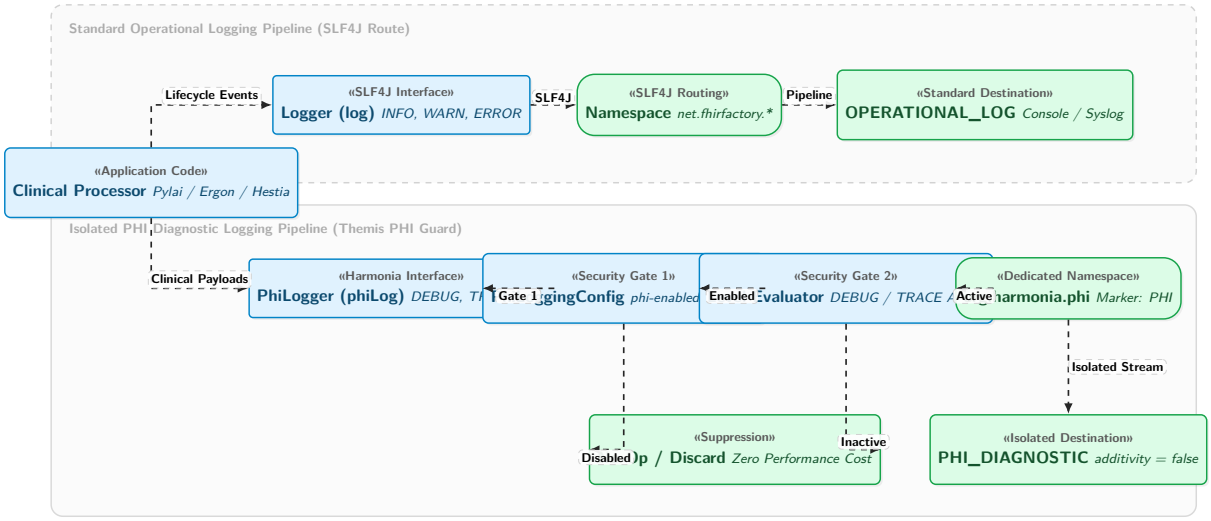


Figure 40.1: ArchiMate 3.2 View: PHI-Aware Logging Security Architecture and Appender Isolation

40.2.1 Dual-Gate Security Evaluation Flow

The security evaluation pipeline operates as follows:

1. **Dual-Logger Invocation**: Application components instantiate standard SLF4J `Logger` for operational events and Harmonia `PhiLogger` for diagnostic data.
2. **Gate 1 (Global Platform Switch)**: `PhiLogger` intercepts calls and consults `PhiLoggingConfig.isPhiLoggingEna` If disabled (default `false`), the method returns immediately, dropping all evaluation.
3. **Gate 2 (Active Logger Level)**: If Gate 1 passes, the underlying SLF4J delegate verifies whether `isDebugEnabled()` or `isTraceEnabled()` is active for the target namespace. If inactive, execution terminates.
4. **Dedicated Namespace Routing**: When both gates pass, log events are published to the dedicated SLF4J namespace `org.harmonia.phi` with an attached SLF4J Marker (`Marker: PHI`).
5. **Logback Non-Additivity**: The Logback configuration declares `additivity="false"` on the `org.harmonia.phi` category, ensuring clinical log entries are directed exclusively to the `PHI_DIAGNOSTIC` appender and never leak into operational consoles or system aggregators.

40.3 Configuration & Environment Variables

The PHI logging framework is centrally configured through standard system properties and environment variables. Table 40.2 summarizes the configuration keys.

System Property	Environment Variable	Default	Description
`harmonia.logging.phi-enabled`	`HARMONIA_LOGGING_PHI_ENABLED`	`false`	Global security gate enabling evaluation and emission of PHI diagnostic logs.

Table 40.2: PHI Logging Configuration Properties

40.3.1 Startup Warning Banner

When `harmonia.logging.phi-enabled=true` is detected at runtime initialization, `PhiLoggingConfig` emits a prominent, single-instance warning banner to standard operational logs (`INFO` level) to alert system operators:

```
================================================================================
PHI diagnostic logging is ENABLED.
DEBUG/TRACE diagnostic logs may contain protected health information.
Ensure this setting is NOT active in untrusted or production environments without
appropriate PHI security controls and restricted destination appenders.
================================================================================
```

Listing 40.1: Operational Startup Warning Banner for PHI Logging

40.4 Developer Guide & API Usage Blueprint

This section provides actionable blueprints for software engineers developing components across Harmonia subsystems.

40.4.1 The Dual-Logger Pattern

Every service, camel processor, ergon activity, or repository class handling clinical data must declare two distinct logger instances:

1. **Standard SLF4J Logger** (`org.slf4j.Logger`): Bound to the class name; used strictly for operational events (`info`, `warn`, `error`).
2. **Harmonia PhiLogger** (`net.fhirfactory.harmonia.logging.PhiLogger`): Obtained via `PhiLoggerFactory.get` used exclusively for clinical diagnostics (`debug`, `trace`).

```
1  package net.fhirfactory.harmonia.pylai.ingress;
2
3  import net.fhirfactory.harmonia.logging.PhiLogger;
4  import net.fhirfactory.harmonia.logging.PhiLoggerFactory;
5  import org.hl7.fhir.r5.model.Patient;
6  import org.hl7.fhir.r5.model.Task;
7  import org.slf4j.Logger;
8  import org.slf4j.LoggerFactory;
9
10 public class PatientIngressProcessor {
11
12     // 1. Standard SLF4J logger for operational, non-PHI logging
13     private static final Logger log = LoggerFactory.getLogger(PatientIngressProcessor.
       class);
14
15     // 2. Harmonia PhiLogger for conditionally permitted PHI diagnostics
16     private static final PhiLogger phiLog = PhiLoggerFactory.getLogger(
       PatientIngressProcessor.class);
17
18     public void processIngress(Task task, Patient patient, String rawHl7) {
19         // Operational log: Non-PHI metadata, identifiers, status enums only
20         log.info("Inbound patient message accepted: taskId={}, status={}",
21                 task.getIdPart(), task.getStatus());
22
23         // PHI Diagnostic log: Names, MRNs, DOBs, clinical demographics
24         phiLog.debug("Extracted patient demographics: id={}, name={}, mrn={}",
25                 patient.getIdPart(),
26                 patient.getNameFirstRep().getNameAsSingleString(),
27                 patient.getIdentifierFirstRep().getValue());
28
```

```
29          // Low-level wire trace: Raw HL7 ER7 payload
30          phiLog.trace("Inbound raw HL7 ER7 wire payload: {}", rawHl7);
31      }
32  }
```

Listing 40.2: Dual-Logger Instantiation and Usage Pattern

40.4.2 Compile-Time Safety and API Contract

The `PhiLogger` interface is deliberately designed without `info(...)`, `warn(...)`, or `error(...)` methods. Any developer attempt to log clinical information via `phiLog.info(...)` will fail at compile-time, preventing accidental leaking of PHI into operational streams.

Table 40.3 details the method signatures provided by `PhiLogger`.

Method Signature	Usage & Evaluation Semantics
`boolean isDebugEnabled()`	Returns `true` iff Gate 1 and Gate 2 (DEBUG) are both satisfied.
`void debug(String msg)`	Emits a static debug message with originating class prefix.
`void debug(String format, Object arg)`	Emits a parameterized debug message with single argument.
`void debug(String format, Object arg1, Object arg2)`	Emits a parameterized debug message with two arguments.
`void debug(String format, Object... args)`	Emits a parameterized debug message with variable arguments.
`void debug(String msg, Throwable t)`	Emits a debug message along with exception throwable.
`void debug(String format, Supplier<?>... suppliers)`	Lazily evaluates argument suppliers and emits debug message.
`boolean isTraceEnabled()`	Returns `true` iff Gate 1 and Gate 2 (TRACE) are both satisfied.
`void trace(String msg)`	Emits a static trace message with originating class prefix.
`void trace(String format, Object arg)`	Emits a parameterized trace message with single argument.
`void trace(String format, Object arg1, Object arg2)`	Emits a parameterized trace message with two arguments.
`void trace(String format, Object... args)`	Emits a parameterized trace message with variable arguments.
`void trace(String msg, Throwable t)`	Emits a trace message along with exception throwable.
`void trace(String format, Supplier<?>... suppliers)`	Lazily evaluates argument suppliers and emits trace message.

Table 40.3: PhiLogger Interface API Contract

40.4.3 Lazy Evaluation with Functional Suppliers

In high-throughput HIE pipelines, serializing complex FHIR Bundles or HL7 message models to JSON/XML strings creates substantial CPU overhead and garbage collection pressure. To achieve zero-cost execution when PHI logging is disabled, developers must use `Supplier<?>` functional lambdas:

```
1 // Zero overhead when harmonia.logging.phi-enabled=false:
2 // The lambda expression is NEVER invoked and JSON serialization is bypassed
      completely.
3 phiLog.debug("Transformed clinical FHIR Bundle: {}",
4     () -> fhirContext.newJsonParser().setPrettyPrinting().encodeResourceToString(
      bundle));
5
6 phiLog.trace("Generated HL7 ACK segment: {}",
7     () -> hl7Message.encode());
```

Listing 40.3: Lazy Evaluation Using Functional Suppliers

40.4.4 Categorization: PHI vs Non-PHI Operational Metadata

Table 40.4 provides concrete guidance distinguishing between operational metadata and protected clinical data.

Category	Permitted in Operational Logs (`log`)	Restricted to PHI Diagnostics (`phiLog`)
Identifiers	System UUIDs, Correlation IDs, Pragma IDs, Task IDs, Message Control IDs (MSH-10)	Medical Record Numbers (MRN), Medicare Numbers, Social Security Numbers (SSN), Driver's License
Demographics	Age band (e.g., `ADULT`, `PEDIATRIC`), Administrative gender enum	Full Patient Names, Given/Family Names, Street Address, Phone, Email, Exact Date of Birth (DOB)
Clinical Data	Resource Type (e.g., `Observation`, `Condition`), LOINC code identifier	Diagnostic text, Lab values, Reference ranges, Doctor clinical notes, Diagnosis narratives
Payloads	Message byte size, Segment count, Processing duration (ms)	Full ER7 HL7 string, Full FHIR JSON/XML resource payload

Table 40.4: Classification of Operational Metadata vs. Restricted PHI

40.4.5 Absolute Prohibition on Secrets and Credentials

Architectural Note

Zero-Trust Secrets Prohibition: While PHI diagnostic logging conditionally permits protected clinical data for troubleshooting, **security secrets are strictly prohibited across all logging channels, levels, and appenders without exception**.

The following must **never** appear in any log:

- Passwords, hashed passwords, and database connection strings.
- OAuth2 Access Tokens, Refresh Tokens, and Bearer JSON Web Tokens (JWTs).
- API Keys, Client Secrets, and HMAC signatures.
- TLS/SSL private keys, Java Keystore passwords, and PKCS#12 certificates.
- ActiveMQ Artemis broker credentials and messaging tokens.

40.4.6 Exception Handling & Stack Trace Sanitization

Because unhandled exceptions and stack traces frequently surface in operational error aggregation dashboards, developers must ensure that exception messages do not embed clinical values or patient identifiers:

```
// INCORRECT: Clinical data in exception message propagates to operational logs
throw new ValidationException("Patient " + patient.getName() + " has invalid Medicare
    number " + mrn);

// CORRECT: Non-PHI error message with diagnostic logging prior to throw
phiLog.debug("Validation failed for patient name={}, mrn={}",
        patient.getNameFirstRep().getNameAsSingleString(), mrn);
throw new ValidationException("Demographic validation rule failed: ERR_VAL_042", "
    ERR_VAL_042");
```

Listing 40.4: Exception Message Sanitization Pattern

40.4.7 Mapped Diagnostic Context (MDC) Invariants

Mapped Diagnostic Context (MDC) keys are propagated across asynchronous thread boundaries and printed in standard log prefixes.

- **Allowed Operational MDC Keys**: `correlationId`, `pragmaId`, `praxisId`, `ergonId`, `taskId`, `messageControlId`, `interfaceId`, `component`.

- **Strictly Forbidden MDC Keys**: Any patient name, MRN, date of birth, clinical finding, or raw message payload.

40.5 Subsystem Integration Patterns

40.5.1 Gateway Tier: Pylai Ingress/Egress

In the **Pylai** gateway tier, inbound HL7 MLLP and FHIR HTTP streams are received. Non-PHI metadata (e.g., source IP, endpoint URI, message control ID) is logged via `log.info`, while raw wire frames and unparsed segments are dispatched to `phiLog.trace`.

40.5.2 Workflow Tier: Energeia & Ergon Activities

In the **Energeia** processing tier, Ergon modules (`ErgonBase`) handle clinical data transformations and LOINC/SNOMED enrichment. Task state progressions and checkpoint records are logged via standard operational loggers, while clinical diffs and enriched FHIR structures are captured via `phiLog.debug`.

40.5.3 Persistence Tier: Hestia & Mnemosyne

In the **Hestia / Mnemosyne** persistence tier, database transaction commits, table partition metrics, and cache hit rates use `log.info`. Deep SQL statements containing embedded clinical parameters are isolated to `phiLog.trace`.

40.6 Logback Configuration & Appender Isolation

To guarantee fail-safe security, Harmonia microservices configure Logback with dedicated diagnostic file appenders and explicit non-additivity.

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration scan="true" scanPeriod="30 seconds">

    <!-- 1. Operational Console Appender (Strictly Non-PHI) -->
    <appender name="OPERATIONAL_CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} [%X{
    correlationId}] - %msg%n</pattern>
        </encoder>
    </appender>

    <!-- 2. Dedicated PHI Diagnostic Appender (Restricted Encrypted Storage) -->
    <appender name="PHI_DIAGNOSTIC" class="ch.qos.logback.core.rolling.
    RollingFileAppender">
        <file>/var/log/harmonia/phi-diagnostic.log</file>
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <fileNamePattern>/var/log/harmonia/phi-diagnostic-%d{yyyy-MM-dd}.log.gz</
    fileNamePattern>
            <!-- Strict retention: max 7 days -->
            <maxHistory>7</maxHistory>
            <totalSizeCap>10GB</totalSizeCap>
        </rollingPolicy>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level [%marker] %logger
    {36} [%X{correlationId}] - %msg%n</pattern>
        </encoder>
    </appender>

    <!-- Root Logger: Routes all operational logs to console -->
    <root level="INFO">
        <appender-ref ref="OPERATIONAL_CONSOLE" />
    </root>

    <!-- PHI Logger Category: additivity="false" prevents leakage to Root/Operational
    Console -->
    <logger name="org.harmonia.phi" level="DEBUG" additivity="false">
        <appender-ref ref="PHI_DIAGNOSTIC" />
    </logger>

</configuration>
```

Listing 40.5: Production Logback Configuration (logback.xml)

40.6.1 Physical Storage & Retention Invariants

1. **Disk Encryption**: Storage directories hosting `phi-diagnostic.log` must reside on AES-256 encrypted volumes.
2. **Strict Retention Limit**: PHI diagnostic logs must enforce a maximum retention window of 7 days (`<maxHistory>7</maxHistory>`).
3. **Access Control**: Access to diagnostic files is restricted strictly to authorized Privacy/Security Officers and senior support engineers via POSIX file permissions (`chmod 0600`).

40.7 Testing & Verification Strategies

Unit and integration test suites can programmatically manipulate the PHI logging configuration using `PhiLoggingConfig` to verify conditional emission and suppression behaviors.

```
package net.fhirfactory.harmonia.logging;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.concurrent.atomic.AtomicBoolean;

import static org.assertj.core.api.Assertions.assertThat;

class PhiLoggingIntegrationTest {

    private final PhiLogger phiLog = PhiLoggerFactory.getLogger(
    PhiLoggingIntegrationTest.class);

    @BeforeEach
    void setUp() {
        PhiLoggingConfig.reset(); // Resets to system property defaults
    }

    @AfterEach
    void tearDown() {
        PhiLoggingConfig.reset();
    }

    @Test
    @DisplayName("Should suppress lambda evaluation when PHI logging is disabled")
    void testSupplierSuppressedWhenDisabled() {
        PhiLoggingConfig.setPhiLoggingEnabled(false);
        AtomicBoolean evaluated = new AtomicBoolean(false);

        phiLog.debug("Diagnostic check: {}", () -> {
            evaluated.set(true);
            return "Sensitive Clinical Payload";
        });

        assertThat(evaluated).isFalse();
    }

    @Test
    @DisplayName("Should execute lambda evaluation when PHI logging is enabled")
    void testSupplierExecutedWhenEnabled() {
        PhiLoggingConfig.setPhiLoggingEnabled(true);
        AtomicBoolean evaluated = new AtomicBoolean(false);

        phiLog.debug("Diagnostic check: {}", () -> {
            evaluated.set(true);
            return "Sensitive Clinical Payload";
        });

        assertThat(evaluated).isTrue();
```

```
52         }
53 }
```

Listing 40.6: Unit Testing Pattern with PhiLoggingConfig

Appendix A

Glossary of Classical Nomenclature & Terminology

This appendix provides the authoritative glossary of classical Greek architectural naming metaphors, mythological etymologies, healthcare standards, and platform abbreviations utilized across the Harmonia Health Integration Environment.

A.1 Core Greek Metaphor Register

A.2 Healthcare & Enterprise Integration Terminology

Concept	Classical Etymology	Mythological / Classical Role	Harmonia Architectural Responsibility	Status
Harmonia	*harmonia* (agreement, concord)	Goddess of harmony, order, and bringing discordant elements into union.	Overarching Health Integration Environment (HIE) uniting all subsystems.	[IMPLEMENTED]
Hestia	*hestia* (hearth, altar fire)	Virgin goddess of the hearth, family stability, and the sacred flame.	Persistence foundation subsystem governing both ephemeral caching and relational storage.	[IMPLEMENTED]
Pylai	*pylai* (gates, portals)	The great ceremonial and defensive city gates controlling entry and egress.	Boundary protocol gateways (inbound/outbound MLLP, FHIR REST Provider Registry).	[IMPLEMENTED]
Petasos	*petasos* (winged sun hat)	The broad-brimmed winged hat of Hermes, messenger god of swift travel.	Resilient messaging abstraction layer isolating broker clients behind pure transport contracts.	[IMPLEMENTED]
Energeia	*energeia* (action, actuality)	Aristotelian concept of activity, energy, and bringing potential into reality.	Task execution and workflow orchestration subsystem (workers, activity units, blueprints).	[IMPLEMENTED]
Ponos	*ponos* (toil, hard labor)	Personification of strenuous work, physical labor, and ceaseless exertion.	High-concurrency WorkEngine daemon consuming tasks from message queues and invoking activities.	[IMPLEMENTED]
Ergon	*ergon* (work, deed, artifact)	A discrete, purposeful piece of work or single finished deed.	Atomic, single-responsibility activity execution unit (parsing ADT, mapping FHIR, egress dispatch).	[IMPLEMENTED]
Praxis	*praxis* (practical action)	Practical human action guided by reason; deliberate execution of a plan.	Workflow blueprint definition governing multi-step sequential or parallel task execution stages.	[IMPLEMENTED]
Pragma	*pragma* (matter, deed done)	A tangible thing done, a matter of fact, or a concrete transaction.	Canonical task execution state envelope carrying metadata, checkpoints, and security context.	[IMPLEMENTED]
Mneme	*mneme* (memory, recall)	One of the original three Boeotian Muses, personifying short-term memory.	In-memory distributed cache grid (Infinispan 15.0.3 Hot Rod) providing sub-millisecond coordination.	[IMPLEMENTED]
Mnemosyne	*mnemosyne* (remembrance)	Titaness of memory and mother of the Muses; personifies permanent historical memory.	Authoritative relational persistence tier (HAPI FHIR R5 JPA and Spring Data JPA on PostgreSQL 16).	[IMPLEMENTED]
Calliope	*calliope* (beautiful-voiced)	Eldest of the nine Muses, presiding over eloquence and epic poetry.	Canonical data models, shared DTOs, HL7 v2-to-FHIR converters, and standard event envelopes.	[IMPLEMENTED]
Themis	*themis* (divine law, custom)	Titaness of divine order, justice, and the unwritten cosmic law.	Default-deny policy evaluation engine, role-to-authority mappings, and non-PHI security audit logging.	[IMPLEMENTED]
Iris	*iris* (rainbow messenger)	Goddess of the rainbow and swift winged messenger connecting sea and sky.	Presentation tier delivering the WildFly BEFE REST gateway and three decoupled Vue 3 web SPAs.	[IMPLEMENTED]
Agora	*agora* (assembly, market)	Ancient Greek public gathering place for debate, civic assembly, and commerce.	Collaboration gateway projecting clinical events into Matrix Synapse Spaces, child rooms, and care team chats.	[IMPLEMENTED]
Paradeigma	*paradeigma* (model, exemplar)	A model, archetype, or pattern used to demonstrate a principle or test a hypothesis.	Synthetic clinical simulation framework generating personas, encounters, and multi-system scenarios.	[IMPLEMENTED]

Table A.1: Harmonia Classical Architectural Nomenclature and Etymology Register

Term	Full Expansion	Harmonia Definition & Context
HIE	Health Integration Environment	Comprehensive enterprise software platform facilitating secure, standardized health data exchange across hospital networks.
HL7 v2.x	Health Level Seven Version 2	Pipe-and-hat delimited messaging protocol utilized by legacy hospital information systems (ADT, ORM, ORU).
FHIR R5	Fast Healthcare Interoperability Resources (R5)	Modern RESTful, JSON/XML-based healthcare standard utilized for Harmonia's internal canonical clinical models.
MLLP	Minimal Lower Layer Protocol	TCP framing protocol using Start Block `0x0B` and End Block `0x1C 0x0D` to delimit HL7 v2 messages.
PHI	Protected Health Information	Individually identifiable health information strictly protected under HIPAA, GDPR, and Australian Privacy Principles.
PAS	Patient Administration System	Core hospital operational system managing patient admission, discharge, transfer, and demographics.
EMR / EHR	Electronic Medical / Health Record	Clinical repository storing longitudinal patient encounter charts, orders, observations, and notes.
LMS / LIS	Laboratory Management / Information System	Specialized laboratory diagnostic system managing pathology orders, specimens, and analytical results.
RIS / PACS	Radiology Information System / Picture Archiving	Systems managing medical imaging orders, DICOM studies, and diagnostic radiology reports.
Hot Rod	Infinispan Hot Rod Protocol	Binary, topology-aware client-server RPC protocol used to communicate with Mneme in-memory cache nodes.
AS	Matrix Application Service	Privileged integration protocol connecting the Agora gateway to the Matrix Synapse homeserver.
REC-001	Ingress Dual-Write Safety	Architecture invariant: gateways must guarantee Petasos queue acceptance before returning an `AA` ACK over MLLP.
REC-002	Granular Fan-Out Tracking	Architecture invariant: outbound dispatch activities must record destination-specific delivery checkpoints in `Task.output`.

Table A.2: Healthcare and Enterprise Integration Terminology Register

Appendix B

Authoritative Harmonia System Inventory

This appendix provides the authoritative system inventory across all 9 core subprojects and 46 Maven modules (37 leaf modules and 9 parent POMs) of the Harmonia platform, accompanied by infrastructure workload definitions and an end-to-end 11-dimension architectural traceability matrix.

B.1 Subproject and Leaf Module Inventory

Subproject	Module Path	Artifact ID	Container / Image	Workload	Role & Primary Technology
Calliope (Canonical Models & Domain Schemas)					
Calliope	`calliope`	`calliope`	Embedded JAR	Library	Java 21, HAPI FHIR R5 Structures, Jackson
Themis (Security Governance & Policy Enforcement)					
Themis	`themis/themis-api`	`themis-api`	Embedded JAR	Library	Jakarta Security Contracts, Policy Interfaces
Themis	`themis/themis-core`	`themis-core`	Embedded JAR	Service	Java 21 ABAC/RBAC Deterministic Evaluator
Themis	`themis/themis-audit`	`themis-audit`	Embedded JAR	Service	Non-PHI Security Audit Logging Service
Hestia (Distributed Caching & Authoritative Persistence)					
Hestia	`hestia/mneme-cluster`	`mneme-cluster`	`localhost:32000/.../mneme-cluster`	StatefulSet	Infinispan 15.0.3 Data Grid (JGroups TCP)
Hestia	`hestia/mneme-persistence`	`mneme-persistence`	Packaged in `mneme-cluster`	Library	Infinispan NonBlockingStore Write-Behind SPI
Hestia	`hestia/mnemosyne-clinical`	`mnemosyne-clinical`	`localhost:32000/.../mnemosyne-clinical`	Deployment	Spring Boot 3.2.5, HAPI FHIR R5 JPA Server
Hestia	`hestia/mnemosyne-operations`	`mnemosyne-operations`	`localhost:32000/.../mnemosyne-operations`	Deployment	Spring Boot 3.2.5, Operations JPA Server
Hestia	`hestia/hie-operations-cli`	`hie-operations-cli`	CLI Binary / JAR	CLI Utility	Java 21 CLI, Picocli / Operations REST
Petasos (Resilient Clustered Transport & Broker Adapters)					
Petasos	`petasos/petasos-api`	`petasos-api`	Embedded JAR	Library	Pure Java Transport Abstractions (Zero JMS)
Petasos	`petasos/petasos-core`	`petasos-core`	Embedded JAR	Library	Message Dedup Window & Topic Routers
Petasos	`petasos/petasos-artemis`	`petasos-artemis`	Embedded JAR	Driver	ActiveMQ Artemis 2.33.0 Core Client Adapter
Petasos	`petasos/petasos-test`	`petasos-test`	Test Harness JAR	Test Harness	Embedded Artemis Cluster Testing Harness

Table B.1: Harmonia System Inventory: Foundation, Security, Caching, and Transport Modules

Subproject	Module Path	Artifact ID	Container / Image	Workload	Role & Primary Technology
Energeia (Workflow Orchestration & Erga Task Processing)					
Energeia	energeia/erga	erga	Embedded JAR	Library	Apache Camel 4.4, HL7/FHIR Activity Units
Energeia	energeia/praxis	praxis	Embedded JAR	Library	TaskSequence Route Orchestration & Seeding
Energeia	energeia/ponos	ponos	localhost:32000/.../ponos-task-processor	Deployment	WildFly 31 Jakarta EE 10 WAR Task Processor
Energeia	energeia/ponos-cli	ponos-cli	CLI Binary / JAR	CLI Utility	Java 21 CLI, Ponos Management Controller
Pylai (Inbound & Outbound Protocol Gateways)					
Pylai	pylai/pylai-mllp-base	pylai-mllp-base	Embedded JAR	Library	Netty / Camel MLLP Base & Destination Registry
Pylai	pylai/pylai-mllp-in	pylai-mllp-in	localhost:32000/.../pylai-mllp-in	Deployment	WildFly 31 Jakarta EE 10 WAR Inbound Gateway
Pylai	pylai/pylai-mllp-out	pylai-mllp-out	localhost:32000/.../pylai-mllp-out	Deployment	WildFly 31 Jakarta EE 10 WAR Outbound Dispatcher
Pylai	pylai/pylai-fhir-registry	pylai-fhir-registry	Standalone Spring JAR	Service	Spring Boot 3.2.5 FHIR R5 Provider Registry
Pylai	pylai/pylai-mllp-cli	pylai-mllp-cli	CLI Binary / JAR	CLI Utility	Java 21 MLLP Synthetic Test Injector
Iris (Presentation Gateways & Portal Applications)					
Iris	iris/iris-befe	iris-befe	localhost:32000/.../iris-befe	Deployment	WildFly 31.0.1 Jakarta EE 10 BEFE Gateway
Iris	iris/iris-clinical	iris-clinical	localhost:32000/.../iris-clinical	Deployment	Vue 3 / Vite Clinical Exploration SPA
Iris	iris/iris-console	iris-console	localhost:32000/.../iris-console	Deployment	Vue 3 / Vite Operations Telemetry SPA
Iris	iris/iris-administration	iris-administration	localhost:32000/.../iris-administration	Deployment	Vue 3 / Vite Provider Registry Admin SPA
Agora (Matrix Collaboration Gateway & Synapse Integration)					
Agora	agora/agora-api	agora-api	Embedded JAR	Library	Pure Java Domain Models, Events, Topics
Agora	agora/agora-matrix	agora-matrix	Embedded JAR	Library	Spring WebClient, Matrix v1.11 / Admin DTOs
Agora	agora/agora-core	agora-core	Embedded JAR	Library	Java 21, JPA Persistence, Petasos Bridges
Agora	agora/agora-service	agora-service	localhost:32000/.../agora	Deployment	Spring Boot 3.2.5, AS REST Controller
Paradeigma (Synthetic Clinical Simulation & Architecture Guardrails)					
Paradeigma	paradeigma/paradeigma-common	paradeigma-common	Embedded JAR	Library	Synthetic Personas, Generators, Clinical DTOs
Paradeigma	paradeigma/paradeigma-emr	paradeigma-emr	localhost:32000/.../paradeigma-emr	Simulation	Synthetic EMR Simulator (HL7 ORM / ADT)
Paradeigma	paradeigma/paradeigma-lms	paradeigma-lms	localhost:32000/.../paradeigma-lms	Simulation	Synthetic LMS Simulator (HL7 ORU Labs)
Paradeigma	paradeigma/paradeigma-pas	paradeigma-pas	localhost:32000/.../paradeigma-pas	Simulation	Synthetic PAS Simulator (HL7 ADT A01–A40)
Paradeigma	paradeigma/paradeigma-rispac	paradeigma-rispac	localhost:32000/.../paradeigma-rispac	Simulation	Synthetic RIS-PAC Simulator (Imaging Orders)
Paradeigma	paradeigma/paradeigma-scenarios	paradeigma-scenarios	localhost:32000/.../paradeigma-scenarios	Simulation	Orchestrated Multi-System Scenario Driver
Paradeigma	paradeigma/paradeigma-test	paradeigma-test	Test Harness JAR	ArchUnit	ArchUnit Invariant Guardrail Suite (24 Rules)

Table B.2: Harmonia System Inventory: Workflow, Gateways, Presentation, and Simulation Modules

B.2 Infrastructure and Middleware Workloads

Workload Name	Component Type	Container Image	Kind	Persistent Storage / Scope
artemis-primary-a	ActiveMQ Artemis Broker	apache/activemq-artemis:2.33.0	StatefulSet	2 GiB PVC (artemis-primary-a-data), Group A Live
artemis-backup-a	ActiveMQ Artemis Broker	apache/activemq-artemis:2.33.0	StatefulSet	2 GiB PVC (artemis-backup-a-data), Group A Replicating
artemis-primary-b	ActiveMQ Artemis Broker	apache/activemq-artemis:2.33.0	StatefulSet	2 GiB PVC (artemis-primary-b-data), Group B Live
artemis-backup-b	ActiveMQ Artemis Broker	apache/activemq-artemis:2.33.0	StatefulSet	2 GiB PVC (artemis-backup-b-data), Group B Replicating
infinispan-1	Infinispan Cache Grid	localhost:32000/harmonia/mneme-cluster:1.0.0-SNAPSHOT	StatefulSet	1 GiB PVC (mneme-data-1), Node 1 Hot Rod :11222
infinispan-2	Infinispan Cache Grid	localhost:32000/harmonia/mneme-cluster:1.0.0-SNAPSHOT	StatefulSet	1 GiB PVC (mneme-data-2), Node 2 Hot Rod :11222
postgres-1	PostgreSQL Database	postgres:16-alpine	StatefulSet	2 GiB PVC (postgres-data-1), Clinical Store 1
postgres-2	PostgreSQL Database	postgres:16-alpine	StatefulSet	2 GiB PVC (postgres-data-2), Clinical Store 2
postgres-ops-1	PostgreSQL Database	postgres:16-alpine	StatefulSet	2 GiB PVC (postgres-ops-data-1), Operations Store 1
postgres-ops-2	PostgreSQL Database	postgres:16-alpine	StatefulSet	2 GiB PVC (postgres-ops-data-2), Operations Store 2
postgres-synapse	PostgreSQL Database	postgres:16-alpine	StatefulSet	2 GiB PVC (postgres-synapse-data), Matrix & Agora DB
synapse	Matrix Synapse Homeserver	matrixdotorg/synapse:v1.120.0	StatefulSet	5 GiB PVC (synapse-data), Matrix HTTP :8008
hapi-fhir-jpa-server-1	FHIR R5 Server	localhost:32000/harmonia/mnemosyne-clinical:1.0.0-SNAPSHOT	Deployment	Stateless, Clinical REST API :8080
hapi-fhir-jpa-server-2	FHIR R5 Server	localhost:32000/harmonia/mnemosyne-clinical:1.0.0-SNAPSHOT	Deployment	Stateless, Clinical REST API :8080
hie-operations-jpa-server-1	Operations Server	localhost:32000/harmonia/mnemosyne-operations:1.0.0-SNAPSHOT	Deployment	Stateless, Operations REST API

Appendix C

Authoritative Subproject & Leaf Module Register

This appendix provides the granular architectural and build register across all 9 subprojects and 37 leaf modules within the Harmonia repository reactor.

C.1 Module Taxonomy & Architectural Classification

Subproject	Leaf Module Artifact	Primary Java Package	Architectural Responsibility & Contracts	Status
Calliope (Canonical Domain Models)				
calliope	calliope	net.fhirfactory.harmonia.calliope	Canonical schemas, DTOs, FHIR converter models.	[IMPLEMENTED]
Themis (Default-Deny Policy Enforcement)				
themis	themis-api	net.fhirfactory.harmonia.themis.api	Security contracts, policy decision interfaces.	[IMPLEMENTED]
themis	themis-core	net.fhirfactory.harmonia.themis.core	Deterministic ABAC/RBAC 4-gate evaluator.	[IMPLEMENTED]
themis	themis-audit	net.fhirfactory.harmonia.themis.audit	Non-PHI security audit logging pipeline.	[IMPLEMENTED]
Hestia (Persistence & Caching Subsystem)				
hestia	mneme-cluster	net.fhirfactory.harmonia.mneme	Clustered Infinispan 15.0.3 data grid server.	[CONFIGURED]
hestia	mneme-persistence	net.fhirfactory.harmonia.persistence	NonBlockingStore write-behind cache SPI.	[IMPLEMENTED]
hestia	mnemosyne-clinical	net.fhirfactory.harmonia.hapifhir	HAPI FHIR R5 JPA server backed by PostgreSQL.	[IMPLEMENTED]
hestia	mnemosyne-operations	net.fhirfactory.harmonia.operations	Spring Data JPA operations persistence server.	[IMPLEMENTED]
hestia	hie-operations-cli	net.fhirfactory.harmonia.operationscli	Picocli administrative operational CLI tool.	[IMPLEMENTED]
Petasos (Transport & Messaging Subsystem)				
petasos	petasos-api	net.fhirfactory.harmonia.petasos.api	Pure Java transport abstractions (zero JMS).	[IMPLEMENTED]
petasos	petasos-core	net.fhirfactory.harmonia.petasos.core	Sliding-window deduplication & topic routing.	[IMPLEMENTED]
petasos	petasos-artemis	net.fhirfactory.harmonia.petasos.artemis	ActiveMQ Artemis 2.33.0 Core client adapter.	[IMPLEMENTED]
petasos	petasos-test	net.fhirfactory.harmonia.petasos.test	Embedded Artemis multi-broker test harness.	[IMPLEMENTED]
Energeia (Task Processing & Workflow Orchestration)				
energeia	erga	net.fhirfactory.harmonia.erga	Discrete Camel activity units (AdtDistribution, etc).	[IMPLEMENTED]
energeia	praxis	net.fhirfactory.harmonia.praxis	TaskSequence blueprints and default seeders.	[IMPLEMENTED]
energeia	ponos	net.fhirfactory.harmonia.ponos	WildFly Jakarta EE 10 WAR task worker engine.	[IMPLEMENTED]
energeia	ponos-cli	net.fhirfactory.harmonia.workflowcli	Administrative CLI for Ponos queue inspection.	[IMPLEMENTED]
Pylai (Interface & Boundary Gateways)				
pylai	pylai-mllp-base	net.fhirfactory.harmonia.pylai.base	Netty/Camel MLLP base and destination registry.	[IMPLEMENTED]
pylai	pylai-mllp-in	net.fhirfactory.harmonia.pylai.inbound	Inbound MLLP receiver WAR (Port 2575).	[IMPLEMENTED]
pylai	pylai-mllp-out	net.fhirfactory.harmonia.pylai.outbound	Outbound MLLP dispatcher WAR (Egress queues).	[IMPLEMENTED]
pylai	pylai-fhir-registry	net.fhirfactory.harmonia.pylai.registry	Spring Boot FHIR R5 Provider Registry gateway.	[IMPLEMENTED]
pylai	pylai-mllp-cli	net.fhirfactory.harmonia.pylai.cli	MLLP synthetic message injection CLI tool.	[IMPLEMENTED]
Iris (Presentation Tier & BEFE Gateways)				
iris	iris-befe	net.fhirfactory.harmonia.iris.befe	WildFly 31.0.1 dual-port BEFE REST gateway.	[IMPLEMENTED]
iris	iris-clinical	src/(Vue3/TypeScript)	Clinical exploration and timeline web SPA.	[IMPLEMENTED]
iris	iris-console	src/(Vue3/TypeScript)	Operations telemetry and queue monitoring SPA.	[IMPLEMENTED]
iris	iris-administration	src/(Vue3/TypeScript)	Provider Registry administration web SPA.	[IMPLEMENTED]
Agora (Collaboration Gateway & Matrix AS Integration)				
agora	agora-api	net.fhirfactory.harmonia.agora.api	Collaboration domain models and topics.	[IMPLEMENTED]
agora	agora-matrix	net.fhirfactory.harmonia.agora.matrix	Matrix Client-Server and Synapse Admin client.	[IMPLEMENTED]
agora	agora-core	net.fhirfactory.harmonia.agora.core	Collaboration lifecycle and identity service.	[IMPLEMENTED]
agora	agora-service	net.fhirfactory.harmonia.agora.service	Matrix Application Service REST microservice.	[IMPLEMENTED]
Paradeigma (Synthetic Clinical Simulation & Testbed)				
paradeigma	paradeigma-common	net.fhirfactory.harmonia.paradeigma.common	Synthetic personas, identifiers, and clinical DTOs.	[IMPLEMENTED]
paradeigma	paradeigma-pas	net.fhirfactory.harmonia.paradeigma.pas	PAS simulator generating ADT A01–A40 feeds.	[IMPLEMENTED]
paradeigma	paradeigma-emr	net.fhirfactory.harmonia.paradeigma.emr	EMR simulator generating encounter records.	[IMPLEMENTED]
paradeigma	paradeigma-lms	net.fhirfactory.harmonia.paradeigma.lms	LMS laboratory simulator generating ORU results.	[IMPLEMENTED]

Appendix D

Middleware Capability Matrix

This appendix documents the exact capabilities, clustering modes, and architectural features of each middleware technology integrated into Harmonia, contrasting features actively relied upon against deliberately avoided capabilities.

D.1 Middleware Technology Summary

Technology	Version	Clustering / HA Mode	Harmonia Platform Capabilities Relied Upon
ActiveMQ Artemis	2.33.0	Symmetric mesh + Journal Replication	Durable queues, client failover URL, `_AMQ_DUPL_ID` deduplication, `ON_DEMAND` redistribution.
Infinispan	15.0.3	JGroups TCP Discovery	Hot Rod binary protocol, 17 replicated caches, text/plain encoding, write-behind SPI.
PostgreSQL	16.2	Separate Clinical, Ops & Synapse DBs	Relational transactions, JSONB indexing, HikariCP connection pooling, schema isolation.
WildFly	31.0.1	Standalone Microservices	Jakarta EE 10 CDI, JAX-RS REST endpoints, dual HTTP listener ports (8080/8090).
Spring Boot	3.2.5	Stateless Microservices	Apache Camel routing, Actuator health probes, HAPI FHIR R5 JPA structures, Netty TCP server.
Matrix Synapse	1.120.0	Closed Non-Federated StatefulSet	Application Service v1 API, Client-Server v1.11, Spaces hierarchy, 90-day retention cleanup.

Table D.1: Harmonia Middleware Technology Capability Matrix

D.2 Middleware Capabilities: Used vs. Avoided Analysis

D.2.1 ActiveMQ Artemis 2.33.0 Messaging Engine

Dimension	Used / Relied Upon	Avoided / Excluded	Architectural Rationale
Clustering	Clustered mesh (`ON_DEMAND` redistribution, discovery)	Static discovery via fixed XML IP lists only	Dynamic mesh prevents message starvation and single points of failure.
High Availability	Replicated journal HA (`<replicated/>`)	Shared-store HA (NFS / SAN file lock clustering)	Eliminates shared storage controller bottlenecks and split-brain lock corruption.
Transport Protocol	Artemis Core / Netty Protocol (port 61616)	OpenWire, AMQP 1.0, MQTT, STOMP	Pure Netty transport minimizes protocol translation and connection pool overhead.
Broker Hosting	Dedicated StatefulSet pods in Kubernetes	Embedded in-VM Artemis inside production apps	Isolates broker crashes and memory churn from business workflow logic.
Coordination	Native Artemis cluster discovery protocol	Apache ZooKeeper / HashiCorp Consul	Reduces operational dependencies and external infrastructure complexity.
Dead-Letter Handling	Automatic DLQ (`DLQ`) & Expiry (`ExpiryQueue`)	Silent message drops or infinite redelivery loops	Bounded retry (max 3 attempts) with dead-letter preservation guarantees zero clinical message loss.

Table D.2: ActiveMQ Artemis Capability Analysis: Used vs. Avoided

D.2.2 Infinispan 15.0.3 Distributed In-Memory Data Grid

Dimension	Used / Relied Upon	Avoided / Excluded	Architectural Rationale
Synchronization	Synchronous replication (`REPL_SYNC`) across all 17 caches	Asynchronous replication (`REPL_ASYNC`) without ack	Synchronous caching guarantees read-your-writes consistency across pods.
Transport Protocol	Hot Rod Binary Protocol (port 11222)	REST, Memcached, WebSocket protocols	Hot Rod binary RPC minimizes serialization latency and payload overhead.
Discovery	JGroups TCP discovery (port 7800) over headless service	JGroups UDP multicast discovery	UDP multicast is unreliable or prohibited in standard Kubernetes container networks.
Query & Indexing	Key-based lookup over text/plain payloads	Lucene / Hibernate Search on cache entries	Search indexing is delegated to PostgreSQL; avoids memory exhaustion.
Transactions	Non-transactional caches with write-behind SPI	2-Phase Commit (2PC) / XA distributed transactions	Eliminates 2PC distributed locking overhead; relies on idempotent workflows.
Cross-Site HA	Single-cluster multi-pod grid	Cross-site replication (x-site across data centers)	Local HA handles node recovery; cross-site deferred to future phases.

Table D.3: Infinispan Capability Analysis: Used vs. Avoided

D.2.3 PostgreSQL 16 Relational Persistence

D.2.4 WildFly 31.0.1 Jakarta EE Runtime (Iris BEFE Gateway & Ponos)

D.2.5 Matrix Synapse 1.120.0 Collaboration Homeserver

Dimension	Used / Relied Upon	Avoided / Excluded	Architectural Rationale
DB Isolation	Strictly separated clinical, operations, and Synapse DBs	Single unified monolithic database schema	Prevents operational telemetry queries from impacting clinical ACID transaction throughput.
Persistence Schema	Relational tables with immutable composite primary keys	In-place row updates without version tracking	Guarantees complete historical auditability and temporal reproducibility of FHIR records.
Indexing Strategy	Targeted composite B-Tree indexes on lookup and timestamps	Full-database automated or unchecked index sprawl	Balances fast sub-millisecond retrieval with high-throughput write performance.
Logic Placement	Schema definitions, constraints, and standard SQL queries	Database triggers, PL/pgSQL stored procedures, DB rules	Business logic remains in testable Java/Spring services (Themis, Energeia, Mnemosyne).
Replication	Independent node instances partitioned by domain	Native PostgreSQL streaming replication / Patroni	Simplifies base container orchestration; HA provided by application-tier redundancy and K8s.

Table D.4: PostgreSQL Capability Analysis: Used vs. Avoided

Dimension	Used / Relied Upon	Avoided / Excluded	Architectural Rationale
Component Model	Jakarta RESTful Web Services (JAX-RS 3.1) & CDI 4.0	Stateful Enterprise Java Beans (EJB) & JMS MDBs	Stateless HTTP REST and Hot Rod binary clients prevent session state stickiness.
Gateway Pattern	Decoupled Backend-For-Frontend (BEFE) proxying Mneme	Direct JPA / Hibernate persistence in WildFly	Strictly enforces Invariant 3 (Iris Presentation Decoupling).
Clustering	Standalone container instances in Kubernetes	WildFly Domain Mode / mod_cluster multicast	Kubernetes handles lifecycle, rolling updates, and liveness/readiness probes.
Messaging	External ActiveMQ Artemis cluster via Netty / REST	Embedded WildFly Artemis messaging broker subsystem	Prevents broker memory exhaustion and GC pauses from disrupting HTTP gateway responsiveness.
Security	Themis REST security context propagation	Container-managed JAAS realms with fixed database auth	Allows centralized default-deny authorization via Themis API.

Table D.5: WildFly Capability Analysis: Used vs. Avoided

Dimension	Used / Relied Upon	Avoided / Excluded	Architectural Rationale
Federation	Closed, non-federated deployment (`federation.enabled:false`)	Public inter-homeserver federation	Enforces strict healthcare security boundary; isolates collaboration contexts.
Room Encryption	Transport TLS 1.3 across hops; unencrypted server-side rooms	Olm/Megolm end-to-end encryption (E2EE)	Allows automated clinical governance, indexing, and Petasos workflow dispatch (AGORA-ADR-002).
User Provisioning	Deterministic accounts provisioned via Synapse Admin API	Public open user self-registration	Only authenticated Harmonia principals can access collaboration spaces (AGORA-ADR-001).
Room Discovery	Invite-only private Spaces and child rooms	Public room directory and world-readable aliases	Prevents unauthorized browsing of clinical threads (AGORA-ADR-005).
Event Retention	90-day automated cleanup policy	Unbounded permanent event retention	Matrix is ephemeral; authoritative data lives in Mnemosyne (AGORA-ADR-004).
Transactions	Durable AS transaction deduplication in PostgreSQL	Stateless transaction ingestion	Prevents duplicate Petasos task dispatch on network retries (AGORA-ADR-007).

Table D.6: Matrix Synapse Capability Analysis: Used vs. Avoided

Appendix E

Configuration Property Register

This appendix provides the authoritative cross-reference of environment variables, Spring Boot properties, and JVM system flags across all subsystems of the Harmonia platform.

E.1 Dual-Dimension Configuration Discipline

Harmonia strictly bifurcates its operational parameter space into:

1. **Infrastructure Configuration**: Kubernetes CPU/memory requests, limits, PVC sizes, node selectors, readiness/liveness probe thresholds, and network policies (governed via Kustomize in `deployment/kubernetes/base/`).
2. **Application Configuration**: Spring application YAML, WildFly standalone subsystems, ActiveMQ Artemis broker XML, Infinispan cache definitions, and Themis policy thresholds.

E.2 Global Infrastructure & Environment Parameters

Configuration Key	Environment Variable	Default Value	Architectural Description
`harmonia.k8s.namespace`	`HARMONIA_NAMESPACE`	`harmonia`	Target Kubernetes namespace for cluster deployment.
`harmonia.storage.class`	`HARMONIA_STORAGE_CLASS`	`microk8s-hostpath`	PersistentVolume provisioner storage class.
`harmonia.logging.phi-enabled`	`HARMONIA_LOGGING_PHI_ENABLED`	`false`	Global security gate for diagnostic PHI logging.
`harmonia.artemis.url`	`PETASOS_BROKER_URL`	`tcp://artemis-primary-a:61616...`	Multi-broker high-availability client connection URL.
`harmonia.infinispan.servers`	`HARMONIA_CACHE_SERVERS`	`infinispan-1:11222;...`	Hot Rod cluster endpoints for Mneme cache grid.
`harmonia.agora.hs-token`	`AGORA_HS_TOKEN`	*Vault Injected*	Shared secret authenticating Synapse AS transactions.
`harmonia.agora.as-token`	`AGORA_AS_TOKEN`	*Vault Injected*	Shared secret authenticating Agora calls to Synapse.

Table E.1: Harmonia Global Infrastructure Configuration Parameters

E.3 Subsystem Application Parameters

Subsystem	Property Key	Default Value	Operational Behavior & Invariant
Petasos	`petasos.dedup.window-size`	`50000`	Sliding-window message deduplication cache capacity.
Petasos	`petasos.broker.reconnect-attempts`	`-1` (infinite)	Automatic failover reconnection loop bound.
Ponos	`ponos.concurrency.workers`	`10`	Concurrent Camel task processing consumer threads.
Ponos	`ponos.task.timeout-seconds`	`300`	Maximum execution bound before task cancellation.
Pylai	`pylai.mllp.port`	`2575`	Inbound TCP socket port for HL7 v2 messages.
Pylai	`pylai.mllp.backlog`	`1024`	Netty TCP server connection backlog queue.
Themis	`themis.policy.default-deny`	`true`	Enforcement invariant: unmatched requests evaluate to DENY.
Themis	`themis.audit.enabled`	`true`	Emits non-PHI security audit records to ops store.
Mneme	`mneme.cache.mode`	`REPL_SYNC`	Synchronous cache replication across 17 data grid caches.
Mneme	`mneme.store.write-behind-queue`	`1024`	Asynchronous buffer size for REST cache store flushing.
Agora	`agora.matrix.server-url`	`http://synapse:8008`	Synapse homeserver REST endpoint.
Agora	`agora.retention.days`	`90`	Transient collaboration room retention period.

Table E.2: Harmonia Subsystem Application Configuration Properties

Appendix F

Port and Network Protocol Register

This appendix registers the authoritative 18 platform network listeners across all subsystems, transport protocols, traffic directions, and TLS termination boundaries in the Harmonia cluster.

F.1 The 18 Platform Network Listeners

#	Port	Protocol	Workload Binding	Traffic Flow	Operational Purpose & Security Boundary
1	**2575**	TCP / MLLP	`pylai-mllp-in`	Ingress: External → Gateway	Inbound HL7 v2 triggers (ADT, ORM, ORU).
2	**80**	TCP / HTTP	Iris SPAs	Ingress → Nginx	Static web assets delivery (Plaintext HTTP).
3	**443**	TCP / HTTPS	Ingress Controller	External → Cluster	Master TLS 1.2/1.3 entrypoint for SPAs & APIs.
4	**8080**	TCP / HTTP	`iris-befe`	Ingress → BEFE	Clinical FHIR R5 REST API (`/clinical/*`).
5	**8090**	TCP / HTTP	`iris-befe`	Ingress → BEFE	Operations telemetry & queues (`/operations/*`).
6	**9990**	TCP / HTTP	WildFly Containers	Admin → Management	Jakarta EE management CLI and console.
7	**61616**	TCP / Netty	Artemis Brokers	Pods ↔ Brokers	Petasos CORE transport & HA journal replication.
8	**8161**	TCP / HTTP	Artemis Brokers	Admin → Management	ActiveMQ Artemis web console and metrics.
9	**11222**	TCP / Hot Rod	Infinispan Grid	BEFE / Ponos → Grid	Hot Rod binary RPC cache protocol (17 caches).
10	**7800**	TCP / JGroups	Infinispan Grid	Peer ↔ Peer	JGroups TCP cluster peer discovery & sync.
11	**5432**	TCP / Native PG	`postgres-1`	JPA → Clinical DB	Primary clinical FHIR database (`fhir_node_1`).
12	**5433**	TCP / Native PG	`postgres-2`	JPA → Clinical DB	Secondary clinical FHIR database (`fhir_node_2`).
13	**5434**	TCP / Native PG	`postgres-ops-1`	JPA → Ops DB	Primary operational database (`ops_node_1`).
14	**5435**	TCP / Native PG	`postgres-ops-2`	JPA → Ops DB	Secondary operational database (`ops_node_2`).
15	**5436**	TCP / Native PG	`postgres-synapse`	Synapse → Synapse DB	Matrix Synapse & Agora relational database.
16	**8008**	TCP / HTTP	`synapse`	Agora / Ingress → Synapse	Matrix Client-Server, Admin & AS APIs.
17	**8092**	TCP / HTTP	`agora`	Synapse → Agora	AS transaction ingestion (`PUT/transactions`).
18	**9992**	TCP / HTTP	`agora`	Probes → Agora	Spring Boot Actuator health, liveness & metrics.

Table F.1: Authoritative Harmonia 18-Port Network Listener Register

Appendix G

Container and Image Register

This appendix provides the authoritative register of all container images, base distributions, build targets, and runtime security privileges across the Harmonia platform.

G.1 Container Image Register

Workload	Container Image Repository / Tag	Base Distribution	Build Profile	Runtime User & Security Context
Presentation Tier (Iris)				
iris-clinical	localhost:32000/harmonia/iris-clinical:1.0.0-SNAPSHOT	Nginx 1.25 Alpine	Node/Vite Build	Non-root (1001:1001), Read-only rootfs.
iris-console	localhost:32000/harmonia/iris-console:1.0.0-SNAPSHOT	Nginx 1.25 Alpine	Node/Vite Build	Non-root (1001:1001), Read-only rootfs.
iris-administration	localhost:32000/harmonia/iris-administration:1.0.0-SNAPSHOT	Nginx 1.25 Alpine	Node/Vite Build	Non-root (1001:1001), Read-only rootfs.
iris-befe	localhost:32000/harmonia/iris-befe:1.0.0-SNAPSHOT	WildFly 31.0.1 JDK 21	Maven WAR	Non-root (jboss), Port 8080/8090.
Gateways & Workflows (Pylai, Energeia, Agora)				
pylai-mllp-in	localhost:32000/harmonia/pylai-mllp-in:1.0.0-SNAPSHOT	WildFly 31.0.1 JDK 21	Maven WAR	Non-root (jboss), MLLP Port 2575.
pylai-mllp-out	localhost:32000/harmonia/pylai-mllp-out:1.0.0-SNAPSHOT	WildFly 31.0.1 JDK 21	Maven WAR	Non-root (jboss), Outbound socket client.
task-processor	localhost:32000/harmonia/ponos-task-processor:1.0.0-SNAPSHOT	WildFly 31.0.1 JDK 21	Maven WAR	Non-root (jboss), Camel WorkEngine.
agora	localhost:32000/harmonia/agora:1.0.0-SNAPSHOT	Eclipse Temurin 21 Alpine	Spring Boot JAR	Non-root (appuser), Port 8092.
Stateful Middleware & Storage				
artemis-*	apache/activemq-artemis:2.33.0	Red Hat UBI 9 Minimal	Upstream Image	Non-root (artemis), Netty Port 61616.
infinispan-*	localhost:32000/harmonia/mneme-cluster:1.0.0-SNAPSHOT	Quay Infinispan 15.0.3	Custom Config	Non-root (infinispan), Port 11222.
postgres-*	postgres:16-alpine	Alpine Linux 3.19	Upstream Image	Non-root (postgres), Port 5432.
synapse	matrixdotorg/synapse:v1.120.0	Debian Bookworm Slim	Upstream Image	Non-root (synapse), Port 8008.
mnemosyne-clinical	localhost:32000/harmonia/mnemosyne-clinical:1.0.0-SNAPSHOT	Eclipse Temurin 21 Alpine	Spring Boot JAR	Non-root (appuser), FHIR JPA Port 8080.
mnemosyne-operations	localhost:32000/harmonia/mnemosyne-operations:1.0.0-SNAPSHOT	Eclipse Temurin 21 Alpine	Spring Boot JAR	Non-root (appuser), Ops JPA Port 8080.

Table G.1: Harmonia Container Image and Runtime Security Register

Appendix H

Kubernetes Workload and Resource Register

This appendix provides the definitive Kubernetes manifest inventory across all 25 workloads, headless services, ingress objects, and persistent storage claims defined in `deployment/kubernetes/base/`.

H.1 Workload Allocations & Persistent Claims

Workload Name	Kind	CPU Req / Lim	Mem Req / Lim	PVC Claim	Storage Class / Notes
Tier 1: Presentation Tier (Vue 3 SPAs)					
iris-clinical	Deployment	50m / 500m	64Mi / 256Mi	None	Stateless Nginx SPA, Port 80.
iris-console	Deployment	50m / 500m	64Mi / 256Mi	None	Stateless Nginx SPA, Port 80.
iris-administration	Deployment	50m / 500m	64Mi / 256Mi	None	Stateless Nginx SPA, Port 80.
Tier 2: Boundary Protocol Gateways & Presentation Backend					
befe	Deployment	250m / 1000m	512Mi / 1536Mi	None	WildFly dual-port BEFE (8080/8090).
mllp-gateway	Deployment	250m / 1000m	512Mi / 1536Mi	None	Netty/Camel Inbound MLLP (Port 2575).
mllp-outbound-his	Deployment	250m / 1000m	512Mi / 1536Mi	None	Outbound MLLP HIS Egress Dispatcher.
mllp-outbound-lis	Deployment	250m / 1000m	512Mi / 1536Mi	None	Outbound MLLP LIS Egress Dispatcher.
agora	Deployment	250m / 1000m	512Mi / 1536Mi	None	Matrix AS REST Gateway (Port 8092).
Tier 3: Task Execution & Collaboration Core					
task-processor	Deployment	250m / 1000m	512Mi / 1536Mi	None	Ponos Camel WorkEngine (Port 8080).
synapse	StatefulSet	250m / 1000m	512Mi / 2048Mi	5 GiB	synapse-data, Matrix HTTP 8008.
Tier 4: Resilient Messaging & Distributed Cache Grid					
artemis-primary-a	StatefulSet	200m / 1000m	512Mi / 1024Mi	2 GiB	artemis-primary-a-data, Netty 61616.
artemis-backup-a	StatefulSet	200m / 1000m	512Mi / 1024Mi	2 GiB	artemis-backup-a-data, Netty 61616.
artemis-primary-b	StatefulSet	200m / 1000m	512Mi / 1024Mi	2 GiB	artemis-primary-b-data, Netty 61616.
artemis-backup-b	StatefulSet	200m / 1000m	512Mi / 1024Mi	2 GiB	artemis-backup-b-data, Netty 61616.
infinispan-1	StatefulSet	250m / 1000m	512Mi / 1536Mi	1 GiB	mneme-data-1, Hot Rod 11222.
infinispan-2	StatefulSet	250m / 1000m	512Mi / 1536Mi	1 GiB	mneme-data-2, Hot Rod 11222.
Tier 5: Durable Relational Persistence & JPA Servers					
postgres-1	StatefulSet	100m / 1000m	256Mi / 1024Mi	2 GiB	postgres-data-1, Clinical DB 1.
postgres-2	StatefulSet	100m / 1000m	256Mi / 1024Mi	2 GiB	postgres-data-2, Clinical DB 2.
postgres-ops-1	StatefulSet	100m / 1000m	256Mi / 1024Mi	2 GiB	postgres-ops-data-1, Ops DB 1.
postgres-ops-2	StatefulSet	100m / 1000m	256Mi / 1024Mi	2 GiB	postgres-ops-data-2, Ops DB 2.
postgres-synapse	StatefulSet	100m / 1000m	256Mi / 1024Mi	2 GiB	postgres-synapse-data, Synapse DB.
hapi-fhir-jpa-1	Deployment	250m / 1000m	512Mi / 1536Mi	None	HAPI FHIR R5 JPA Server 1.
hapi-fhir-jpa-2	Deployment	250m / 1000m	512Mi / 1536Mi	None	HAPI FHIR R5 JPA Server 2.
hie-ops-jpa-1	Deployment	250m / 1000m	512Mi / 1536Mi	None	Operations JPA Server 1.
hie-ops-jpa-2	Deployment	250m / 1000m	512Mi / 1536Mi	None	Operations JPA Server 2.

Table H.1: Harmonia Kubernetes 25-Workload Resource Allocations and Storage Claims

Appendix I

Persistence Register: Relational Databases & Cache Grid

This appendix provides the authoritative persistence register detailing relational schemas, database tables, primary keys, indexing strategies, and distributed in-memory cache definitions across the Harmonia platform.

I.1 Relational Database Schema Register

Harmonia provisions three decoupled PostgreSQL 16 database clusters:

Database Cluster	Port (Mapped)	Schema / DB Name	Owning Server	Primary Entities & Operational Role
`postgres-1`	5432	`fhir_node_1`	`mnemosyne-clinical`	Clinical FHIR R5 resources (`hie_fhir_resources`).
`postgres-2`	5433	`fhir_node_2`	`mnemosyne-clinical`	Clinical FHIR R5 secondary replica store.
`postgres-ops-1`	5434	`ops_node_1`	`mnemosyne-operations`	Workflow state & audit lineage (`hie_operations_resources`).
`postgres-ops-2`	5435	`ops_node_2`	`mnemosyne-operations`	Workflow state & audit secondary replica store.
`postgres-synapse`	5436	`synapse_db`	Synapse / Agora	Matrix state, `agora_resource_mappings`, `agora_as_transactions`.

Table I.1: Harmonia Relational Database Register

I.1.1 Relational Table Definitions

1. **`hie_fhir_resources`**: Primary key `(resource_type,resource_id,version_id)`. Stores raw JSON resource payload, optimistic lock version, last-updated timestamp, and JSONB search indexes.
2. **`hie_operations_resources`**: Primary key `(operation_id,sequence_id)`. Stores `Pragma` state envelopes, task checkpoint progressions, and execution latencies.
3. **`agora_resource_mappings`**: Primary key UUID `id`. Maps Harmonia resource references to Matrix room/space IDs with `status` and timestamp columns.
4. **`agora_as_transactions`**: Primary key `transaction_id`. Enforces AS push transaction idempotency with receive/process timestamps.

I.2 Distributed Cache Grid Register (Infinispan 15.0.3)

Table I.2 details the 17 synchronously replicated caches configured in `infinispan.xml`:

Cache Name	Mode	Encoding	Write-Behind SPI	Flushing REST Target
person-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
relatedperson-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
practitioner-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
practitionerrole-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
organization-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
location-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
healthcareservice-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
group-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
provenance-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
auditevent-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
consent-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
task-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
communication-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
documentreference-cache	REPL_SYNC	text/plain	FhirRestCacheStore	http://mnemosyne-clinical-1:8080/fhir/r5
tasksequence-cache	REPL_SYNC	text/plain	OperationsRestCacheStore	http://mnemosyne-operations-1:8080/api/operations
messagequeue-cache	REPL_SYNC	text/plain	OperationsRestCacheStore	http://mnemosyne-operations-1:8080/api/operations
modulestatus-cache	REPL_SYNC	text/plain	OperationsRestCacheStore	http://mnemosyne-operations-1:8080/api/operations

Table I.2: Infinispan In-Memory Replicated Cache Grid Register

Appendix J

Module Dependency Matrix

This appendix defines the allowed unidirectional dependency flows across all 9 subprojects of the Harmonia repository.

Subproject	Permitted Dependencies	Architectural Constraints & Forbidden Dependencies
Calliope	JDK, HAPI FHIR Structures	Pure domain models; zero dependencies on any other Harmonia subproject.
Themis	JDK, Calliope, HAPI FHIR	Zero dependencies on storage, messaging, or application tiers.
Hestia	Calliope, Themis, Infinispan, HAPI FHIR, PostgreSQL	Persistent storage layer; zero dependencies on gateways or UI.
Petasos	JDK, Calliope, Themis API	`petasos-api` contains zero JMS/Artemis dependencies.
Energeia	Calliope, Themis, Hestia, Petasos	Workflow engine; zero dependencies on gateways or UI.
Pylai	Calliope, Themis, Petasos, Hestia	Boundary gateways; zero dependencies on Iris or Paradeigma.
Iris	Calliope, Themis API, Hot Rod Client	Presentation only; zero direct database or JPA dependencies.
Agora	Calliope, Themis API, Petasos API, Mnemosyne Operations	Zero dependencies on Ponos (`Agora->Ponos` forbidden); Matrix DTOs encapsulated.
Paradeigma	Production APIs (MLLP, REST)	**Leaf / Simulation Only**. Production modules must NEVER depend on Paradeigma.

Table J.1: Harmonia Subproject Allowed Dependencies and Unidirectional Invariants

Appendix K

Normal Deployment Verification Checklist

This appendix provides the operational checklist for executing and verifying a standard 25-workload Harmonia Normal Deployment on MicroK8s.

K.1 Phase 1: Host Preparation & MicroK8s Setup

- ☐ Verify Ubuntu Linux 22.04 LTS or 24.04 LTS on minimum 8-core, 16 GB RAM host.
- ☐ Configure sysctl parameters (`net.ipv4.ip_forward=1`, `vm.max_map_count=262144`).
- ☐ Install MicroK8s snap via channel `1.28/stable` with `--classic`.
- ☐ Enable required add-ons: `dns`, `hostpath-storage`, `ingress`, `registry`.
- ☐ Verify cluster readiness via `microk8sstatus--wait-ready`.

K.2 Phase 2: Persistent Storage & Volume Claims

- ☐ Verify 12 PersistentVolumeClaims are created and bound in namespace `harmonia`.
- ☐ Confirm 4x Artemis volumes (`artemis-primary-a/b`, `artemis-backup-a/b`) mount 2 GiB each.
- ☐ Confirm 2x Infinispan volumes (`mneme-data-1`, `mneme-data-2`) mount 1 GiB each.
- ☐ Confirm 5x PostgreSQL volumes (`postgres-data-1/2`, `postgres-ops-data-1/2`, `postgres-synapse-data`) mount 2 GiB each.
- ☐ Confirm 1x Synapse volume (`synapse-data`) mounts 5 GiB.

K.3 Phase 3: Stateful Tier Verification

- ☐ Verify ActiveMQ Artemis 4-broker cluster formed via `artemischeckcluster`.
- ☐ Verify Infinispan 2-node grid formed on port 7800 and 17 caches initialized.
- ☐ Verify PostgreSQL databases report `pg_isready` on ports 5432, 5434, and 5436.
- ☐ Verify Synapse homeserver responds on port 8008 (`/_matrix/client/versions`).

K.4 Phase 4: Stateless Workloads & Gateways

- ☐ Verify Inbound MLLP Gateway (`mllp-gateway`) is listening on TCP port 2575.
- ☐ Verify Outbound MLLP Gateways (`mllp-outbound-his`, `mllp-outbound-lis`) are active.
- ☐ Verify Ponos WorkEngine connected to Petasos queue and Camel routes running.
- ☐ Verify Agora gateway connected to Synapse and listening on port 8092.
- ☐ Verify Iris BEFE gateway active on ports 8080 and 8090.
- ☐ Verify Iris SPAs (`iris-clinical`, `iris-console`, `iris-administration`) serve HTTP 200 on port 80.

Appendix L

Paradeigma Simulation Deployment Checklist

This appendix provides the operational checklist for deploying and verifying a 30-workload Harmonia Paradeigma Simulation cluster with strict production isolation.

L.1 Phase 1: Production Isolation Pre-Flight Checks

- ☐ Run ArchUnit isolation tests: `mvntest-plparadeigma/paradeigma-test-am-Dtest="*ArchitectureTest"`.
- ☐ Verify 0 production POMs declare dependencies on `paradeigma-*`.
- ☐ Verify 0 production Java classes import `net.fhirfactory.harmonia.paradeigma.*`.
- ☐ Verify zero simulation flags exist in production source code.
- ☐ Confirm production container images exclude simulation classes.

L.2 Phase 2: Simulation Workload Deployment

- ☐ Apply simulation Kustomize overlay: `kubectlapply-kdeployment/kubernetes/environments/simulation`.
- ☐ Verify 5 decoupled simulator pods reach Running state:
  - `paradeigma-pas`: HTTP port 8091.
  - `paradeigma-emr`: HTTP port 8092, MLLP port 2201.
  - `paradeigma-lms`: HTTP port 8093, MLLP ports 2202 and 2204.
  - `paradeigma-rispac`: HTTP port 8094, MLLP ports 2203 and 2205.
  - `paradeigma-scenarios`: HTTP port 8090.
- ☐ Verify simulator pods request 0 PersistentVolumeClaims.

L.3 Phase 3: Scenario Execution & Journey Verification

- ☐ Trigger single Patient Journey: `curl-XPOSThttp://localhost:8090/api/scenarios/journey` (no request body).
- ☐ Verify PAS generates and transmits `ADT^A04` / `ADT^A01` to `pylai-mllp-in:2575`.
- ☐ Verify Pylai returns synchronous `AA` acknowledgment to simulator.
- ☐ Verify EMR, LMS, and RIS-PAC receive fan-out event notifications on ports 2201–2205.
- ☐ Query scenario status: `curlhttp://localhost:8090/api/scenarios/status`.
- ☐ Verify dual-gate logging invariant: zero PHI in operational logs; diagnostic PHI gated by `harmonia.logging.phi-enabled`.

Appendix M

Architecture Requirements

This appendix records the enduring architecture requirements for Harmonia. It intentionally describes *what the architecture must achieve*, not the implementation tasks currently required to achieve it.

Implementation defects, architecture-conformance findings, remediation activities, and delivery tasks are maintained separately in the Architecture Backlog. Architectural choices that constrain how these requirements are realised are maintained separately as Architecture Decisions.

M.1 Requirement Conventions

Requirements use a stable domain identifier:

- `CORE-nnn` — cross-cutting platform requirements;
- `SEC-nnn` — security requirements;
- `PROV-nnn` — provenance and audit requirements;
- `DATA-nnn` — persistence, consistency, and recovery requirements;
- `INT-nnn` — integration, messaging, and interoperability requirements;
- `DIR-nnn` — Provider Registry requirements;
- `CLIN-nnn` — clinical information and longitudinal-record requirements;
- `UX-nnn` — human-facing presentation requirements;
- `OPS-nnn` — operational and resilience requirements.

The intended traceability model is:

Driver → Goal → Principle → Requirement → Decision → Architecture → Backlog → Verification

M.2 Core Platform Requirements

M.2.1 CORE-001 — Information Authority Classification

Harmonia shall support classification of information sources, ingress packets, and individual assertions as `AUTHORITATIVE`, `INFORMATIONAL`, or `ANECDOTAL`. Authority shall be contextual and independent of provenance, security classification, and access control. An authoritative classification shall express how Harmonia may treat an assertion within a defined information context and shall not independently assert clinical truth.

M.2.2 CORE-002 — Separation of Architectural Concerns

Harmonia shall maintain clear separation between interface/protocol handling, messaging, workflow orchestration, domain processing, canonical modelling, persistence, security, collaboration, and presentation so that each capability can evolve without assuming ownership of another capability's internal runtime.

M.2.3 CORE-003 — Governed Canonical Semantics

Information used for internal processing shall be represented using governed canonical semantics. Transformation between external formats and internal representations shall preserve clinically and operationally material meaning and shall be traceable.

M.2.4 CORE-004 — Correlation and Causation

Harmonia shall retain sufficient correlation and causation information to relate an originating interaction to subsequent messages, tasks, workflow activities, persistence operations, and egress interactions.

M.3 Security Requirements

M.3.1 SEC-001 — Trusted Authentication

Protected Harmonia services shall establish caller identity through a trusted authentication boundary using recognised authentication standards. Application components shall not trust caller-supplied identity claims that have not been authenticated by that boundary.

M.3.2 SEC-002 — Default-Deny Authorisation

Protected operations shall be denied unless an authenticated principal is explicitly authorised to perform the requested action in the applicable security domain.

M.3.3 SEC-003 — Authentication and Authorisation Separation

Authentication and authorisation shall remain distinct concerns. Authentication establishes identity; authorisation determines whether that identity may perform an action against a resource.

M.3.4 SEC-004 — Contextual Resource Authorisation

Authorisation decisions shall be capable of considering the principal, security domain, requested action, resource type, resource identity where applicable, effective authorities, and other governed security attributes.

M.3.5 SEC-005 — End-to-End Security Context

Security context established at ingress shall remain available across synchronous calls, asynchronous processing, persistence, retry, replay, and recovery where required to preserve accountability and authorisation semantics.

M.3.6 SEC-006 — Initiating and Executing Identity

Harmonia shall distinguish the principal that initiated an operation from the service or process currently executing it. Automated processing shall not erase or replace the initiating identity.

M.3.7 SEC-007 — Anti-Spoofing

External callers shall not be able to establish or elevate identity, roles, authorities, security domains, or internal trust by supplying untrusted headers or payload fields.

M.3.8 SEC-008 — Security Metadata Preservation

Security-domain and resource-security metadata required for access-control decisions shall survive transformation, messaging, caching, persistence, replay, and recovery.

M.3.9 SEC-009 — Least Privilege

Human users, services, processes, and infrastructure identities shall receive only the permissions required for their defined responsibilities.

M.3.10 SEC-010 — PHI-Safe Operational Logging

Routine operational logging shall not expose PHI, credentials, authentication tokens, or sensitive clinical payloads. Diagnostic logging capable of containing PHI shall be explicitly governed, access-controlled, and isolated.

M.4 Provenance and Audit Requirements

M.4.1 PROV-001 — End-to-End Provenance

Harmonia shall retain sufficient provenance to determine the origin, actors, transformations, processing history, and resulting artefacts associated with clinically or operationally material information.

M.4.2 PROV-002 — Source Context

Where applicable, persisted information shall retain its source system, source organisation, source identifier, effective date or period, recorded/authored date, Harmonia receipt time, and relevant transformation history.

M.4.3 PROV-003 — Dual-Actor Provenance

Where automated processing occurs on behalf of a human or originating system, provenance shall be capable of representing both the initiating actor and the executing service/process.

M.4.4 PROV-004 — Provenance and Authority Independence

Provenance shall describe origin and processing history; Information Authority shall describe how Harmonia may treat an assertion. Neither shall substitute for the other.

M.4.5 PROV-005 — Immutable Audit

Security and operational audit records required for accountability shall be system-generated, durable, and immutable under ordinary application operations.

M.4.6 PROV-006 — Audit Discoverability

Authorised users and services shall be able to locate relevant audit and provenance records using governed search criteria including resource, actor, correlation, and time where applicable.

M.5 Data, Persistence and Recovery Requirements

M.5.1 DATA-001 — Durable Governed State

Information intended to survive as Harmonia state shall be stored in an explicitly designated durable persistence service. Volatile process memory shall not become the authoritative store for governed clinical, directory, security, provenance, or workflow state.

M.5.2 DATA-002 — Authoritative Read Semantics

Read and search operations shall operate over the authoritative persisted information set irrespective of cache population, cache eviction, process restart, or node replacement.

M.5.3 DATA-003 — Cache Transparency

Caching shall improve performance or availability without changing the semantic existence, correctness, or discoverability of persisted information.

M.5.4 DATA-004 — Search Scalability

Search and collection retrieval shall support bounded server-side filtering and paging appropriate to the information domain and shall not require consumers to retrieve or scan entire datasets.

M.5.5 DATA-005 — Concurrency and Version Integrity

Governed mutable information shall support appropriate versioning and concurrency controls to prevent silent overwriting of newer state.

M.5.6 DATA-006 — Referential Integrity

Where resources contain governed references to other resources, Harmonia shall validate required referential integrity before committing changes that would create invalid or unsafe state.

M.5.7 DATA-007 — Persistence Failure Safety

Acknowledged governed state shall not be silently lost because of asynchronous persistence, cache failure, process restart, or infrastructure failover. Persistence failure and recovery semantics shall be explicit and observable.

M.5.8 DATA-008 — Recovery Fidelity

Recovery and replay shall preserve the contextual metadata necessary to maintain the original security, provenance, authority, correlation, causation, and processing semantics.

M.6 Integration and Interoperability Requirements

M.6.1 INT-001 — Standards-Based Healthcare Interoperability

Harmonia shall support healthcare interoperability through standards-based interfaces including HL7 v2 and HL7 FHIR R5, while permitting governed adapters for proprietary source-system interfaces where required.

M.6.2 INT-002 — Reliable Messaging

Asynchronous messaging used for governed processing shall provide durable at-least-once delivery. Consumers shall be designed to tolerate duplicate delivery safely.

M.6.3 INT-003 — Messaging Recovery

Acknowledged in-flight work shall survive broker restart, network interruption, process failure, and supported failover scenarios without silent loss.

M.6.4 INT-004 — FHIR R5 Boundary

FHIR-facing APIs shall use HL7 FHIR Release 5 resource structures, search semantics, metadata, and error representations except where a documented constraint requires otherwise.

M.6.5 INT-005 — Opaque Resource Identity

FHIR logical resource identifiers shall be treated as opaque identifiers and shall not expose or assume business identifiers such as MRNs as resource identity.

M.6.6 INT-006 — Governed Validation

Information entering governed Harmonia state shall be validated against applicable structural, semantic, profile, referential, and business rules before authoritative state is changed.

M.6.7 INT-007 — Terminology Abstraction

Terminology-dependent clinical capabilities shall consume governed terminology services through a Harmonia abstraction rather than coupling presentation components directly to a particular terminology-server product.

M.7 Provider Registry Requirements

M.7.1 DIR-001 — Standards-Based Provider Directory

Harmonia shall provide a standards-based FHIR R5 representation of practitioners, practitioner roles, organisations, locations, healthcare services, endpoints, groups, and their governed relationships.

M.7.2 DIR-002 — Provider Directory Discovery

Authorised consumers shall be able to read and search approved provider-directory information through standards-based interfaces.

M.7.3 DIR-003 — Governed Directory Change

Changes to authoritative Provider Registry state shall pass through governed validation and approval/stewarding processes rather than uncontrolled direct mutation.

M.7.4 DIR-004 — Directory Referential Integrity

Provider-directory changes shall preserve valid relationships between practitioners, roles, organisations, locations, services, endpoints, and groups.

M.7.5 DIR-005 — Healthcare Identifier Governance

The Provider Registry shall support governance of applicable national and organisational healthcare identifiers and prevent invalid or conflicting authoritative identifier state.

M.7.6 DIR-006 — Provider Self-Service Governance

Where providers or departmental administrators can request directory changes, self-service shall initiate governed change processing rather than bypass authoritative stewardship.

M.8 Clinical Information and Longitudinal Record Requirements

M.8.1 CLIN-001 — Vendor-Neutral Clinical Information Layer

Harmonia shall maintain a durable, vendor-neutral representation of clinically relevant information independently of proprietary source-system interfaces while preserving the identity, provenance, and authority of source information.

M.8.2 CLIN-002 — FHIR-Based Longitudinal Clinical Representation

The externally consumable longitudinal clinical information model shall use FHIR R5 and support the resources required to represent patient identity, flags, allergies/intolerances, episodes, encounters, problems, observations, diagnostic requests/results, referrals, medications, and necessary supporting entities.

M.8.3 CLIN-003 — Medication as a First-Class Clinical Domain

The longitudinal record shall distinguish medication definition, prescribing/ordering, dispensing, administration, and reported medication use rather than collapsing them into a single undifferentiated medication list.

M.8.4 CLIN-004 — Source-System Independence

Downstream consumers shall be able to consume Harmonia's governed clinical representation without requiring direct knowledge of, or dependency upon, proprietary EMR, PAS, LIS, RIS/PACS, or departmental data models.

M.8.5 CLIN-005 — Source Authority Preservation

A source system may remain authoritative for the clinical or administrative activity it manages while Harmonia maintains the governed interoperability representation of that information.

M.8.6 CLIN-006 — Authority-Aware Clinical Reconciliation

Harmonia shall be capable of applying governed reconciliation behaviour based on source, Information Authority, provenance, resource type, existing state, and processing context. Authority classifications shall not imply fixed reconciliation outcomes.

M.8.7 CLIN-007 — Preservation of Conflicting Assertions

Where clinically appropriate, Harmonia shall be capable of retaining multiple differing assertions with their provenance, authority, and verification state rather than forcing an unjustified single truth.

M.8.8 CLIN-008 — Clinical Source Context

Authorised consumers shall be able to determine, where relevant, what an item represents, when it was clinically effective, who or what recorded it, where it originated, when Harmonia received it, its authority classification, and its verification/reconciliation state.

M.8.9 CLIN-009 — Governed Patient Identity

Clinical information shall be associated with patients through governed identity matching and reconciliation that preserves source identifiers and avoids unsafe silent duplication or merging.

M.8.10 CLIN-010 — Read-Oriented Initial LHR

The initial longitudinal-record capability shall prioritise reliable read and search access while preserving an architecture that can support selectively governed clinical authoring later.

M.8.11 CLIN-011 — Selective Clinical Authoring

Clinical create/update capability shall be enabled by information domain only where the required terminology, ontology, validation, security, governance, reconciliation, and workflow controls are available.

M.9 Human-Facing Presentation Requirements

M.9.1 UX-001 — Consistent Iris Experience

Harmonia's Iris applications shall use a common design system and consistent interaction patterns while allowing Clinical, Monitoring, and Administration applications to retain domain-appropriate information architecture.

M.9.2 UX-002 — Information-Dense Operational Design

Iris shall favour readable, information-dense, action-oriented presentation with restrained visual decoration and clear navigation.

M.9.3 UX-003 — Honest System State

User interfaces shall distinguish observed runtime information from configured/static information and shall not fabricate plausible runtime state when data is unavailable.

M.9.4 UX-004 — Explicit Availability States

Material user-interface views shall distinguish loading, loaded, empty, unavailable, partial, and error states so infrastructure failure is not misrepresented as an empty healthy dataset.

M.9.5 UX-005 — Patient Search and Workspace

Iris-Clinical shall provide patient search/selection and shall open selected patients in clearly identified patient workspaces. Multiple patient workspaces may be open concurrently.

M.9.6 UX-006 — Longitudinal Clinical Navigation

The initial Iris-Clinical patient workspace shall provide access to Demographics, Flags, Allergies/Intolerances, Episodes/Encounters, Problems, Medications, Observations, Diagnostics, and Referrals.

M.9.7 UX-007 — Persistent Patient Context

Iris-Clinical shall maintain clear patient identity context while a patient workspace is active and shall support prominent display of clinically important active warnings such as flags and allergies/intolerances.

M.9.8 UX-008 — Provenance-Aware Clinical Presentation

Clinical views shall expose relevant source, effective/recorded/received dates, authority, provenance, and verification/reconciliation context without independently asserting clinical truth.

M.9.9 UX-009 — Initial Clinical Scope

Appointment management, general tasking, and comprehensive dynamic-EMR workflow are not required for the initial Iris-Clinical longitudinal-record capability.

M.10 Operational and Resilience Requirements

M.10.1 OPS-001 — Reproducible Deployment

Harmonia shall support repeatable deployment in development and target runtime environments using externalised configuration and automated deployment mechanisms.

M.10.2 OPS-002 — Durable Infrastructure State

Stateful platform services shall use explicit durable storage appropriate to their recovery requirements.

M.10.3 OPS-003 — Meaningful Health and Readiness

Services shall expose health and readiness information that reflects material dependencies rather than merely indicating that a process or network port exists.

M.10.4 OPS-004 — Operational Observability

Harmonia shall expose sufficient operational telemetry to diagnose subsystem, interface, queue, workflow, persistence, event, and task health without requiring direct inspection of implementation internals.

M.10.5 OPS-005 — Production Resilience

The target architecture shall support appropriate high availability, recovery, and horizontal or vertical scaling for messaging, persistence, caching, and stateless services. Development simplifications shall not prevent deployment of the production resilience model.

M.10.6 OPS-006 — Architecture Conformance

Critical architectural boundaries and invariants shall be verifiable through automated tests or equivalent repeatable controls where practical.

M.11 Requirement Evolution

Requirements in this appendix are deliberately implementation-neutral. When implementation analysis identifies a defect or missing capability, that observation shall be recorded as an architecture finding and/or backlog item and linked to the relevant requirement rather than rewriting the requirement around the current codebase.

New requirements shall be added when Harmonia acquires a new enduring architectural obligation. Backlog work may be added, completed, replaced, or abandoned without changing the requirement unless the architectural obligation itself changes.

Appendix N

Architecture Conformance Findings

This appendix records implementation observations discovered during architecture and code reviews. Findings describe the implementation at a point in time; they are not themselves architecture requirements. A finding may create one or more backlog items and may later be closed without changing the underlying requirement.

N.1 Open / Recently Identified Findings

N.1.1 FND-001 — Clinical Search Has Been Cache-Dependent

Clinical/FHIR search was identified as scanning in-memory cache values rather than querying the authoritative persisted dataset. This can produce false-negative searches after restart, eviction, or cold start.

Related: DATA-002, DATA-003; BL-DATA-002, BL-DATA-003.

N.1.2 FND-002 — Volatile Clinical Fallback Path

A clinical persistence path was identified that could fall back to unbacked JVM memory, allowing state to disappear on process/pod restart.

Related: DATA-001, DATA-007; BL-DATA-001.

N.1.3 FND-003 — Clinical API Authorisation Gap

Review identified Clinical FHIR endpoints that did not consistently pass through Themis authorisation.

Related: SEC-002–SEC-004; BL-SEC-001.

N.1.4 FND-004 — Clinical Browser Authentication Gap

Review identified an Iris-Clinical path without a complete authenticated browser-to-BEFE identity flow.

Related: SEC-001; BL-SEC-001.

N.1.5 FND-005 — Credentialed Wildcard CORS

A BEFE CORS configuration was identified that permitted wildcard origins while supporting credentials.

Related: SEC-007; BL-SEC-004.

N.1.6 FND-006 — Mutable AuditEvent Surface

AuditEvent update/delete operations and corresponding UI controls were identified despite the target immutable audit model.

Related: PROV-005; BL-PROV-002, BL-IRIS-004.

N.1.7 FND-007 — Unbounded Iris Collection Loading

Iris was identified as discarding useful Bundle total metadata and performing broad/unbounded parallel resource searches in some views.

Related: DATA-004; BL-DATA-003, BL-IRIS-005.

N.1.8 FND-008 — Pylai Header Trust Risk

A Pylai security interceptor path was identified that could derive principal/authority context from raw HTTP headers unless protected by a trusted upstream boundary.

Related: SEC-007; BL-SEC-003.

N.1.9 FND-009 — Legacy Embedded-Broker Assumptions

Historical Ponos/Pylai deployment and configuration paths contained assumptions that Ponos hosted Artemis. The runtime has been converged toward standalone Petasos ownership, but remaining legacy assumptions should continue to be removed when encountered.

Related: `CORE-002`, `INT-002`; `BL-MSG-001`, `BL-MSG-002`.

N.1.10 FND-010 — Partial Queue Telemetry Population

Operational contracts exist for queue depth and producer/consumer metrics, but the development topology has not always populated all live queue metrics.

Related: `OPS-004`; `BL-MSG-003`.

N.2 Finding Lifecycle

A finding should be marked closed when verification demonstrates that the implementation conforms to the relevant requirement. Closing a finding does not remove or retire the requirement. Where useful, closed findings may be retained as architecture history.

Appendix O

Architecture and Implementation Backlog

This appendix records identified work required to converge the Harmonia implementation toward the target architecture. Unlike the Architecture Requirements catalogue, backlog items are intentionally implementation-aware and may be completed, replaced, decomposed, or retired as the codebase evolves.

O.1 Backlog Conventions

Backlog identifiers use the form `BL-<DOMAIN>-nnn`. Each item should identify the requirement(s) it supports and, where relevant, the architecture finding that caused the work to be raised.

Suggested lifecycle states are `IDENTIFIED`, `PLANNED`, `IN-PROGRESS`, `BLOCKED`, `DONE`, and `SUPERSEDED`.

O.2 Foundation Convergence

O.2.1 BL-SEC-001 — Complete Clinical Authorisation Boundary

Ensure all Clinical FHIR endpoints pass through the trusted Themis authorisation boundary using authenticated container identity, deterministic policy evaluation, correct action/resource mapping, default-deny behaviour, and 401/403 semantics.

Supports: `SEC-001`, `SEC-002`, `SEC-003`, `SEC-004`.

O.2.2 BL-SEC-002 — Complete Security Context Propagation

Propagate the canonical Themis security context across Iris BEFE, Pylai, Petasos messages, Ponos/Erga processing, Mnemosyne persistence, retry, replay, and recovery while retaining initiating and executing principals.

Supports: `SEC-005`, `SEC-006`, `SEC-008`, `PROV-003`.

O.2.3 BL-SEC-003 — Close Internal Trust-Boundary Spoofing Paths

Remove or protect code paths that accept caller-controlled principal/role headers. Establish a trusted internal service-to-service context propagation mechanism before accepting forwarded security context.

Supports: `SEC-007`, `SEC-009`.

O.2.4 BL-SEC-004 — Restrict Browser CORS

Replace wildcard credentialed CORS behaviour with configured trusted Iris origins.

Supports: `SEC-007`, `SEC-009`.

O.2.5 BL-PROV-001 — Complete End-to-End Provenance Framework

Ensure ingress, workflow, transformation, persistence, and egress produce/retain sufficient provenance, including source context and initiating/executing actor relationships.

Supports: `PROV-001`–`PROV-004`.

O.2.6 BL-PROV-002 — Enforce AuditEvent WORM Behaviour

Remove ordinary update/delete behaviour for AuditEvent from service and UI layers; establish durable immutable audit persistence and governed search.

Supports: `PROV-005`, `PROV-006`.

O.2.7 BL-DATA-001 — Remove Volatile Clinical Persistence Fallback

Eliminate any in-JVM fallback that can accept governed clinical state without durable backing. Fail explicitly when the durable path is unavailable.

Supports: `DATA-001`, `DATA-007`.

O.2.8 BL-DATA-002 — Make Clinical Search Authoritative-Backed

Replace cache-value scanning with authoritative Mnemosyne-backed search so persisted resources remain discoverable after restart, eviction, or cold start.

Supports: `DATA-002`, `DATA-003`.

O.2.9 BL-DATA-003 — Implement Server-Side FHIR Paging and Totals

Push search filtering/paging to persistence and return appropriate FHIR Bundle paging/total metadata. Update Iris clients to consume it.

Supports: `DATA-004`, `INT-004`.

O.2.10 BL-DATA-004 — Validate Write-Behind Failure and Recovery

Define and test retry, ordering, idempotency, recovery, and observability for Mneme-to-Mnemosyne write-behind persistence.

Supports: `DATA-007`, `DATA-008`.

O.2.11 BL-DATA-005 — Preserve Context During Persistence Recovery

Ensure cache-store/recovery paths retain security, provenance, authority, correlation, and causation metadata.

Supports: `DATA-008`, `SEC-005`, `PROV-001`.

O.3 Architecture Technical Debt

O.3.1 Execution Context and Context Propagation

BL-ARCH-01 — Replace ThreadLocal CORS Context in OperationsServerManager

Description Review and refactor OperationsServerManager so the validated CORS origin is propagated explicitly as request scoped state rather than through CURRENT_ALLOWED_ORIGIN ThreadLocal..

Priority Low.

Driver Maintainability and future concurrency safety.

Rationale The current implementation is safe because the value is established and cleared within a finally block. However, correctness depends on the server's current thread-per-request execution model. Explicit request-context propagation would remove that coupling and make future asynchronous/executor-based processing safer.

Acceptance Criteria No CORS/security state stored in ThreadLocal; validated origin travels explicitly with request/response processing; existing trusted/untrusted/no-origin/preflight behaviour unchanged; existing CORS tests remain green; no changes to authentication or Themis authorization semantics.

Supports

Related CORS Hardenning (see xxxx).

O.4 Messaging and Runtime Backlog

O.4.1 BL-MSG-001 — Preserve Petasos/Ponos Separation

Maintain the verified architecture in which Petasos owns standalone Artemis messaging and Ponos is a messaging client/work executor. Remove remaining legacy embedded-broker assumptions where found.

Supports: CORE-002, INT-002, INT-003.

O.4.2 BL-MSG-002 — Standardise Broker Configuration

Converge runtime configuration on the approved Petasos broker connection contract across Ponos, Pylai, Agora, and related clients.

Supports: CORE-002, OPS-001.

O.4.3 BL-MSG-003 — Complete Petasos Runtime Telemetry

Populate and expose live broker/queue health, depth, producer/consumer, retry, and DLQ telemetry through the existing operational monitoring abstractions.

Supports: OPS-003, OPS-004.

O.4.4 BL-MSG-004 — Verify Broker Persistence and Failover

Exercise restart, reconnection, redelivery, persistence, and supported HA/failover scenarios with context preservation and no acknowledged message loss.

Supports: INT-002, INT-003, DATA-008.

O.4.5 BL-MSG-005 — Petasos Message Replay

Implement governed Petasos message replay only after the current security, provenance, persistence, and UI convergence baseline is complete.

Supports: INT-002, PROV-001, DATA-008.

O.5 Provider Registry Backlog

O.5.1 BL-DIR-001 — Complete Provider Registry Validation

Complete resource-specific validation and referential checks for Practitioner, PractitionerRole, Organization, Location, HealthcareService, Endpoint, and Group.

Supports: DIR-001, DIR-004, INT-006.

O.5.2 BL-DIR-002 — Complete Governed Change Workflow

Ensure Provider Registry POST/PUT requests consistently create governed change work, validation, approval/stewarding, persistence, versioning, and structured outcomes.

Supports: DIR-003, DATA-005.

O.5.3 BL-DIR-003 — Provider Registry Syndication

Implement outbound Provider Registry syndication after the stable UI and foundational processing/security baseline is complete.

Supports: DIR-001, DIR-002, INT-001.

O.6 Iris Baseline Backlog

O.6.1 BL-IRIS-001 — Maintain Shared Iris Design System

Continue convergence of Clinical, Monitor, and Administration applications on iris-befe/PrimeVue shared design patterns and consistent navigation.

Supports: UX-001, UX-002.

O.6.2 BL-IRIS-002 — Remove Fabricated Runtime Presentation

Ensure operational UI values are sourced from real runtime telemetry or explicitly marked unavailable/configured; do not generate plausible fake values.

Supports: UX-003, UX-004, OPS-004.

O.6.3 BL-IRIS-003 — Complete Monitor Runtime Drill-Down

Complete subsystem/component/runtime drill-down for pods/state, useful operational detail, and event/task/statistics panes using real backend data.

Supports: UX-001, UX-003, OPS-004.

O.6.4 BL-IRIS-004 — Make Audit UI Read-Only

Remove create/update/delete affordances for immutable audit records and retain search/detail inspection.
Supports: PROV-005.

O.6.5 BL-IRIS-005 — Eliminate Unbounded Dashboard Searches

Replace broad parallel unbounded resource loads with bounded overview queries and server-side counts/paging.
Supports: DATA-004, UX-004.

O.7 Clinical / Longitudinal Health Record Backlog

O.7.1 BL-CLIN-001 — Clinical FHIR Information Layer

Establish the vendor-neutral, independently persisted Clinical FHIR information service across Pylai, Mnemosyne, and Calliope. Source systems remain authoritative for their defined activities while Harmonia becomes the governed interoperability representation.
Supports: CLIN-001, CLIN-004, CLIN-005.

O.7.2 BL-CLIN-002 — Core Clinical Resource Model

Implement the initial LHR resource set: Patient, Flag, AllergyIntolerance, EpisodeOfCare, Encounter, Condition, Observation, ServiceRequest, DiagnosticReport, and required supporting references.
Supports: CLIN-002, INT-004, INT-006.

O.7.3 BL-CLIN-003 — Medication Information Model

Implement Medication, MedicationRequest, the appropriate R5 medication-use representation, MedicationAdministration, MedicationDispense, and their longitudinal relationships.
Supports: CLIN-003.

O.7.4 BL-CLIN-004 — Information Authority Framework

Implement source-, packet-, and assertion-level Information Authority metadata and governance using AUTHORITATIVE, INFORMATIONAL, and ANECDOTAL classifications.
Supports: CORE-001, CLIN-005, CLIN-006.

O.7.5 BL-CLIN-005 — LHR Reconciliation and Update Processing

Implement Ponos/Erga rules that use source, authority, provenance, resource type, and current state to accept, retain, reconcile, supersede, review, quarantine, or reject incoming assertions without hard-coding outcomes to authority categories.
Supports: CLIN-006, CLIN-007.

O.7.6 BL-CLIN-006 — Clinical Source-System Ingestion

Implement governed ingestion patterns for EMR, PAS, LIS, RIS/PACS, and other clinical systems, preserving source identifiers, provenance, authority, and clinical semantics.
Supports: CLIN-001, CLIN-004, CLIN-005, PROV-002.

O.7.7 BL-CLIN-007 — Iris-Clinical LHR Viewer

Implement the read-oriented patient viewer with Find/Select Patient, multiple patient workspaces, Demographics, Flags, Allergies/Intolerances, Episodes/Encounters, Problems, Medications, Observations, Diagnostics, Referrals, and provenance/source context.
Supports: CLIN-010, UX-005–UX-009.

O.7.8 BL-CLIN-008 — Clinical Authoring and Terminology Services

Introduce selective clinical create/update only after terminology lookup/search, ValueSet expansion/validation, profile governance, security, reconciliation, and workflow controls exist. Keep Iris decoupled from the terminology-server product.

Supports: CLIN-011, INT-006, INT-007.

O.7.9 BL-CLIN-009 — Patient Identity / Golden Record Processing

Elaborate and implement governed patient matching, identifier reconciliation, source history, merge handling, and golden-record behaviour appropriate to the LHR.

Supports: CLIN-009.

O.8 Agora Backlog

O.8.1 BL-AGORA-001 — Complete Matrix/Synapse Clinical Mapping

Complete the governed Patient-to-hidden-room/sub-room, PractitionerRole-to-room, Group-to-room, and Practitioner-to-space mappings with membership managed by Agora.

Supports: CORE-002, SEC-004.

O.8.2 BL-AGORA-002 — Align Agora Security Context

Ensure Agora collaboration access respects Themis identity/authority and relevant resource/Pragma security tags.

Supports: SEC-004, SEC-005, SEC-008.

O.9 Deployment and Operations Backlog

O.9.1 BL-OPS-001 — Maintain Docker Compose Known-Good Baseline

Keep the verified Docker Compose runtime reproducible while architectural convergence work proceeds; avoid UI work silently modifying backend topology.

Supports: OPS-001, OPS-006.

O.9.2 BL-OPS-002 — Complete MicroK8s Deployment Automation

Complete Maven image build/publish and Ansible deployment/undeployment workflows for the target MicroK8s host, including required persistent storage and registry configuration.

Supports: OPS-001, OPS-002, OPS-005.

O.9.3 BL-OPS-003 — Architecture Regression Guardrails

Expand automated architecture tests to protect subsystem ownership, security boundaries, dependency direction, Paradeigma isolation, and other critical invariants.

Supports: OPS-006, CORE-002.

O.10 Harmonia - Audit

O.10.1 BL-AUDIT-01 — Introduce a typed Harmonia Security Domain

Replace the current string-based securityDomain representation (`"CLINICAL"`, `"AUDIT"`, `"OPERATIONS"`, `"PROVIDER_REGISTRY"`, etc.) with a canonical typed Harmonia/Themis security-domain value or enum, and propagate it consistently through `ThemisSecurityContext`, `ThemisResource`, `ThemisAuditEvent`, policies and related contracts.

Rationale: Task 06 correctly retained String securityDomain because the repository currently only has string constants and changing this would expand Step 01 substantially. The exploration explicitly identified this as the smallest compatible choice.

Timing: Foundations cleanup after Tasks 06–10, unless another task creates a natural migration point.

O.10.2 BL-AUDIT-02 — Relocate immutable audit evidence to dedicated Operations/Audit persistence

Logical ownership is AUDIT/operations rather than clinical; defer physical movement so Task 06 doesn't turn into the persistence redesign covered by later tasks.

O.10.3 BL-AUDIT-03 — Database-level audit immutability hardening

Consider PostgreSQL permissions/triggers or equivalent append-only/WORM controls after the application/persistence architecture settles. This adds defence-in-depth beneath the service and persistence invariants.

O.11 Backlog Sequencing

The current preferred sequence is:

1. complete Security convergence;
2. complete Provenance and immutable Audit convergence;
3. complete persistence/search/recovery convergence;
4. complete validation/governance and Ponos/Erga processing foundations;
5. preserve and verify Petasos/runtime convergence;
6. commence the Clinical/LHR backlog (`BL-CLIN-001` onward);
7. implement Iris-Clinical against the stable Clinical FHIR service;
8. introduce selective clinical authoring only after terminology and governance prerequisites exist.

Large capability backlog items, particularly `BL-CLIN-002` and `BL-CLIN-003`, should remain capability-sized until their prerequisite foundations converge. They can then be decomposed into bounded Junie-sized implementation work packages.

Appendix P

Architecture Decisions

This appendix records architecture choices that constrain how Harmonia requirements are realised. Decisions are distinct from requirements: a requirement states what Harmonia must achieve; a decision records the architectural approach selected to achieve or protect that outcome.

P.1 Current Decisions

P.1.1 ADR-001 — Petasos Owns Messaging Runtime

Petasos is the standalone Harmonia messaging capability and owns the ActiveMQ Artemis runtime. Ponos consumes Petasos messaging services and owns workflow/work execution; it does not own broker lifecycle.

P.1.2 ADR-002 — Pylai Owns External Interface and Protocol Behaviour

Pylai owns external protocol/interface behaviour, including HL7/FHIR ingress and egress. Domain workflow logic remains outside the protocol boundary.

P.1.3 ADR-003 — Mnemosyne Owns Durable Application State

Mnemosyne is the durable persistence capability. Mneme/Infinispan provides operational/cache state and must not redefine the existence of durable governed information.

P.1.4 ADR-004 — Calliope Owns Canonical Models and Semantic Governance

Calliope owns canonical model definitions, profiles, terminology bindings, and semantic governance used by Harmonia processing.

P.1.5 ADR-005 — Themis Owns Security Policy Decisions

Themis is the common Harmonia authorisation/policy capability. Authentication boundaries establish trusted identity; Themis evaluates permissions.

P.1.6 ADR-006 — FHIR R5 Is the Clinical Interoperability Boundary

The durable clinical interoperability service is represented and exposed using HL7 FHIR R5. Proprietary source models are translated into the governed representation rather than propagated as the enterprise integration contract.

P.1.7 ADR-007 — Harmonia Is an Interoperability Authority, Not Universal Clinical Truth

Source systems may remain authoritative for the activities they manage. Harmonia maintains the governed, vendor-neutral interoperability representation and preserves provenance and Information Authority rather than claiming universal clinical source-of-truth status.

P.1.8 ADR-008 — Iris-Clinical Begins as a Longitudinal Record Viewer

The initial Iris-Clinical capability is read-oriented. The architecture permits selective clinical authoring later, but authoring is introduced only where terminology, validation, security, governance, and workflow prerequisites are satisfied.

P.1.9 ADR-009 — Information Authority Is Cross-Cutting

AUTHORITATIVE, INFORMATIONAL, and ANECDOTAL classification is a core Harmonia concept rather than an Iris or Clinical-only feature.

P.1.10 ADR-010 — Clinical Search Must Be Authoritative-Backed

Cache contents are not the definition of the clinical record. Clinical read/search must remain correct from durable state after cache loss or restart.

P.1.11 ADR-011 — Agora Uses Restricted Self-Hosted Matrix/Synapse

The initial Agora collaboration environment is self-hosted, restricted, and non-federated. Patient collaboration rooms and membership are governed by Harmonia rather than unmanaged public federation.

P.1.12 ADR-012 — Paradeigma Is an Isolated Exemplar/Test Subsystem

Paradeigma exists to exercise Harmonia interfaces and workflows with deterministic synthetic clinical scenarios. It is not a production clinical dependency and must remain isolated from production architecture.

P.1.13 ADR-013 — Kleio Owns Audit Evidence and Provenance

Kleio is the Harmonia capability responsible for immutable audit evidence and governed provenance. Themis, Pylai, Ponos, Ergon, Iris, and other Harmonia subsystems may produce evidence, but do not independently own the enterprise audit model or durable audit record. Accepted audit evidence is append-only and shall not be updated, replaced, patched, deleted, or resurrected through ordinary application persistence mechanisms.

P.1.14 ADR-014 — Petasos Owns Durable Processing Transition Boundaries

Petasos governs the durable transfer of work between independently recoverable Harmonia processing activities. Ponos owns execution and orchestration of those activities; Petasos establishes the durable boundary at which responsibility for work is transferred. ActiveMQ Artemis is an implementation mechanism used by Petasos and does not itself define Harmonia processing semantics.

P.1.15 ADR-015 — Replay Is Anchored to Explicit Durable Transitions

Replay is supported only from Petasos-governed durable transition points explicitly designated as replayable. Harmonia does not provide arbitrary replay from internal implementation states. A deliberate replay creates a new processing attempt linked to the original transition through correlation, causation, and provenance; it does not modify, replace, or impersonate the original event or processing attempt. Replay, deterministic reprocessing, transport redelivery, and redelivery of an established result are distinct operations and shall not be treated as equivalent.

P.1.16 ADR-016 — Audit Is Anchored to Architecturally Significant Transitions

Kleio AuditEvents are generated for architecturally significant actions and transitions, including acceptance or rejection of work, significant commitment or authority transitions, external dispatch, and deliberate replay where policy requires evidence. Internal implementation steps such as parsing, mapping helpers, individual validation rules, and ordinary method execution are not independently audited merely because they occur. Audit, provenance, and replay remain distinct concepts but may share a durable Petasos transition as their architectural anchor.

P.1.17 ADR-017 — Processing Pressure Is Expressed as Durable Backlog

Where processing demand exceeds downstream capacity, Harmonia shall preferentially express pressure as bounded, observable, durable Petasos/Artemis backlog rather than unbounded growth in JVM heap, worker threads, database connections, or volatile work state. Queue depth, oldest-transition age, processing rate, retry/replay count, dead-letter count, and backlog drain rate form part of the operational model for transition processing.

P.1.18 ADR-018 — Mnemosyne Defines the Authoritative Durable State Boundary

Status: Accepted
Date: 24 September 2026

Context

Harmonia requires application state to remain available and recoverable across process, node and infrastructure failures while supporting high throughput, horizontal scaling and deployment across multiple

failure domains.

The original Mneme architecture considered using Infinispan as more than a conventional cache. Under this model, application state would be synchronously replicated across multiple Infinispan instances, potentially distributed across independent sites or data centres. Each site could then independently persist replicated state through its local cache-store mechanism into a local Mnemosyne persistence service.

The intended benefits were:

- high availability at the application and middleware tier rather than relying primarily upon database or storage-tier high availability;
- low write latency by allowing application processing to complete following distributed cache acceptance rather than waiting for physical database persistence;
- resilience to failure of an individual persistence store;
- independent persistence of replicated state at multiple sites;
- reduced dependence on expensive database replication and highly available storage infrastructure; and
- eventual reconciliation of persistence stores following temporary database, site or network failure.

Conceptually, the distributed Mneme fabric would therefore have formed the immediate acceptance boundary, with Mnemosyne persistence occurring asynchronously behind it.

This approach is technically feasible. However, for replicated cache state to constitute authoritative durable state, Harmonia would also need to guarantee recovery following cache, process, site and network failures. This would require Harmonia to own mechanisms for durable outstanding-write tracking, persistence retry, restart recovery, cross-site reconciliation, version consistency, conflict resolution, partition handling and eventual convergence of independent persistence stores.

The resulting capability would effectively introduce a distributed persistence and consistency layer above the existing durable persistence technologies.

Harmonia already provides distinct technologies appropriate to the principal resilience requirements:

- **Mnemosyne/PostgreSQL** provides durable application-state persistence;
- **Petasos/Artemis** provides durable transfer of work between independently recoverable processing activities; and
- **Mneme/Infinispan** provides distributed caching and efficient access to reconstructable state.

The additional complexity required to make Mneme itself an authoritative persistence boundary is therefore not justified by the expected performance, availability or infrastructure-cost benefits.

Decision

Mnemosyne defines the authoritative durable application-state boundary for Harmonia.

Application state that requires durable persistence is not considered durably accepted solely because it has been written to, replicated by, or is currently available from Mneme/Infinispan.

A successful Mneme cache operation does not constitute durable application-state acceptance.

State requiring durable persistence is considered accepted when:

1. it has been successfully committed through Mnemosyne to its authoritative durable persistence mechanism; or
2. responsibility for further processing has been successfully transferred across an explicitly defined durable Petasos transition, where the state necessary to complete or reconstruct that processing is durably recoverable.

Petasos/Artemis may therefore provide a durable **work acceptance** boundary, but does not replace Mnemosyne as the authoritative repository for committed application state.

Mneme/Infinispan will remain a distributed caching capability. State held by Mneme must be reconstructable from authoritative durable state or from an explicitly defined durable processing transition where reconstruction is part of that transition's contract.

Volatile or cached state must never be the sole recoverable representation of application state or work that Harmonia has acknowledged as durably accepted.

Architectural Responsibilities

Subsystem	Responsibility	Primary Technology	Durability Role
Mneme	Distributed caching and efficient state access	Infinispan	Non-authoritative, reconstructable state
Mnemosyne	Durable application-state persistence	PostgreSQL	Authoritative durable application state
Petasos	Transfer of work between independently recoverable processing activities	ActiveMQ Artemis	Durable processing transitions
Kleio	Immutable audit and provenance evidence	PostgreSQL	Durable immutable evidence

These responsibilities are deliberately distinct. Harmonia must not introduce implicit overlap between cache availability, processing durability and authoritative persistence.

Consequences

The decision simplifies Harmonia's failure and recovery semantics.

Positive Consequences

- Harmonia does not need to implement its own distributed database consistency and convergence mechanisms.
- Persistence failure can be represented explicitly rather than hidden behind successful cache operations.
- Database recovery and replication can use established PostgreSQL capabilities appropriate to each deployment topology.
- Petasos/Artemis can absorb processing pressure and temporary downstream outages as durable backlog rather than requiring volatile cache state to act as a persistence substitute.
- Mneme remains independently scalable and distributable without carrying an authoritative durability obligation.
- Cache contents may safely be evicted, restarted or reconstructed without causing loss of acknowledged authoritative state.
- Different deployment profiles may provide different levels of PostgreSQL, Artemis and Infinispan redundancy without changing Harmonia's logical durability semantics.
- Failure behaviour becomes easier to test and reason about.

Negative Consequences

- authoritative synchronous writes may incur greater latency than memory/cache acceptance;
- PostgreSQL remains part of the critical path for operations requiring immediate durable application-state commitment;
- deployments requiring high availability of authoritative state must provide an appropriate PostgreSQL HA/DR topology;
- Infinispan replication cannot by itself be used to mask loss of the authoritative persistence service; and
- some existing Harmonia persistence and cache interactions must be changed because they currently allow cache or JVM-local state to be treated as successfully persisted state.

These costs are accepted in preference to Harmonia assuming responsibility for distributed persistence, reconciliation and consistency.

Failure Semantics

The following invariant applies:

> **Accepted state must be durably recoverable.**

This does **not** imply:

> Accepted state must always have already been committed to PostgreSQL.

A durable Petasos transition may constitute acceptance of work where the information necessary to continue processing is durably recoverable from that transition.

However:

> **Cache presence, cache replication, JVM-local state or an asynchronous persistence attempt does not by itself constitute durable acceptance.**

Consequently:

- failure to persist required authoritative state must be propagated as failure;
- a local in-memory fallback must not convert unavailable durable persistence into apparent success;
- asynchronous processing may return acceptance only after the corresponding durable work boundary has been established;
- failure after a durable Petasos transition must result in recoverable backlog, redelivery or controlled failure rather than silent loss;
- cache loss must be recoverable from Mnemosyne or another explicitly defined durable source; and
- transient processing state such as a Pragma may remain volatile where it can be reconstructed without loss of acknowledged work.

Rejected Alternative — Mneme as a Distributed Persistence Fabric

The rejected architecture would have treated replicated Infinispan state as Harmonia's primary immediate persistence/availability boundary.

Multiple independent Mneme instances or sites would replicate state between them. Each site would independently write replicated state through its cache-store implementation to a local Mnemosyne persistence service. Failure of an individual database or persistence site would not prevent application acceptance, with failed persistence operations subsequently retried and stores gradually reconciled.

This model was rejected **not because it is technically invalid**, but because completing it safely would require Harmonia to provide substantial additional distributed-systems capability, including:

- durable tracking of outstanding persistence operations;
- recovery of pending writes following process or site restart;
- persistence retry and backoff;
- store convergence monitoring;
- version and ordering semantics;
- duplicate and idempotency handling;
- conflict detection and resolution;
- network-partition behaviour;
- reconciliation of independently persisted copies; and
- operational tooling for detecting and repairing divergence.

These responsibilities would duplicate capabilities already available through established database and durable-messaging technologies while materially increasing implementation, testing and operational complexity.

The likely reduction in persistence latency and infrastructure cost does not justify that complexity for Harmonia's expected workload and availability requirements.

Relationship to Other Decisions

This decision reinforces:

- **ADR-003 — Mnemosyne Owns Durable Application State**
- **ADR-010 — Clinical Search Must Be Authoritative-Backed**
- **ADR-014 — Petasos Owns Durable Processing Transition Boundaries**
- **ADR-015 — Replay Is Anchored to Explicit Durable Transitions**
- **ADR-017 — Processing Pressure Is Expressed as Durable Backlog**

It also establishes the architectural basis for the current implementation sequence:

```
Task 07   Remove volatile persistence fallback
             |
```

```
Task 08   Establish authoritative Clinical writes
             |
Task 09   Establish cache-aside point reads
             |
Task 10   Establish authoritative Clinical search
```

Task 07 must remove cases where cache or JVM-local state is incorrectly treated as durable acceptance without prematurely redesigning the authoritative Clinical write path that is addressed by Task 08.

In alignment with ADR-020, Task 08 establishes authoritative CREATE and UPDATE write semantics, treating domain-appropriate lifecycle changes as conditional updates, without providing a physical authoritative DELETE path.

Architectural Principle

Petasos makes work recoverable.
Mnemosyne makes application state durable.
Mneme makes active state available, coordinated and fast.
Kleio makes evidence permanent.

Harmonia will prefer these explicit responsibilities over introducing a distributed persistence protocol spanning them.

P.1.19 ADR-019 — Mneme Owns Distributed Resource Access and Coordination

Status: Accepted
Date: 25 September 2026

Context

Harmonia processes resources and data objects through validation, transformation, workflow, integration and persistence activities. Access to these objects is expected to be uneven, highly repetitive and strongly read-dominant.

At any point in time, only a comparatively small subset of the total persisted resource population is expected to be actively involved in processing. That active subset may be read repeatedly by different Harmonia components while it is validated, referenced, transformed, enriched, coordinated or progressed through workflows. A useful design expectation is approximately a 10:1 read-to-write ratio for this active working set; the ratio is indicative rather than a contractual limit.

Repeated authoritative retrieval from Mnemosyne for every such interaction would unnecessarily increase persistence traffic, database connection utilisation and processing latency.

Mneme was introduced to provide more than a convenient local cache. Using Infinispan and its distributed access mechanisms, Mneme is intended to provide a common low-latency resource-access and coordination capability across the Harmonia ecosystem.

The principal intended capabilities are:

1. **Distributed resource availability** — supported active resources and data objects can be accessed by Harmonia components regardless of which runtime node originally obtained, created or cached their working representation.
2. **Distributed concurrency and version coordination** — concurrent access to a given active resource or data object is coordinated through Mneme rather than independently by each client or JVM. Client code should not need to recreate distributed concurrency/version-management machinery.
3. **Read-dominant performance** — frequently read active state can be accessed at low latency without repeated database retrieval, while the comparatively smaller number of authoritative writes are committed through Mnemosyne.
4. **Efficient internal distribution** — native distributed-cache access and replication can be used within Harmonia without unnecessarily introducing HTTP network boundaries between internal components.

The original Mneme design also contemplated **distributed durability**: replicated Infinispan state could act as an immediate durable acceptance fabric with persistence occurring behind it. ADR-018 deliberately rejects that responsibility because the required recovery, reconciliation, partition, convergence and durable outstanding-write semantics are not justified by the expected benefit.

Rejecting distributed durability does not reduce Mneme to an optional best-effort cache. Its distributed availability, coordination and performance semantics remain deliberate architectural responsibilities.

Decision

> **Mneme owns Harmonia's distributed resource access and coordination capability for active application state.**

Mneme will use Infinispan to expose a distributed, low-latency and reconstructable representation of selected resources and data objects being actively used by Harmonia.

Mneme provides four defining characteristics:

- **distributed availability** — supported active state is accessible across Harmonia runtime instances through the distributed Mneme capability;
- **distributed coordination** — Mneme provides the common coordination point for concurrent access and working versions of an individual active resource or data object;
- **read-dominant performance** — Mneme absorbs high-volume, repetitive and potentially jittery reads against the active working set, reducing repeated access to authoritative persistence; and
- **reconstructability** — Mneme state is non-authoritative and may be rebuilt from authoritative durable state or another explicitly defined durable source.

Mneme does **not** own distributed durability.

A successful Mneme operation, distributed replication, concurrency decision or working version does not by itself constitute authoritative durable acceptance. Authoritative application-state persistence and the final persisted version remain the responsibility of Mnemosyne.

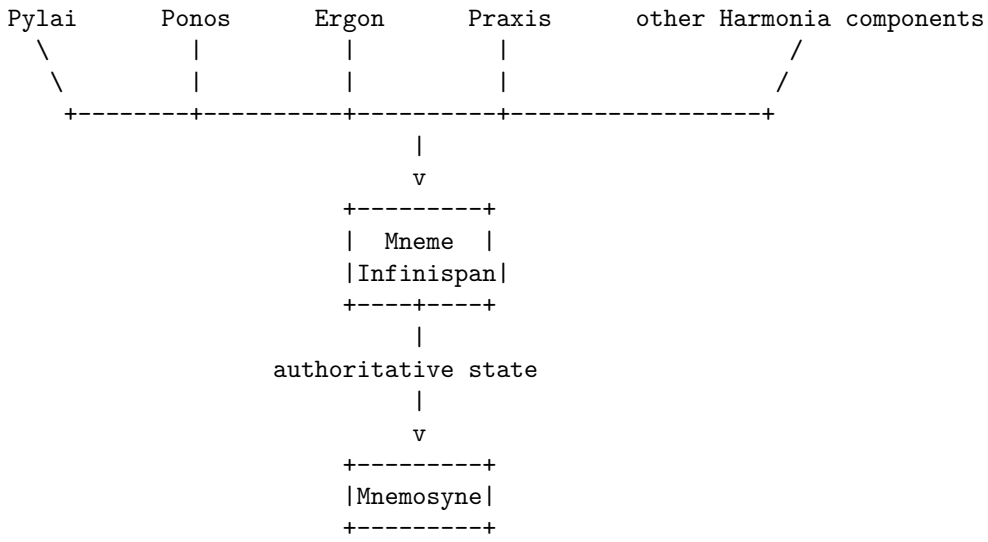
> **Mneme coordinates active distributed state; Mnemosyne commits authoritative durable state.**

The exact write-boundary protocol between Mneme coordination and Mnemosyne authoritative persistence is governed by the relevant write architecture and must preserve authoritative concurrency/version correctness at commit.

Distributed Resource Availability

Where a resource or data object is supported by Mneme, client code should be able to obtain its active representation through the Mneme capability without depending on the JVM or processing node that originally loaded or produced it.

Mneme therefore represents a system-wide resource-access capability, not a collection of unrelated per-process caches.



Distributed Concurrency and Version Coordination

Mneme is the distributed coordination point for concurrent access to an active resource or data object.

Where multiple Harmonia components or runtime nodes operate on the same active object, they must not independently establish incompatible local working versions merely because they execute in different JVMs.

Mneme should use appropriate Infinispan concurrency facilities so that the coordination mechanism is substantially transparent to ordinary client code. The implementation may use versioned or conditional operations, atomic replacement, locking or other appropriate Infinispan mechanisms, but those mechanisms are implementation choices beneath the Mneme contract.

This responsibility does not make Mneme the final authority for a durable write. Mnemosyne must still enforce the authoritative persistence transaction and persisted-version semantics.

Read-Dominant Active Working Set

Mneme is intentionally optimised around a comparatively small active subset of the total persisted resource population.

The expected access pattern is strongly read-dominant, with approximately ten reads for each write being a useful design assumption rather than a fixed service-level guarantee.

Typical processing may therefore involve repeated reads, validation, reference lookups, workflow evaluation and transformation against Mneme before a smaller number of create or update operations require authoritative persistence.

Mneme exists to prevent these repeated reads from degenerating into continuous database lookups while still allowing Mnemosyne to remain the authoritative durability boundary.

Process-Local Fallback

Where a component requires Mneme semantics:

> **Loss of Mneme must be visible as loss of distributed resource availability or coordination. It must not silently degrade into process-local application state.**

Production services must therefore not use `ConcurrentHashMap`, local collections or equivalent process-memory structures as transparent substitutes for Mneme-backed application state.

Such a fallback would lose not only distributed caching, but also Mneme's resource-availability and concurrency/version-coordination semantics.

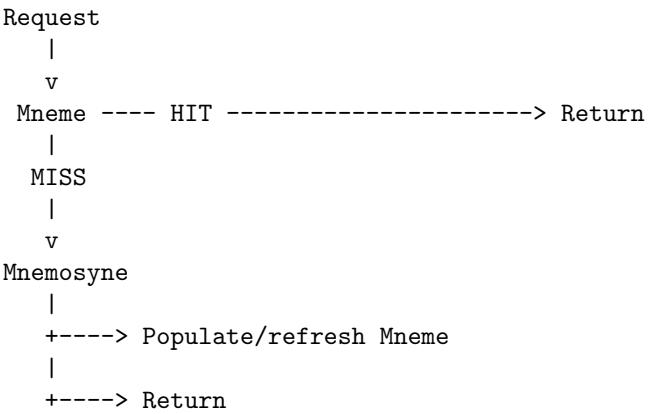
A component may respond to Mneme unavailability by failing the operation, or by using an explicitly designed degraded path where the architecture permits one. A degraded path must be deliberate and visible; it must not establish an alternative JVM-local Mneme.

This decision does **not** prohibit ordinary process-local memory. Local collections remain appropriate for temporary computation, immutable configuration, logger caches, local registrations, implementation metadata and other genuinely process-local concerns.

> **Process-local state is permitted. Process-local substitution for Mneme is not.**

Relationship Between Mneme and Mnemosyne

For read-dominant access, the intended model is:



For authoritative writes:

```
active distributed state / coordination
                  |
                  v
                Mneme
                  |
          authoritative write
                  |
                  v
             Mnemosyne
                  |
          durable commit/version
```

The precise authoritative Clinical write path is addressed separately and must not be inferred solely from this diagram.

There is deliberately no transparent substitution of the form:

```
Mneme unavailable
       |
       v
ConcurrentHashMap
       |
       v
Pretend distributed semantics still exist
```

Consequences

Positive Consequences

- supported active resources are available through a common distributed access capability across Harmonia runtime nodes;
- concurrency and working-version coordination are not independently reinvented by each client or process;
- high-volume and jittery reads can be absorbed without continuously querying Mnemosyne;
- native distributed-cache access can minimise unnecessary internal HTTP latency and protocol overhead;
- Mneme can be scaled and distributed independently of authoritative persistence;
- cache contents remain reconstructable and may be evicted or restarted without redefining durable truth; and
- the distinction between distributed coordination and durable authority is explicit.

Negative Consequences

- Mneme/Infinispan remains an important runtime dependency even though it is not a durability boundary;
- loss of Mneme may make distributed resource access or coordination unavailable until restored or an explicitly designed degraded path is used;
- concurrency/version behaviour must be designed and tested deliberately rather than assumed from cache replication alone;
- authoritative write paths must correctly bridge Mneme coordination and Mnemosyne persisted-version semantics; and
- appropriate monitoring of Mneme availability, topology and concurrency behaviour is operationally important.

These costs are accepted because distributed availability, coordination and read-dominant access are intentional Harmonia capabilities rather than incidental cache optimisations.

Rejected Alternative — Mneme as Distributed Durability

ADR-018 records the rejected distributed-durability model. The idea remains technically attractive: replicated Infinispan state could form an immediate acceptance fabric and allow persistence stores to converge behind it.

It is rejected for Harmonia because doing it safely would require durable outstanding-write tracking, restart recovery, partition semantics, reconciliation, conflict resolution, convergence monitoring and associated operational tooling. Those costs are not justified by the expected workload and availability benefit.

This rejection applies specifically to **durability**. It does not reject Mneme's distributed availability, coordination, version-management or read-performance responsibilities.

Rejected Alternative — Process-Local Cache Fallback

A JVM-local fallback may make an individual process appear available while silently destroying Mneme's distributed semantics.

```
Node A                  Node B                  Node C

Person/123 v7           Person/123 v6           Person/123 absent
Task/456 PROCESSING     Task/456 RECEIVED       Task/456 absent
```

None of these copies need be authoritative for the situation to be incorrect. The nodes no longer share the resource availability or coordination model that Mneme exists to provide.

The apparent availability benefit therefore does not justify process-local substitution.

Relationship to Other Decisions

This decision complements:

- **ADR-003 — Mnemosyne Owns Durable Application State**
- **ADR-010 — Clinical Search Must Be Authoritative-Backed**
- **ADR-014 — Petasos Owns Durable Processing Transition Boundaries**
- **ADR-017 — Processing Pressure Is Expressed as Durable Backlog**
- **ADR-018 — Mnemosyne Defines the Authoritative Durable State Boundary**

Together:

- **Mnemosyne** answers: Where does authoritative application state live and what version was durably committed?
- **Petasos** answers: Will accepted work survive until processing can continue?
- **Mneme** answers: How is active state made available, coordinated and read efficiently across Harmonia?
- **Kleio** answers: What durable evidence exists of significant actions and transitions?

Architectural Principle

> **Mneme makes active state available, coordinated and fast across Harmonia — reconstructable, never authoritative, and never secretly local.**

P.1.20 ADR-020 — Governed Information Uses Lifecycle State Rather Than Physical Deletion

Status: Accepted
Date: 25 September 2026

Context

In healthcare integration and interoperability environments, persisted resources (including patients, encounters, practitioners, clinical observations, documents, and processing tasks) represent clinical, legal, regulatory, and evidentiary facts.

Traditional persistence architectures often treat physical deletion (`DELETE` in SQL or HTTP `DELETE` in REST APIs) as a standard CRUD primitive that permanently removes records from durable storage. In a distributed health integration platform such as Harmonia, physical deletion introduces severe operational, clinical, and governance failures:

1. **Loss of Clinical and Legal History** — destroying persisted records erases the longitudinal record of care, compromises traceability, and violates healthcare data retention and evidentiary standards.

2. **Referential Integrity Destruction** — physical deletion of an entity breaks references in downstream systems, clinical documents, immutable audit records, and related resources across the enterprise.
3. **Audit and Provenance Invalidation** — accepted audit evidence governed by Kleio (ADR-013) must point to verifiable historical and provenance contexts; deleting the underlying entities destroys the audit graph.
4. **Distributed State Ambiguity** — in an ecosystem combining distributed caching (Mneme) and durable persistence (Mnemosyne), physical deletion creates unsolvable race conditions, stale read anomalies, and ambiguity regarding whether an absent cache record was deleted, evicted, or never created.

In clinical and administrative domains, when an entity is retired, deactivated, cancelled, completed, entered in error, or otherwise logically removed, this action constitutes a domain-significant lifecycle transition rather than data destruction.

Furthermore, lower-level distributed cache capabilities (such as Infinispan's version-aware conditional removal primitive `removeWithVersion()` characterized in laboratory evaluations) must not be conflated with governed resource deletion. Similarly, long-term data archival and regulatory purge have sometimes been improperly assumed to be standard in-band application persistence features.

A definitive architectural decision is required to establish Harmonia's platform invariant regarding governed persisted information, the write semantics for authoritative Clinical operations (Task 08), the boundary between cache eviction and resource state, and the formal scope of archival and purge.

Decision

> **Harmonia does not provide physical DELETE semantics for governed persisted information. Logical deletion is a domain-appropriate lifecycle transition and therefore an authoritative UPDATE.**

Harmonia establishes the following normative platform guardrail:

> **Harmonia SHALL NOT expose physical deletion as a normal operation for governed persisted information. Logical deletion SHALL be represented as a domain-appropriate lifecycle transition and processed as a concurrency-controlled authoritative update. Mneme cache eviction SHALL NOT be interpreted as deletion of authoritative state. Archival, retention-based purge and physical disposal are outside the current Harmonia framework scope.**

Four Core Distinctions

To ensure conceptual clarity across architecture, design, and implementation, Harmonia strictly distinguishes four operational concepts:

1. **Resource Lifecycle Transition**: An authoritative state change in Mnemosyne reflecting domain progression (illustrative examples include transitioning to `inactive`, `entered-in-error`, `cancelled`, `completed`, or `superseded`). A lifecycle transition is committed to Mnemosyne as a concurrency-controlled authoritative `UPDATE`, subject to version advancement, provenance capture, and Kleio audit logging.
2. **Mneme Eviction**: The removal, invalidation, or expiration of a reconstructable active-state entry from the distributed cache (Mneme/Infinispan) according to cache capacity, TTL/idle policies, explicit invalidation, or cache-clearing operations. Cache eviction affects only the non-authoritative active cache working set; it does NOT alter, mutate, or delete authoritative durable state in Mnemosyne.
3. **Archival**: The long-term migration, cold-storage management, or offline tiering of historical records. Archival is formally designated as outside the scope of the current Harmonia framework and must be managed by external operational and database lifecycle procedures.
4. **Purge / Physical Disposal**: The physical destruction, cryptographic erasure, or permanent scrubbing of persisted records under statutory data-retention schedules or legal mandates. Retention-based purge and physical disposal are formally designated as outside the scope of the current Harmonia framework.

Write Semantics and Task 08 Consequence

Harmonia categorizes authoritative write operations into distinct primitives:

- **CREATE**: Establishes a new authoritative resource in Mnemosyne, establishes initial durable state and provenance, and participates in the version and coordination protocol defined by Task 08.
- **UPDATE**: Modifies existing authoritative resource attributes or lifecycle state in Mnemosyne, advances authoritative state under the version and concurrency control protocol defined by Task 08, and coordinates state with Mneme.
- **LIFECYCLE TRANSITION**: A semantic specialization of **UPDATE** that alters lifecycle attributes (illustrative examples include domain-appropriate fields such as FHIR `status`, `active`, or `verificationStatus`). It executes strictly through the authoritative `UPDATE` write pipeline.
- **DELETE (Physical)**: **Not supported.** Harmonia does not provide an authoritative physical delete path or database `DELETE` protocol for governed persisted information.

Implementation sequence Task 08 ("Establish authoritative Clinical writes") is consequently scoped to design and implement authoritative `CREATE` and `UPDATE` write protocols between Mneme coordination and Mnemosyne persistence. Task 08 does not design, implement, or expose a physical authoritative `DELETE` path.

Concurrency and Distributed Cache Semantics

All lifecycle transitions execute through concurrency-controlled `UPDATE` protocols in Mnemosyne:

- A lifecycle transition participates in the same concurrency-control and conflict-rejection mechanisms as any other authoritative update in Mnemosyne. Stale updates that conflict with committed durable state are rejected according to the concurrency protocol established in Task 08.
- Invalid lifecycle state transitions (such as attempting an illegal transition on an already terminal resource) are rejected by domain validation rules.
- In Mneme, Infinispan's `removeWithVersion()` is a version-aware conditional removal capability that Harmonia may use for reconstructable cache management and cluster coordination. It operates strictly within the non-authoritative active cache working set and does not define or execute governed resource deletion semantics.

FHIR and Domain Lifecycle Semantics

At the protocol and API surface:

- External protocol interactions conceptually mapped to deletion (such as HTTP `DELETE` in FHIR REST APIs) do not execute physical database deletions in Mnemosyne.
- Instead, inbound protocol adapters and workflow pipelines translate such interactions into domain-appropriate lifecycle updates (illustrative examples include updating `active = false` or setting `status = 'entered-in-error'`), committing an authoritative update with associated provenance and audit evidence.
- Where domain or protocol specifications dictate specific responses (such as HTTP `410 Gone`) for retired or entered-in-error resources, the response is determined by evaluating governed lifecycle attributes in authoritative persistence, not by the physical absence of a record.

Audit and Provenance

In conformance with ADR-013 (Kleio Owns Audit Evidence and Provenance):

- Accepted audit evidence in Kleio is append-only, permanent, and immutable.
- Every lifecycle transition produces immutable Kleio `AuditEvent` records capturing the actor, authorization context, transition timestamp, prior state, and resulting state.
- Because persisted entities remain permanently in Mnemosyne, audit trails and provenance graphs remain intact and verifiable throughout the system's life.

Consequences

Positive Consequences

- **Information Preservation**: Clinical and operational history is preserved without loss of longitudinal record integrity.

- **Referential Integrity**: Cross-resource references, patient links, and downstream integrations remain structurally sound without dangling foreign keys.
- **Audit Compliance**: Kleio audit evidence and provenance graphs remain anchored to permanent, verifiable historical records.
- **Concurrency Safety**: Lifecycle transitions are protected by concurrency control and version checks, preventing lost updates and race conditions.
- **Clear Architectural Boundaries**: Distinguishes non-authoritative Mneme cache eviction from authoritative Mnemosyne state transitions, and excludes complex archival/purge engines from platform scope.

Negative Consequences

- **Monotonic Storage Growth**: Persistent database tables grow monotonically, requiring operational database capacity monitoring and storage planning.
- **Query Filtering Discipline**: Search queries and read pipelines must consistently filter on domain lifecycle state (e.g. `active = true`, `status != 'entered-in-error'`) to prevent retired records from appearing in active clinical queries.
- **API Mapping Rigor**: Inbound protocol gateways must carefully map incoming deletion requests to domain-specific lifecycle state changes.

Rejected Alternatives

- **Physical DELETE as Persistence Primitive**: Rejected because physical deletion destroys clinical history, invalidates foreign keys, breaks audit chains, and creates severe synchronization failures across distributed nodes.
- **Universal Generic Soft-Delete Flag (`is_deleted = true`)**: Rejected as the primary mechanism for domain lifecycle governance in favor of domain-native lifecycle attributes (e.g., FHIR `status`, `active`, `verificationStatus`, `period.end`). Generic boolean flags obscure domain semantics, fail to capture clinical context or reason for retirement, and complicate state validation. Any database-level columns (such as `is_deleted`) are implementation artefacts and do not replace domain-level lifecycle representation.
- **In-Band Platform Purge and Archival Subsystem**: Rejected for the current Harmonia framework. Implementing compliant multi-jurisdictional data retention, cold-storage migration, and tombstone synchronization would add significant operational complexity not justified by the core integration mission.

Relationship to Other Decisions

This decision complements and reinforces:

- **ADR-003 — Mnemosyne Owns Durable Application State**
- **ADR-006 — FHIR R5 Is the Clinical Interoperability Boundary**
- **ADR-010 — Clinical Search Must Be Authoritative-Backed**
- **ADR-013 — Kleio Owns Audit Evidence and Provenance**
- **ADR-018 — Mnemosyne Defines the Authoritative Durable State Boundary**
- **ADR-019 — Mneme Owns Distributed Resource Access and Coordination**

Architectural Principle

> **Governed information progresses through lifecycle states; it is never erased. State changes are authoritative updates; cache evictions are not deletions; archival and purge belong to external governance.**